

Top 200 Java Backend Developer Interview Questions and Answers

For developers with 3-4 years of experience

Java 17 and 21 • Spring Boot 3 • REST and HTTP • PostgreSQL • JPA and Hibernate
JUnit 5 and Mockito • Maven and Gradle • Redis • Kafka • Docker • Observability

200 questions numbered Q001 to Q200, each with a priority, a spoken answer of roughly 45 to 90 seconds, an in-depth explanation, a practical backend example, common follow-ups, mistakes to avoid and a production perspective.

Generated from the Markdown sources on 2026-09-29

Contents

How to Use This Book 7

Chapter 1. Core Java, OOP, exceptions, and language fundamentals 9

Q001. How would you apply encapsulation, abstraction, inheritance and polymorphism in an order service?	9
Q002. When would you choose an interface, abstract class or concrete class?	10
Q003. Why prefer composition over inheritance in service design?	11
Q004. What is the equals and hashCode contract, and how can it break a HashSet?	12
Q005. How is Comparable different from Comparator, and what must ordering agree with?	13
Q006. Why are Strings immutable, and how do StringBuilder and the pool affect code?	14
Q007. What is pass-by-value in Java, including object references?	14
Q008. How do checked and unchecked exceptions influence API design?	15
Q009. How would you design exception handling for a Spring service?	16
Q010. How does try-with-resources work and what happens with suppressed exceptions?	17
Q011. What do final, finally and finalize mean in modern Java?	18
Q012. How do you make a class truly immutable?	19
Q013. When are records suitable for backend DTOs, and when are they not?	20
Q014. How should Optional be used at service boundaries?	21
Q015. What is the difference between overloading and overriding?	22
Q016. How do access modifiers and packages shape a module's API?	23
Q017. What happens during class loading and static initialization?	24
Q018. How do primitives, wrappers and autoboxing create bugs?	25
Q019. How would you handle dates, times and time zones in an API?	26
Q020. How should BigDecimal be used for money?	27
Q021. When should you use enums rather than strings or booleans?	28
Q022. What does a sealed hierarchy buy you in Java 17?	29
Q023. How do switch expressions and pattern matching differ across Java 17 and 21?	30
Q024. What do annotations do, and when is reflection appropriate?	31
Q025. How would you design a value object for an email address?	32
Q026. What are the risks of mutable keys and shallow copies?	33
Q027. How do you avoid null-related bugs without hiding invalid states?	34
Q028. What is the difference between fail-fast validation and accumulating errors?	35
Q029. How would you refactor a long conditional into a maintainable strategy?	36
Q030. What makes a Java API backward-compatible for callers?	37

Chapter 2. Collections, generics, streams, and functional Java 39

Q031. How do you choose among List, Set and Map in a service?	39
Q032. How does HashMap locate a key, handle collisions and resize?	40
Q033. When would you use ArrayList versus LinkedList?	40
Q034. Which Set implementation fits deduplication, order or sorting?	41

Q035. How do iterators and fail-fast behavior work?	42
Q036. What do generic type parameters and wildcards protect?	43
Q037. What does type erasure mean for overloads and runtime checks?	44
Q038. How would you group and aggregate orders with streams?	44
Q039. What is the difference between map and flatMap?	45
Q040. How do lazy streams and terminal operations affect side effects?	46
Q041. When should you avoid parallel streams?	47
Q042. How do collectors handle grouping, ordering and duplicate keys?	48
Q043. When is a loop clearer or faster than a stream?	48
Q044. What is a functional interface, and how do lambdas capture variables?	49
Q045. How would you safely build an immutable collection result?	50
Q046. How do you use computeIfAbsent safely?	51
Q047. How would you count duplicate events efficiently?	52
Q048. How do you handle nulls in stream pipelines and collectors?	52
Q049. What is the cost of sorting and what comparator mistakes matter?	53
Q050. How would you choose a bounded in-memory data structure?	54

Chapter 3. Concurrency, asynchronous programming, and JVM 56

Q051. What makes a race condition, and how do visibility and atomicity differ?	56
Q052. What happens-before guarantees does synchronized provide?	57
Q053. When is volatile sufficient and when is it not?	58
Q054. When would you use AtomicInteger, LongAdder or a lock?	58
Q055. How do you size and manage an ExecutorService?	59
Q056. How do CompletableFuture failures and timeouts propagate?	60
Q057. How is ConcurrentHashMap different from synchronizedMap?	61
Q058. How do you identify and prevent deadlocks?	62
Q059. What are safe ways to publish and share mutable objects?	63
Q060. When are thread-local values helpful and dangerous?	64
Q061. How would you implement bounded producer-consumer work?	65
Q062. What do CountdownLatch and Semaphore solve?	65
Q063. How should interruption be handled?	66
Q064. What are virtual threads in Java 21 and when should you use them?	67
Q065. How do you avoid oversubscription when composing async calls?	68
Q066. What memory areas does the JVM use for an application?	69
Q067. How does garbage collection impact latency and throughput?	70
Q068. How would you investigate increasing heap usage?	71
Q069. How do heap OOM and native/thread exhaustion differ?	72
Q070. What do JIT warm-up and profiling mean for benchmarks?	72
Q071. How do you read a thread dump during a latency incident?	73
Q072. What happens if a task throws inside execute versus submit?	74
Q073. How do you prevent lost updates in a shared in-memory counter?	75
Q074. What are the trade-offs between blocking and nonblocking APIs?	76

Q075. How would you diagnose a service with high CPU but low throughput?	77
--	----

Chapter 4. Spring Framework and Spring Boot 78

Q076. What does dependency injection solve, and why prefer constructor injection?	78
Q077. How are component scanning and bean registration different?	79
Q078. What happens during bean creation and lifecycle?	79
Q079. What are bean scopes and singleton thread-safety implications?	80
Q080. How does Spring Boot auto-configuration decide what to create?	81
Q081. How should profiles and external configuration be managed?	82
Q082. What is the Spring MVC request lifecycle?	83
Q083. How do @RequestParam, @PathVariable and @RequestBody differ?	84
Q084. How would you validate requests and return consistent errors?	85
Q085. How does @Transactional work through proxies?	86
Q086. What do propagation modes REQUIRED and REQUIRES_NEW actually do?	87
Q087. What determines transaction rollback and isolation in Spring?	88
Q088. Why can a read-only transaction still write?	89
Q089. How do Spring filters and interceptors differ?	90
Q090. How do you avoid circular dependency and oversized services?	91
Q091. When would you use @ConfigurationProperties instead of @Value?	92
Q092. How do application events work, and what do they not guarantee?	93
Q093. What are the risks of @Async in a web application?	94
Q094. How should scheduled tasks be designed for multiple replicas?	95
Q095. How does Spring Security's filter chain fit into a request?	96
Q096. How do you test a Spring MVC controller slice?	97
Q097. What happens on application startup and how do you debug failure?	98
Q098. How do you troubleshoot a bean not found or ambiguous bean?	99
Q099. How do you configure database connections and pool limits?	100
Q100. What causes slow Spring endpoints besides slow Java code?	101
Q101. How do you handle validation of business rules versus DTO shape?	102
Q102. What is the difference between @Controller and @RestController?	103
Q103. How do you implement graceful shutdown for a Boot service?	104
Q104. How would you structure a small Spring feature end to end?	105
Q105. What should you check before upgrading a Boot 2 app to Boot 3?	106

Chapter 5. SQL, transactions, JPA, and Hibernate 108

Q106. Write a join to find customers and their latest orders.	108
Q107. How do INNER, LEFT and FULL joins differ?	109
Q108. When do WHERE and HAVING apply in grouped SQL?	110
Q109. How do subqueries, EXISTS and joins compare?	110
Q110. How would you page through a large table safely?	111
Q111. How do B-tree indexes help and when are they not used?	112
Q112. How do you read EXPLAIN ANALYZE without overreacting?	113

Q113. What are ACID properties in a money transfer?	114
Q114. What anomalies do isolation levels prevent?	115
Q115. How do database locks and deadlocks arise?	116
Q116. When choose optimistic versus pessimistic locking?	117
Q117. What are JPA entity lifecycle states?	118
Q118. How do persistence context and dirty checking work?	119
Q119. When does flush happen, and how is it different from commit?	120
Q120. How do lazy and eager loading affect endpoint performance?	120
Q121. How do you diagnose and fix an N+1 query?	121
Q122. How should entities and DTOs be separated?	122
Q123. What do cascade and orphanRemoval mean?	123
Q124. How do one-to-many mappings affect SQL and ownership?	124
Q125. How do JPQL, native SQL and projections compare?	125
Q126. What happens when a transaction calls an external service?	126
Q127. How do you batch writes without exhausting memory?	127
Q128. How should schema migrations be deployed?	128
Q129. What do unique constraints protect that application checks cannot?	129
Q130. How do you make a transfer atomic in SQL and service code?	130
Q131. Why can a composite index's column order matter?	131
Q132. How do connection pools interact with database transactions?	132
Q133. How do you detect duplicate rows from a join and fix the query?	132
Q134. How would you store and query audit history?	133
Q135. When would you choose SQL over JPA for a complex report?	134

Chapter 6. HTTP, REST APIs, and microservices 136

Q136. How do GET, POST, PUT, PATCH and DELETE differ?	136
Q137. Which HTTP status codes should an API use for common outcomes?	137
Q138. How would you design a resource-oriented order API?	138
Q139. How should validation and error responses be structured?	138
Q140. How would you implement pagination, filtering and stable sorting?	139
Q141. How does idempotency make payment creation retry-safe?	140
Q142. What are ETag and Cache-Control used for?	141
Q143. How do timeouts, retries and backoff interact?	142
Q144. When should you add a circuit breaker or bulkhead?	143
Q145. How do synchronous calls compare with asynchronous messaging?	144
Q146. How would you version an API without breaking clients?	145
Q147. How do correlation IDs and trace context travel across services?	146
Q148. What failure modes occur with a chain of service calls?	147
Q149. What is eventual consistency and when is it acceptable?	148
Q150. How would you design an order placement workflow end to end?	149
Q151. How do API gateways differ from service-to-service concerns?	150
Q152. How do you prevent duplicate requests from creating duplicate resources?	151

Q153. What should happen when a downstream service times out after committing?	152
Q154. How do you make a webhook consumer safe?	153
Q155. How would you rate-limit an API?	154
Q156. How do content negotiation and media types affect an API?	155
Q157. What is the difference between liveness and readiness?	156
Q158. How do you handle file upload and download safely?	157
Q159. How would you design a small notification service?	158
Q160. When should you split a service, and when keep a modular monolith?	159

Chapter 7. Security 160

Q161. How do authentication and authorization differ?	160
Q162. When choose server sessions versus JWT bearer tokens?	160
Q163. How should passwords be stored and verified?	161
Q164. What are OAuth2 and OpenID Connect at a practical level?	162
Q165. How do CORS and CSRF differ?	163
Q166. How do you prevent SQL injection and unsafe deserialization?	164
Q167. How would you secure a Spring Boot API endpoint?	165
Q168. How do you design resource-level authorization?	166
Q169. What should an access token contain and how is it validated?	167
Q170. How do you manage secrets across environments?	168
Q171. How do you handle sensitive data in logs and responses?	169
Q172. What security checks belong in a public file-upload endpoint?	170

Chapter 8. Testing, debugging, and coding exercises 172

Q173. How do unit, slice, integration and end-to-end tests differ?	172
Q174. How would you test a service with Mockito without over-mocking?	173
Q175. How do you test repository queries and database constraints?	174
Q176. How do you test MVC status, validation and security?	175
Q177. How would you test transaction rollback and concurrency?	176
Q178. What makes a flaky test and how do you diagnose one?	177
Q179. Write a function to find the first non-repeating character.	178
Q180. Write a function to merge overlapping intervals.	179
Q181. Write SQL for top customers by monthly order total.	180
Q182. How do you debug a slow endpoint systematically?	181
Q183. How do you investigate a sporadic 500 response?	182
Q184. What test data and clock strategies make tests deterministic?	183
Q185. How do you review a PR for correctness and maintainability?	183

Chapter 9. Messaging and caching 185

Q186. What do Kafka topics, partitions and consumer groups do?	185
Q187. What does at-least-once delivery require from a consumer?	186
Q188. How do retries and dead-letter handling work for poison messages?	187

Q189. How does the transactional outbox reduce dual-write risk?	188
Q190. How do you preserve useful event ordering?	189
Q191. What is cache-aside and when should Redis be used?	190
Q192. How do TTL and invalidation affect correctness?	191
Q193. How do you prevent a cache stampede or hot key?	192
Q194. How do Redis data structures and atomic operations help?	193
Q195. How do you choose between an event and a synchronous request?	194

Chapter 10. Deployment, observability, and production troubleshooting 195

Q196. What belongs in a production-ready Docker image?	195
Q197. Which logs, metrics and traces would you add to an API?	196
Q198. How do readiness, liveness and graceful shutdown work together?	197
Q199. How would you triage rising latency and errors after a deploy?	198
Q200. How would you investigate an intermittent production order failure end to end?	199

Top 40 Questions to Revise First 201

7-Day Interview Preparation Plan 203

Practical Mini-Exercises 205

Glossary 208

Final Coverage Checklist 212

Chapter counts 213

How to Use This Book

This book contains 200 interview questions with complete answers, aimed at a backend developer with roughly three to four years of Java experience. It assumes Java 17 or 21, Spring Boot 3, PostgreSQL with JPA and Hibernate, JUnit 5 with Mockito, Maven or Gradle, and the usual production surroundings of Redis, Kafka, Docker and an observability stack.

What each question contains

Every question follows the same nine-part structure, so you can navigate by section rather than reading linearly.

Section	What it is for
Priority	Must Know, Important or Bonus — used to triage when time is short.
Why interviewers ask it	The signal the question is probing, in one sentence.
Interview-ready answer	A spoken answer of roughly 45–90 seconds. This is the part to rehearse aloud.
In-depth explanation	The understanding behind the spoken answer, for follow-up questions and for real work.
Practical backend example	A compact, correct snippet in Java, SQL, YAML or a short checklist.
Common follow-ups	The questions that usually come next, with brief answers.
Mistakes to avoid	Answers and implementations that cost candidates offers or cause incidents.
Production perspective	What the topic looks like when a real system depends on it.
Related concepts covered	Threads to pull if you want to go deeper.

A study method that works

1. **Read the question, then answer it out loud before reading on.** Recall is what builds fluency; rereading builds only familiarity.
2. **Time yourself against 45–90 seconds.** Interview answers that run past two minutes without a pause lose the room, and answers under twenty seconds sound thin.
3. **Compare with the interview-ready answer and note only the gaps.** Do not memorise the wording — the phrasing should be yours.
4. **Answer the follow-ups without looking.** Interviewers rarely stop at the first question; the follow-up is usually where the decision is made.
5. **Run the example.** Type the snippet into a scratch project or a SQL console. Reading code and writing code produce very different levels of retention.
6. **Mark a question done only when you can state one trade-off and one production implication in your own words.** That is the standard the answers were written to.

Triage when you have limited time

- **Two evenings:** the 40 questions in `top-40.md`, spoken aloud.
- **One week:** the 7-day plan in `study-plan.md`, which assigns all 200 questions across seven sessions.
- **Longer runway:** read chapter by chapter and do the exercises in `mini-exercises.md`, which are tied to specific question IDs.

If you are interviewing for a specific role, reweight accordingly: a data-heavy role puts Chapter 5 first, a platform role puts Chapters 3, 9 and 10 first, and a product-team role puts Chapters 4, 6 and 8 first.

How to answer well in the room

- **Start with the direct answer, then justify it.** Do not narrate your way to the point.
- **Name your assumptions.** "Assuming PostgreSQL and a single service" is a strong opening, not a hedge.
- **Prefer "it depends, and here is what it depends on".** Then pick a default and defend it — an unresolved "it depends" reads as evasion.
- **Say when you do not know.** Then describe how you would find out. That answer beats a confident invention every time.
- **Use concrete examples from your own work.** The answers here give you structure; your experience gives them credibility.

What this book deliberately does not do

- It contains no interview-frequency statistics, no benchmark numbers and no citations to studies. Nothing here is invented to sound authoritative, so where a claim would need measurement, the text tells you to measure instead.
- It avoids trivia that has no bearing on real work.
- It does not present version-dependent behaviour as universal. Where something changed between Java or Spring Boot versions, the version is stated.
- It does not tell you that one technology is always better than another. Where a choice exists, the text explains when each option is the right one.

Chapter 1. Core Java, OOP, exceptions, and language fundamentals

Assume Java 17 unless a question explicitly mentions Java 21. Examples omit imports when the types are unambiguous, and use a small order-service domain so the ideas stay connected to backend work rather than to textbook shapes.

Q001. How would you apply encapsulation, abstraction, inheritance and polymorphism in an order service?

Priority: Must Know

Why interviewers ask it: They want design decisions applied to a real service, not four memorised definitions.

Interview-ready answer: Encapsulation means the Order owns its invariants: no caller sets status directly, they call markPaid or cancel, and the object rejects an illegal transition. Abstraction means checkout depends on a PaymentGateway interface describing what I need — capture, refund — not on a provider SDK. Polymorphism is what makes that useful: a Stripe implementation in production and a stub in tests satisfy the same contract, and the service does not change. Inheritance I use sparingly, only when there is a genuine is-a relationship with substitutable behaviour; sharing code is not a reason to extend a class, because composition is easier to change and to test. The practical goal is that business rules live in one place and replaceable integrations sit behind small interfaces.

In-depth explanation: Encapsulation is about valid state, not about the private keyword — a class with a private field and a public setter for every field is not encapsulated. The useful test is whether an object can ever be observed in a state your business rules forbid. Inheritance carries the Liskov obligation: a subtype must be usable wherever the supertype is, so an override that strengthens preconditions or weakens postconditions is a bug even when it compiles. Polymorphism resolves at runtime through the reference's dynamic type, which is what allows dependency inversion: high-level policy depends on an interface, and the low-level detail implements it. In a Spring application this shows up as constructor-injected interfaces, and the payoff is that swapping a provider or writing a fast test touches one line of wiring.

Practical backend example: Invariants inside the object, integration behind an interface:

```
public class Order {
    private OrderStatus status = OrderStatus.NEW;
    private String receiptId;

    public void markPaid(String receiptId) {
        if (status != OrderStatus.NEW) {                // the object protects its own rules
            throw new IllegalStateException("cannot pay an order in status " + status);
        }
        this.receiptId = Objects.requireNonNull(receiptId);
        this.status = OrderStatus.PAID;
    }
}

public interface PaymentGateway { Receipt capture(Money amount); } // abstraction I own

// polymorphism: the service is identical in production and in tests
public class CheckoutService {
    private final PaymentGateway payments;
    public CheckoutService(PaymentGateway payments) { this.payments = payments; }
}
```

Common follow-ups:

- Where is inheritance appropriate? Where the subtype genuinely satisfies the supertype's contract — a stable taxonomy or a framework extension point — not for code reuse.
- Is a private field enough encapsulation? No; public setters can still drive the object into an invalid state. Expose intent-revealing methods instead.
- How does this relate to dependency inversion? The service depends on an interface it defines; the provider adapter implements it, so the dependency points inward.

Mistakes to avoid: Reciting definitions with no example; calling every interface an abstraction while all logic sits in the controller; inheriting from a base service for two helper methods; anaemic entities that are just field bags with a service mutating them.

Production perspective: Narrow interfaces around external providers are what make a migration or an outage workaround tractable — you write one adapter instead of editing twenty call sites. The cost is indirection, so do not create an interface per class by reflex; introduce one when there is a real second implementation, including a test double that meaningfully simplifies testing.

Related concepts covered: Domain invariants, Liskov substitution, dependency inversion, test doubles, anaemic domain model.

Q002. When would you choose an interface, abstract class or concrete class?

Priority: Must Know

Why interviewers ask it: It shows how you shape extension points, and whether you add them speculatively.

Interview-ready answer: I use an interface for a capability that unrelated types can provide — `PaymentGateway`, `TaxCalculator` — because a class can implement many interfaces and that keeps the contract narrow. I use an abstract class when several closely related implementations genuinely share state or a template algorithm, and I want to fix the skeleton while subclasses fill in steps. I use a concrete class when there is no extension point, which is most of the time. Since Java 8 interfaces can have default and static methods, so "interfaces cannot have behaviour" is outdated, but they still cannot hold per-instance state. The rule I apply is not to invent an abstraction for a second implementation that does not exist yet; a concrete class is easy to extract an interface from later.

In-depth explanation: Default methods exist mainly for API evolution — adding a method to a published interface without breaking implementers — and a class inheriting conflicting defaults from two interfaces must override to disambiguate. Abstract classes buy you constructors, protected state and final template methods, at the price of single inheritance and tighter coupling: subclasses become sensitive to the base class's internals. Sealed interfaces (Java 17) add a third option when you want a closed set of implementations with exhaustive handling. A practical heuristic: if the shared thing is a contract, use an interface; if it is a partially implemented algorithm with shared state, an abstract class is honest; if it is neither, keep it concrete and let the need for abstraction emerge from a second real caller.

Practical backend example: A capability contract, a template, and a value type that needs neither:

```
public interface TaxCalculator {                // capability: unrelated implementations
    Money calculate(Order order);
    default boolean supports(Country country) { return true; } // evolvable default
}

public abstract class AbstractImportJob {      // template: shared skeleton + state
    protected final Clock clock;
    protected AbstractImportJob(Clock clock) { this.clock = clock; }
    public final ImportReport run() {          // final: the skeleton is not overridable
        var started = Instant.now(clock);
        var rows = fetch();                  // subclass supplies the steps
        return new ImportReport(rows.size(), started, Instant.now(clock));
    }
    protected abstract List<Row> fetch();
}

public record TaxRate(String region, BigDecimal percentage) {} // concrete: no extension point
```

Common follow-ups:

- Can an interface hold state? Only `public static final` constants; instance state requires a class.
- Can an abstract class implement an interface? Yes, and it may leave some methods abstract for subclasses.
- Why might you prefer a sealed interface? When you want a fixed set of implementations and exhaustive switch handling without a default branch.

Mistakes to avoid: Claiming interfaces cannot have method bodies; creating an interface with exactly one implementation for testability when a hand-written fake or a real object would do; deep abstract class hierarchies where a subclass must read the parent's source to work.

Production perspective: A published interface is a maintenance commitment — every implementer breaks when you add an abstract method. If a library-style interface must evolve, a default method with a sensible fallback avoids a breaking change; in internal code, changing all implementations is usually cheaper than accumulating defaults.

Related concepts covered: Default and static methods, template method pattern, sealed types, API evolution, speculative generality.

Q003. Why prefer composition over inheritance in service design?

Priority: Must Know

Why interviewers ask it: It probes whether you can keep change local as a codebase grows.

Interview-ready answer: Composition delegates to injected collaborators, so each behaviour can be replaced or tested on its own; inheritance binds a subclass to a base class's implementation, including details that are not part of any documented contract. The classic failure is a base service whose method calls another overridable method: a harmless-looking change to the base silently changes every subclass. With composition the dependencies are explicit in the constructor, which also makes an oversized class obvious — five collaborators is a design smell you can see. I still use inheritance where the subtype relationship is genuine and substitutable, or where a framework requires it. But "I need these two helper methods" is a reason to inject a collaborator, not to extend a class.

In-depth explanation: The fragile base class problem is the core argument: subclasses depend on the base's internal call sequence, so behaviour-preserving refactors in the base can break them, and the compiler will not warn you. Composition also composes — you can wrap a collaborator in a decorator to add retries, caching or metrics without touching either class, which inheritance cannot do at runtime. The counterweight is that composition can be over-applied: extracting every conditional into a strategy interface produces indirection with no benefit. A reasonable progression is to write concrete code first, then extract a collaborator when you have an independent policy, an external integration, or a genuine second implementation. Where you do inherit, making the class or the methods `final` unless designed for extension prevents accidental overriding.

Practical backend example: Injected policies, and a decorator that inheritance could not give you:

```
// composition: each policy replaceable and independently testable
public class CheckoutService {
    private final FraudChecker fraud;
    private final PaymentGateway payments;
    public CheckoutService(FraudChecker fraud, PaymentGateway payments) { ... }
}

// decoration: add behaviour around an existing implementation without subclassing it
public class MeteredPaymentGateway implements PaymentGateway {
    private final PaymentGateway delegate;
    private final Timer timer;
    public Receipt capture(Money amount) {
        return timer.record(() -> delegate.capture(amount));
    }
}
```

Common follow-ups:

- When is inheritance still right? Stable taxonomies, framework contracts, and sealed hierarchies where substitutability is real and checked.
- Does composition remove coupling? No; it converts implicit coupling to the base class into explicit coupling to a contract you control.
- How do you protect a class you do own from bad subclassing? Make it `final`, or make overridable methods explicitly documented and everything else `final`.

Mistakes to avoid: A `BaseService` that every service extends for logging; overriding a method to disable inherited behaviour (a substitutability violation); extracting a strategy interface for a single `if` statement.

Production perspective: Inheritance chains make incidents harder: you read three classes to learn what one endpoint actually does. Explicit constructor dependencies show the blast radius of a change at a glance, and they make it obvious when a class has quietly grown into a god object.

Related concepts covered: Fragile base class, decorator pattern, dependency injection, final classes, strategy extraction.

Q004. What is the equals and hashCode contract, and how can it break a HashSet?

Priority: Must Know

Why interviewers ask it: Violating it produces silent data loss in collections and caches, and it is the most common Java correctness trap.

Interview-ready answer: The contract is: equal objects must have equal hash codes; equals must be reflexive, symmetric, transitive, consistent, and false for null. Hash-based collections rely on it — HashMap locates the bucket by hash, then compares with equals inside the bucket. If two objects are equal but hash differently, they land in different buckets and a HashSet happily stores both duplicates. The subtler failure is mutating a field used in hashCode after insertion: the entry stays in the old bucket, so contains returns false and the entry is unreachable but still consuming memory. That is why I build keys from immutable fields, and prefer records or IDE-generated implementations over hand-written ones.

In-depth explanation: Unequal objects may share a hash code — that is a collision, handled by the bucket's chain or tree — so hashCode is about distribution, not uniqueness. Symmetry breaks easily when a subclass adds a field to equals: sub.equals(base) is false while base.equals(sub) is true, which is why getClass() comparison and instanceof comparison behave differently, and why composition is safer than inheritance for value types. For JPA entities the usual advice is to base equals on a business key rather than the generated ID, because the ID is null before persist, so an entity added to a Set before flushing changes its hash afterwards. Java 16 records generate both methods from their components, which makes them the least error-prone option for keys and DTOs.

Practical backend example: A correct key, and the mutation trap:

```
// record: equals/hashCode generated from immutable components
public record OrderKey(long tenantId, String externalReference) {
    public OrderKey {
        Objects.requireNonNull(externalReference);
    }
}

// the trap: a mutable field participating in hashCode
class MutableKey {
    String value; // mutated after insertion
    @Override public int hashCode() { return Objects.hash(value); }
    @Override public boolean equals(Object o) {
        return o instanceof MutableKey k && Objects.equals(value, k.value);
    }
}

Set<MutableKey> set = new HashSet<>();
MutableKey key = new MutableKey(); key.value = "a";
set.add(key);
key.value = "b"; // hash changes; entry stays in the old bucket
set.contains(key); // false - the entry is lost but still in memory
```

Common follow-ups:

- Must unequal objects have different hash codes? No; collisions are legal and expected. Equal objects must agree.
- What happens with a constant hashCode? Everything collides into one bucket: correct but degenerate, O(n) lookups until the bin treeifies.
- How should JPA entities implement it? Prefer a stable business key; using a generated ID breaks for unsaved entities whose ID is still null.

Mistakes to avoid: Overriding equals without hashCode; including mutable or lazily loaded fields; using == for String or Integer comparison inside equals; forgetting the null and type checks.

Production perspective: These bugs surface as caches that never hit, deduplication that lets duplicates through, or a Set whose size keeps growing — all of which look like data problems, not code problems. Records for keys and a test that inserts, mutates and re-looks-up are cheap protection.

Related concepts covered: Hash bucket distribution, records, symmetry and inheritance, JPA entity identity, cache correctness.

Q005. How is Comparable different from Comparator, and what must ordering agree with?

Priority: Important

Why interviewers ask it: Sorting contracts are easy to violate, and a violation can throw at runtime under load.

Interview-ready answer: Comparable defines a type's single natural ordering, implemented by the class itself in `compareTo`. Comparator is an external, reusable ordering you pass to a sort or a `TreeMap`, and you can have as many as you need — by total, by date, by status then date. The contract matters: the comparison must be consistent — if $a > b$ and $b > c$ then $a > c$ — antisymmetric, and it must not depend on mutable state that changes during the sort. Java's sort detects a broken comparator and throws `IllegalArgumentException: Comparison method violates its general contract!`, often only on larger inputs. I also make sure `compareTo` is consistent with `equals` for anything used in a sorted set or map, because those use comparison, not `equals`, to decide membership.

In-depth explanation: `TreeSet` and `TreeMap` consider two elements duplicates when the comparison returns zero, so an ordering inconsistent with `equals` silently drops entries — the classic case is `BigDecimal`, where `1.0` and `1.00` are unequal by `equals` but compare equal. The runtime exception comes from `TimSort`, which relies on a consistent total order to merge runs; small arrays use insertion sort and may not detect the violation, which is why these bugs appear in production rather than in tests. Subtraction-based comparators (`a.value - b.value`) overflow for large or negative values and are a frequent source of inconsistency; `Integer.compare` is both safe and clearer. Modern comparator construction — `Comparator.comparing(...).thenComparing(...).reversed()` — is composable and avoids most hand-written mistakes, and `nullsFirst/nullsLast` handles nullable fields explicitly.

Practical backend example: Composed comparators and the overflow trap:

```
// natural ordering: one per type, on the type itself
public record Money(long minorUnits, String currency) implements Comparable<Money> {
    public int compareTo(Money other) {
        if (!currency.equals(other.currency)) throw new IllegalArgumentException("currency mismatch");
        return Long.compare(minorUnits, other.minorUnits); // never subtraction
    }
}

// external orderings: as many as the use cases need
Comparator<Order> byStatusThenNewest =
    Comparator.comparing(Order::status)
        .thenComparing(Order::createdAt, Comparator.reverseOrder())
        .thenComparing(Order::id); // stable tiebreaker

orders.sort(Comparator.comparing(Order::customerName,
    Comparator.nullsLast(String.CASE_INSENSITIVE_ORDER))); // explicit null policy
```

Common follow-ups:

- What causes "Comparison method violates its general contract"? An inconsistent or non-transitive comparator, often from subtraction overflow or comparing on mutable state.
- Why can a `TreeSet` lose elements? It treats a zero comparison as equality, so an ordering inconsistent with `equals` merges distinct objects.
- Is `Collections.sort` stable? Yes for objects — equal elements keep their relative order — which is why a tiebreaker only matters for determinism across different inputs.

Mistakes to avoid: `a.getX() - b.getX()` on ints that can overflow; comparators that read a field mutated by another thread during the sort; implementing Comparable inconsistently with `equals` and then using a `TreeMap`.

Production perspective: Sorting bugs scale with data: a comparator that works on ten rows throws on ten thousand. If you sort user-controlled data, add a deterministic final tiebreaker such as the ID so that pagination does not shuffle rows between pages.

Related concepts covered: `TimSort` contract, `TreeMap` semantics, comparator composition, null handling, deterministic pagination.

Q006. Why are Strings immutable, and how do StringBuilder and the pool affect code?

Priority: Must Know

Why interviewers ask it: String behaviour touches memory, security and correctness in everyday code.

Interview-ready answer: String is immutable so it can be shared safely: it is thread-safe without synchronisation, its hash code can be cached — which is why it is a good map key — and a value cannot change after a security or validation check. The compiler interns string literals in a pool so identical literals share one object, which is exactly why comparing with `==` is unreliable: a literal and a runtime-built string are different objects with equal contents, so you compare with `equals`. Immutability means every concatenation creates a new object, so building a string in a loop with `+` allocates garbage proportional to the loop; `StringBuilder` mutates one buffer instead. A single concatenation expression is fine — the compiler handles it — it is the loop that matters.

In-depth explanation: Since Java 9, String stores bytes with an encoding flag (compact strings), so Latin-1 text uses one byte per character; this is an implementation detail, not a guarantee, but it matters for heap sizing. `String.intern()` explicitly adds to the pool, which is occasionally useful for massive duplicate data but can hurt: the pool is a hash table, and interning many distinct strings costs time and native memory. `StringBuilder` is unsynchronised and fast; `StringBuffer` is its synchronised ancestor and almost never needed. Text blocks (Java 15) make multi-line SQL and JSON readable without concatenation. One security note: because strings are immutable and pooled, a password held in a String stays in memory until garbage collected, which is why sensitive values are conventionally held in `char[]` or cleared buffers.

Practical backend example: Where the allocation actually happens:

```
// fine: the compiler compiles a single expression into efficient concatenation
String message = "order " + orderId + " confirmed";

// costly: a new String (and builder) per iteration
String csv = "";
for (OrderLine line : lines) csv += line.sku() + ";";           // O(n^2) copying

// correct: one buffer, resized geometrically
StringBuilder sb = new StringBuilder(lines.size() * 16);
for (OrderLine line : lines) sb.append(line.sku()).append(';');
String result = sb.toString();

// comparison: never ==
String fromRequest = new String("PAID".getBytes(StandardCharsets.UTF_8));
boolean wrong = fromRequest == "PAID";                       // false: different objects
boolean right = "PAID".equals(fromRequest);                   // true, and null-safe on the literal
```

Common follow-ups:

- Why does `==` sometimes work for strings? Both sides may be the same pooled literal; that is an implementation coincidence, not a rule.
- Is `String.intern()` a good idea? Rarely; it helps only with massive duplication and otherwise adds pool pressure and latency.
- `StringBuilder` or `StringBuffer`? `StringBuilder` unless you genuinely share the builder across threads, which is itself a design smell.

Mistakes to avoid: Concatenating in a loop; using `==` for value comparison; building SQL by concatenation (an injection risk as well as slow); holding secrets in long-lived String fields.

Production perspective: String duplication is a common source of heap pressure in services that parse many similar payloads; a heap dump usually shows it immediately as thousands of identical values. Fixing it with a small canonicalisation map or an enum is often a bigger win than tuning the garbage collector.

Related concepts covered: String pool and interning, compact strings, `StringBuilder` sizing, text blocks, heap analysis, secret handling.

Q007. What is pass-by-value in Java, including object references?

Priority: Must Know

Why interviewers ask it: It is a precision question: the right answer requires distinguishing a reference from the object it points to.

Interview-ready answer: Java is strictly pass-by-value. For a primitive, the value is copied. For an object, the reference is copied — so the method gets its own copy of a pointer to the same object. That means a method can mutate the object and the caller sees the change, but reassigning the parameter inside the method has no effect on the caller's variable. The practical consequence is that passing a mutable collection or entity to a method hands out the ability to modify it, which is why I return unmodifiable views or defensive copies at boundaries, and prefer immutable types for anything shared. People sometimes call this "pass-by-reference" for objects, but it is not: a true pass-by-reference language would let the method reassign the caller's variable.

In-depth explanation: The distinction matters for API design more than for trivia. If a service method accepts a `List` and stores it in a field, the caller can still mutate that list afterwards and silently change the service's state — a classic aliasing bug that also shows up with arrays returned from getters. The same applies to JPA entities handed to another layer: whoever holds the reference can change a managed entity, and dirty checking will write those changes at flush. Defensive copying on input and unmodifiable wrappers on output remove the ambiguity; `List.copyOf` both copies and rejects nulls. Note also that `final` on a parameter or field prevents reassignment of the reference, not mutation of the object — `final List` can still have elements added.

Practical backend example: Reassignment versus mutation, and the boundary fix:

```
void demonstrate(Order order, int count) {
    order.markPaid("rcp_1");           // visible to the caller: same object
    order = new Order();               // NOT visible: only this local copy is rebound
    count = 99;                       // NOT visible: primitive copy
}

// aliasing bug: the caller can still mutate the stored list
class Basket {
    private final List<OrderLine> lines;
    Basket(List<OrderLine> lines) { this.lines = List.copyOf(lines); } // defensive copy
    List<OrderLine> lines() { return Collections.unmodifiableList(lines); } // no write access out
}
```

Common follow-ups:

- Can a method change which object the caller's variable refers to? No; only the object's internal state can be changed.
- What does `final` on a parameter guarantee? Only that the parameter is not reassigned inside the method; the object remains mutable.
- Why copy collections at the boundary? Because a stored reference lets the caller mutate your internal state later, which is a bug that appears far from its cause.

Mistakes to avoid: Saying Java passes objects by reference; returning the internal collection or array from a getter; assuming an immutable wrapper makes the underlying elements immutable; treating `final` as immutability.

Production perspective: Aliasing bugs are hard to reproduce because the mutation happens somewhere else entirely. Immutable value types and copy-on-input boundaries eliminate a whole class of "who changed this?" incidents, at the cost of some allocation that is almost always irrelevant.

Related concepts covered: Aliasing, defensive copying, unmodifiable views, `final` semantics, JPA entity exposure.

Q008. How do checked and unchecked exceptions influence API design?

Priority: Must Know

Why interviewers ask it: The choice shapes every caller's code, and most candidates have only a stylistic opinion.

Interview-ready answer: A checked exception is part of the method signature: callers must catch it or declare it, so it says "this failure is expected and you should decide what to do". An unchecked exception says "this is a programming error or an unrecoverable condition". My rule is to use unchecked exceptions for almost everything in application code — a business rule violation, a missing resource — because there is usually one place, a global handler, that turns them into responses, and checked exceptions force intermediate layers to handle or propagate failures they cannot act on. I use checked exceptions where the caller genuinely has a recovery path and I want the compiler to enforce the decision. What matters more than the category is that the exception carries meaning: a typed `OrderNotFoundException` with the ID, not a bare `RuntimeException` with a string.

In-depth explanation: The real costs of checked exceptions are the pressure to write `catch (Exception e) {}` to make code compile, and the fact that they leak through abstractions — a `SQLException` in a service signature exposes persistence to the web layer. Spring's design is instructive: it wraps JDBC's checked `SQLException` in the unchecked `DataAccessException` hierarchy precisely so that callers are not forced to handle what they cannot fix, while still allowing specific subtypes such as `DuplicateKeyException` to be caught where a recovery exists. When translating, always keep the cause (`throw new OrderProcessingException(msg, e)`) or the stack trace of the original failure is lost, which makes production debugging much harder. Also design for the handler: exceptions that carry structured data — the resource type, the ID, a stable error code — let you produce consistent API errors without parsing messages.

Practical backend example: Typed unchecked exceptions and translation that preserves the cause:

```
public class OrderNotFoundException extends RuntimeException {    // unchecked: no recovery here
    private final UUID orderId;
    public OrderNotFoundException(UUID orderId) {
        super("order not found: " + orderId);
        this.orderId = orderId;    // structured, not just text
    }
    public UUID orderId() { return orderId; }
}

// translate at the boundary, keep the cause
try {
    return paymentClient.capture(request);
} catch (IOException e) {    // checked, provider-specific
    throw new PaymentUnavailableException("payment provider unreachable", e);    // cause preserved
}
```

Common follow-ups:

- Why does Spring translate `SQLException`? So callers are not forced to handle low-level failures, while typed subclasses still allow targeted recovery.
- When is a checked exception the right call? When the caller has a realistic, specific recovery action and you want the compiler to insist on it.
- What must never be lost when wrapping? The cause. Without it you lose the original stack trace and usually the actual reason.

Mistakes to avoid: Swallowing exceptions with an empty catch; throws `Exception` on service methods; catching `Throwable`; using exceptions for normal control flow; losing the cause when rethrowing.

Production perspective: Exception types are an operational interface: they drive alerting, retry decisions and error dashboards. A hierarchy that distinguishes "client sent something invalid" from "a dependency is down" lets you alert on the second without being paged for the first.

Related concepts covered: Exception translation, cause chaining, `DataAccessException`, error codes, alerting on failure classes.

Q009. How would you design exception handling for a Spring service?

Priority: Must Know

Why interviewers ask it: Consistent error responses are a visible sign of API maturity, and the layering is easy to get wrong.

Interview-ready answer: I throw meaningful domain exceptions where the problem is detected, and translate them to HTTP in exactly one place — a `@RestControllerAdvice` with `@ExceptionHandler` methods. Controllers do not contain try/catch for business failures, and services do not know about status codes. The advice maps each exception type to a status and builds a consistent body; in Spring 6 I use `ProblemDetail`, the standard problem-details format (RFC 7807, updated by RFC 9457), with a stable machine-readable code, a human-readable message, and a correlation ID so support can find the request in the logs. Validation failures produce per-field errors. Unexpected exceptions get a 500 with a generic message — logged in full server-side, with no stack trace or internal detail in the response.

In-depth explanation: Order of specificity matters: Spring picks the most specific handler, so a broad `ExceptionHandler` handler is a safety net, not the main path. Log levels should differ by class of failure — client errors at debug or info,

dependency failures at warn, unexpected errors at error with the stack trace — otherwise your error dashboard is dominated by users typing bad input. Some failures never reach `@ExceptionHandler`: anything thrown in a filter or during security processing is handled by the filter chain, so authentication and authorization errors need their own entry points to produce the same body shape. Keep the error contract documented and stable; clients branch on it, so changing a code is a breaking change.

Practical backend example: One translation point, one body shape:

```
@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(OrderNotFoundException.class)
    ProblemDetail handleNotFound(OrderNotFoundException ex) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        problem.setProperty("code", "ORDER_NOT_FOUND"); // stable, machine-readable
        problem.setProperty("correlationId", MDC.get("correlationId"));
        return problem; // 404
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    ProblemDetail handleValidation(MethodArgumentNotValidException ex) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, "Validation failed");
        problem.setProperty("errors", ex.getBindingResult().getFieldErrors().stream()
            .map(f -> Map.of("field", f.getField(), "message", f.getDefaultMessage())).toList());
        return problem; // 400 with per-field detail
    }

    @ExceptionHandler(Exception.class)
    ProblemDetail handleUnexpected(Exception ex) {
        log.error("unhandled exception correlationId={}", MDC.get("correlationId"), ex); // full detail in logs
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.INTERNAL_SERVER_ERROR, "Unexpected error"); // generic to the client
        problem.setProperty("correlationId", MDC.get("correlationId"));
        return problem;
    }
}
```

Common follow-ups:

- Why not catch in the controller? It scatters mapping logic and produces inconsistent bodies; one advice keeps the contract in one place.
- What about exceptions thrown in filters? They bypass `@ExceptionHandler`; security and filter errors need their own entry points to match the format.
- Should the client see the stack trace? Never. Log it with a correlation ID and return that ID instead.

Mistakes to avoid: Returning 200 with an error field; leaking exception messages containing SQL or internal hostnames; logging the same exception at every layer; using 500 for validation failures.

Production perspective: A stable error contract with codes and correlation IDs is what makes support tractable: a customer quotes an ID, you find the exact request. Without it, every investigation starts with a timestamp and a guess.

Related concepts covered: `ProblemDetail` (RFC 7807, updated by RFC 9457), `@RestControllerAdvice`, validation errors, correlation IDs, log-level discipline, information disclosure.

Q010. How does try-with-resources work and what happens with suppressed exceptions?

Priority: Important

Why interviewers ask it: Resource leaks are a real production failure mode, and the suppression detail shows depth.

Interview-ready answer: Try-with-resources declares resources that implement `AutoCloseable` in the parentheses, and the compiler generates the `close()` calls in a finally block, in reverse order of declaration, whether the body completes normally or throws. It removes the classic bug where an exception in the body hides the close, or an exception in close replaces the real failure. That last case is what suppression handles: if the body throws and `close()` also throws, the body's exception propagates and the close exception is attached to it, retrievable via `getSuppressed()`. Before this construct, the close exception would overwrite the original one and you would debug

the wrong problem. In backend code it matters for streams, JDBC objects and any client with a connection.

In-depth explanation: Since Java 9 you can name an existing effectively-final variable in the resource list, which avoids a redundant local. Multiple resources are closed in reverse order, which is what you want for a `ResultSet` inside a `Statement` inside a `Connection`. It is worth knowing which objects actually need closing: in a Spring application, `JdbcTemplate` and `JPA` manage connections for you, so manual closing is limited to things like `Files.lines`, `Stream` over I/O, `HTTP` response bodies, and files you open directly. A frequently missed case is a `Stream` returned from a repository method — Spring Data's `Stream` results hold a database cursor and must be closed, so they belong in a `try-with-resources`. Note that the suppressed exception is easy to lose in logs if you log only `getMessage()`; logging the throwable prints suppressed entries too.

Practical backend example: Closing in reverse order, and reading the suppressed cause:

```
// resources closed in reverse order: rs, then ps, then connection
try (Connection connection = dataSource.getConnection();
    PreparedStatement ps = connection.prepareStatement("SELECT id FROM \"order\" WHERE status = ?")) {
    ps.setString(1, "PAID");
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) { ... }
    }
}

// Spring Data streaming results hold a cursor: close them
try (Stream<Order> orders = orderRepository.streamAllByStatus(OrderStatus.PAID)) {
    orders.forEach(this::export);
}

// suppression: the body's exception wins, close's exception is attached
try (var resource = new FailingResource()) {
    throw new IllegalStateException("primary failure");
} catch (Exception e) {
    log.error("failed: {} suppressed={}", e.getMessage(), Arrays.toString(e.getSuppressed()), e);
}
```

Common follow-ups:

- What if both the body and `close()` throw? The body's exception propagates; the close exception is added to its suppressed list.
- Can you use an existing variable? Yes, since Java 9, provided it is final or effectively final.
- Does Spring still need this? For managed resources no, but streaming repository results, file I/O and raw JDBC all still require it.

Mistakes to avoid: Closing inside the `try` body instead of using the construct; ignoring a streaming repository result's cursor; catching and logging only the message so suppressed exceptions vanish; assuming `finally` alone is equivalent.

Production perspective: Leaked connections and cursors manifest as pool exhaustion under load — requests queue and time out while the database looks idle. Because the leak accumulates, the failure typically appears hours after deployment, which makes the cause easy to misattribute.

Related concepts covered: `AutoCloseable`, suppressed exceptions, reverse close order, streaming result sets, connection pool exhaustion.

Q011. What do final, finally and finalize mean in modern Java?

Priority: Important

Why interviewers ask it: Three similar names with unrelated meanings, one of which is deprecated — it is a quick precision check.

Interview-ready answer: `final` is a modifier: a final variable cannot be reassigned, a final method cannot be overridden, a final class cannot be extended. It does not make an object immutable — a `final List` can still have elements added — but final fields do have a memory-model guarantee: correctly constructed objects publish their final fields safely to other threads. `finally` is the block that runs after `try/catch` regardless of outcome, used for cleanup, though `try-with-resources` is better for closeable resources. `finalize` was a method the garbage collector could call before reclaiming an object; it was unpredictable, harmful to performance, and is deprecated for removal — `Cleaner` or

explicit `close()` is the modern approach. In practice I use `final` constantly, `finally` occasionally, and `finalize` never.

In-depth explanation: The `final`-field guarantee is genuinely useful: if an object's fields are `final` and the reference does not escape during construction, other threads observing the reference see fully initialised values without synchronisation, which is the basis of safe immutable value objects. `finally` has a sharp edge: a `return` or a thrown exception inside `finally` discards whatever the `try` block was returning or throwing, silently swallowing failures — most static analysers flag it. On `finalize`, the reasons for deprecation are worth knowing: finalisers run on an unspecified thread at an unspecified time, can resurrect objects, and delay reclamation by at least one GC cycle. `Cleaner` registers a cleanup action tied to reachability without those hazards, but even it is a backstop; deterministic `close()` in `try-with-resources` is the correct design.

Practical backend example: The useful one, the sharp edge, and the modern replacement:

```
public final class Money {                // final class: no subclass can break invariants
    private final long minorUnits;        // final fields: safely published to other threads
    private final String currency;
    public Money(long minorUnits, String currency) {
        this.minorUnits = minorUnits;
        this.currency = Objects.requireNonNull(currency);
    }
}

int broken() {
    try { return 1; }
    finally { return 2; }                // returns 2 and discards the try result - never do this
}

// instead of finalize: deterministic close, with Cleaner only as a safety net
public class NativeBuffer implements AutoCloseable {
    private static final Cleaner CLEANER = Cleaner.create();
    private final Cleaner.Cleanable cleanable;
    public void close() { cleanable.clean(); }
}
```

Common follow-ups:

- Does `final` make an object immutable? No; it prevents reassignment of the variable, not mutation of the object.
- Can `finally` swallow an exception? Yes, if it returns or throws — a well-known source of hidden failures.
- What replaced `finalize`? Deterministic `close()` with `try-with-resources`, and `Cleaner` when a safety net is genuinely needed.

Mistakes to avoid: Claiming `final` gives immutability; returning from `finally`; relying on finalisers to release resources; assuming `System.gc()` makes any of this deterministic.

Production perspective: Marking value classes and their fields `final` is free defensive design: it documents intent, enables safe sharing across threads, and lets the JIT make stronger assumptions. Relying on finalisers, by contrast, has caused real outages by holding native resources far longer than expected.

Related concepts covered: Final field semantics, safe publication, `try-with-resources`, `Cleaner`, control-flow hazards.

Q012. How do you make a class truly immutable?

Priority: Must Know

Why interviewers ask it: Immutability is the cheapest concurrency and correctness tool, and "truly" hides the interesting details.

Interview-ready answer: Make the class `final` so behaviour cannot be overridden, make every field `private` and `final`, set them all in the constructor, and provide no method that changes state. The part people miss is deep immutability: if a field is a mutable object — a list, a date, an array — you must copy it on the way in and never hand out the internal reference on the way out, otherwise callers can mutate your "immutable" object from outside. Validate invariants in the constructor so an instance can never exist in an invalid state. Records give you most of this for free, but not deep immutability: a record component holding a `List` is still mutable unless you copy it in the compact constructor.

In-depth explanation: The payoff is that immutable objects are inherently thread-safe and can be shared, cached and used as map keys without defensive thinking. `Final` fields also carry the memory-model guarantee that other threads

see fully constructed values, provided the object does not leak this during construction — for example by registering a listener in the constructor. `List.copyOf` is the concise way to copy and freeze in one step, and it rejects nulls; `Collections.unmodifiableList` wraps without copying, so the underlying list can still change beneath you. For "modification", provide with-style methods that return a new instance. The cost is allocation, which is usually irrelevant, but for very hot paths or very large structures a mutable builder used during construction and frozen afterwards is the standard compromise.

Practical backend example: Deep immutability with a record, including the copy step:

```
public record Order(UUID id, Money total, List<OrderLine> lines, Instant createdAt) {
    public Order {
        Objects.requireNonNull(id);
        lines = List.copyOf(lines);
        if (lines.isEmpty()) throw new IllegalArgumentException("an order needs at least one line");
    }
    public Order withLineAdded(OrderLine line) {
        var updated = new ArrayList<>(lines);
        updated.add(line);
        return new Order(id, total.plus(line.amount()), updated, createdAt);
    }
}

List<OrderLine> caller = new ArrayList<>(List.of(line));
Order order = new Order(id, total, caller, now);
caller.add(otherLine);
```

// compact constructor
// defensive copy: caller's list is detached
// "modify" by producing a new instance
// no effect: the record copied the list

Common follow-ups:

- Do records guarantee immutability? They give shallow immutability; mutable components must be copied in the compact constructor.
- Why copy on the way out too? Returning the internal collection lets a caller mutate your state; return an unmodifiable view or a copy.
- What is the memory-model benefit? Final fields are safely published, so other threads see fully initialised values without extra synchronisation.

Mistakes to avoid: Storing a caller's list directly; exposing an array from a getter; a setter "just for the framework"; leaking this from a constructor; treating `Collections.unmodifiableList` over a field you keep mutating as immutable.

Production perspective: Immutable domain values remove an entire class of concurrency bugs and make reasoning about caches straightforward — if the value cannot change, a cached copy cannot go stale by mutation. Where allocation genuinely matters, measure before trading that away.

Related concepts covered: Deep versus shallow immutability, defensive copying, safe publication, records, builder pattern, with-style updates.

Q013. When are records suitable for backend DTOs, and when are they not?

Priority: Important

Why interviewers ask it: Records are common in modern Java code, and knowing their limits is more useful than knowing the syntax.

Interview-ready answer: Records are an excellent fit for data carriers: request and response DTOs, value objects, query projections, event payloads. They give you a canonical constructor, accessors, equals, hashCode and toString from the component list, and the compact constructor is a natural place to validate. They are a poor fit for JPA entities, which need a no-argument constructor, non-final fields for proxying and dirty checking, and an identity that is not value-based. They are also awkward when you need partial construction, many optional fields or framework-mandated setters. Jackson supports records natively in recent versions, and Bean Validation annotations work on components, so for API DTOs they are usually the better default.

In-depth explanation: A record's equals is value-based over all components, which is right for DTOs and wrong for entities: two Order entities with the same field values are not the same order, and an entity's identity must survive field changes. The compact constructor runs before fields are assigned, so it is the place to normalise (trim, lowercase) and to copy mutable components. Records cannot extend a class, though they can implement interfaces, and they combine

well with sealed interfaces for closed result hierarchies. For JSON, Jackson 2.12+ handles records including constructor binding, and adding `@JsonProperty` on components lets you keep wire names stable while renaming Java fields. Where a DTO has fifteen optional fields, a builder is friendlier than a fifteen-argument canonical constructor — that is a signal about the DTO's design as much as about records.

Practical backend example: Record DTO with validation and normalisation, versus an entity that should not be one:

```
// good: API request DTO - validated, normalised, immutable
public record CreateOrderRequest(
    @NotBlank String customerReference,
    @NotEmpty List<Valid OrderLineRequest> lines,
    @NotNull @Positive Long totalMinorUnits) {
    public CreateOrderRequest {
        customerReference = customerReference == null ? null : customerReference.trim();
        lines = lines == null ? List.of() : List.copyOf(lines);
    }
}

// good: projection from a query
public record CustomerRevenue(UUID customerId, String name, long netMinorUnits) {}

// not a record: JPA needs a no-arg constructor, mutable fields and identity-based equality
@Entity
public class Order {
    @Id @GeneratedValue private UUID id;
    private OrderStatus status;
    protected Order() { } // required by JPA
}
```

Common follow-ups:

- Why not use records for entities? JPA requires a no-argument constructor and mutable state for proxies and dirty checking, and entity identity is not value-based.
- Can records be validated? Yes; Bean Validation annotations on components work, and the compact constructor can enforce invariants directly.
- Can a record implement an interface? Yes, and sealed interface plus records is a clean way to model a closed set of results.

Mistakes to avoid: Mapping a record directly to a database table; assuming a record with a `List` component is deeply immutable; using records for objects with rich behaviour and lifecycle; exposing entity records over the API and coupling the wire format to the schema.

Production perspective: Records make the entity/DTO boundary easy to keep, which is what prevents schema changes leaking into your API and mass-assignment bugs leaking into your schema. That boundary costs a mapping layer and pays for itself the first time a column is renamed.

Related concepts covered: DTO versus entity, compact constructors, Jackson binding, Bean Validation, sealed hierarchies, projections.

Q014. How should Optional be used at service boundaries?

Priority: Must Know

Why interviewers ask it: `Optional` is widely misused, and the misuse patterns are easy to spot in an answer.

Interview-ready answer: `Optional` is designed as a return type for a method whose result may legitimately be absent — a lookup that finds nothing. I use it there, and I let the caller decide: `orElseThrow` for a mandatory resource, `map` to transform, `orElse` for a default. I avoid it as a field type or as a method parameter — it is not serialisable, it adds an allocation and a wrapper for no benefit, and an optional parameter is better expressed as an overload. I never call `get()` without checking, because that is just a `NoSuchElementException` waiting to happen. And absence is not the same as an error: a missing optional record is `Optional.empty()`, whereas a corrupt one should throw.

In-depth explanation: Spring Data repositories return `Optional` from `findById` precisely so the missing case is explicit at the type level, and `orElseThrow(() -> new OrderNotFoundException(id))` turns it into a domain exception that the error handler maps to 404. Chaining reads well — `repository.findById(id).map(Mapper::toDto).orElseThrow(...)` — but nested optionals or `Optional<List>`

are smells: an empty list already expresses "nothing found". Serialisation is a practical constraint: Jackson can be configured to handle Optional fields, but the resulting JSON contract is confusing, so DTOs should use nullable fields or omit them. In streams, flatMap(Optional::stream) (Java 9+) is the idiomatic way to drop empties. As a rule of thumb: Optional at the boundary of a lookup, plain types everywhere else.

Practical backend example: Idiomatic use, and the patterns to avoid:

```
// repository returns Optional; the caller chooses the policy
public OrderDto get(UUID id) {
    return orderRepository.findById(id)
        .map(OrderDto::from)
        .orElseThrow(() -> new OrderNotFoundException(id));    // mandatory -> 404
}

public ShippingPreference preferenceFor(UUID customerId) {
    return preferences.findById(customerId)
        .orElse(ShippingPreference.DEFAULT);    // optional -> default
}

// collecting present values from a stream
List<Order> found = ids.stream()
    .map(orderRepository::findById)
    .flatMap(Optional::stream)    // Java 9+
    .toList();

// avoid: Optional fields, Optional parameters, and unchecked get()
// record Dto(Optional<String> note) {}    // not serialisation-friendly
// void save(Optional<Order> order) {}    // use an overload instead
// Order o = repository.findById(id).get();    // throws NoSuchElementException
```

Common follow-ups:

- Should an Optional field appear in a DTO? No; use a nullable field or omit it. Optional is not designed for serialisation or for state.
- What about Optional<List<T>>? Redundant — an empty list already means "none".
- orElse or orElseGet? orElseGet when the default is expensive, because orElse evaluates its argument eagerly even when a value is present.

Mistakes to avoid: get() without isPresent; Optional parameters; isPresent/get pairs where map/orElseThrow reads better; using Optional to signal errors rather than absence.

Production perspective: The value is that "not found" becomes a compile-time consideration rather than a null that fails three frames away. Combined with a typed not-found exception and a single error handler, it is what makes 404 versus 500 correct across an API without per-endpoint effort.

Related concepts covered: Null-safety, Spring Data lookups, stream flatMap, serialisation constraints, eager versus lazy defaults.

Q015. What is the difference between overloading and overriding?

Priority: Important

Why interviewers ask it: It separates compile-time from runtime dispatch, which explains several surprising bugs.

Interview-ready answer: Overloading is multiple methods with the same name and different parameter lists in the same class; the compiler picks one from the static types of the arguments, so it is resolved at compile time. Overriding is a subclass replacing a superclass method with the same signature; the JVM picks the implementation from the object's runtime type, which is dynamic dispatch and the basis of polymorphism. The practical consequence is that overloading with a null literal or with an Object versus a specific type can bind to a method you did not expect, and that overload choice does not change just because the runtime type is a subclass. Overriding has rules: the signature must match, the return type may be covariant, access cannot be narrowed, and a checked exception cannot be broadened. I use @Override so the compiler catches a typo that would otherwise create an accidental overload.

In-depth explanation: Overload resolution follows a specificity order — exact match, widening, boxing, varargs — which is why remove(int) and remove(Object) on List behave so differently: list.remove(1) removes the element at index 1, while list.remove(Integer.valueOf(1)) removes the value. A null argument binds to the most specific applicable type and is ambiguous when two unrelated types apply, which is a compile error. Overriding is

also where the private/static distinction bites: private and static methods are not overridden, they are hidden, so calling a static method through a reference uses the reference's static type. For substitutability, an override must honour the original contract, and if you need to reuse the parent's behaviour, `super.method()` makes that explicit rather than implicit.

Practical backend example: The classic overload surprise and a correct override:

```
List<Integer> ids = new ArrayList<>(List.of(10, 20, 30));
ids.remove(1);           // overload on int: removes INDEX 1 -> [10, 30]
ids.remove(Integer.valueOf(10)); // overload on Object: removes the VALUE 10

class NotificationService {
    void send(Object payload) { ... }
    void send(String message) { ... }
}
Object payload = "hello";
service.send(payload); // binds to send(Object): chosen from the STATIC type

class EmailGateway implements NotificationGateway {
    @Override // compiler verifies this really overrides something
    public DeliveryResult send(Notification notification) { ... } // covariant return allowed
}
```

Common follow-ups:

- Can a static method be overridden? No; it is hidden. Dispatch uses the reference's static type.
- Can an override change the return type? Yes, to a covariant (more specific) type, but not to an unrelated one.
- Why is `@Override` recommended? It turns a misspelled or mismatched signature into a compile error instead of a silent new method.

Mistakes to avoid: Overloading with types that differ only by boxing or by `Object`; relying on runtime type to pick an overload; narrowing access or broadening checked exceptions in an override; omitting `@Override`.

Production perspective: Overload ambiguity causes bugs that are invisible in review because the call site looks obviously correct — the `List.remove` case has caused real data bugs. Distinct method names (`removeAt`, `removeValue`) are often better than clever overloading in application code.

Related concepts covered: Static versus dynamic dispatch, overload resolution rules, covariant returns, method hiding, `@Override` safety.

Q016. How do access modifiers and packages shape a module's API?

Priority: Important

Why interviewers ask it: Visibility is an architectural tool, and most developers use only public and private.

Interview-ready answer: The four levels are private (class only), package-private (the default, same package), protected (package plus subclasses), and public (everywhere). Used deliberately they define a module's surface: I put a feature's classes in one package, keep helpers and implementation classes package-private, and expose only the few types other features need. That makes the public surface small enough to reason about and refactor. Package structure matters as much as the keywords — layer-by-layer packages (controller, service, repository) force almost everything to be public because collaborators live in different packages, whereas feature packages (orders, payments) let internals stay package-private. The Java module system can enforce this at the JAR level with exported packages, but most Spring applications rely on package discipline plus a build-time check.

In-depth explanation: protected is often misunderstood: it allows access from a subclass, but through a reference of the subclass type, not to arbitrary instances of the parent from another package. Spring adds practical constraints: proxied methods must be visible to the proxy, so `@Transactional` on a private method does nothing at all with the default proxying, and CGLIB subclass proxies cannot override `final` or `private` methods. That is a framework constraint, not a language rule, and it is a reason to keep proxied methods public or at least package-private rather than making everything public reflexively. For enforcing boundaries over time, ArchUnit tests ("no class in `orders.internal` may be accessed from `payments`") turn conventions into failing builds, which is the only mechanism that survives team growth.

Practical backend example: Feature packaging with a narrow public surface:

```
com.example.orders
■■■■ OrderService.java      (public - the feature's API)
■■■■ OrderController.java   (public - web entry point)
■■■■ Order.java             (public - domain type used by the API)
■■■■ internal
    ■■■■ OrderTotalsCalculator.java (package-private - implementation detail)
    ■■■■ OrderRepository.java      (package-private - nobody outside orders needs it)
```

```
class OrderTotalsCalculator {          // package-private: invisible outside com.example.orders
    .internal
    Money calculate(List<OrderLine> lines) { ... }
}

// ArchUnit: make the boundary a build failure, not a convention
@ArchTest
static final ArchRule internals_are_not_used_elsewhere = noClasses()
    .that().resideOutsideOfPackage("com.example.orders..")
    .should().accessClassesThat().resideInAPackage("com.example.orders.internal..");
```

Common follow-ups:

- What is the default modifier for a top-level class? Package-private; only members can be private.
- Is protected effectively public? No; it grants subclass access through the subclass's own type, plus package access.
- Why can visibility break Spring proxies? Proxies wrap or subclass the bean, so private and final methods are not intercepted — `@Transactional` on a private method silently does nothing.

Mistakes to avoid: Making everything public because the test is in another package; treating protected as a documentation hint; layer-only packaging that defeats package-private entirely; relying on comments to mark "internal" classes.

Production perspective: A small public surface is what makes refactoring safe: you can change an internal class without auditing the whole codebase. Teams that enforce boundaries with automated rules keep that property; teams that rely on conventions lose it within a year.

Related concepts covered: Package-private design, feature packaging, Spring proxy visibility rules, ArchUnit enforcement, JPMS exports.

Q017. What happens during class loading and static initialization?

Priority: Important

Why interviewers ask it: Startup failures and mysterious `NoClassDefFoundErrors` come from this area.

Interview-ready answer: A class goes through loading (finding and reading the bytecode), linking (verification, preparation of static fields with default values, and resolution), and initialisation (running static field initialisers and static blocks in textual order, after the superclass is initialised). Initialisation is lazy and happens on first active use — creating an instance, calling a static method, reading a non-constant static field. Compile-time constants are inlined by the compiler, so reading one does not trigger initialisation at all. The operational detail that matters: if a static initialiser throws, you get `ExceptionInInitializerError`, the class is marked erroneous, and every later use throws `NoClassDefFoundError` — which is confusing because the second error names a class that clearly exists. So I keep static initialisation trivial and put anything that can fail into managed startup code.

In-depth explanation: Class identity is the pair (class loader, binary name), so the same class loaded by two loaders produces two distinct types and a `ClassCastException` that looks impossible — a real issue in application servers and plugin systems, less so in a plain Spring Boot fat jar. Static state is per class loader, which also means "singletons" are not global in a container with several loaders. In a Spring application, prefer a `@Bean` or `@PostConstruct` over a static block for anything that reads configuration or performs I/O: the container controls ordering, failures are reported with context, and the work is testable. Static initialisers that call network services are a known cause of hangs at startup with no useful log line, because they run before most logging is configured.

Practical backend example: The failure mode and the managed alternative:

```

class Rules {
    // dangerous: I/O in a static initialiser
    static final List<Rule> RULES = fetchRemoteRules();    // throws -> ExceptionInInitializerError
    static final int MAX_RETRIES = 3;                    // compile-time constant: inlined by callers
}

// later, somewhere unrelated:
Rules.RULES.size();    // NoClassDefFoundError: Could not initialize class Rules (the real cause is gone)

```

```

// managed: ordering, failure reporting and testability handled by the container
@Configuration
class RuleConfiguration {
    @Bean
    RuleSet ruleSet(RuleClient client) {                // failure surfaces as a clear startup error
        return client.fetchRules();
    }
}

```

Common follow-ups:

- Do static fields belong to instances? No; they belong to the class as loaded by a particular class loader.
- What happens after a static initialiser fails? The class is unusable for that loader; subsequent access throws `NoClassDefFoundError`.
- Why can two "identical" classes be incompatible? Class identity includes the loader, so different loaders yield different types.

Mistakes to avoid: Network or file I/O in static blocks; assuming `static final` always means compile-time constant; relying on static mutable state for caching in a container; catching `ExceptionInInitializerError` instead of fixing the cause.

Production perspective: Startup failures need a visible cause and a deployment policy — a container that crash-loops with `NoClassDefFoundError` tells you nothing about the original exception unless it was logged when it first happened. Moving initialisation into beans makes the first failure the one you see.

Related concepts covered: Loading, linking and initialisation, lazy initialisation, constant inlining, class loaders and identity, Spring startup lifecycle.

Q018. How do primitives, wrappers and autoboxing create bugs?

Priority: Must Know

Why interviewers ask it: Boxing bugs are subtle, common, and produce wrong results rather than exceptions.

Interview-ready answer: Primitives hold values and cannot be null; wrappers are objects, so they can be null and have identity. Autoboxing converts between them automatically, and that convenience creates three traps. First, unboxing a null throws `NullPointerException` at a line that looks like arithmetic — typical when a nullable database column maps to `Long` and you assign it to a `long`. Second, `==` on wrappers compares references: it appears to work for small values because the JVM caches `Integer` between -128 and 127, then fails for 128 and above, so you must use `equals`. Third, boxing in a loop allocates objects and is measurably slower than primitives. My rules are: primitives for arithmetic and internal state, wrappers only where null is meaningful, `equals` for wrapper comparison, and explicit null handling when mapping nullable columns.

In-depth explanation: The `Integer` cache is specified for -128..127 (its upper bound is adjustable with `-XX:AutoBoxCacheMax`), and `Boolean`, `Byte`, `Short`, `Character` and `Long` have similar caching for small values — which is exactly why the bug is intermittent: tests use small IDs, production uses large ones. Ternary expressions have a nasty variant: mixing a primitive and a wrapper in the two branches forces unboxing of the wrapper, so `flag ? 1 : nullableInteger` can throw even when the chosen branch is the primitive. In collections everything is boxed, so `Map<String, Integer>` counters allocate per increment; `merge` or `computeIfAbsent` with `AtomicInteger`, or a primitive-specialised structure, avoids that where it matters. `Optional.orElse(null)` assigned to a primitive is another quiet unboxing hazard.

Practical backend example: All three traps in code you might actually write:

```

Integer a = 127, b = 127;
Integer c = 128, d = 128;
System.out.println(a == b);           // true - cached instances
System.out.println(c == d);           // false - different objects (use equals)
System.out.println(c.equals(d));       // true

// nullable column -> primitive: NPE at the assignment, not at the query
Long discountId = resultSet.getObject("discount_id", Long.class); // may be null
long id = discountId;                                           // NullPointerException

// ternary unboxing: throws even though the int branch is selected
Integer nullable = null;
int value = true ? 1 : nullable;    // compiler unboxes both branches -> NPE risk

// safe mapping
long safeId = discountId != null ? discountId : 0L;

```

Common follow-ups:

- Why does == sometimes work on Integer? The small-value cache returns the same object; outside that range you get distinct objects.
- Where does the NullPointerException come from in arithmetic? Implicit unboxing of a null wrapper.
- When are wrappers the right choice? When absence is meaningful — a nullable column, an unset filter — otherwise prefer primitives.

Mistakes to avoid: == on wrappers; declaring counters as Integer; mixing primitives and wrappers in ternaries; mapping nullable columns to primitives; boxing inside hot loops.

Production perspective: These bugs pass tests with small fixtures and fail with production-sized identifiers, which makes them a classic "works in staging" incident. A static analysis rule for wrapper == comparisons catches most of them automatically.

Related concepts covered: Integer cache, unboxing NPEs, ternary type promotion, collection boxing costs, nullable column mapping.

Q019. How would you handle dates, times and time zones in an API?

Priority: Must Know

Why interviewers ask it: Time handling is where real systems quietly produce wrong data, and the correct approach is a set of concrete decisions.

Interview-ready answer: I store and transmit instants in UTC, using Instant in Java and timestampz in PostgreSQL, and format them as ISO-8601 with an offset — 2026-03-14T09:15:30Z. Conversion to a local zone happens at the edge, for display, using the user's zone rather than the server's default. I choose the type by meaning: Instant for a moment that happened, LocalDate for a calendar date such as a birthday or an invoice date, LocalDateTime only with a separate zone for future scheduled events, and ZonedDateTime when the zone is part of the data. I never use the legacy Date and Calendar, and I never rely on the JVM default zone — I inject a clock, which also makes time testable.

In-depth explanation: Future events are the case that catches people: a meeting scheduled for "09:00 Europe/Berlin next March" must be stored as local time plus zone, because if a government changes the daylight-saving rules the stored UTC instant would now be the wrong local time. Recurring schedules have the same property. Daylight-saving transitions also make some local times nonexistent or ambiguous, and ZonedDateTime resolves them with defined rules rather than throwing — worth knowing when you compute "the same time tomorrow". Storage matters too: PostgreSQL timestampz stores a UTC instant and converts on input and output, while timestamp stores wall-clock text with no zone, which is almost never what you want for events. Finally, durations and periods differ: Duration is exact time, Period is calendar-aware, and adding one month is not adding thirty days.

Practical backend example: Types, storage, and an injected clock:

```

@Entity
public class Order {
    private Instant createdAt;           // timestamptz in PostgreSQL: a moment in time
    private LocalDate invoiceDate;      // a calendar date: no zone, no time
}

@Entity
public class ScheduledMeeting {
    private LocalDateTime localStart;   // future event: store local time...
    private String zoneId;              // ...plus the zone, so DST rule changes stay correct
    public Instant startInstant() {
        return localStart.atZone(ZoneId.of(zoneId)).toInstant();
    }
}

@Service
public class OrderService {
    private final Clock clock;          // injected: never Instant.now() directly
    public Order create(CreateOrderRequest request) {
        return new Order(UUID.randomUUID(), Instant.now(clock));
    }
}

```

```
{ "createdAt": "2026-03-14T09:15:30Z", "invoiceDate": "2026-03-14" }
```

Common follow-ups:

- Why store future events as local time plus zone? Because time-zone rules change; the intended local time is the invariant, not the derived UTC instant.
- timestamp or timestamptz in PostgreSQL? timestamptz for instants; timestamp has no zone and silently reinterprets values.
- Why inject a Clock? It removes the hidden dependency on the system clock and makes boundary conditions testable.

Mistakes to avoid: Using Date/SimpleDateFormat (also not thread-safe); relying on the server's default zone; storing local times for events; assuming every day has 24 hours; formatting with a locale-dependent pattern in an API.

Production perspective: Time bugs cluster around midnight, month ends, daylight-saving transitions and year boundaries — exactly when reports are produced and people notice. Pin the JVM and container time zone to UTC, log timestamps with offsets, and test the transition dates explicitly.

Related concepts covered: java.time type selection, UTC storage, PostgreSQL timestamptz, DST transitions, Clock injection, ISO-8601 formatting.

Q020. How should BigDecimal be used for money?

Priority: Must Know

Why interviewers ask it: Money errors are visible to customers and auditors, and the correct approach has specific rules.

Interview-ready answer: Never use double or float for money: binary floating point cannot represent 0.1 exactly, so sums drift. Use BigDecimal constructed from a String or from integer minor units — `new BigDecimal("0.1")` captures the binary error, `new BigDecimal(0.1)` does not. BigDecimal is immutable, so every operation returns a new value, and division requires an explicit scale and RoundingMode or it throws ArithmeticException on a non-terminating result. Comparison must use `compareTo`, because `equals` also compares scale, so 1.0 and 1.00 are not equal. The alternative many systems use is to store integer minor units — cents — in a long and only format for display; that removes rounding ambiguity entirely and is attractive for high-volume ledgers. Either way, currency travels with the amount, and rounding is decided by the business, not left to a default.

In-depth explanation: Rounding policy deserves explicit thought: `RoundingMode.HALF_UP` matches common commercial expectations, while `HALF_EVEN` (banker's rounding) reduces cumulative bias across many operations and is required in some domains. The order of operations matters for taxes and discounts — rounding each line then summing gives a different total from summing then rounding — so the rule must be written down and tested. In PostgreSQL, `numeric(19,4)` maps to BigDecimal without loss, while double precision reintroduces the problem at the storage layer. In JSON, serialising a BigDecimal as a number can lose precision in JavaScript clients, so many

APIs send a string, or send integer minor units plus a currency code. Finally, a Money value object carrying amount and currency prevents the two classic bugs: adding different currencies, and passing a raw number where a currency-aware value belongs.

Practical backend example: Construction, division, comparison and a value object:

```
BigDecimal wrong = new BigDecimal(0.1);           // 0.1000000000000000055511151231257827...
BigDecimal right = new BigDecimal("0.1");         // exactly 0.1
BigDecimal fromCents = BigDecimal.valueOf(1999, 2); // 19.99 without string parsing

BigDecimal share = total.divide(new BigDecimal("3"), 2, RoundingMode.HALF_UP); // scale required

new BigDecimal("1.0").equals(new BigDecimal("1.00")); // false - scale differs
new BigDecimal("1.0").compareTo(new BigDecimal("1.00")); // 0 - use this

public record Money(long minorUnits, Currency currency) { // integer minor units
    public Money plus(Money other) {
        if (!currency.equals(other.currency)) throw new IllegalArgumentException("currency mismatch");
        return new Money(Math.addExact(minorUnits, other.minorUnits), currency); // overflow-safe
    }
    public BigDecimal toAmount() {
        return BigDecimal.valueOf(minorUnits, currency.getDefaultFractionDigits());
    }
}
```

```
-- storage: exact, never double precision
ALTER TABLE "order" ADD COLUMN total numeric(19,4) NOT NULL;
```

Common follow-ups:

- Why is `new BigDecimal(0.1)` wrong? The double argument already carries binary error, which `BigDecimal` faithfully preserves.
- Why does `equals` surprise people? It compares scale as well as value; `compareTo` compares numeric value only.
- Minor units or `BigDecimal`? Minor units avoid rounding ambiguity and are convenient for ledgers; `BigDecimal` is better when you need fractional-cent intermediate results such as tax rates.

Mistakes to avoid: double for money anywhere in the path, including JSON and the database; dividing without a scale and rounding mode; `equals` for amount comparison; mixing currencies silently; rounding in a different order from the specification.

Production perspective: Money discrepancies get escalated, not triaged — a one-cent difference in a daily report triggers an investigation. Write the rounding and ordering rules down, test them with known totals, and keep amounts exact end to end, including the JSON representation.

Related concepts covered: Floating-point representation, rounding modes, scale versus value, numeric storage types, Money value objects, JSON precision.

Q021. When should you use enums rather than strings or booleans?

Priority: Important

Why interviewers ask it: Enums are a cheap way to make invalid states unrepresentable, and their persistence trade-offs are practical knowledge.

Interview-ready answer: I use an enum whenever a value comes from a small, known, closed set — order status, payment method, channel. The benefits are compile-time checking, exhaustive switch handling, no typos, and a natural place to attach behaviour or metadata to each constant. Strings for the same purpose invite typos and inconsistent casing, and require validation everywhere. Multiple booleans are worse: three flags imply eight states when only three are legal, and nothing prevents an impossible combination. The one place to be careful is persistence and the wire format: I store enums as strings (`@Enumerated(EnumType.STRING)`), never as ordinals, because ordinals break the moment someone reorders or inserts a constant.

In-depth explanation: Enums can carry state and behaviour — a `PaymentMethod` can know whether it supports refunds — which keeps per-constant logic next to the constant instead of in a scattered switch. In Java 17, an exhaustive switch over an enum in expression form does not need a default branch, so adding a constant produces a compile error at every place that must handle it: that is a feature, and adding a default throws it away. Across

service boundaries, enums need a tolerant-reader policy: a consumer built before a new constant existed must not crash on it, so deserialisation should map unknown values to a defined fallback rather than failing. For database storage, a string column with a check constraint is the common pairing; `@Enumerated(EnumType.ORDINAL)` is the default in JPA when you forget the annotation, which is precisely the dangerous choice.

Practical backend example: Behaviour on the constant, safe persistence, tolerant deserialisation:

```
public enum PaymentMethod {
    CARD(true), BANK_TRANSFER(true), STORE_CREDIT(false);

    private final boolean refundable;
    PaymentMethod(boolean refundable) { this.refundable = refundable; }
    public boolean isRefundable() { return refundable; } // behaviour lives with the constant
}

@Entity
public class Payment {
    @Enumerated(EnumType.STRING) // never ORDINAL
    @Column(nullable = false, length = 32)
    private PaymentMethod method;
}

// exhaustive switch: adding a constant becomes a compile error here
String label = switch (order.status()) {
    case NEW -> "Awaiting payment";
    case PAID -> "Paid";
    case CANCELLED -> "Cancelled";
}; // no default: the compiler enforces completeness

// tolerant reading at the boundary
@JsonCreator
static PaymentMethod fromWire(String value) {
    return Arrays.stream(values()).filter(m -> m.name().equalsIgnoreCase(value))
        .findFirst().orElse(UNKNOWN);
}
```

Common follow-ups:

- Why store the name rather than the ordinal? Ordinals depend on declaration order, so reordering or inserting a constant silently corrupts existing rows.
- What happens when a producer adds a constant? Consumers must tolerate unknown values — map to a fallback instead of throwing.
- When is an enum the wrong choice? When the set is open or configurable at runtime, such as user-defined categories; use a reference table instead.

Mistakes to avoid: Ordinal persistence; parsing enums with `valueOf` on untrusted input without catching `IllegalArgumentException`; adding a default branch that hides missing cases; several booleans representing one state machine.

Production perspective: Enum changes are schema and contract changes. Adding a constant is usually safe if consumers are tolerant; removing or renaming one requires a migration on both sides, so treat the constant name as a published value.

Related concepts covered: Closed sets, exhaustive switch, enum persistence, tolerant readers, state modelling, check constraints.

Q022. What does a sealed hierarchy buy you in Java 17?

Priority: Bonus

Why interviewers ask it: It is a modern language feature whose value shows up in domain modelling, not syntax.

Interview-ready answer: A sealed interface or class declares exactly which types may implement or extend it, using `permits`, and every permitted subtype must be `final`, `sealed` or explicitly `non-sealed`. The payoff is a closed set the compiler knows about: a switch over a sealed type can be checked for exhaustiveness, so adding a new subtype makes every incomplete switch fail to compile instead of falling into a default branch at runtime. That makes it a good fit for modelling results and domain events — a `PaymentResult` that is either `Approved`, `Declined` or `Pending`, each a record with its own data. Compared with an enum, sealed types carry per-case data; compared with an open

interface, you keep control over the set.

In-depth explanation: Sealed types plus records plus pattern matching form a coherent trio for algebraic-style modelling: the sealed interface names the choices, the records carry each choice's payload, and pattern matching in switch deconstructs them. In Java 17 pattern matching for switch is a preview feature; it became standard in Java 21, so a book targeting both should be explicit about which version a snippet requires. Permitted subtypes must be in the same module, or the same package for the unnamed module, which is also a packaging decision. The alternative — an interface with a type field and casts — loses both exhaustiveness and clarity. Where the set genuinely needs to be open for plugins, sealing is the wrong tool.

Practical backend example: A closed result type with per-case data (exhaustive switch shown in Java 21 form):

```
public sealed interface PaymentResult permits Approved, Declined, Pending {}

public record Approved(String receiptId, Money captured) implements PaymentResult {}
public record Declined(String reasonCode, boolean retryable) implements PaymentResult {}
public record Pending(String providerReference, Duration retryAfter) implements PaymentResult {}

// Java 21: exhaustive, no default branch needed; adding a case breaks compilation here
String describe(PaymentResult result) {
    return switch (result) {
        case Approved a -> "captured " + a.captured();
        case Declined d -> "declined: " + d.reasonCode();
        case Pending p -> "pending, retry after " + p.retryAfter();
    };
}
```

Common follow-ups:

- Why not an enum? Enums cannot carry different data per constant; sealed records can.
- What must permitted subtypes declare? Each must be final, sealed or non-sealed, and live in the same module or package.
- Is pattern matching for switch available in 17? As a preview; it is standard from Java 21, so check the target version before relying on it.

Mistakes to avoid: Sealing a type that genuinely needs external implementations; adding a default branch and losing exhaustiveness checking; assuming Java 17 supports the full Java 21 pattern-matching syntax.

Production perspective: The practical benefit is that adding a new case — a new payment outcome — produces compile errors at exactly the places that must handle it, instead of a runtime surprise in one forgotten branch. That is valuable in code that evolves across teams.

Related concepts covered: Algebraic data modelling, exhaustive switch, records, pattern matching, Java version differences.

Q023. How do switch expressions and pattern matching differ across Java 17 and 21?

Priority: Important

Why interviewers ask it: Version precision matters when a team is migrating, and vague answers signal second-hand knowledge.

Interview-ready answer: Switch expressions with arrow labels and yield are standard from Java 14, so they are fully available in 17: they return a value, do not fall through, and must be exhaustive for enums and sealed types. Pattern matching for instanceof is also standard in 16, so if (o instanceof Order order) binds the variable directly. What changed in Java 21 is that pattern matching for switch became standard — matching on type patterns with guards (case Order o when o.total().isPositive()) — along with record patterns for destructuring. The two had different timelines: pattern matching for switch first previewed in 17, record patterns only from 19, and both were finalised in 21. In a Java 17 codebase I therefore use switch expressions and instanceof patterns freely, but type patterns in switch still require preview flags; on 21 I can use the full set.

In-depth explanation: Beyond syntax, the semantics are worth knowing: arrow labels eliminate accidental fall-through, and a switch expression over an enum or sealed type that covers all cases needs no default, which is what makes new cases a compile error. When the selector can be null, classic switch throws NullPointerException; Java 21 allows an explicit case null so the handling is visible rather than accidental. Record patterns in 21 destructure nested

data in one line, which reads far better than a chain of accessors. The migration angle matters in interviews: knowing that a preview feature requires `--enable-preview` and produces class files tied to that exact JDK version explains why teams avoid previews in production.

Practical backend example: What compiles where:

```
// Java 17 (standard): switch expression + instanceof pattern
int retryDelaySeconds = switch (status) {
    case NEW, PENDING -> 5;
    case PAID -> 0;
    case CANCELLED -> {
        audit.record(status);
        yield -1; // yield returns a value from a block
    }
};

if (event instanceof PaymentEvent payment && payment.amount().isPositive()) {
    ledger.record(payment); // binding variable, standard since 16
}

// Java 21 (standard): type patterns in switch, guards, record patterns, case null
String describe(Object event) {
    return switch (event) {
        case null -> "no event";
        case PaymentEvent(var id, Money amount) when amount.isPositive() -> "payment " + id;
        case PaymentEvent p -> "non-positive payment " + p.id();
        case RefundEvent r -> "refund " + r.id();
        default -> "unknown";
    };
}
```

Common follow-ups:

- Which parts are not standard in 17? Pattern matching for switch is preview there; record patterns do not exist at all until their 19 preview. Switch expressions and instanceof patterns are standard in 17.
- What happens with a null selector? Classic switch throws; Java 21 lets you write `case null` explicitly.
- Why avoid preview features in production? They require `--enable-preview`, can change between releases, and tie class files to one JDK version.

Mistakes to avoid: Claiming Java 17 has full pattern matching for switch; adding default to an exhaustive sealed switch; assuming arrow labels fall through; using preview features in a production build.

Production perspective: These features reduce boilerplate in mapping and dispatch code, which is where bugs hide. During a migration from 17 to 21, they are also a good early win: switching a visitor-style chain to a pattern switch usually removes code and adds exhaustiveness checking.

Related concepts covered: Switch expressions, instanceof patterns, record patterns, guarded patterns, preview feature policy, JDK migration.

Q024. What do annotations do, and when is reflection appropriate?

Priority: Important

Why interviewers ask it: Spring is built on both, and understanding the mechanics separates users of the framework from people who can debug it.

Interview-ready answer: Annotations are metadata attached to code elements. They do nothing by themselves — something has to read them: the compiler, an annotation processor at build time, or a framework at runtime through reflection, which requires `RetentionPolicy.RUNTIME`. Spring reads annotations at startup to build bean definitions, wire dependencies and create proxies, so the cost is mostly paid once rather than per request. Reflection is appropriate for framework-level work — mapping, serialisation, dependency injection, test tooling — where you genuinely cannot know the types at compile time. In application code it is rarely the right answer: it bypasses compile-time checking, breaks refactoring and IDE navigation, is slower, and can fail at runtime in ways the compiler would have caught. If I find myself reflecting in business logic, I look for a design that uses an interface or a map of handlers instead.

In-depth explanation: Retention policies matter: SOURCE annotations vanish after compilation (@Override), CLASS is retained in the class file but not loaded, and RUNTIME is visible to reflection. Meta-annotations compose — Spring's @RestController is itself annotated with @Controller and @ResponseBody, which is why the framework searches annotation hierarchies rather than exact types. Reflection has real costs beyond speed: it defeats dead-code analysis, complicates native compilation with GraalVM (which needs explicit reflection configuration or build-time hints), and can throw InaccessibleObjectException under the module system when accessing non-exported internals. MethodHandle and VarHandle are faster alternatives for repeated access, and annotation processors move work to compile time entirely — which is how Lombok, MapStruct and Micronaut avoid runtime reflection.

Practical backend example: A custom runtime annotation and the reflection that reads it:

```
@Retention(RetentionPolicy.RUNTIME)           // required for runtime reflection
@Target(ElementType.METHOD)
public @interface AuditLogged {
    String action();
}

@Service
public class OrderService {
    @AuditLogged(action = "ORDER_CANCELLED")
    public void cancel(UUID orderId) { ... }
}

// a Spring AOP aspect reads the metadata - framework-level use of reflection
@Aspect @Component
public class AuditAspect {
    @Around("@annotation(audited)")
    public Object record(ProceedingJoinPoint joinPoint, AuditLogged audited) throws Throwable {
        Object result = joinPoint.proceed();
        auditLog.write(audited.action(), SecurityContextHolder.getContext().getAuthentication());
        return result;
    }
}
```

Common follow-ups:

- Why must retention be RUNTIME? Otherwise the annotation is discarded and reflection cannot see it.
- Why is reflection discouraged in business code? It removes compile-time safety and refactoring support, and it is slower than a direct call.
- What is the GraalVM implication? Reflective access needs explicit configuration or hints, or the native image fails at runtime.

Mistakes to avoid: Assuming an annotation has behaviour on its own; using reflection to reach private fields in application code; forgetting @Retention; relying on reflection in code destined for a native image without registering it.

Production perspective: Reflection-heavy startup is a measurable part of a Spring Boot application's boot time, which matters for autoscaling and serverless deployments. Compile-time approaches — annotation processors, explicit configuration — trade a little flexibility for faster, more predictable startup.

Related concepts covered: Retention policies, meta-annotations, Spring AOP, MethodHandles, annotation processors, GraalVM native images.

Q025. How would you design a value object for an email address?

Priority: Important

Why interviewers ask it: It is a small design exercise that exposes validation, normalisation, immutability and boundary thinking all at once.

Interview-ready answer: I make it an immutable record with a single String component, validated and normalised in the compact constructor so an instance cannot exist in an invalid state — trim, lowercase the domain part, reject blanks and anything failing a pragmatic format check. Then the type replaces String in signatures: sendWelcome(Email email) cannot be called with a phone number, which is the real value. I keep validation pragmatic rather than attempting a fully RFC-compliant regex, because the only definitive validation is sending a message and confirming receipt. For persistence I map it with an attribute converter or store the string and reconstruct, and for equality I decide explicitly whether comparison is case-insensitive — normalising on construction makes that decision once.

In-depth explanation: Normalisation deserves care: the domain is case-insensitive, but the local part is technically case-sensitive per the standard, even though most providers treat it as insensitive. Lowercasing everything is a common product decision, and it must be applied consistently, including in the database's unique index, or duplicates slip in. `toLowerCase(Locale.ROOT)` avoids the Turkish dotless-i problem that `toLowerCase()` with a default locale can introduce. Validation belongs in the constructor so every path — API, import job, message consumer — gets the same rule; a Bean Validation `@Email` annotation on a DTO field is a good outer layer but does not protect the domain. The wider pattern is primitive obsession: replacing bare strings and longs with small types eliminates a class of argument-order bugs that the compiler otherwise cannot see.

Practical backend example: The value object and its persistence:

```
public record Email(String value) {
    private static final Pattern PATTERN =
        Pattern.compile("^[^@\\s]+@[^@\\s.]+(\\.^[^@\\s.]++)+$"); // pragmatic, not RFC-complete

    public Email {
        Objects.requireNonNull(value, "email is required");
        value = value.trim().toLowerCase(Locale.ROOT); // normalise once, here
        if (value.length() > 254 || !PATTERN.matcher(value).matches()) {
            throw new IllegalArgumentException("invalid email address");
        }
    }
    public String domain() { return value.substring(value.indexOf('@') + 1); }
}

@Converter(autoApply = true)
public class EmailConverter implements AttributeConverter<Email, String> {
    public String convertToDatabaseColumn(Email email) { return email == null ? null : email.value(); }
    public Email convertToEntityAttribute(String column) { return column == null ? null : new Email
        (column); }
}
```

```
CREATE UNIQUE INDEX uq_customer_email ON customer (email); -- values are already normalised
```

Common follow-ups:

- Why not a full RFC 5322 regex? It is enormous, still cannot prove deliverability, and rejects valid addresses; verification by sending a message is the real check.
- Should comparison be case-insensitive? Normalise on construction and make the database index agree; then equality is simply value equality.
- Where does validation belong? In the constructor, so every entry path is covered; DTO-level `@Email` is an additional outer layer.

Mistakes to avoid: Passing raw strings through every layer; validating in the controller only; `toLowerCase()` without a locale; storing unnormalised values behind a unique index; making the value object mutable.

Production perspective: Normalised value objects prevent the duplicate-account class of support tickets, where `User@Example.com` and `user@example.com` become two customers. They also make audit and export code simpler, because there is one canonical representation.

Related concepts covered: Primitive obsession, compact constructor validation, locale-safe normalisation, JPA attribute converters, unique constraints.

Q026. What are the risks of mutable keys and shallow copies?

Priority: Important

Why interviewers ask it: Both produce data corruption that no exception announces.

Interview-ready answer: A mutable key is dangerous because hash-based and sorted collections place an entry according to the key's value at insertion time. Mutate a field that participates in `hashCode` or `compareTo`, and the entry stays where it was: `contains` returns false, `get` returns null, and the entry is unreachable but still retained — a silent leak and a silent data loss at once. A shallow copy is the related problem: copying an object but sharing its inner mutable objects means the "copy" and the original change together, which surfaces as one customer's change appearing in another's data. The fixes are the same in spirit: immutable keys, deep copies where nesting is mutable,

and unmodifiable views at boundaries so callers cannot reach in.

In-depth explanation: Java's own APIs illustrate the shallow-copy trap: `clone()` on an array copies references, not elements; `new ArrayList<>(other)` copies the list structure but shares the element objects; `Collections.unmodifiableList` prevents adding and removing but not mutating the elements themselves. So "defensive copy" only helps if the elements are immutable or are themselves copied. For keys, the practical rule is that anything used as a Map key or Set element should be immutable — a record over immutable components is ideal. A subtle related case is a JPA entity used as a key before it is persisted: its ID is null at insertion and set at flush, which changes the hash if `hashCode` uses the ID. Where a key must derive from mutable state, remove the entry before mutating and re-insert afterwards, which makes the cost explicit.

Practical backend example: The shared-nested-object bug and the deep copy:

```
// shallow copy: the two "independent" baskets share the same lines
class Basket {
    private final List<OrderLine> lines;
    Basket(Basket other) { this.lines = new ArrayList<>(other.lines); } // new list, same elements
}
Basket copy = new Basket(original);
copy.lines().get(0).setQuantity(99); // also changes original's first line if OrderLine is mutable

// safer: immutable elements, so sharing them is harmless
public record OrderLine(String sku, int quantity, Money amount) {}
Basket copy2 = new Basket(List.copyOf(original.lines()));

// mutable key: remove, mutate, re-insert if you truly must
map.remove(key);
key.setRegion("EU");
map.put(key, value);
```

Common follow-ups:

- Why is the entry unreachable after mutation? The lookup hashes the new value and probes a different bucket from the one holding the entry.
- Does `unmodifiableList` make the contents immutable? No; it only blocks structural changes to the list. Element mutation still works.
- How do you copy safely? Make the elements immutable, or copy them individually — a deep copy — rather than copying only the container.

Mistakes to avoid: Entities as `HashMap` keys before persist; `clone()` on arrays of mutable objects assuming isolation; returning an internal list wrapped as `unmodifiable` while still mutating the elements; mutating a key "just this once".

Production perspective: These bugs are found by users, not by monitoring: a cache that never hits, a report that shows another tenant's value, a set that grows without bound. Preferring immutable value types for keys and payloads removes the whole category rather than patching instances of it.

Related concepts covered: Hash bucket placement, deep versus shallow copies, unmodifiable views, entity identity, memory retention.

Q027. How do you avoid null-related bugs without hiding invalid states?

Priority: Must Know

Why interviewers ask it: Null handling shows whether you design for correctness or just add defensive checks.

Interview-ready answer: The goal is not to remove every null check but to reduce the places where null is possible. I validate at the boundary — `Objects.requireNonNull` in constructors, Bean Validation on request DTOs — so invalid input never reaches the domain. I prefer empty collections over null, `Optional` as a return type for genuine lookups, and value objects that cannot be constructed in an invalid state. What I avoid is silently substituting defaults: replacing a missing required field with an empty string or zero turns a clear failure into wrong data that someone discovers a month later. So the rule is fail fast and loudly for invalid states, and represent legitimate absence explicitly with `Optional` or an empty collection. Nullability annotations plus static analysis catch the remainder at build time.

In-depth explanation: There is a real distinction between "this value is missing and that is fine" and "this value should exist and does not". The first is modelling; the second is a bug or invalid input, and it deserves an exception at the earliest point, with the field name in the message. `Objects.requireNonNull` in constructors is the cheapest

enforcement, and it fails at construction rather than at first use, which is much easier to debug. `@NonNull/@Nullable` annotations (JSpecify, or the framework's own) let tools such as Error Prone or IntelliJ inspections flag violations without runtime cost; Kotlin interop respects them too. For collections, returning `List.of()` rather than `null` removes an entire class of caller-side checks. In deeply nested data, optional chaining or a null-safe accessor is preferable to a pyramid of `if (x != null)`.

Practical backend example: Boundary validation, explicit absence, and no silent defaults:

```
public record CreateOrderRequest(
    @NotBlank String customerReference,           // rejected at the boundary with 400
    @NotEmpty List<@Valid OrderLineRequest> lines) {}

public class Order {
    private final UUID id;
    private final Money total;
    public Order(UUID id, Money total) {
        this.id = Objects.requireNonNull(id, "id");           // fails at construction, not later
        this.total = Objects.requireNonNull(total, "total");
    }
}

public List<Discount> discountsFor(UUID customerId) {
    return discountRepository.findById(customerId); // returns List.of(), never null
}

public Optional<ShippingAddress> defaultAddress(UUID customerId) { // legitimate absence
    return addressRepository.findDefault(customerId);
}

// avoid: hiding a missing required value
// String reference = request.customerReference() != null ? request.customerReference() : "";
```

Common follow-ups:

- Should a method ever return null for a collection? No; return an empty collection so callers need no check.
- Is `Optional` a replacement for validation? No; it expresses absence in a return type. Invalid input should be rejected with an error.
- What do nullability annotations buy you? Build-time detection through static analysis and better IDE and Kotlin interop, with no runtime cost.

Mistakes to avoid: `if (x != null)` scattered through business logic instead of at the boundary; substituting defaults for missing required values; returning null collections; catching `NullPointerException` as flow control; `Optional` fields in entities.

Production perspective: A `NullPointerException` deep in a service usually means a check was missing several layers earlier, and the stack trace points at the symptom. Boundary validation with the field name in the message turns those incidents into a 400 response with a clear reason, which is both cheaper to support and safer.

Related concepts covered: Boundary validation, `Optional` semantics, empty collections, nullability annotations, fail-fast construction, static analysis.

Q028. What is the difference between fail-fast validation and accumulating errors?

Priority: Important

Why interviewers ask it: It is a user-experience and API-design decision, not only a coding style.

Interview-ready answer: Fail-fast stops at the first problem and throws immediately; accumulation collects every problem and reports them together. For an API request, accumulation is almost always right: a client submitting a form wants all invalid fields in one response, not a round trip per mistake. Bean Validation does this natively — `@Valid` gathers every constraint violation and the handler maps them to a list of field errors. For internal invariants, fail-fast is right: a constructor should throw on the first impossible argument, because there is no user to inform and continuing would mean operating on a broken object. So my rule is accumulate at the boundary where a human or client will act on the results, and fail fast inside the domain where the only correct action is to stop.

In-depth explanation: There is a middle case worth naming: cross-field and business-rule validation that cannot be expressed as field constraints — "delivery date must be after the order date", "this SKU is not available in this country".

Those often need the same accumulated treatment, which means a validation service that returns a list of violations rather than throwing on the first. Order also matters: cheap syntactic checks first, then expensive ones such as database lookups, so an obviously malformed request does not cost a query. Do not accumulate across a security boundary — an authorization failure should stop processing immediately rather than being reported alongside field errors, since continuing may leak information about resources the caller cannot see. For batch processing the same distinction applies at a different scale: a per-item result list is usually more useful than aborting the whole batch, provided partial success is well defined.

Practical backend example: Accumulating at the boundary, failing fast inside:

```
// boundary: Bean Validation accumulates, the handler returns every field error at once
@PostMapping("/api/orders")
ResponseEntity<OrderResponse> create(@Valid @RequestBody CreateOrderRequest request) { ... }

// business rules that constraints cannot express: accumulate too
public List<Violation> validate(CreateOrderRequest request) {
    List<Violation> violations = new ArrayList<>();
    if (request.deliveryDate().isBefore(LocalDate.now(clock))) {
        violations.add(new Violation("deliveryDate", "must not be in the past"));
    }
    if (!catalogue.availableIn(request.sku(), request.country())) {
        violations.add(new Violation("sku", "not available in " + request.country()));
    }
    return violations;
    // caller decides: 422 with all of them
}

// domain: fail fast, because there is no one to negotiate with
public Order(UUID id, Money total, List<OrderLine> lines) {
    Objects.requireNonNull(id);
    if (lines.isEmpty()) throw new IllegalArgumentException("an order needs at least one line");
}
```

Common follow-ups:

- Which status code for a failed business rule? 400 for a malformed request; 422 is a common choice for a syntactically valid request that violates a rule — pick one and document it.
- Should validation hit the database? Only after the cheap checks pass, and be aware that a check-then-act validation is still racy; a unique constraint is the real guarantee.
- Do you accumulate authorization failures? No; stop immediately, and avoid revealing what exists.

Mistakes to avoid: Returning only the first field error to a form client; running expensive validation before basic checks; treating validation as a substitute for database constraints; mixing authorization results into validation output.

Production perspective: Accumulated, field-level errors with stable codes cut support volume, because the client can render the problem next to the input. They also make client-side retries unnecessary, which reduces load during peak submission periods.

Related concepts covered: Bean Validation, cross-field rules, 400 versus 422, check-then-act races, batch partial success, information disclosure.

Q029. How would you refactor a long conditional into a maintainable strategy?

Priority: Important

Why interviewers ask it: It tests pragmatic refactoring judgement — including knowing when not to.

Interview-ready answer: First I check what the conditional really is. If it is a chain selecting behaviour by a type or key — shipping cost per carrier, fee per payment method — I replace it with a map from key to strategy, so adding a case means adding a class and a registration rather than editing a method everyone touches. In Spring, injecting `Map<String, PricingStrategy>` gives you the registry for free from the bean names. If the branches are just data — different rates per region — a table or configuration is better than classes. And if it is a short, cohesive conditional, I leave it alone: replacing three lines with three classes and a factory makes the code harder to follow, not easier. Before refactoring I make sure there are tests covering each branch, then refactor, then verify behaviour is unchanged.

In-depth explanation: The signal that a conditional should become polymorphic is repetition: the same switch on the same key appearing in several methods, which means each new case requires edits in multiple places. The

map-of-strategies approach makes the open-closed principle concrete, and with sealed types plus an exhaustive switch you get a compile-time guarantee that every case is handled — sometimes better than a registry, because a missing strategy in a map is a runtime failure. Keep the default path explicit: an unknown key should throw a clear exception rather than silently doing nothing. Also weigh the testing cost: strategies are individually testable, but the wiring needs a test too, so the total number of tests goes up while each gets simpler.

Practical backend example: From chain to registry, with Spring wiring:

```
// before: every new method repeats the same switch
public Money shippingCost(Order order) {
    if (order.carrier() == Carrier.DHL) return dhlRate(order);
    else if (order.carrier() == Carrier.UPS) return upsRate(order);
    else if (order.carrier() == Carrier.LOCAL) return localRate(order);
    throw new IllegalStateException("unknown carrier");
}

// after: one small class per behaviour, registered by key
public interface ShippingRate {
    Carrier carrier();
    Money calculate(Order order);
}

@Service
public class ShippingCostService {
    private final Map<Carrier, ShippingRate> strategies;

    public ShippingCostService(List<ShippingRate> rates) {           // Spring injects all
        implementations
        this.strategies = rates.stream()
            .collect(Collectors.toMap(ShippingRate::carrier, Function.identity()));
    }

    public Money cost(Order order) {
        ShippingRate rate = strategies.get(order.carrier());
        if (rate == null) throw new UnsupportedOperationException(order.carrier()); // explicit default
        return rate.calculate(order);
    }
}
```

Common follow-ups:

- When should you not refactor? When the conditional is short, appears once, and is unlikely to grow — indirection has a real readability cost.
- Strategy map or sealed types with exhaustive switch? Sealed types give compile-time completeness; a map is better when implementations are plugged in dynamically.
- How do you refactor safely? Characterisation tests for each branch first, then refactor, then confirm the tests still pass unchanged.

Mistakes to avoid: Creating a strategy per branch for data differences that belong in configuration; a silent no-op default; scattering strategy registration so no one can find the implementations; refactoring without tests.

Production perspective: The measurable benefit is that a new case is an additive change — a new class and a new test — instead of an edit to a method several teams depend on. That is what reduces merge conflicts and regression risk as a codebase grows.

Related concepts covered: Strategy pattern, open-closed principle, Spring collection injection, sealed types, characterisation tests, configuration versus code.

Q030. What makes a Java API backward-compatible for callers?

Priority: Important

Why interviewers ask it: Shared libraries and modules break teams when compatibility is treated casually.

Interview-ready answer: There are two kinds of compatibility, and they differ. Source compatibility means existing code still compiles; binary compatibility means existing compiled code still links and runs without recompiling. Adding a method to a class is usually fine; adding an abstract method to an interface breaks every implementer, which is why default methods exist. Changing a method's parameter or return type, renaming anything public, narrowing visibility, or

adding a checked exception are all breaking. Some changes are source-compatible but not binary-compatible — widening a return type or changing a constant's value, which is inlined at compile time. My approach is to keep the public surface small, add rather than change, deprecate with a replacement and a removal timeline, and use semantic versioning so callers can see the risk in the version number.

In-depth explanation: Several specific cases are worth knowing. Adding an overload can silently change which method a recompiled caller binds to, since overload resolution happens at compile time. Changing a public static final primitive or String constant does not affect already-compiled callers at all, because the old value was inlined — a genuinely surprising source of inconsistency. Reordering enum constants breaks anything persisting ordinals. Serialization adds another dimension: changing fields without managing serialVersionUID breaks deserialisation of old data, which matters for caches and message payloads. For services rather than libraries, the same thinking applies to the wire contract: adding an optional field is safe, removing or renaming one is not, and consumers should be tolerant readers. Tooling helps — japicmp or revapi can fail the build on an incompatible change, which is more reliable than review.

Practical backend example: Compatible evolution of a published interface:

```
// breaking: every implementer stops compiling
public interface PaymentGateway {
    Receipt capture(Money amount);
    Refund refund(String receiptId);           // newly added abstract method - breaks implementers
}

// compatible: a default implementation keeps existing implementers working
public interface PaymentGateway {
    Receipt capture(Money amount);
    default Refund refund(String receiptId) {
        throw new UnsupportedOperationException("refunds are not supported by this gateway");
    }
}

// compatible: add an overload rather than changing the signature
public Receipt capture(Money amount) { return capture(amount, IdempotencyKey.random()); }
public Receipt capture(Money amount, IdempotencyKey key) { ... }

// deprecate with a migration path and a timeline
@Deprecated(since = "2.4", forRemoval = true) // removal announced for 3.0
public Receipt charge(Money amount) { return capture(amount); }
```

Common follow-ups:

- Is adding a method to an interface safe? Only with a default implementation; otherwise every implementer breaks.
- What is source-compatible but not binary-compatible? Changing an inlined constant, or widening a return type — old compiled callers can fail to link or keep the old value.
- How do you signal a breaking change? Semantic versioning plus @Deprecated(forRemoval = true) with a documented replacement and removal release.

Mistakes to avoid: Renaming public methods "for clarity" in a shared library; changing constant values assuming callers see the new one; removing a deprecated method without a release note; letting internal classes become public by accident.

Production perspective: In a multi-team codebase, an incompatible library release turns into blocked deployments across services. Automated compatibility checks in CI plus a short deprecation window is the combination that keeps the cost predictable; the same discipline applied to API and event schemas prevents the equivalent problem at runtime.

Related concepts covered: Source versus binary compatibility, default methods, deprecation policy, semantic versioning, constant inlining, schema evolution.

Chapter 2. Collections, generics, streams, and functional Java

Assume Java 17 unless a question explicitly mentions Java 21. Complexity claims describe the standard OpenJDK implementations; where behaviour is an implementation detail rather than a specification guarantee, the text says so.

Q031. How do you choose among List, Set and Map in a service?

Priority: Must Know

Why interviewers ask it: Collection choice is the first design decision in almost every backend method, and a wrong one shows up later as duplicate rows, quadratic loops or unstable output.

Interview-ready answer: I start from semantics, not performance. If order matters and duplicates are legal I use a List. If the collection answers "have I already seen this?" I use a Set. If I need lookup by key I use a Map. Then I refine the implementation: ArrayList for almost all lists, HashSet for membership, LinkedHashSet when I must keep insertion order, TreeSet/TreeMap when I need sorted traversal or range queries, and EnumMap for enum keys. The last question is concurrency: if multiple threads write, I move to ConcurrentHashMap or an immutable copy rather than wrapping with Collections.synchronizedMap.

In-depth explanation: The interface expresses a contract; the implementation expresses a cost model. List guarantees positional access and allows duplicates; contains is $O(n)$. Set guarantees uniqueness by equals/hashCode (or by comparator for TreeSet), with $O(1)$ average membership for hash sets. Map is a key-to-value association with the same hashing rules for keys. A very common backend smell is scanning a list inside a loop — for (Order o : orders) if (ids.contains(o.id())) with ids as a List is $O(n*m)$; converting ids to a Set makes it effectively $O(n)$. Sorted structures cost $O(\log n)$ per operation but give you firstKey, headMap, subMap and ordered iteration, which hash structures cannot provide at any price. Also consider iteration order stability: HashMap iteration order is unspecified and may change between versions, so never let API output depend on it — sort explicitly or use LinkedHashMap.

Practical backend example: Filtering a batch of incoming order events against a set of blocked customer IDs:

```
Set<UUID> blocked = Set.copyOf(blockedCustomerRepository.findAllIds()); // O(1) membership
Map<UUID, List<OrderEvent>> byCustomer = events.stream()
    .filter(e -> !blocked.contains(e.customerId()))
    .collect(Collectors.groupingBy(OrderEvent::customerId));
```

Common follow-ups:

- How do you keep insertion order in a map? Use LinkedHashMap; HashMap order is unspecified and must not leak into an API response.
- When is TreeMap worth the log-n cost? When you need sorted iteration, a floor/ceiling lookup or range scans, such as tiered pricing bands.
- What about arrays? Use them only for fixed-size primitive data in hot paths; they lack the API and safety of collections.

Mistakes to avoid: Using List.contains inside a loop over another list; assuming HashSet preserves insertion order; sorting a HashMap by "just iterating it"; choosing Collections.synchronizedList for a hot path and expecting scalability.

Production perspective: Collection choice changes the shape of CPU profiles far more often than micro-optimisations do. It also affects memory: each HashMap entry carries an object header, hash, key, value and next pointer, so caching millions of small entries on-heap is usually worse than an external store such as Redis.

Related concepts covered: Big-O reasoning, equals/hashCode, iteration order, enum collections, concurrent collections, memory footprint.

Q032. How does HashMap locate a key, handle collisions and resize?

Priority: Must Know

Why interviewers ask it: It reveals whether you understand hashing well enough to debug missing keys, pathological performance and broken key classes.

Interview-ready answer: HashMap keeps an array of buckets whose length is a power of two. On put it takes the key's hashCode, applies a spreading function that XORs the high bits into the low bits, and masks it to an index. If the bucket is empty it stores the entry; otherwise it walks the bucket comparing hashes first and then equals. Colliding entries form a linked list, and in OpenJDK a heavily collided bin converts to a balanced tree so worst-case lookup degrades to $O(\log n)$ instead of $O(n)$. When size exceeds capacity times the 0.75 load factor the table doubles and entries are redistributed. All of that assumes hashCode and equals are consistent and stable — if a key mutates after insertion, the entry becomes unreachable.

In-depth explanation: The contract-level facts are: equal objects must have equal hash codes, unequal objects may share one, and average $O(1)$ lookup depends on a reasonable hash distribution. The rest — the $h \oplus (h \ggg 16)$ spread, treeification at eight entries in a bin when the table has at least 64 slots, the 0.75 default load factor and the default capacity of 16 — are OpenJDK implementation details worth knowing but worth labelling as such. Resizing is not free: it allocates a new array and rehashes every entry, so pre-sizing with `new HashMap<>(expectedSize / 0.75f + 1)` matters for large bulk loads. A poor hashCode (for example, returning a constant) is functionally correct but turns the map into a list. Treeification only helps when keys are Comparable, otherwise the tree falls back to comparing hash codes and identity ordering.

Practical backend example: Building a lookup for a nightly reconciliation job over roughly 500,000 rows:

```
Map<String, PaymentRow> byReference = new HashMap<>(700_000); // avoid repeated resizes
for (PaymentRow row : rows) {
    byReference.put(row.reference(), row); // reference is an immutable String key
}
```

Common follow-ups:

- What happens if two keys collide? They share a bucket; the map compares hash then equals to distinguish them, so correctness is preserved and only speed suffers.
- Is iteration order guaranteed? No. It depends on hashes and capacity and can change after a resize or a JDK upgrade.
- What breaks if a key is mutated? The entry stays in its old bucket, so get, remove and containsKey can all fail to find it. Prefer immutable keys.
- Is HashMap thread-safe? No. Concurrent writes can corrupt it or produce lost updates; use ConcurrentHashMap.

Mistakes to avoid: Saying collisions overwrite values; claiming treeification is part of the Java specification; using a mutable entity with a database-generated ID as a map key; overriding equals without hashCode.

Production perspective: Pathological hashing has been a real denial-of-service vector for user-controlled keys, which is why the JDK added treeification. In services, the more common problem is memory: a HashMap used as an unbounded cache is a classic slow leak, because nothing evicts entries.

Related concepts covered: equals/hashCode contract, load factor and capacity, treeified bins, immutable keys, ConcurrentHashMap, memory leaks.

Q033. When would you use ArrayList versus LinkedList?

Priority: Important

Why interviewers ask it: It separates candidates who recite Big-O tables from those who understand memory locality and real measurements.

Interview-ready answer: In practice I use ArrayList almost always. It stores elements in a contiguous array, so indexed access is $O(1)$ and iteration is cache-friendly. LinkedList gives $O(1)$ insertion and removal only when you already hold the position — typically via a ListIterator — because finding the position is still $O(n)$. Its per-element node objects add memory overhead and pointer chasing, so even for queue-like workloads ArrayDeque usually beats it. I would consider LinkedList only for a genuine deque-with-frequent-middle-splicing pattern, and I would measure

first.

In-depth explanation: `ArrayList.add` at the end is amortised $O(1)$: when the array is full it grows by roughly 50% and copies. Insert or remove in the middle is $O(n)$ because of the arraycopy, but arraycopy is a fast intrinsic, so for lists of a few thousand elements it frequently outperforms a linked traversal. `LinkedList` implements both `List` and `Deque`; its `get(int)` walks from the nearer end, which makes an index-based loop over a `LinkedList` accidentally $O(n^2)$. Memory matters too: each `LinkedList` node holds item, next and previous references plus a header, typically far more bytes per element than an array slot. `ArrayDeque` is the recommended stack and queue implementation; `Stack` and `Vector` are legacy synchronised classes you should not introduce in new code.

Practical backend example: Batching outbound webhook payloads for delivery:

```
List<WebhookPayload> batch = new ArrayList<>(BATCH_SIZE); // sized once, no resize churn
Deque<RetryTask> retryQueue = new ArrayDeque<>(); // better than LinkedList for FIFO
```

Common follow-ups:

- Is `LinkedList` insertion really $O(1)$? Only with an existing iterator or at the ends; index-based insertion pays $O(n)$ to locate the node.
- Why is `ArrayList` faster despite equal Big-O for iteration? Contiguous memory means fewer cache misses and no pointer chasing.
- What replaces `Stack` and `Vector`? `ArrayDeque` and `ArrayList`, plus explicit synchronisation or a concurrent collection if needed.

Mistakes to avoid: Choosing `LinkedList` because "inserts are $O(1)$ " without considering the search cost; looping with `get(i)` over a `LinkedList`; forgetting that `Arrays.asList` returns a fixed-size view whose `add` throws `UnsupportedOperationException`.

Production perspective: Pre-sizing an `ArrayList` when the count is known removes copy churn in high-throughput paths. Conversely, an `ArrayList` built from an unbounded query result is a memory risk: stream or paginate instead.

Related concepts covered: Amortised complexity, cache locality, `ArrayDeque`, legacy collections, defensive sizing.

Q034. Which Set implementation fits deduplication, order or sorting?

Priority: Important

Why interviewers ask it: Choosing the wrong `Set` silently changes output ordering or, with `TreeSet`, silently changes what "duplicate" means.

Interview-ready answer: `HashSet` is the default for pure membership and deduplication, with average $O(1)$ operations and no order guarantee. `LinkedHashSet` keeps insertion order for a small extra cost per element, which is what I use when the result is returned to a client and needs to be stable. `TreeSet` keeps elements sorted and offers navigation methods like `first`, `ceiling` and `headSet`, at $O(\log n)$. The important subtlety is that `TreeSet` decides equality with `compareTo` or the supplied `Comparator`, not with `equals`, so an ordering that ignores a field will treat distinct objects as duplicates.

In-depth explanation: `HashSet` is backed by a `HashMap`, so everything about hashing, resizing and mutable keys applies to it directly. `LinkedHashSet` adds a doubly linked list through the entries, giving predictable iteration at the cost of two extra references per element. `TreeSet` is backed by a `TreeMap` red-black tree. Its contract asks for an ordering "consistent with equals"; when it is not, the set still works but its notion of duplicates comes from `compareTo(...)` `== 0`. That is occasionally useful — for example a case-insensitive `TreeSet<>(String.CASE_INSENSITIVE_ORDER)` that intentionally treats "Alice" and "alice" as the same. `EnumSet` is a specialised, extremely compact bitset for enum elements and should be preferred for flags. For immutable results, `Set.of(...)` and `Set.copyOf(...)` reject nulls and duplicates at construction and have an unspecified, deliberately randomised iteration order.

Practical backend example: Deduplicating notification recipients while keeping the order the business rules produced:

```
Set<String> recipients = new LinkedHashSet<>();
recipients.add(order.customerEmail());
recipients.addAll(account.billingContacts()); // duplicates dropped, order preserved
Set<Channel> channels = EnumSet.of(Channel.EMAIL, Channel.PUSH);
```

Common follow-ups:

- Why did my TreeSet drop a distinct element? Its comparator returned 0, so the set considered the element a duplicate.
- Does Set.of() keep insertion order? No; its iteration order is unspecified and intentionally varies between runs.
- Can a HashSet contain null? Yes, one null; TreeSet and Set.of reject null.

Mistakes to avoid: Returning a HashSet from an API and assuming stable JSON array order; writing a comparator on one field and being surprised at lost elements; using a TreeSet of mutable objects whose sort keys change.

Production perspective: Unstable ordering makes responses non-deterministic, which breaks client caching, snapshot tests and diff-based debugging. If the order is part of the contract, make it explicit with a sort or a linked implementation.

Related concepts covered: Comparator consistency, EnumSet, immutable collections, deduplication, deterministic API output.

Q035. How do iterators and fail-fast behavior work?

Priority: Important

Why interviewers ask it: ConcurrentModificationException is a rite of passage, and the right answer shows you know it is a bug detector, not a thread-safety feature.

Interview-ready answer: Most java.util collections return fail-fast iterators. They record a modification count when created and check it on each step; if the collection was structurally modified through anything except the iterator itself, they throw ConcurrentModificationException. The safe ways to remove during iteration are Iterator.remove(), Collection.removeIf(...), or building a new collection. Importantly, fail-fast is best effort: the exception is a debugging aid, not a guarantee, so it must never be relied on to detect concurrent access between threads. Concurrent collections like ConcurrentHashMap and CopyOnWriteArrayList instead give weakly consistent iterators that never throw but may not see the newest updates.

In-depth explanation: "Structural modification" means adding or removing elements, or in the case of a map, changing the key set; replacing a value with set or put on an existing key is not structural. The check lives in modCount and is not synchronised, so in a multithreaded scenario it can be missed entirely — this is precisely why the documentation calls the behaviour best effort. CopyOnWriteArrayList iterates over an immutable snapshot taken at iterator creation: reads never block and never throw, but each write copies the whole array, so it suits listener lists and rarely changing configuration, not high-write data. ConcurrentHashMap's iterators traverse live state with weak consistency: they reflect some updates made after creation and never throw ConcurrentModificationException.

Practical backend example: Removing expired sessions from an in-memory registry:

```
sessions.entrySet().removeIf(e -> e.getValue().expiresAt().isBefore(Instant.now())); // safe
// unsafe: for (var e : sessions.entrySet()) if (expired(e)) sessions.remove(e.getKey());
```

Common follow-ups:

- Is ConcurrentModificationException always thrown for concurrent modification? No; the check is best effort and can be missed, especially across threads.
- Can you modify values while iterating a map? Yes — entry.setValue(...) and put on an existing key are not structural changes.
- What does CopyOnWriteArrayList cost? Every mutation copies the backing array, so it is only appropriate for read-dominated collections.

Mistakes to avoid: Catching ConcurrentModificationException and retrying instead of fixing the iteration; treating a fail-fast iterator as a concurrency guard; calling list.remove(index) inside an enhanced for loop.

Production perspective: A `ConcurrentModificationException` in a log is usually a shared-mutable-state bug: a collection in a singleton bean is being modified by request threads. The fix is normally a concurrent collection or a per-request copy, not a try/catch.

Related concepts covered: `modCount` and structural modification, `removeIf`, copy-on-write collections, weakly consistent iterators, shared mutable state.

Q036. What do generic type parameters and wildcards protect?

Priority: Must Know

Why interviewers ask it: Generic APIs are where library-quality code shows, and PECS is the compact test of whether you can design one.

Interview-ready answer: Generics move type checking to compile time and remove casts. The key rule is that generics are invariant: `List<Dog>` is not a `List<Animal>`, because if it were, you could insert a `Cat` into a list of dogs. Wildcards restore flexibility safely. `? extends T` means "some unknown subtype of T", so you can read T values but cannot add anything except null — a producer. `? super T` means "some unknown supertype of T", so you can add T values but reads come back as `Object` — a consumer. That is PECS: producer extends, consumer super. I use a bounded type parameter when several positions in the signature must agree, and a wildcard when only one position needs flexibility.

In-depth explanation: Invariance exists because Java arrays made the opposite choice — `Object[] a = new String[1]` compiles and then throws `ArrayStoreException` at runtime. Generics chose compile-time safety instead. A method that only reads from a collection should take `Collection<? extends T>` so callers can pass `List<SubType>`. A method that only writes should take `Collection<? super T>`. `Collections.copy(List<? super T> dest, List<? extends T> src)` is the canonical illustration. Bounded type parameters such as `<T extends Comparable<? super T>>` let the compiler relate several arguments and the return type. An unbounded wildcard `List<?>` is useful for code that only needs `size`, `isEmpty` or iteration as `Object`. Note that `? extends` does not mean "class only"; it also accepts interfaces, and multiple bounds are written `<T extends Number & Comparable<T>>`.

Practical backend example: A reporting helper that must accept any subtype collection:

```
public static Money sum(Collection<? extends Invoice> invoices) { // producer -> extends
    return invoices.stream().map(Invoice::total).reduce(Money.ZERO, Money::plus);
}

public static void drainTo(List<? super AuditEvent> sink, Queue<AuditEvent> source) { // consumer ->
    super
    AuditEvent event;
    while ((event = source.poll()) != null) sink.add(event);
}
```

Common follow-ups:

- Why is `List<Dog>` not a `List<Animal>`? Because assignment would permit inserting a `Cat`, breaking the dog list at runtime.
- What can you add to a `List<? extends Animal>`? Only null; the concrete element type is unknown.
- When do you prefer `<T>` over a wildcard? When the same type must appear in more than one place, such as parameter and return type.

Mistakes to avoid: Declaring `List<Object>` and casting; using wildcards on return types, which forces callers to deal with unknown types; believing arrays and generics behave alike.

Production perspective: Wildcards mostly pay off in shared internal libraries and SPIs. In ordinary application services, a clear concrete type is more readable than a clever signature; generic gymnastics should earn their keep.

Related concepts covered: Invariance versus covariance, PECS, bounded type parameters, array store exceptions, API design.

Q037. What does type erasure mean for overloads and runtime checks?

Priority: Important

Why interviewers ask it: Erasure explains a family of confusing compiler errors and unchecked warnings that appear in real code.

Interview-ready answer: Generic type arguments exist at compile time and are erased in the bytecode: `List<String>` and `List<Integer>` are both `List` at runtime. Consequences are that you cannot overload two methods whose parameters differ only by type argument, you cannot write `new T()` or `new T[10]`, `instanceof List<String>` is illegal, and a cast to a generic type produces an unchecked warning because the runtime cannot verify it. The compiler inserts casts and sometimes bridge methods to keep polymorphism working. When I need the type at runtime I pass a `Class<T>` token or a framework-provided type reference.

In-depth explanation: Erasure was chosen for migration compatibility with pre-generics code, which is also why raw types still compile with warnings. Because of erasure, a `List<String>` can be polluted with a non-String through a raw reference, and the failure surfaces later at the implicit cast — this is heap pollution, and `@SafeVarargs` exists to document generic varargs methods that do not cause it. Generic arrays are forbidden for the same reason: an array knows its component type at runtime, a generic does not. Frameworks work around erasure by capturing type information in class literals or subclassed type tokens, for example Jackson's `TypeReference` or Spring's `ParameterizedTypeReference`, which read the generic supertype metadata that the class file does retain.

Practical backend example: Deserialising a generic response body in a Spring client:

```
ResponseEntity<List<OrderDto>> response = restClient.get()
    .uri("/orders")
    .retrieve()
    .toEntity(new ParameterizedTypeReference<List<OrderDto>>() {}); // keeps the generic type
```

Common follow-ups:

- Can you write `new T()`? No; pass a `Supplier<T>` or a `Class<T>` and use reflection if you truly need instantiation.
- Why does `instanceof List<String>` not compile? The runtime cannot distinguish it from any other `List`; only `instanceof List<?>` is allowed.
- What are bridge methods? Compiler-generated methods that preserve overriding when erasure changes a signature, for example in `Comparable` implementations.

Mistakes to avoid: Using raw types to silence errors; suppressing unchecked warnings without a comment explaining why the cast is safe; expecting reflection to reveal a collection's element type from an instance.

Production perspective: Erasure surprises typically appear at integration boundaries — JSON mapping, caching libraries and message converters — where a value is stored as `Object` and cast back incorrectly, producing a `ClassCastException` far from the original code.

Related concepts covered: Raw types, heap pollution, `@SafeVarargs`, type tokens, reflection limits, bridge methods.

Q038. How would you group and aggregate orders with streams?

Priority: Must Know

Why interviewers ask it: Grouping and aggregation is the most common real stream task, and it exposes how comfortable you are with collectors.

Interview-ready answer: I build a pipeline of filter, map and then a collector. For "total revenue per customer" I use `Collectors.groupingBy` with a downstream `Collectors.reducing` or `summingLong`. For counts I use `counting()`. If I need an ordered result I supply a map factory such as `LinkedHashMap` or `TreeMap`, because the default is a `HashMap` with unspecified order. If the grouping is over a large dataset that lives in the database, I prefer to push the aggregation into SQL with `GROUP BY` and only stream the summarised rows — moving a million rows into the JVM to sum them is the classic mistake.

In-depth explanation: `groupingBy(classifier)` returns `Map<K, List<T>>`; the two- and three-argument forms let you replace the downstream collector and the map supplier. Useful downstreams are `counting()` (returns `Long`), `summingInt/Long/Double`, `averagingDouble`, `mapping(fn, downstream)`, `filtering(...)`, `toSet()`, `reducing(...)` and `teeing(...)` for two aggregates at once. `partitioningBy` is the boolean-key specialisation and

always contains both true and false keys. Two sharp edges matter: the classifier must not return null or `groupBy` throws a `NullPointerException`, and money should be summed with `BigDecimal` via `reducing` rather than with `summingDouble`, which introduces binary floating point error. For ordering, remember that `groupBy` does not preserve encounter order in its keys, while the lists inside each group do preserve it for an ordered source.

Practical backend example: Monthly revenue per customer for a small in-memory report:

```
Map<UUID, BigDecimal> revenueByCustomer = orders.stream()
    .filter(o -> o.status() == OrderStatus.PAID)
    .collect(Collectors.groupingBy(
        Order::customerId,
        TreeMap::new, // deterministic key order
        Collectors.reducing(BigDecimal.ZERO, Order::total, BigDecimal::add)));

Map<OrderStatus, Long> countByStatus = orders.stream()
    .collect(Collectors.groupingBy(Order::status, () -> new EnumMap<>(OrderStatus.class),
        Collectors.counting()));
```

Common follow-ups:

- What if two records map to the same key with `toMap`? Without a merge function `toMap` throws `IllegalStateException`; supply one, for example `(a, b) -> b`.
- Is `groupBy` ordered? The returned `HashMap` is not; pass a map factory if order matters.
- Why not `summingDouble` for money? Doubles cannot represent decimal fractions exactly; use `BigDecimal` reduction or integer minor units.
- Should this run in Java or SQL? If the data is in a table and the result is an aggregate, `GROUP BY` in SQL is almost always cheaper.

Mistakes to avoid: Loading whole tables to aggregate in memory; a classifier that can return null; using `Collectors.toMap` on data with duplicate keys; assuming `counting()` returns an `int`.

Production perspective: In-memory aggregation is bounded by heap and GC; it also hides the cost from database metrics, making slow endpoints harder to attribute. Aggregate at the source, cache the summary if it is read often, and keep the stream code for shaping the response.

Related concepts covered: `Collectors` API, `EnumMap`, `BigDecimal` arithmetic, SQL `GROUP BY` trade-offs, deterministic ordering.

Q039. What is the difference between `map` and `flatMap`?

Priority: Must Know

Why interviewers ask it: It is a quick check that you understand stream shape, and the same idea recurs in `Optional`, reactive types and `CompletableFuture`.

Interview-ready answer: `map` transforms each element one-to-one, so a stream of `N` elements stays `N` elements — if the function returns a collection you end up with a stream of collections. `flatMap` transforms each element into a stream and concatenates the results, flattening one level, so `N` elements can become any number. I use `map` for field extraction and conversion, `flatMap` when each element contains many child elements, such as orders containing line items. The same distinction applies to `Optional.map` versus `Optional.flatMap`, where `flatMap` avoids an `Optional<Optional<T>>`.

In-depth explanation: `flatMap` takes a function returning a `Stream`, and the returned streams are consumed and closed as the pipeline runs; returning a null stream throws, so return `Stream.empty()` instead. It is also the idiomatic way to drop empties: `stream.flatMap(o -> o.map(Stream::of).orElseGet(Stream::empty))` in Java 8, or simply `stream.flatMap(Optional::stream)` from Java 9 onwards. `mapMulti` (Java 16+) is a lower-allocation alternative when each element produces few results, because it pushes results into a consumer rather than creating a stream per element. Note that `flatMap` can weaken laziness: short-circuit operations like `findFirst` may still consume an entire inner stream in older JDKs, although modern implementations improved this.

Practical backend example: Collecting all SKUs across a batch of orders, and unwrapping optional lookups:

```

List<String> skus = orders.stream()
    .flatMap(order -> order.lines().stream()) // one order -> many lines
    .map(OrderLine::sku)                     // one line -> one sku
    .distinct()
    .toList();

List<Customer> found = ids.stream()
    .map(customerRepository::findById)        // Stream<Optional<Customer>>
    .flatMap(Optional::stream)               // drops the empties
    .toList();

```

Common follow-ups:

- What does `map` return when the function returns a list? `Stream<List<T>>`, which you then have to flatten.
- Does `flatMap` flatten deeply? No, one level per call; nest calls for deeper structures.
- What is `Optional.flatMap` for? Chaining a lookup that itself returns an `Optional` without nesting.

Mistakes to avoid: Returning null from a `flatMap` function; calling `.map(...).collect(...)` then flattening with nested loops; using `flatMap` where a simple `map` would do, which adds allocation.

Production perspective: `flatMap` over a lazily loaded JPA association triggers N+1 queries — the stream looks elegant while issuing one query per parent. Fetch the children with a join or an entity graph first.

Related concepts covered: Optional composition, stream laziness, `mapMulti`, N+1 query risk, `distinct` and `toList`.

Q040. How do lazy streams and terminal operations affect side effects?

Priority: Must Know

Why interviewers ask it: Misunderstanding laziness produces code that silently does nothing, or does work twice.

Interview-ready answer: Intermediate operations such as `filter`, `map` and `peek` only build a pipeline; nothing runs until a terminal operation such as `collect`, `forEach`, `count` or `findFirst` is invoked. Elements are then pulled through the pipeline one at a time, which is why `findFirst` can stop early. Two practical consequences: a pipeline with no terminal operation performs zero work, and `peek` is not a reliable place for side effects because it may be skipped or partially executed. Streams are also single-use — a second terminal operation on the same stream throws `IllegalStateException`.

In-depth explanation: The pull model means the order of operations is per element, not per stage: for `filter().map().findFirst()` the JDK processes one element through both stages before touching the next. Short-circuit operations (`findFirst`, `anyMatch`, `limit`) exploit this. `count()` may skip the pipeline entirely when it can compute the size from the source without executing the stages, so counting a stream that has a `peek` logging statement can log nothing at all. Stateful operations — `sorted`, `distinct`, `limit after sorted` — must buffer, which removes much of the laziness benefit and can be expensive on large sources. Side effects inside pipelines are discouraged because they interact badly with laziness, ordering and parallelism; the JDK documentation explicitly warns against them. Streams over resources (`Files.lines`) hold file handles and should be closed with `try-with-resources`.

Practical backend example: A pipeline that appears to audit every rejected order but may not:

```

long rejected = orders.stream()
    .peek(o -> auditLog.record(o)) // unreliable: may be skipped or short-circuited
    .filter(Order::isRejected)
    .count();

// do this instead
List<Order> rejectedOrders = orders.stream().filter(Order::isRejected).toList();
rejectedOrders.forEach(auditLog::record); // explicit, always runs

```

Common follow-ups:

- Why did nothing happen? There was no terminal operation, so the pipeline never executed.
- Can a stream be reused? No; create a new stream from the source, or collect once into a list.
- Is `forEach` ordered? Not for parallel streams; use `forEachOrdered` when encounter order matters.
- Why did `peek` not print? `count()` can elide stages, and short-circuiting stops traversal early.

Mistakes to avoid: Using peek for logging or persistence; mutating an external collection inside map; forgetting to close streams backed by I/O; assuming a stream is a data structure you can iterate twice.

Production perspective: Side effects hidden inside pipelines make behaviour dependent on optimisations, which is very hard to reproduce in a debugger. Keep pipelines pure and perform I/O in explicit statements where retries, metrics and error handling are visible.

Related concepts covered: Short-circuiting, stateful operations, stream closing, forEachOrdered, pure functions.

Q041. When should you avoid parallel streams?

Priority: Must Know

Why interviewers ask it: Parallelism is the easiest place to give a confidently wrong answer, and production incidents follow.

Interview-ready answer: Parallel streams are not a general speed switch. By default they run on the shared common ForkJoinPool sized to the number of cores minus one, so any blocking work inside them — a JDBC call, an HTTP request, a lock — starves every other user of that pool, including other parallel streams in the same JVM. They only help for large, CPU-bound, independent work over a source that splits evenly, such as an array or ArrayList. They hurt for small collections, for sources that split poorly like LinkedList or Iterator-based streams, for order-sensitive pipelines, and for anything touching shared mutable state. In a web service, requests are already parallel across threads, so intra-request parallelism usually adds contention rather than throughput.

In-depth explanation: parallelStream() splits the source with a Spliterator, processes segments in fork-join tasks, and combines results. The combine step matters: collect with a mutable container needs a proper combiner, and collectors like groupingBy must merge maps, which costs allocation. If you must parallelise blocking work, submit the pipeline to your own ForkJoinPool — pool.submit(() -> list.parallelStream()).get() — so you do not contaminate the common pool, though a plain ExecutorService with explicit tasks is usually clearer. Encounter-order-sensitive operations (findFirst, forEachOrdered, limit, sorted) add synchronisation overhead in parallel mode. Any claim about speedup must come from a measurement on representative data, ideally with JMH, not from intuition: JIT warm-up alone can make a naive timing loop misleading.

Practical backend example: Hashing a large in-memory batch of documents is a reasonable candidate; enriching records over HTTP is not.

```
// reasonable: CPU-bound, independent, large, ArrayList source
List<String> digests = documents.parallelStream().map(this::sha256).toList();

// unsafe in a server: blocking I/O on the shared common pool
// List<Profile> profiles = ids.parallelStream().map(httpClient::fetchProfile).toList();
ExecutorService io = Executors.newFixedThreadPool(16); // bounded, dedicated pool instead
```

Common follow-ups:

- Which pool runs a parallel stream? The common ForkJoinPool, unless the pipeline is submitted from inside another ForkJoinPool.
- How do you benchmark fairly? Use JMH with warm-up iterations, realistic data sizes and the same JVM flags as production.
- Is Collectors.toMap safe in parallel? It works, but prefer toConcurrentMap for unordered concurrent collection; either way the merge function must be associative.
- Does parallel() guarantee ordering? Results of ordered collectors stay ordered, but forEach execution order does not.

Mistakes to avoid: Claiming parallel streams are always faster; doing I/O or locking inside them; mutating a shared ArrayList from a parallel pipeline; parallelising a 50-element list.

Production perspective: Common-pool starvation shows up as unrelated endpoints slowing together, which is very hard to diagnose. If you need concurrency in a service, use an explicit, bounded, named executor whose queue and rejection policy you can monitor.

Related concepts covered: ForkJoinPool, spliterators, JMH benchmarking, thread pool isolation, shared mutable state.

Q042. How do collectors handle grouping, ordering and duplicate keys?

Priority: Important

Why interviewers ask it: Collector edge cases — duplicate keys, nulls and map types — are where stream code fails in production rather than in tests.

Interview-ready answer: `Collectors.toMap(keyFn, valueFn)` throws `IllegalStateException` when two elements produce the same key, so I supply a merge function when duplicates are possible, and a map supplier when I need a specific map type such as `LinkedHashMap`. `toMap` also rejects null values with a `NullPointerException`, which surprises people migrating from a loop with `map.put`. `groupingBy` never has the duplicate-key problem because it accumulates into a downstream collector, but its classifier must not return null. If I need a specific iteration order I pass a `TreeMap` or `LinkedHashMap` factory, since the default `HashMap` order is unspecified.

In-depth explanation: The four-argument `toMap(key, value, merge, mapSupplier)` is the form worth memorising. The merge function receives the existing and the new value and decides the winner — `(first, second) -> first` for "keep earliest", or a real merge for accumulation. `Collectors.toList()` historically returns an `ArrayList` but the API deliberately does not guarantee mutability; `Stream.toList()` (Java 16+) returns an unmodifiable list that does allow nulls, while `Collectors.toUnmodifiableList()` rejects nulls. `joining`, `summarizingInt`, `flatMaping`, `filtering` and `teeing` cover most remaining reporting needs. For concurrency, `groupingByConcurrent` and `toConcurrentMap` are unordered concurrent collectors — meaningful only for parallel, unordered pipelines.

Practical backend example: Indexing the latest price per SKU from an unsorted feed:

```
Map<String, PriceRow> latestBySku = feed.stream()
    .collect(Collectors.toMap(
        PriceRow::sku,
        Function.identity(),
        (a, b) -> a.updatedAt().isAfter(b.updatedAt()) ? a : b, // duplicate policy
        LinkedHashMap::new)); // stable order
```

Common follow-ups:

- What happens without a merge function? A duplicate key throws `IllegalStateException` at collection time, often only with production data.
- Can `toMap` hold null values? No — it calls `map.merge`, which rejects nulls; use `groupingBy` or a manual loop.
- Is `Collectors.toList()` mutable? In practice yes today, but the contract does not promise it; use `toList()` or `toCollection(ArrayList::new)` deliberately.

Mistakes to avoid: Assuming test data has no duplicates; returning `Collectors.toList()` results and mutating them; expecting `groupingBy` keys in encounter order; using concurrent collectors on sequential streams for no benefit.

Production perspective: A `toMap` without a merge function is a latent outage: it works until one duplicate arrives from an upstream system. Decide the duplicate policy explicitly and log when a merge actually happens if the duplicates are unexpected.

Related concepts covered: `toMap` merge semantics, unmodifiable results, null handling, concurrent collectors, deterministic ordering.

Q043. When is a loop clearer or faster than a stream?

Priority: Important

Why interviewers ask it: They want engineering judgement, not style dogma in either direction.

Interview-ready answer: I use streams when the code reads as a declarative transformation — filter, map, group, collect — because the intent is clearer than the mechanics. I use a plain loop when I need early exit with complex conditions, when I must mutate several accumulators at once, when I need the index, when exceptions make lambda code awkward, or when the body is easier to step through in a debugger. On performance, a simple loop over an `ArrayList` is often slightly faster because it avoids lambda and boxing overhead, but the difference is usually irrelevant compared with a database call; I would only optimise after profiling shows a hot path.

In-depth explanation: Streams add allocation: a pipeline object per stage, boxed values unless you use `IntStream/LongStream/DoubleStream`, and lambda capture objects when they are not stateless. The JIT inlines simple pipelines well, so the gap is small for straightforward work, but megamorphic call sites — the same pipeline

shape used with many different lambdas — can inhibit inlining in very hot code. Checked exceptions are a genuine ergonomic problem: a lambda cannot throw a checked exception unless the functional interface declares it, which leads to wrapper helpers that obscure stack traces. Conversely, loops encourage accidental complexity: nested loops with flags and manual index arithmetic are where off-by-one bugs live. A good rule is to keep the pipeline flat and short; if you need a comment to explain a stream, a loop is probably clearer.

Practical backend example: Validation that must collect several results and stop early on a fatal error reads better as a loop:

```
List<String> problems = new ArrayList<>();
for (OrderLine line : order.lines()) {
    if (line.quantity() <= 0) problems.add("quantity must be positive: " + line.sku());
    if (line.sku() == null) return List.of("fatal: missing sku"); // early exit
}
```

Common follow-ups:

- Are streams slower? Sometimes marginally, mostly due to allocation and boxing; measure before caring.
- How do you avoid boxing? Use `IntStream`, `LongStream` and `mapToInt` for primitive pipelines.
- How do you handle checked exceptions in a lambda? Wrap them in an unchecked exception at the boundary, or use a loop.

Mistakes to avoid: Rewriting readable loops as unreadable one-liners; claiming streams are "always slower"; hiding I/O inside a pipeline; using `Optional` chains where an `if` is clearer.

Production perspective: Readability wins in code that on-call engineers must debug at 3 a.m. Reserve micro-optimised loops for measured hot paths and comment why they exist so a future refactor does not undo the reason.

Related concepts covered: Primitive streams, boxing costs, JIT inlining, exception handling in lambdas, code review judgement.

Q044. What is a functional interface, and how do lambdas capture variables?

Priority: Important

Why interviewers ask it: Capture rules explain compile errors, memory retention and a real class of concurrency bugs.

Interview-ready answer: A functional interface has exactly one abstract method, so a lambda or method reference can implement it; `@FunctionalInterface` makes that intent compile-checked. The standard set is `Function`, `BiFunction`, `Supplier`, `Consumer`, `Predicate`, `UnaryOperator` and the primitive variants. A lambda can capture local variables only if they are `final` or `effectively final`, because it captures the value, not the variable. Instance and static fields are captured by reference through `this`, so they can change and are not thread-safe by default. Unlike an anonymous class, a lambda does not create a new `this` — inside it, `this` refers to the enclosing instance.

In-depth explanation: The `effectively-final` rule avoids the confusing semantics of mutable capture and supports safe publication to other threads. The common workaround — capturing an `AtomicInteger` or a one-element array to mutate it — compiles but reintroduces the shared-state problem the rule was avoiding; in a stream, prefer a proper reduction. Lambdas are compiled to `invokedynamic` with a bootstrap that creates the implementation at runtime, so a non-capturing lambda is typically a reused singleton while a capturing lambda allocates per call; that matters only in very hot loops. A capturing lambda stored in a long-lived structure keeps the captured objects — including an implicit `this` — reachable, which is a subtle memory leak when listeners are registered and never removed. Method references come in four flavours: static, bound instance, unbound instance and constructor.

Practical backend example: A retry helper parameterised by a supplier, plus a listener that must be removed to avoid retention:

```
public <T> T withRetry(int attempts, Supplier<T> action) {
    RuntimeException last = null;
    for (int i = 0; i < attempts; i++) {
        try { return action.get(); } catch (RuntimeException e) { last = e; }
    }
    throw last;
}

Money total = withRetry(3, () -> pricingClient.quote(order)); // captures 'order' (effectively final)
```

Common follow-ups:

- Why must captured locals be effectively final? The lambda copies the value; allowing reassignment would make the semantics ambiguous and unsafe across threads.
- What does this mean inside a lambda? The enclosing instance, not the lambda itself, unlike an anonymous inner class.
- Can a lambda throw a checked exception? Only if the functional interface declares it; otherwise wrap it.

Mistakes to avoid: Mutating an array or AtomicInteger purely to bypass the capture rule; assuming each lambda evaluation creates a new object; leaking this into a long-lived callback registry.

Production perspective: Callback-heavy code with captured state is a frequent source of heap growth in long-running services. When registering listeners or cache loaders, make ownership and deregistration explicit.

Related concepts covered: invokedynamic, method references, effectively final, memory retention, reductions versus mutation.

Q045. How would you safely build an immutable collection result?

Priority: Important

Why interviewers ask it: Returning internal mutable state is one of the quietest ways to break encapsulation in a service.

Interview-ready answer: I return an unmodifiable copy, not the internal collection. `List.copyOf(...)`, `Set.copyOf(...)`, `Map.copyOf(...)` and `Stream.toList()` all produce collections that reject modification. The important caveat is that these are shallow: the elements themselves can still be mutable, so an immutable list of mutable entities does not protect the entities. `Collections.unmodifiableList(internal)` is a view, not a copy — if the backing list changes, the view changes with it, which is usually not what a caller expects. For truly immutable data I combine an immutable container with immutable elements, typically records or value objects.

In-depth explanation: There are three distinct concepts: a defensive copy (a new collection with the same references), an unmodifiable view (a wrapper that throws on mutation but reflects backing changes) and a deep copy (new elements too). `List.of` and `copyOf` reject null elements and throw `NullPointerException`, while `Stream.toList()` permits nulls — a difference that bites when mapping data that may be absent. Mutating through an unmodifiable wrapper throws `UnsupportedOperationException` at runtime, not compile time, so the safety is a contract, not a type guarantee. For JPA entities, returning the live collection is worse than a style issue: Hibernate tracks that collection for dirty checking, and callers modifying it can trigger unexpected inserts or deletes at flush time.

Practical backend example: An aggregate exposing its lines safely:

```
@Entity
public class Order {
    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true)
    private final List<OrderLine> lines = new ArrayList<>();

    public List<OrderLine> lines() { return List.copyOf(lines); } // callers cannot mutate
    persistence state
    public void addLine(OrderLine line) { line.assignTo(this); lines.add(line); } // one controlled
    entry point
}
```

Common follow-ups:

- Does `List.copyOf` deep-copy elements? No; the elements are shared, so mutable elements remain mutable.

- What is the difference between a view and a copy? A view reflects later changes to the backing collection; a copy does not.
- Can `List.of` contain null? No; it throws `NullPointerException`, unlike `Stream.toList()`.

Mistakes to avoid: Returning the internal `ArrayList` field directly; assuming `unmodifiableList` freezes the data; using `List.of` on a mapped stream that may contain nulls.

Production perspective: Immutable returns make concurrency reasoning much simpler and remove a class of "who changed this list?" bugs. The cost is one copy per call, which is negligible for small collections and worth measuring for large hot ones.

Related concepts covered: Defensive copying, unmodifiable views, records, JPA collection management, null handling.

Q046. How do you use `computeIfAbsent` safely?

Priority: Important

Why interviewers ask it: It is the idiomatic map-populating method and also the easiest way to corrupt a map or hang a thread.

Interview-ready answer: `computeIfAbsent(key, fn)` computes and inserts a value only when the key is absent or mapped to null, and returns the current value either way, which makes it perfect for multimaps and lazy initialisation. The rule is that the mapping function must be short, side-effect free and must not modify the same map. In `HashMap`, modifying the map inside the function can corrupt it — modern JDKs detect many cases and throw `ConcurrentModificationException`. In `ConcurrentHashMap` the computation runs while the bin is locked, so a recursive update on the same map can deadlock or livelock, and a slow function blocks other writers to that bin.

In-depth explanation: The related methods form a family worth knowing: `putIfAbsent` (value already computed), `computeIfAbsent` (lazy creation), `computeIfPresent`, `compute` and `merge` (accumulate). `merge(key, one, Integer::sum)` is the cleanest counter idiom. Returning null from the `computeIfAbsent` function means no mapping is recorded and null is returned — useful for "absent means genuinely missing". In `ConcurrentHashMap`, these methods are atomic, which is exactly why compound sequences like `if (!map.containsKey(k)) map.put(k, v)` should be replaced by them. For caches, `computeIfAbsent` on a plain `HashMap` is not a cache: nothing bounds or evicts it. Use Caffeine or Redis when you need TTL, size limits or metrics.

Practical backend example: Building an index of lines per order, and an atomic per-tenant counter:

```
Map<UUID, List<OrderLine>> byOrder = new HashMap<>();
for (OrderLine line : lines) {
    byOrder.computeIfAbsent(line.orderId(), id -> new ArrayList<>()).add(line);
}

ConcurrentMap<String, AtomicLong> hits = new ConcurrentHashMap<>();
hits.computeIfAbsent(tenantId, t -> new AtomicLong()).incrementAndGet(); // atomic creation
```

Common follow-ups:

- Can the mapping function modify the map? No. `HashMap` may throw `ConcurrentModificationException` and `ConcurrentHashMap` can deadlock on the same bin.
- What if the function returns null? No entry is stored and the call returns null.
- How is it different from `putIfAbsent`? `putIfAbsent` requires the value up front; `computeIfAbsent` creates it only when needed.

Mistakes to avoid: Doing I/O or a database call inside the mapping function of a `ConcurrentHashMap`; using a plain map as an unbounded cache; replacing atomic map methods with check-then-act sequences.

Production perspective: Long-running mapping functions in `ConcurrentHashMap` serialise writers on the same bin, which appears as unexplained latency under load. Keep the function to object construction and perform the expensive work outside the map.

Related concepts covered: `merge` and `compute` family, atomic map operations, multimap idiom, cache boundaries, `ConcurrentHashMap` locking.

Q047. How would you count duplicate events efficiently?

Priority: Important

Why interviewers ask it: It is a small coding exercise that reveals collection fluency, null handling and awareness of scale limits.

Interview-ready answer: For an in-memory list I build a frequency map: either `map.merge(key, 1L, Long::sum)` in a loop, or `Collectors.groupingBy(key, Collectors.counting())` with a stream. Both are $O(n)$ with $O(\text{distinct})$ memory. If the input is a large stream of events I would not hold it all in memory: I would count in the database with `GROUP BY ... HAVING COUNT(*) > 1`, or use a Redis counter per key with a TTL for a rolling window. If the question is "have I already processed this?", a persisted idempotency key is the right structure, not a map.

In-depth explanation: The main trade-off is memory versus fidelity. A `HashMap` of counts is exact but grows with cardinality; for very high-cardinality streams, approximate structures such as a count-min sketch or HyperLogLog trade a bounded error for constant memory. Keys must be immutable and have correct `equals/hashCode`, otherwise counts silently split across entries. For concurrent producers, use `ConcurrentHashMap` with `merge` or `LongAdder` values — `LongAdder` scales better than `AtomicLong` under heavy contention. If you also need the top N, avoid sorting the whole map: use a bounded min-heap of size N, which is $O(n \log N)$.

Practical backend example: Detecting duplicate payment references in a batch, with a bounded top-offenders list:

```
Map<String, Long> counts = new HashMap<>();
for (PaymentEvent e : events) counts.merge(e.reference(), 1L, Long::sum);

List<Map.Entry<String, Long>> worst = counts.entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
    .limit(10)
    .toList();
```

```
-- the same job pushed to PostgreSQL when the data already lives there
SELECT reference, COUNT(*) AS occurrences
FROM payment_event
WHERE received_at >= now() - interval '1 day'
GROUP BY reference
HAVING COUNT(*) > 1
ORDER BY occurrences DESC
LIMIT 10;
```

Common follow-ups:

- What if the input does not fit in memory? Aggregate in the database, or use a streaming/approximate counter with bounded memory.
- How do you make it thread-safe? `ConcurrentHashMap` with `merge`, or values of type `LongAdder`.
- How do you get the top N without full sorting? Keep a size-N min-heap while scanning.

Mistakes to avoid: Using `map.get` plus null checks instead of `merge`; using mutable keys; sorting a huge map just to take ten rows; conflating duplicate detection with idempotency guarantees.

Production perspective: Duplicate detection usually exists to protect a side effect — do not send the email twice. The durable answer is a unique constraint or an idempotency key in the database; in-memory counting only helps within one process and is lost on restart.

Related concepts covered: `merge` idiom, `groupingBy` with counting, `LongAdder`, approximate counting, idempotency, SQL aggregation.

Q048. How do you handle nulls in stream pipelines and collectors?

Priority: Important

Why interviewers ask it: Null handling differs between apparently interchangeable APIs, and the failures appear only with real data.

Interview-ready answer: I filter nulls explicitly and early with `Objects::nonNull`, or better, I stop producing them at the source. The rules worth knowing are: `Stream.of(null)` is fine but `List.of(null)` and `Map.of` reject nulls;

`Collectors.toMap` throws `NullPointerException` on a null value; `groupingBy` throws if the classifier returns null; `Collectors.toUnmodifiableList()` rejects nulls while `Stream.toList()` allows them; and `Optional.of(null)` throws while `Optional.ofNullable(null)` gives an empty `Optional`. For mapping functions that may find nothing, I return `Optional` and flatten with `Optional::stream`.

In-depth explanation: These differences come from the immutable collection factories deliberately treating null as a programming error, while older APIs tolerated it. The practical consequence is that a refactor from `Collectors.toList()` to `toUnmodifiableList()` can start throwing on data that previously worked. `Comparator.nullsFirst/nullsLast` wrap a comparator so sorting does not throw on null keys. `Objects.requireNonNullElse` and `Optional.orElseGet` give explicit defaults. When nulls come from a database column, the cleanest fix is to decide at the mapping layer whether absence means "unknown", "not applicable" or "zero" and encode that in the domain type rather than propagating null into the pipeline.

Practical backend example: Building a lookup from rows where an optional column may be null:

```
Map<String, String> emailByUser = users.stream()
    .filter(u -> u.email() != null) // decide the policy explicitly
    .collect(Collectors.toMap(User::username, User::email)); // toMap would throw on a null value

List<String> sortedNames = users.stream()
    .map(User::displayName)
    .sorted(Comparator.nullsLast(Comparator.naturalOrder()))
    .toList();
```

Common follow-ups:

- Why did `toMap` throw a `NullPointerException`? It delegates to `merge`, which forbids null values; filter or map them to a sentinel first.
- Can `Stream.toList()` contain null? Yes; `List.copyOf` and `toUnmodifiableList` cannot.
- How do you sort data containing nulls? Wrap the comparator with `Comparator.nullsFirst` or `nullsLast`.

Mistakes to avoid: Assuming every immutable-collection factory tolerates null; hiding nulls with `Optional` fields on entities; using `Optional.of` where `ofNullable` is meant.

Production perspective: Null-related pipeline failures usually appear as a `NullPointerException` deep inside JDK collector code, with a stack trace that does not name your field. Fail fast at the boundary — validate or normalise on input — so the error message points at the source system.

Related concepts covered: Immutable collection factories, `Optional` semantics, null-safe comparators, boundary validation, error diagnosis.

Q049. What is the cost of sorting and what comparator mistakes matter?

Priority: Important

Why interviewers ask it: Sorting is everywhere in APIs, and a broken comparator throws an exception that most developers have never debugged.

Interview-ready answer: Sorting objects is $O(n \log n)$ and uses a stable merge sort (TimSort), so equal elements keep their relative order; primitive arrays use a dual-pivot quicksort that is not stable. The comparator must be a total order: consistent, antisymmetric and transitive. If it is not, `Arrays.sort` may throw `IllegalArgumentException: Comparison method violates its general contract!`, usually because someone subtracted ints and overflowed, compared floating point with tolerance, or returned inconsistent results for equal objects. I build comparators with `Comparator.comparing(...).thenComparing(...)` and `Integer.compare` rather than hand-written arithmetic, and I add a tie-breaker so paging is deterministic.

In-depth explanation: `a - b` overflows when the difference exceeds `Integer.MAX_VALUE`, silently inverting the sign; `Integer.compare(a, b)` never does. Comparing doubles with an epsilon breaks transitivity, which is exactly what TimSort detects. Sorting mutable objects whose sort keys change after insertion into a `TreeSet` or `PriorityQueue` corrupts the structure. For large result sets, the important realisation is architectural rather than algorithmic: sorting in the database with an index that matches the `ORDER BY` avoids loading and sorting everything in the JVM, and keyset pagination avoids the deep-offset problem entirely. Also remember that stability is what makes multi-pass sorting work, and that `Comparator.reverseOrder()` versus `reversed()` differ in what they reverse.

Practical backend example: A deterministic, overflow-safe comparator for a paged listing:

```
Comparator<Order> byRecency = Comparator
    .comparing(Order::createdAt, Comparator.reverseOrder())
    .thenComparing(Order::id);           // tie-breaker keeps paging stable

orders.sort(byRecency);
// avoid: (a, b) -> (int) (a.amountCents() - b.amountCents()) // can overflow
```

Common follow-ups:

- Why does sorting throw "Comparison method violates its general contract"? TimSort detected an inconsistent comparator, commonly from subtraction overflow or epsilon comparison.
- Is Java's object sort stable? Yes, TimSort is stable; primitive sorts are not, which is invisible because primitives are indistinguishable.
- Why add a tie-breaker for pagination? Without a unique final key, rows can repeat or vanish between pages.

Mistakes to avoid: Subtracting to compare; sorting in memory what the database can sort with an index; mutating sort keys of elements inside a TreeSet; relying on an unstable sort for multi-key ordering.

Production perspective: Deep OFFSET pagination combined with in-memory sorting is a classic slow endpoint. Prefer indexed ORDER BY with keyset pagination, and sort in Java only for small, already-bounded result sets.

Related concepts covered: TimSort stability, comparator contracts, integer overflow, pagination determinism, database sorting and indexes.

Q050. How would you choose a bounded in-memory data structure?

Priority: Bonus

Why interviewers ask it: Unbounded in-memory state is one of the most common causes of memory exhaustion in long-running services.

Interview-ready answer: The first question is whether the state must be in memory at all. If it must, it has to be bounded: a maximum size, a TTL, or both. LinkedHashMap in access order with an overridden removeEldestEntry gives a simple LRU; Caffeine gives size and time eviction, refresh, and hit-rate statistics without writing that logic yourself. If the data must be shared across instances or survive a restart, that is Redis, not a local map. I also make sure the eviction policy matches the access pattern and that cache misses are cheap enough to absorb, because every local cache is also a source of stale data after a deployment or a write on another node.

In-depth explanation: A local cache multiplies memory by the number of instances and gives each one a different view of the truth; that is acceptable for reference data that changes rarely and intolerable for balances. Sizing must be expressed in entries or weight, not vague intent, because heap pressure appears as longer GC pauses before it appears as OutOfMemoryError. WeakHashMap and soft references look attractive but make behaviour depend on GC timing and are rarely the right tool for caching. Bounded queues matter for the same reason: an unbounded LinkedBlockingQueue in an executor turns backpressure into heap growth. Finally, measure hit rate; a cache below a useful hit rate adds complexity, memory and staleness with no benefit.

Practical backend example: A small LRU for feature flags plus a Caffeine cache with statistics:

```
Map<String, FeatureFlag> lru = new LinkedHashMap<>(256, 0.75f, true) {
    @Override protected boolean removeEldestEntry(Map.Entry<String, FeatureFlag> eldest) {
        return size() > 1_000;           // hard bound
    }
};

Cache<String, ExchangeRate> rates = Caffeine.newBuilder()
    .maximumSize(10_000)
    .expireAfterWrite(Duration.ofMinutes(5))    // bounded staleness
    .recordStats()
    .build();
```

Common follow-ups:

- When should Redis replace a local map? When data must be shared across replicas, survive restarts, or exceed comfortable heap size.

- Is a HashMap cache a memory leak? Effectively yes if nothing bounds or evicts it; it grows with distinct keys until the heap is exhausted.
- What eviction policy should you choose? Size plus TTL is a safe default; access-order LRU suits skewed access, and per-key TTL suits data with known freshness rules.

Mistakes to avoid: Caching per-user data locally in a multi-replica deployment; using WeakHashMap as a cache; unbounded queues in executors; caching without measuring hit rate or staleness impact.

Production perspective: Local caches change failure modes: a deploy invalidates them all at once, which can produce a thundering herd against the database. Stagger TTLs with jitter and make sure the origin can survive a cold cache.

Related concepts covered: LRU eviction, Caffeine, Redis trade-offs, GC pressure, backpressure, cache staleness and stampedes.

Chapter 3. Concurrency, asynchronous programming, and JVM

Baseline is Java 17 on a modern HotSpot JVM with G1 as the default collector. Java 21 features are named explicitly where they change the answer. Garbage collector and JIT behaviour are implementation characteristics of HotSpot, not language guarantees.

Q051. What makes a race condition, and how do visibility and atomicity differ?

Priority: Must Know

Why interviewers ask it: Every later concurrency answer depends on separating "another thread cannot see my write" from "two threads interleaved mid-operation".

Interview-ready answer: A race condition is when the correctness of a result depends on the timing of threads. Two distinct mechanisms cause it. Atomicity failures happen when an operation that looks like one step is several: `count++` is a read, an add and a write, so two threads can both read 5 and both write 6. Visibility failures happen when a thread writes a value and another thread never sees it, because the Java Memory Model allows values to be kept in registers or caches and allows reordering unless a happens-before relationship exists. `volatile` fixes visibility and ordering but not atomicity; `synchronized`, locks and atomic classes fix both. Immutability and confinement avoid the problem entirely.

In-depth explanation: The JMM defines happens-before: if action A happens-before action B, then A's effects are visible to B. Program order gives it inside one thread; across threads you need a synchronisation edge — unlocking a monitor happens-before a later lock of the same monitor, a `volatile` write happens-before a later read of the same field, `Thread.start()` happens-before everything in the started thread, and everything in a thread happens-before another thread's successful `join()`. Without such an edge the code has a data race, and a data race means the result is not merely "sometimes stale" but undefined by the model. That is why the classic "stop flag" loop can spin forever: the JIT may hoist a non-volatile field read out of the loop, which is a legal transformation for race-free semantics. Note the vocabulary distinction interviewers appreciate: a data race is a specific JMM-level condition, while a race condition is any timing-dependent correctness bug — a check-then-act on a perfectly synchronised map is still a race condition.

Practical backend example: A shutdown flag and a counter in a Spring singleton:

```
@Component
public class ImportJob {
    private volatile boolean running = true;           // visibility only
    private final AtomicLong processed = new AtomicLong(); // atomicity for the increment

    public void stop() { running = false; }

    public void run(List<Row> rows) {
        for (Row row : rows) {
            if (!running) return; // reliably observes stop() because the field is volatile
            handle(row);
            processed.incrementAndGet();
        }
    }
}
```

Common follow-ups:

- Is `count++` atomic? No. It is read-modify-write; use `AtomicLong`, `LongAdder` or a lock.
- Does `volatile` make a counter safe? No; it guarantees each read sees the latest write but not that the increment is indivisible.
- Do 64-bit long and double have special rules? Non-volatile 64-bit writes are not guaranteed atomic by the JMM, so tearing is theoretically possible; declaring them `volatile` removes that.

Mistakes to avoid: Saying `volatile` "synchronises" access; assuming a missing lock only causes occasional staleness rather than undefined behaviour; adding `synchronized` to random methods without identifying the invariant being protected.

Production perspective: Concurrency bugs rarely reproduce under a debugger and are timing sensitive, so they appear as rare data corruption or as a job that hangs after a deploy. Design shared state out of existence where possible — stateless beans, immutable objects, per-request data — instead of hunting the bug later.

Related concepts covered: Java Memory Model, happens-before, data race versus race condition, volatile semantics, atomic classes, thread confinement.

Q052. What happens-before guarantees does synchronized provide?

Priority: Must Know

Why interviewers ask it: It tests whether you know that a monitor gives visibility as well as mutual exclusion, and where the boundaries of that guarantee lie.

Interview-ready answer: `synchronized` acquires the monitor of an object. Only one thread holds it at a time, so the block is mutually exclusive, and releasing the monitor happens-before any later acquisition of the same monitor, so everything written inside the block is visible to the next thread that enters. Monitors are reentrant: a thread already holding the lock can re-enter. The critical detail is that the guarantee is per monitor object — `synchronized` on an instance method locks `this`, on a static method it locks the `Class` object, and two threads locking different objects exclude nobody. I keep critical sections small, never call external services inside them, and never lock on a mutable or shared-by-accident object such as a `String` literal or a boxed `Integer`.

In-depth explanation: The lock's memory effects are as important as the exclusion: entering flushes the thread's view so it reads fresh values, exiting publishes writes. This is why a field only ever accessed inside blocks synchronised on the same monitor does not additionally need `volatile`. Choosing the monitor is a design decision: a private `final Object lock = new Object()` prevents callers from locking your instance and causing surprising interactions, whereas `synchronized` methods expose the lock as part of your API. `wait/notifyAll` must be called while holding the monitor and `wait` must always be used in a loop, because spurious wakeups are permitted. `ReentrantLock` adds `tryLock` with timeout, interruptible acquisition, fairness and multiple conditions; plain `synchronized` is simpler and benefits from JVM optimisations, so use the lock class only when you need its extra capabilities. In Java 21, blocking inside `synchronized` can pin a virtual thread to its carrier, which is a reason to prefer `ReentrantLock` in virtual-thread-heavy code.

Practical backend example: Guarding a compound invariant in a rate limiter:

```
public class WindowCounter {
    private final Object lock = new Object(); // private monitor, not exposed to callers
    private long windowStart = System.nanoTime();
    private int count;

    public boolean tryAcquire(int limit, long windowNanos) {
        synchronized (lock) { // exclusion + visibility for both fields
            long now = System.nanoTime();
            if (now - windowStart > windowNanos) { windowStart = now; count = 0; }
            return count++ < limit;
        }
    }
}
```

Common follow-ups:

- Is `synchronized` reentrant? Yes; the same thread can acquire the same monitor repeatedly, with a hold count.
- Do two `synchronized` methods on different instances block each other? No, unless they are static, in which case they share the class monitor.
- When would you use `ReentrantLock` instead? For `tryLock` with a timeout, interruptible locking, fairness, or multiple condition queues.

Mistakes to avoid: Locking on `this` in a public class, on a `String` constant, or on a boxed `Integer` from the cache; performing network or database calls inside a synchronised block; using `notify` when several distinct conditions share one monitor.

Production perspective: Long critical sections show up in thread dumps as many threads `BLOCKED` on the same monitor while CPU sits idle. That pattern — high latency, low CPU — usually means contention, not a slow database.

Related concepts covered: Monitor semantics, reentrancy, lock granularity, wait/notify, ReentrantLock, virtual thread pinning.

Q053. When is volatile sufficient and when is it not?

Priority: Must Know

Why interviewers ask it: `volatile` is the most misused keyword in Java concurrency, and misuse produces intermittent, unreproducible bugs.

Interview-ready answer: `volatile` guarantees that every read sees the most recent write and prevents the compiler and CPU from reordering surrounding memory operations across it. It is sufficient for a single variable written by one thread and read by others — a status flag, a cached immutable reference, a "configuration reloaded" pointer. It is not sufficient for compound actions: `volatile int count; count++` is still three steps, and `if (instance == null) instance = create();` is still check-then-act. For those I use an atomic class, a lock, or an immutable swap where the whole new state is published as one reference assignment.

In-depth explanation: The ordering guarantee is what makes the immutable-swap pattern work: writes to a new object's fields happen-before the volatile write of its reference, so a reader that sees the reference sees a fully constructed object. This is the correct way to publish reloaded configuration. Without `volatile`, safe publication requires some other edge — final field semantics, a concurrent collection, a static initialiser or a lock. `volatile` reads are close to free on common hardware; `volatile` writes are more expensive because they involve a memory barrier, but still far cheaper than lock contention. Two related facts are worth stating: `volatile` does not make the referenced object thread-safe, only the reference itself; and the double-checked locking idiom requires the field to be `volatile` to be correct on the Java Memory Model.

Practical backend example: Hot-reloading pricing rules without locking readers:

```
@Component
public class PricingRules {
    private volatile RuleSet current = RuleSet.empty();    // one-writer, many-readers

    public Money price(Order order) { return current.apply(order); } // always a consistent snapshot

    @Scheduled(fixedDelay = 60_000)
    void reload() {
        RuleSet fresh = ruleLoader.load(); // build fully, then publish atomically
        current = fresh;                  // volatile write publishes the whole object graph
    }
}
```

Common follow-ups:

- Is `volatile count++` safe? No; it is a read-modify-write and can lose updates.
- Does `volatile` on a `List` reference make the list thread-safe? No; only the reference assignment is safe. Use a concurrent list or publish an immutable copy.
- Is double-checked locking safe without `volatile`? No; the field must be `volatile` or a reader can see a partially constructed object.

Mistakes to avoid: Using `volatile` to protect invariants spanning two fields; assuming it provides mutual exclusion; sprinkling it on fields as a "just in case" fix.

Production perspective: The immutable-snapshot-plus-volatile-reference pattern gives lock-free reads for configuration, feature flags and reference data, which is valuable on hot request paths where a shared lock would serialise every request.

Related concepts covered: Safe publication, double-checked locking, memory barriers, immutable snapshots, atomic references.

Q054. When would you use AtomicInteger, LongAdder or a lock?

Priority: Important

Why interviewers ask it: It probes whether you understand compare-and-swap, contention and the difference between a counter and an invariant.

Interview-ready answer: Atomic classes use compare-and-swap: read the value, compute the new one, and swap it only if nothing changed, retrying otherwise. `AtomicInteger/AtomicLong` are ideal for a single counter or flag with low to moderate contention, and `AtomicReference` with `compareAndSet` or `updateAndGet` handles a single mutable reference. Under heavy contention, CAS retries waste CPU, and `LongAdder` performs better because it spreads increments across internal cells and only sums them when you read — the trade-off is that `sum()` is not an atomic point-in-time snapshot. When the invariant spans more than one variable — debit one account and credit another — no atomic class is enough and I use a lock or, in a real system, a database transaction.

In-depth explanation: CAS is optimistic: correctness comes from the retry loop, not from blocking, so it scales well until many threads contend on the same cache line. That is the essence of the `LongAdder` design — reduce contention by using more memory. `AtomicReference.updateAndGet` takes a function that may be invoked multiple times, so it must be pure and side-effect free. The classic theoretical hazard is the ABA problem, where a value changes from A to B and back to A and CAS cannot tell; `AtomicStampedReference` adds a version to detect it, and in practice immutable values make ABA irrelevant. Atomics give linearisable single-variable operations only; multi-variable consistency requires a lock, and multi-row consistency requires a database transaction with appropriate isolation.

Practical backend example: Per-endpoint metrics counters and a compare-and-set state transition:

```
private final LongAdder requests = new LongAdder();           // hot path, high contention
private final AtomicReference<JobState> state =
    new AtomicReference<>(JobState.IDLE);

public void onRequest() { requests.increment(); }             // cheap under load
public long requestCount() { return requests.sum(); }          // approximate while updating

public boolean start() {
    return state.compareAndSet(JobState.IDLE, JobState.RUNNING); // only one thread wins
}
```

Common follow-ups:

- Is `LongAdder.sum()` a consistent snapshot? No; it can miss concurrent updates, which is fine for metrics and wrong for balances.
- What is the ABA problem? A value returning to its original between reads makes CAS succeed incorrectly; use a stamped reference or immutable values.
- When do atomics stop helping? When the invariant involves multiple variables, or when contention is so high that CAS retries dominate.

Mistakes to avoid: Using `AtomicInteger.get()` then `set()` instead of an atomic update; assuming atomics coordinate several fields; using a distributed system's in-memory counter as a business source of truth.

Production perspective: In-process counters vanish on restart and differ per replica, so a "count" that matters to the business belongs in the database or a shared store such as Redis. Use atomics for metrics and local coordination, not for money.

Related concepts covered: Compare-and-swap, contention and cache lines, ABA, atomic state machines, distributed counters.

Q055. How do you size and manage an `ExecutorService`?

Priority: Must Know

Why interviewers ask it: Thread pool misconfiguration is one of the most common causes of production outages in Java services.

Interview-ready answer: I choose the pool by workload. For CPU-bound work, roughly the number of cores; for blocking I/O, more threads, sized from the target throughput and the average wait time, and always bounded. The queue matters as much as the thread count: with `ThreadPoolExecutor`, if the queue is unbounded the pool never grows beyond the core size and work simply accumulates in memory until the heap dies. I use a bounded queue with an explicit rejection policy — `CallerRunsPolicy` for natural backpressure, or a rejection that returns 503 — and I name the threads so thread dumps are readable. On shutdown I call `shutdown()`, wait with `awaitTermination`, then `shutdownNow()`, and I never let a pool hold work that has already been acknowledged to a client without a durable record.

In-depth explanation: `ThreadPoolExecutor`'s algorithm is specific: it creates a thread per task until `corePoolSize`, then queues, then creates threads up to `maximumPoolSize` only when the queue is full, then rejects. So `Executors.newFixedThreadPool` (unbounded `LinkedBlockingQueue`) can never reject and never grows — the failure mode is memory. `Executors.newCachedThreadPool` has the opposite problem: unbounded thread creation under load. Little's law gives a starting point for I/O pools — concurrency equals throughput times latency — but the real constraint is usually downstream: a pool of 200 threads hitting a database with 20 connections just moves the queue. Separate pools per dependency (a bulkhead) stop one slow downstream from consuming every thread. In Spring Boot 3, `@Async` and `@Scheduled` use configurable `TaskExecutor/TaskScheduler` beans, and the scheduler defaults to a single thread, which surprises teams whose jobs overlap.

Practical backend example: A bounded, named, observable pool for outbound calls:

```
@Bean(destroyMethod = "shutdown")
public ThreadPoolExecutor notificationExecutor() {
    ThreadPoolExecutor executor = new ThreadPoolExecutor(
        8, 16, // core, max
        60, TimeUnit.SECONDS,
        new ArrayBlockingQueue<>(500), // bounded: backpressure, not heap growth
        new CustomizableThreadFactory("notify-"), // readable thread dumps
        new ThreadPoolExecutor.CallerRunsPolicy()); // slows producers instead of dropping
    return executor;
}
```

Common follow-ups:

- Why are unbounded queues risky? They convert overload into unbounded memory use and hide the pressure until an `OutOfMemoryError`.
- What does `CallerRunsPolicy` do? Runs the task on the submitting thread, throttling the producer naturally.
- Difference between `shutdown()` and `shutdownNow()`? `shutdown()` stops accepting new tasks and drains the queue; `shutdownNow()` also interrupts running tasks and returns the pending ones.
- How many threads for I/O work? Start from target concurrency, verify against downstream limits such as the connection pool, then load test.

Mistakes to avoid: Using `Executors.newFixedThreadPool` for load-sensitive work; unnamed threads; forgetting `shutdown` in tests and in application context close; sizing the pool larger than the database connection pool.

Production perspective: Instrument queue depth, active threads, completed count and rejection count. Rising queue depth is the earliest warning signal that a downstream dependency is slowing, usually well before error rates move.

Related concepts covered: `ThreadPoolExecutor` mechanics, backpressure, bulkheads, Little's law, graceful shutdown, pool metrics.

Q056. How do `CompletableFuture` failures and timeouts propagate?

Priority: Must Know

Why interviewers ask it: Async composition silently swallows errors unless you know exactly which callbacks run on failure.

Interview-ready answer: A `CompletableFuture` completes either normally or exceptionally. If it completes exceptionally, then `apply` and `compose` stages are skipped and the failure propagates down the chain; exceptionally and `handle` can recover, and when `complete` observes both outcomes without changing them unless it throws. `join()` throws an unchecked `CompletionException` wrapping the cause, while `get()` throws a checked `ExecutionException` — in both cases you must unwrap to find the real error. There is no implicit timeout, so I add `orTimeout` or `completeOnTimeout` (Java 9+); otherwise a hung downstream call blocks a thread forever. For fan-out I use `allOf` and remember it completes exceptionally if any input fails, so I attach per-future recovery when partial results are acceptable.

In-depth explanation: Which thread runs a stage matters. The non-async variants (`thenApply`) run on whichever thread completed the previous stage, or on the calling thread if it is already complete — so a heavy continuation can run on a Netty or scheduler thread by accident. The `*Async` variants default to the common `ForkJoinPool` unless you pass an executor, which you usually should. `supplyAsync` without an executor also uses the common pool, so blocking calls there compete with parallel streams elsewhere in the JVM. Exception wrapping differs subtly between

join and get; `CompletableFuture.completedFuture` plus `exceptionallyCompose` (Java 12+) covers retry-style recovery. Cancellation is weak: `cancel(true)` completes the future with `CancellationException` but does not interrupt the underlying work, so cancellation must be cooperative. In Spring Boot 3, `@Async` methods returning `CompletableFuture` integrate with this model, but the thread does not carry the security context, MDC or transaction unless you propagate them explicitly.

Practical backend example: Fan-out with per-call timeout and graceful degradation:

```
CompletableFuture<Profile> profile = CompletableFuture
    .supplyAsync(() -> profileClient.fetch(userId), ioExecutor)    // explicit pool
    .orTimeout(800, TimeUnit.MILLISECONDS)
    .exceptionally(ex -> Profile.unknown(userId));                // degrade, do not fail the page

CompletableFuture<List<Offer>> offers = CompletableFuture
    .supplyAsync(() -> offerClient.fetch(userId), ioExecutor)
    .completeOnTimeout(List.of(), 500, TimeUnit.MILLISECONDS);

Dashboard dashboard = profile.thenCombine(offers, Dashboard::new).join();
```

Common follow-ups:

- Which executor runs continuations? The completing thread for non-async stages, the common pool for `*Async` without an executor, otherwise the executor you pass.
- What does `allOf` do on failure? It completes exceptionally with the first failure; handle each future individually if partial success is acceptable.
- Difference between `handle` and `whenComplete`? `handle` can transform the result or recover; `whenComplete` only observes and re-propagates.
- Does `cancel` stop the running task? No; it completes the future as cancelled but the underlying work continues unless it checks for interruption.

Mistakes to avoid: Calling `join()` on the request thread and destroying the benefit of `async`; omitting timeouts; ignoring the return value of `exceptionally`; assuming MDC, security context or transactions propagate automatically.

Production perspective: Async code without timeouts turns a slow dependency into a thread leak. Always pair `orTimeout` with a bounded executor and a metric on rejection and timeout counts, so degradation is visible rather than silent.

Related concepts covered: Async composition, exception wrapping, executor selection, timeouts and degradation, context propagation.

Q057. How is `ConcurrentHashMap` different from `synchronizedMap`?

Priority: Must Know

Why interviewers ask it: It distinguishes "every method is locked" from "concurrent by design", and exposes the check-then-act trap.

Interview-ready answer: `Collections.synchronizedMap` wraps a map and guards every method with one lock, so all operations serialise and iteration still requires manual synchronisation on the wrapper. `ConcurrentHashMap` allows fully concurrent reads without locking and locks only the affected bin on write, so throughput scales with cores. It also provides atomic compound operations — `putIfAbsent`, `computeIfAbsent`, `merge`, `replace` — which is the key point: with either map, `if (!map.containsKey(k)) map.put(k, v)` is a race, because two threads can pass the check. `ConcurrentHashMap` forbids null keys and values, and its iterators are weakly consistent rather than fail-fast, so they never throw `ConcurrentModificationException`.

In-depth explanation: Aggregate operations like `size()` and `isEmpty()` are approximations while other threads mutate, by design; if you need an exact count you need external coordination. `computeIfAbsent` runs the mapping function while holding the bin lock, so it must be short and must not update the same map. `ConcurrentHashMap` also offers bulk parallel methods (`forEach`, `reduce`, `search`) with a parallelism threshold, which are rarely needed but good to know. For read-mostly lists, `CopyOnWriteArrayList` is the equivalent trade-off in the other direction: free reads, expensive writes. The general design lesson is that thread-safe components do not compose into a thread-safe workflow: a sequence of individually atomic calls is not atomic, so either use a single atomic operation or guard the whole sequence.

Practical backend example: A per-tenant lock registry and an idempotency guard:

```
private final ConcurrentMap<String, Object> tenantLocks = new ConcurrentHashMap<>();
private final ConcurrentMap<String, Boolean> processed = new ConcurrentHashMap<>();

Object lock = tenantLocks.computeIfAbsent(tenantId, t -> new Object()); // atomic creation

if (processed.putIfAbsent(messageId, Boolean.TRUE) == null) { // exactly one thread proceeds
    handle(message); // in-process guard only
}
```

Common follow-ups:

- Is check-then-put atomic? No; use `putIfAbsent`, `computeIfAbsent` or `merge`.
- Can `ConcurrentHashMap` store null? No, neither keys nor values, because null would be ambiguous with "absent".
- Is `size()` exact? It is an estimate under concurrent modification.
- Does `ConcurrentHashMap` make my service correct across replicas? No; it is per JVM. Cross-instance coordination needs the database or Redis.

Mistakes to avoid: Wrapping with `synchronizedMap` and iterating without holding the lock; treating a sequence of atomic calls as atomic; long computations inside `computeIfAbsent`; using a local map to deduplicate work in a multi-instance deployment.

Production perspective: An in-memory registry that grows per key — locks, dedup markers, per-user state — is an unbounded cache in disguise. Bound it or give entries a lifetime, and prefer a database unique constraint when duplication must be prevented for real.

Related concepts covered: Lock striping, atomic map operations, weakly consistent iterators, composition of thread-safe parts, distributed coordination.

Q058. How do you identify and prevent deadlocks?

Priority: Must Know

Why interviewers ask it: Deadlock diagnosis is a concrete operational skill, and prevention shows you can design lock ordering.

Interview-ready answer: A deadlock needs four conditions: mutual exclusion, hold-and-wait, no preemption and a circular wait. The practical prevention is to break the cycle by always acquiring locks in a globally consistent order — for example by sorted entity ID — and to use `tryLock` with a timeout so a thread can back off and retry instead of waiting forever. To diagnose, I take a thread dump with `jstack` or `jcmd`; the JVM explicitly reports "Found one Java-level deadlock" with the participating threads and monitors. The same cycle can occur in the database when two transactions lock rows in opposite order; PostgreSQL detects it and aborts one transaction with a deadlock error, so the application must retry.

In-depth explanation: Java monitors are not preemptible and have no timeout, which is why `ReentrantLock.tryLock(timeout)` is valuable for code that cannot be ordered. Lock ordering is the cheapest structural fix: if every code path locks account A before account B by comparing IDs, a cycle cannot form. Reducing lock scope helps too — never hold a lock across an I/O call, because the window for a cycle grows with the hold time. Beyond classic deadlock, watch for livelock, where threads keep responding to each other and make no progress, and for thread-pool starvation deadlock, where tasks in a pool wait on results computed by tasks queued behind them in the same pool; the fix there is separate pools or non-blocking composition. Database deadlocks deserve explicit handling: catch the specific exception, apply jittered retry, and make the operation idempotent so a retry is safe.

Practical backend example: Ordered locking for a transfer, and the equivalent database-level ordering:

```
public void transfer(Account from, Account to, Money amount) {
    Account first = from.id().compareTo(to.id()) < 0 ? from : to; // global order
    Account second = first == from ? to : from;
    synchronized (first) {
        synchronized (second) { from.debit(amount); to.credit(amount); }
    }
}
```

```
-- the database version: lock rows in a deterministic order inside one transaction
SELECT id, balance FROM account WHERE id IN (:a, :b) ORDER BY id FOR UPDATE;
```

Common follow-ups:

- How do you detect a deadlock in production? A thread dump (`jcmd <pid> Thread.print`) names the deadlocked threads and monitors; `JMX ThreadMXBean.findDeadlockedThreads` can alert automatically.
- Can databases deadlock too? Yes; PostgreSQL detects the cycle and aborts one transaction, so retry with backoff.
- Does `tryLock` eliminate deadlock? It prevents indefinite waiting, but you must handle the failure path and avoid livelock.

Mistakes to avoid: Acquiring locks in data-dependent order; holding a lock during an HTTP or JDBC call; nesting synchronized blocks across layers; retrying a database deadlock without backoff or idempotency.

Production perspective: Deadlocks manifest as a subset of requests hanging while CPU stays low, and they often start after a seemingly unrelated change reorders two calls. Automated deadlock detection through `ThreadMXBean` plus an alert saves hours during an incident.

Related concepts covered: Lock ordering, `tryLock` and backoff, thread dumps, pool starvation, database deadlock retries, idempotency.

Q059. What are safe ways to publish and share mutable objects?

Priority: Important

Why interviewers ask it: Safe publication explains why "the object exists but its fields look empty" happens, which is otherwise mystifying.

Interview-ready answer: Safe publication means that when another thread sees a reference, it also sees the object's fully initialised state. The guaranteed mechanisms are: initialise it from a static initialiser; store it in a `volatile` field or `AtomicReference`; store it in a final field of a properly constructed object; or place it in a concurrent collection or guard it with a lock. Simply assigning to a plain field is not safe publication, and the reader can observe default values. The simplest strategy is to avoid the problem: make the object immutable with final fields, or confine it to one thread and publish only copies.

In-depth explanation: Final field semantics are a specific JMM guarantee: if an object is properly constructed — meaning the `this` reference does not escape during construction — then any thread that sees the reference sees the correctly initialised final fields, with no extra synchronisation. That is what makes immutable value objects and records naturally thread-safe. Escaping `this` breaks it: registering a listener, starting a thread or passing `this` to a collaborator inside the constructor lets another thread see a half-built object. For mutable shared objects there is no shortcut — every access, read and write, must go through the same synchronisation. A useful middle ground is the copy-on-write style: keep the shared state immutable, and replace the whole reference through a `volatile` field when it changes.

Practical backend example: Immutable snapshot plus controlled mutation:

```
public record TaxTable(Map<String, BigDecimal> ratesByRegion) {           // effectively immutable
    public TaxTable {
        ratesByRegion = Map.copyOf(ratesByRegion);                      // defensive, unmodifiable
        copy
    }
}

private volatile TaxTable table = new TaxTable(Map.of());                // safe publication point
public void refresh(Map<String, BigDecimal> fresh) { table = new TaxTable(fresh); }
```

Common follow-ups:

- Is a final reference enough? Final field semantics cover the fields of a properly constructed object; they do not make a mutable object it points to thread-safe.
- What does "this escaping" mean? Publishing the instance before the constructor finishes, so other threads can observe partial state.
- Are records thread-safe? Their components are final, so yes for the record itself, provided the component values are themselves immutable.

Mistakes to avoid: Starting a thread or registering a callback in a constructor; publishing a builder's mutable internals; assuming that a synchronised setter makes an object safe when getters are unsynchronised.

Production perspective: Unsafe publication typically fails rarely and only under load or on certain CPU architectures, so it survives testing and appears in production. Immutability is the cheapest insurance and also makes caching and sharing safe by construction.

Related concepts covered: Final field semantics, immutability, this-escape, volatile publication, defensive copies.

Q060. When are thread-local values helpful and dangerous?

Priority: Important

Why interviewers ask it: `ThreadLocal` underpins MDC, security context and transaction management, and it leaks in pooled environments when misused.

Interview-ready answer: `ThreadLocal` gives each thread its own copy of a value, which is how frameworks carry ambient context: Spring's `SecurityContextHolder`, request attributes, the transaction-bound `EntityManager`, and SLF4J's MDC. The danger is that application servers reuse threads, so a value left behind is visible to the next unrelated request, which can leak a previous user's identity, and the retained object never becomes garbage. Every set needs a matching remove in a finally block, normally in a filter or interceptor. With Java 21 virtual threads, thread locals still work but the "few threads, reused" assumption disappears, and scoped values are the direction of travel for immutable context.

In-depth explanation: Internally each `Thread` holds a `ThreadLocalMap` whose keys are weak references to the `ThreadLocal` object but whose values are strong. So if the `ThreadLocal` itself is a static field — the normal case — the entry survives as long as the thread does, and in a pool that is the lifetime of the application. This is a classic container class-loader leak during redeloys. `InheritableThreadLocal` copies values to child threads, which sounds convenient but propagates stale context into pool threads unpredictably. The async case is the trickiest: a value set on the request thread is not visible in a `@Async` or `CompletableFuture` continuation, which is why frameworks provide context-propagating decorators. If you only need to pass data down a call chain, a method parameter is simpler, more testable and impossible to leak.

Practical backend example: A correlation-ID filter with guaranteed cleanup:

```
@Component
public class CorrelationIdFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                   FilterChain chain) throws ServletException, IOException {
        String id = Optional.ofNullable(request.getHeader("X-Correlation-Id"))
                           .orElseGet(() -> UUID.randomUUID().toString());
        MDC.put("correlationId", id);           // ThreadLocal under the hood
        try {
            chain.doFilter(request, response);
        } finally {
            MDC.remove("correlationId");        // mandatory: threads are pooled
        }
    }
}
```

Common follow-ups:

- What happens if you forget `remove()`? The value leaks into the next request handled by that pooled thread and is never collected.
- Does a thread local propagate to `@Async` methods? No, unless you install a task decorator that copies the context.
- What replaces it in Java 21? Scoped values offer immutable, structured context for virtual threads; thread locals still work but do not scale as a per-thread cache.

Mistakes to avoid: Using `ThreadLocal` as a general-purpose cache; skipping `remove` on exceptional paths; relying on `InheritableThreadLocal` in pooled executors; storing large objects per thread.

Production perspective: A missing `remove` is a genuine security issue — it can attribute one user's action to another in logs, or worse, in authorisation checks. Centralise context management in one filter and test it with concurrent requests.

Related concepts covered: MDC and correlation IDs, SecurityContextHolder, context propagation in async code, classloader leaks, virtual threads and scoped values.

Q061. How would you implement bounded producer-consumer work?

Priority: Important

Why interviewers ask it: It is the canonical backpressure design, and it appears in import jobs, batch processing and message consumers.

Interview-ready answer: I use a BlockingQueue with a fixed capacity between producers and consumers. Producers call put, which blocks when the queue is full, or offer with a timeout when I would rather reject than wait; consumers call take. The bound is the entire point: it applies backpressure to the producer instead of letting an unbounded queue consume the heap. For shutdown I use a poison-pill sentinel or interrupt the consumers and drain. If the work must survive a crash, an in-memory queue is the wrong tool and the queue belongs in Kafka, a database table or a broker.

In-depth explanation: ArrayBlockingQueue has a fixed array and optional fairness; LinkedBlockingQueue can be bounded or unbounded and has separate put and take locks, which often gives better throughput; SynchronousQueue has no capacity and hands off directly, which is what newCachedThreadPool uses; PriorityBlockingQueue is unbounded and ordered; DelayQueue releases elements at a time. Choosing the saturation policy is a business decision: block the producer, drop the oldest, drop the newest, or reject with an error to the caller. In a web application the last option is usually correct — return 429 or 503 quickly rather than queue work the user is no longer waiting for. Consumers should handle InterruptedException by restoring the flag and exiting, and any partially consumed work should be idempotent so a restart cannot double-apply it.

Practical backend example: A bounded ingestion pipeline with a poison pill:

```
private static final Row POISON = new Row(null);
private final BlockingQueue<Row> queue = new ArrayBlockingQueue<>(1_000); // hard bound

void produce(Stream<Row> rows) throws InterruptedException {
    for (Row row : (Iterable<Row>) rows::iterator) queue.put(row);           // blocks when full
    for (int i = 0; i < consumerCount; i++) queue.put(POISON);
}

void consume() {
    try {
        for (Row row = queue.take(); row != POISON; row = queue.take()) persist(row);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt(); // restore the flag and stop
    }
}
```

Common follow-ups:

- What should happen at saturation? Choose deliberately: block (backpressure), reject fast (429/503), or shed low-priority work. Silent unbounded growth is never acceptable.
- How do you stop consumers cleanly? A poison pill per consumer, or interruption plus a drain loop.
- Why not an unbounded queue? It converts overload into memory exhaustion and long GC pauses, and it hides the problem from metrics.

Mistakes to avoid: Using add instead of put/offer and getting IllegalStateException on a full queue; swallowing InterruptedException; assuming in-memory queues survive a restart; making the consumer non-idempotent.

Production perspective: Queue depth is a leading indicator: export it as a gauge and alert on sustained growth. A queue that is always full means the consumer is the bottleneck; a queue that is always empty means the producer is.

Related concepts covered: Backpressure, BlockingQueue variants, graceful shutdown, interruption, durability boundaries, queue metrics.

Q062. What do CountdownLatch and Semaphore solve?

Priority: Important

Why interviewers ask it: Choosing the right coordination primitive shows you can express intent rather than improvise with sleeps and flags.

Interview-ready answer: `CountDownLatch` is a one-shot gate: threads wait until a counter reaches zero, which suits "wait for N parallel calls to finish" or "wait until initialisation completes". It cannot be reset. `CyclicBarrier` is the reusable version where a fixed number of threads meet repeatedly. `Semaphore` limits concurrent access to a resource by handing out a fixed number of permits — the right tool for "at most 10 simultaneous calls to this legacy system". `Phaser` generalises barriers with dynamic registration. In modern code I often express the same fan-out with `CompletableFuture.allOf`, but a semaphore remains the clearest way to cap concurrency.

In-depth explanation: The distinction that matters is one-shot versus reusable, and counting-down versus permit-holding. A latch's `await` returns immediately once the count hits zero forever after, which makes it ideal for readiness signals; always use the timed `await` in production code so a lost countdown cannot hang a thread indefinitely. A semaphore's permits must be released in a `finally` block, otherwise permits leak and the system slowly grinds to a halt — a subtle bug that looks like a gradual slowdown. `tryAcquire` with a timeout gives fast failure instead of waiting, which is usually what an HTTP handler wants. Fair mode for both lock and semaphore eliminates starvation at a throughput cost. Note that these are all in-process: limiting concurrency across replicas requires a distributed mechanism such as a Redis-based limiter or a database token.

Practical backend example: Capping concurrency against a fragile downstream system:

```
private final Semaphore legacyPermits = new Semaphore(10); // at most 10 in flight

public Report fetchReport(String id) {
    if (!legacyPermits.tryAcquire()) { // fail fast under load
        throw new TooManyRequestsException("legacy system saturated");
    }
    try {
        return legacyClient.fetch(id);
    } finally {
        legacyPermits.release(); // always release
    }
}
```

Common follow-ups:

- Can a latch be reset? No; use `CyclicBarrier` or a new latch instance.
- What happens if a permit is never released? Capacity shrinks permanently; always release in `finally`.
- How do you limit concurrency across instances? A distributed limiter — Redis counters with expiry, or a gateway rate limit — because these primitives are per JVM.

Mistakes to avoid: Using `Thread.sleep` as a coordination mechanism; untimed `await` calls; releasing a semaphore on a different code path than the `acquire`; assuming in-process limits protect a shared downstream.

Production perspective: Semaphores are the simplest bulkhead: they stop one slow dependency from consuming all request threads. Pair the limit with metrics for rejections so capacity decisions are based on data.

Related concepts covered: Bulkhead pattern, fail-fast versus waiting, `CyclicBarrier` and `Phaser`, distributed rate limiting, fairness.

Q063. How should interruption be handled?

Priority: Must Know

Why interviewers ask it: Swallowed `InterruptedException` is the reason services refuse to shut down, and it is visible in almost every legacy codebase.

Interview-ready answer: Interruption is a cooperative cancellation request, not a forced stop. Calling `interrupt()` sets a flag; blocking methods such as `sleep`, `wait`, `take` and `join` throw `InterruptedException` and clear the flag as they do so. The two correct responses are to propagate the exception, or to catch it and restore the flag with `Thread.currentThread().interrupt()` before returning, so callers up the stack can still see the cancellation. Long CPU-bound loops should check `Thread.currentThread().isInterrupted()` periodically, because nothing will throw for them. Swallowing the exception with an empty catch or a log line destroys shutdown behaviour.

In-depth explanation: `Future.cancel(true)` interrupts the running thread; `ExecutorService.shutdownNow()` interrupts all workers. If those interrupts are ignored, `awaitTermination` times out and the JVM hangs on shutdown, which in Kubernetes becomes a SIGKILL after the grace period and potentially lost in-flight work. Interruption does not

unblock everything: a socket read on a classic blocking stream or a JDBC call will not throw `InterruptedException`, which is why network timeouts must be configured separately. Also note that `InterruptedException` and channel-based I/O behave differently. Inside a task, treat interruption as a request to stop promptly and leave state consistent — commit or roll back, release resources, then exit.

Practical backend example: A cancellable worker that stays responsive during shutdown:

```
public void process(BlockingQueue<Job> queue) {
    while (!Thread.currentThread().isInterrupted()) { // CPU-bound check
        Job job;
        try {
            job = queue.poll(1, TimeUnit.SECONDS); // throws on interrupt
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt(); // restore the flag
            break; // exit promptly
        }
        if (job != null) handle(job);
    }
}
```

Common follow-ups:

- Why restore the interrupt flag? Because catching the exception cleared it, and callers or the pool rely on it to know cancellation was requested.
- Does `interrupt()` stop a running thread? No; it requests cancellation, and code must cooperate.
- What about blocking I/O? Classic socket reads are not interruptible; use socket and HTTP client timeouts instead.

Mistakes to avoid: `catch (InterruptedException e) { };` logging and continuing as if nothing happened; using `Thread.stop()`, which is unsafe and removed; relying on interruption to cancel a JDBC query.

Production perspective: Correct interruption handling is what makes rolling deployments safe: the container sends `SIGTERM`, Spring Boot closes the context, executors stop, and in-flight work finishes or aborts cleanly within the grace period.

Related concepts covered: Cooperative cancellation, `shutdownNow`, graceful shutdown, I/O timeouts, `Future.cancel` semantics.

Q064. What are virtual threads in Java 21 and when should you use them?

Priority: Important

Why interviewers ask it: It checks whether you can evaluate a new platform feature honestly instead of treating it as a universal speed-up.

Interview-ready answer: Virtual threads, finalised in Java 21, are lightweight threads scheduled by the JVM onto a small pool of carrier platform threads. When a virtual thread blocks on most JDK I/O it unmounts from its carrier, so the operating system thread is free to run something else. That makes the thread-per-request model viable at very high concurrency without rewriting code in a reactive style. They do not make CPU-bound work faster — parallelism is still limited by cores — and they are not a replacement for bounded concurrency, because you still need to limit pressure on databases and downstream services. You do not pool virtual threads; you create one per task, typically through `Executors.newVirtualThreadPerTaskExecutor()`.

In-depth explanation: The main caveats in Java 21 are pinning and resource limits. A virtual thread that blocks inside a synchronized block or during a native call is pinned to its carrier and cannot unmount, so heavily synchronised blocking code can exhaust carriers; the guidance is to prefer `ReentrantLock` on such paths (later JDK releases reduce this restriction, so state the version you are assuming). Thread locals still work but per-thread caching patterns become expensive when there are a million threads; scoped values are the intended replacement. Because tasks are cheap, the bottleneck moves downstream: unbounded virtual threads can open more JDBC connections than the pool has, so you still need semaphores or pool limits. Spring Boot 3.2+ can run web requests on virtual threads with a single property, which is the easiest way to benefit for I/O-bound services.

Practical backend example: Fanning out many blocking calls with structured, bounded concurrency:

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) { // Java 21
    List<Future<Quote>> futures = suppliers.stream()
        .map(s -> executor.submit(() -> quoteClient.fetch(s))) // blocking call is fine
        .toList();
    for (Future<Quote> f : futures) collect(f.get());
}
```

```
# Spring Boot 3.2+ : handle web requests on virtual threads
spring.threads.virtual.enabled=true
```

Common follow-ups:

- Do virtual threads speed up CPU-bound work? No; throughput there is bounded by cores, and the extra scheduling adds no benefit.
- Should you pool them? No; create one per task. Pooling exists to amortise expensive platform threads, which virtual threads are not.
- What is pinning? A virtual thread blocked inside a synchronized block or native frame cannot unmount from its carrier in Java 21; prefer `ReentrantLock` there.
- Do they remove the need for limits? No; the database connection pool and downstream services still need bounded concurrency.

Mistakes to avoid: Calling them "faster threads"; keeping a fixed pool of virtual threads; assuming existing synchronized-heavy code scales unchanged; forgetting that the JDBC pool remains the real constraint.

Production perspective: For an I/O-bound Spring service, enabling virtual threads can raise concurrency substantially with minimal code change, but capacity planning shifts to the database and downstream quotas. Measure connection pool saturation and downstream error rates before and after.

Related concepts covered: Thread-per-request scaling, carrier threads and pinning, structured concurrency, scoped values, connection pool limits.

Q065. How do you avoid oversubscription when composing async calls?

Priority: Important

Why interviewers ask it: Fan-out looks harmless in code review and is a common cause of downstream overload.

Interview-ready answer: Oversubscription happens when concurrency multiplies: 200 request threads each firing 10 parallel downstream calls is 2,000 in-flight requests that no downstream system agreed to. I control it with explicit bounded executors per dependency, a semaphore or client-level connection limit, and timeouts on every call so slow work releases capacity. I also avoid sharing one pool across unrelated dependencies, because a bulkhead per dependency prevents one slow service from consuming all capacity. Finally, I make the degradation explicit: return partial results or a cached value instead of queueing indefinitely.

In-depth explanation: The total in-flight work equals request concurrency times per-request fan-out, capped by whichever pool is smallest. Three layers usually enforce limits: the HTTP client connection pool (max connections and per-route maximum), the executor running the tasks, and an explicit semaphore for a fragile dependency. A common hidden multiplier is using the `ForkJoin` common pool by default, which is sized to cores and shared by everything, so it both limits and entangles unrelated work. Timeouts must be smaller than the caller's timeout and should include connect, read and overall request deadlines; without a deadline, retries stack on top of a slow dependency and amplify load. Circuit breakers close the loop by stopping calls entirely when error rates spike, giving the downstream time to recover.

Practical backend example: Bounded fan-out with a dedicated pool, a client limit and a deadline:

```
private final ExecutorService pricingPool =
    new ThreadPoolExecutor(8, 8, 0L, TimeUnit.MILLISECONDS,
        new ArrayBlockingQueue<>(100), new CustomizableThreadFactory("pricing-"),
        new ThreadPoolExecutor.AbortPolicy());    // reject instead of piling up

public List<Price> priceAll(List<Sku> skus) {
    List<CompletableFuture<Price>> calls = skus.stream()
        .map(sku -> CompletableFuture
            .supplyAsync(() -> pricingClient.price(sku), pricingPool)
            .orTimeout(300, TimeUnit.MILLISECONDS)
            .exceptionally(ex -> Price.unavailable(sku)))    // degrade per item
        .toList();
    return calls.stream().map(CompletableFuture::join).toList();
}
```

Common follow-ups:

- How many parallel downstream calls are safe? Whatever the downstream can absorb; derive it from its published limits and verify with load tests, not intuition.
- Why not use the common ForkJoinPool? It is shared JVM-wide and sized for CPU work, so blocking calls there affect unrelated code.
- What happens when the bound is hit? Decide explicitly: reject fast, queue briefly with a bounded queue, or degrade to a cached response.

Mistakes to avoid: Unbounded CompletableFuture fan-out; retries without a budget; sharing one executor for all integrations; timeouts longer than the caller's own timeout.

Production perspective: Fan-out amplification turns a small downstream slowdown into a service-wide outage. Bulkheads plus deadlines plus circuit breakers are the standard defence, and each should emit metrics so saturation is observable.

Related concepts covered: Bulkheads, deadlines and timeouts, circuit breakers, connection pools, graceful degradation, retry amplification.

Q066. What memory areas does the JVM use for an application?

Priority: Must Know

Why interviewers ask it: Without a mental map of JVM memory you cannot interpret an OutOfMemoryError or size a container correctly.

Interview-ready answer: The heap holds all objects and is where garbage collection happens; it is split into young and old generations for generational collectors like G1. Each thread has its own stack holding frames, locals and references — that is where StackOverflowError comes from. Metaspace holds class metadata and lives in native memory, so it is not bounded by -Xmx. There is also the JIT code cache, thread structures, GC bookkeeping, and direct or mapped byte buffers used by NIO and some drivers. That last group matters in containers: total process memory is heap plus all native areas, so a container limit equal to -Xmx will be killed by the OOM killer.

In-depth explanation: Objects are allocated in the young generation's eden space, usually via a thread-local allocation buffer so allocation is a pointer bump. Surviving objects are copied between survivor spaces and eventually promoted to the old generation. Large objects may be allocated directly in old space or, in G1, in humongous regions. Strings are interned in the heap since Java 7. Metaspace grows dynamically and can be capped with -XX:MaxMetaspaceSize; leaks there usually come from dynamic class generation or repeated redeploys. Modern JVMs are container-aware: they read cgroup limits and size the heap from -XX:MaxRAMPercentage, which is the recommended way to configure Kubernetes workloads rather than a fixed -Xmx. Native Memory Tracking (-XX:NativeMemoryTracking=summary with jcmd VM.native_memory) is the tool for attributing non-heap growth.

Practical backend example: Container-aware sizing for a Spring Boot service:

```
ENV JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=70 -XX:+HeapDumpOnOutOfMemoryError \
-XX:HeapDumpPath=/tmp/heap.hprof -XX:NativeMemoryTracking=summary"
# container memory limit 1Gi -> heap ~700Mi, leaving room for metaspace, stacks and buffers
```

Common follow-ups:

- Where do class metadata and locals live? Metadata in metaspace (native memory), locals and frames on the thread stack.
- Why was my container killed although the heap looked fine? Non-heap memory — stacks, metaspace, direct buffers, GC overhead — counts toward the limit.
- What causes `StackOverflowError`? Deep or infinite recursion exhausting one thread's stack, tunable with `-Xss` but usually a bug.

Mistakes to avoid: Setting `-Xmx` equal to the container limit; assuming metaspace is part of the heap; ignoring direct buffer usage from HTTP clients and serialisation libraries; treating thread count as free when each thread reserves stack space.

Production perspective: Always enable `-XX:+HeapDumpOnOutOfMemoryError` with a writable path, and ensure the dump survives pod restarts through a mounted volume. Without the dump, diagnosing a memory incident after the fact is guesswork.

Related concepts covered: Generational heap layout, metaspace, thread stacks, direct buffers, container awareness, heap dumps.

Q067. How does garbage collection impact latency and throughput?

Priority: Must Know

Why interviewers ask it: GC knowledge separates "restart the service" from "understand what the pause graph is telling you".

Interview-ready answer: Garbage collection trades CPU and pause time for automatic memory management. Generational collectors exploit the observation that most objects die young: minor collections of the young generation are frequent but short, while old-generation work is less frequent and more expensive. G1, the default since Java 9, is region-based, collects concurrently and aims at a pause target you can set. ZGC and Shenandoah push pauses to single-digit milliseconds at some throughput cost, which suits latency-sensitive services. The practical points are that a full GC is not automatically a leak, that high allocation rate — not just heap size — drives GC cost, and that the fix for GC pressure is usually to allocate less or bound caches rather than to enlarge the heap.

In-depth explanation: Key signals are allocation rate, promotion rate, pause duration distribution and the proportion of time spent in GC. Continuous full GCs with little memory reclaimed indicate a real leak or an undersized heap; frequent long young collections often mean a very high allocation rate or an oversized young generation. Enlarging the heap reduces collection frequency but can lengthen individual pauses for older collectors; G1 mitigates this by collecting a subset of regions. `-XX:MaxGCPauseMillis` is a goal, not a guarantee. GC logs (`-Xlog:gc*`) and JDK Flight Recorder are the right diagnostic tools; a simple metric such as `jvm.gc.pause` from Micrometer is enough to alert on. Note that GC does not compact native memory, does not free direct buffers promptly, and cannot reclaim objects that are still referenced, which is why a growing cache looks exactly like a leak.

Practical backend example: Enabling useful GC observability without guessing:

```
# JVM flags
-Xlog:gc*:file=/var/log/app/gc.log:time,uptime:filecount=5,filesize=20M
-XX:+UseG1GC -XX:MaxGCPauseMillis=200
```

```
// Micrometer exposes jvm.gc.pause, jvm.memory.used and jvm.gc.memory.promoted automatically
@Bean
MeterBinder jvmGcMetrics() { return new JvmGcMetrics(); }
```

Common follow-ups:

- Is a full GC always a leak? No; it can be triggered by heap pressure, metaspace, or an explicit `System.gc()`. A leak shows as heap-after-GC trending upward over time.
- Which collector should you choose? G1 for general services; ZGC or Shenandoah when tail latency matters more than raw throughput; measure rather than assume.
- Does a bigger heap always help? It reduces frequency but can increase pause length and hide leaks; it also costs container memory.

Mistakes to avoid: Calling `System.gc()`; quoting pause numbers as guarantees; tuning flags before measuring allocation behaviour; blaming GC for latency without checking GC logs.

Production perspective: Track heap used after collection rather than instantaneous heap. A sawtooth returning to the same baseline is healthy; a rising baseline is a leak. Correlate pause spikes with request latency percentiles before tuning anything.

Related concepts covered: Generational hypothesis, G1/ZGC trade-offs, allocation rate, GC logging, leak versus pressure, Micrometer JVM metrics.

Q068. How would you investigate increasing heap usage?

Priority: Must Know

Why interviewers ask it: It is a realistic incident scenario that reveals whether you have actually debugged memory problems.

Interview-ready answer: First I confirm it is a leak rather than normal variation by looking at heap used after full collections over hours or days; a rising floor is the signal. Then I capture a heap dump — either automatically on `OutOfMemoryError` or on demand with `jcmd GC.heap_dump` — and open it in a tool such as Eclipse MAT, which shows the dominator tree and leak suspects. I look for the largest retained sets and ask what is holding the reference: usually an unbounded cache or map, a collection in a static field, accumulating `ThreadLocal` values, unclosed resources, or listeners never deregistered. I fix ownership and bound the structure rather than raising `-Xmx`, which only delays the failure.

In-depth explanation: Retained size is the important metric, not shallow size: it tells you how much memory would be freed if the object were collected. `jcmd <pid> GC.class_histogram` gives a quick class-count view without a full dump, and JFR can record allocation profiles with modest overhead in production. Distinguish leak classes: a genuine leak (references retained forever), a cache with no eviction (bounded by traffic, unbounded in effect), memory pressure from oversized working sets such as loading whole tables, and native memory growth, which will not appear in a heap dump at all — that needs Native Memory Tracking. In Spring applications, common culprits are `@Component` beans with mutable collections, static registries, un-evicted Hibernate second-level or session caches during batch jobs, and streaming responses accumulated in memory instead of streamed.

Practical backend example: Capturing evidence safely and fixing an unbounded cache:

```
jcmd <pid> GC.heap_info           # current occupancy
jcmd <pid> GC.class_histogram | head -25 # quick top classes by size
jcmd <pid> GC.heap_dump /tmp/heap.hprof # full dump (pauses the JVM briefly)
```

```
// before: unbounded, grows with distinct keys forever
private final Map<String, Quote> cache = new ConcurrentHashMap<>();
// after: bounded with eviction and metrics
private final Cache<String, Quote> cache = Caffeine.newBuilder()
    .maximumSize(50_000).expireAfterWrite(Duration.ofMinutes(10)).recordStats().build();
```

Common follow-ups:

- Can a cache be a memory leak? Effectively yes, if nothing bounds or evicts it.
- What if the heap dump looks small but the container still dies? Suspect native memory: direct buffers, metaspace, thread stacks; use Native Memory Tracking.
- Does a heap dump pause the application? Yes, briefly, and it writes a file roughly the size of the live heap, so plan disk and timing.

Mistakes to avoid: Raising `-Xmx` as the fix; restarting before collecting evidence; analysing a dump taken after a restart; ignoring `ThreadLocal` and classloader leaks in redeployed applications.

Production perspective: Make the evidence automatic: heap dump on OOM to a persisted volume, GC logs retained, and memory metrics scraped continuously. In an incident you rarely get a second chance to capture the failing state.

Related concepts covered: Retained size and dominator trees, heap dumps and JFR, unbounded caches, `ThreadLocal` leaks, native memory tracking.

Q069. How do heap OOM and native/thread exhaustion differ?

Priority: Important

Why interviewers ask it: The message text tells you where to look, and picking the wrong remedy wastes an incident.

Interview-ready answer: `OutOfMemoryError: Java heap space` means live objects exceed the heap: look for leaks, oversized working sets or an undersized heap. `Metaspace` means class metadata exhausted native memory, usually from dynamic class generation or repeated redeploys. `unable to create new native thread` means the OS or container refused another thread — too many threads, or a thread/process limit, not a heap problem. `Direct buffer memory` points at NIO buffers, often an HTTP client or serialisation library, governed by `-XX:MaxDirectMemorySize`. `GC overhead limit exceeded` means the collector is running constantly and reclaiming almost nothing, which is a heap problem in disguise. Raising `-Xmx` only helps the first and last, and can make thread exhaustion worse by leaving less room for stacks.

In-depth explanation: Each Java thread reserves stack space (typically around 512 KB to 1 MB), so thousands of platform threads consume gigabytes of native memory before doing any work — one of the motivations for virtual threads. In containers, the cgroup memory limit counts everything: heap, metaspace, stacks, code cache, direct buffers and JVM overhead, so heap should be a percentage of the limit rather than all of it. An `OutOfMemoryError` is an Error, not an Exception; catching it is almost always wrong because the JVM state is unreliable afterwards, and the usual configuration is `-XX:+ExitOnOutOfMemoryError` so an orchestrator can restart a clean instance. Note that a container kill by the Linux OOM killer produces no Java error at all — the process simply disappears with exit code 137, which is itself a diagnostic clue.

Practical backend example: Distinguishing the failure modes from evidence:

<code>java.lang.OutOfMemoryError: Java heap space</code>	-> heap dump, look for retained sets
<code>java.lang.OutOfMemoryError: Metaspace</code>	-> class loading, redeploy leaks
<code>java.lang.OutOfMemoryError: unable to create new native thread</code>	-> thread count, ulimits, pool sizes
<code>java.lang.OutOfMemoryError: Direct buffer memory</code>	-> NIO buffers, client library settings
<code>exit code 137, no Java error</code>	-> container limit exceeded (kernel OOM killer)

Common follow-ups:

- Does raising `-Xmx` fix every OOM? No; it is irrelevant for metaspace, native thread and direct buffer exhaustion, and it can worsen them in a fixed-size container.
- Should you catch `OutOfMemoryError`? Generally no; fail fast and let the orchestrator restart the instance.
- How do you find the thread count? `jcmd Thread.print | grep -c 'java.lang.Thread.State'` or JVM metrics; compare with pool configuration.

Mistakes to avoid: Assuming every OOM is a leak; catching and continuing; setting a huge heap in a small container; creating one thread per task with platform threads.

Production perspective: Alert on both JVM heap metrics and container memory working set, and record exit codes. Exit code 137 with no stack trace is a container-limit problem and needs a different fix from a Java heap error.

Related concepts covered: OOM taxonomy, thread stack cost, container limits and cgroups, direct memory, fail-fast configuration.

Q070. What do JIT warm-up and profiling mean for benchmarks?

Priority: Important

Why interviewers ask it: It guards against confident but unsupported performance claims, which is exactly what this book avoids.

Interview-ready answer: HotSpot starts by interpreting bytecode, then compiles hot methods with C1 and later C2, using runtime profiles to inline, unroll and eliminate code. So the first thousands of iterations are unrepresentative, and a naive timing loop measures the interpreter plus compilation. JMH exists to handle warm-up, forking, dead-code elimination and statistics correctly. Even then, results are specific to the JVM version, flags, hardware and data shape, which is why I avoid quoting benchmark numbers as universal facts. For production questions I prefer profiling the real system — JFR or async-profiler — over microbenchmarks, because real bottlenecks are usually I/O, locks or GC rather than raw code speed.

In-depth explanation: The JIT can also deoptimise: an optimisation based on a profile, such as a monomorphic inline cache, is discarded when a new implementation class appears, so a benchmark that exercises one implementation will overstate speed for a polymorphic production path. Dead-code elimination is the classic microbenchmark trap — if a result is unused, the compiler may remove the computation entirely; JMH's Blackhole prevents that. Other confounders are constant folding of fixed inputs, on-stack replacement, GC interference and CPU frequency scaling. Class loading and framework initialisation also make the first requests to a Spring Boot application slower, which is why readiness probes and warm-up requests matter more than they appear to.

Practical backend example: A correctly structured microbenchmark and a production profile:

```
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.MICROSECONDS)
@Warmup(iterations = 5) @Measurement(iterations = 10) @Fork(2)
@State(Scope.Benchmark)
public class SerializationBenchmark {
    private ObjectMapper mapper; private OrderDto order;
    @Setup public void setup() { mapper = new ObjectMapper(); order = OrderDto.sample(); }
    @Benchmark public String serialize() throws Exception { return mapper.writeValueAsString(order); }
}
```

```
# production profiling instead of guessing
jcmd <pid> JFR.start name=app settings=profile duration=120s filename=/tmp/app.jfr
```

Common follow-ups:

- Why not benchmark with `System.currentTimeMillis()`? It has coarse resolution, ignores warm-up and cannot prevent dead-code elimination; use JMH with `System.nanoTime()` semantics handled for you.
- What is deoptimisation? Reverting a speculative optimisation when its assumption breaks, such as a second implementation class appearing.
- Are microbenchmark results transferable? Only within the same JVM, flags, hardware and data shape; always re-measure in context.

Mistakes to avoid: Timing a cold loop; benchmarking with unrealistic constant inputs; reporting a single run; optimising code paths that the profiler shows are not hot.

Production perspective: Continuous profiling in a canary instance gives better information than any microbenchmark, and JFR's overhead is low enough for routine use. Base optimisation decisions on flame graphs and latency percentiles, not on intuition.

Related concepts covered: Tiered compilation, deoptimisation, JMH methodology, JDK Flight Recorder, profiling versus benchmarking, warm-up effects.

Q071. How do you read a thread dump during a latency incident?

Priority: Important

Why interviewers ask it: Reading a dump is a hands-on skill; it immediately distinguishes people who have handled incidents.

Interview-ready answer: I take two or three dumps a few seconds apart with `jcmd <pid> Thread.print`, because a single snapshot cannot show whether threads are stuck or simply busy. Then I group threads by pool name and state. Many threads `BLOCKED` on the same monitor means lock contention; many `WAITING` on a connection pool means the database is the bottleneck; many `TIMED_WAITING` in a socket read means a slow downstream; `RUNNABLE` threads in application code with high CPU means real work or a hot loop. If the same stack appears in every dump, it is stuck; if it changes, it is progressing. The JVM also reports detected deadlocks explicitly at the end of the dump.

In-depth explanation: Thread names are the most valuable diagnostic asset, which is why executors should always get a thread factory with a meaningful prefix — `http-nio-8080-exec-*`, `hikari-pool-1-connection-adder`, `kafka-consumer-*`. HikariCP starvation appears as request threads waiting in `getConnection`, often with a helpful `HikariPool` log line about pool stats. A pool with all threads in `parkNanos` inside `ForkJoinPool.managedBlock` suggests blocking work on the common pool. Combine the dump with CPU data: `top -H -p <pid>` gives per-thread CPU, and converting the thread ID to hex lets you find the matching `nid=` in the dump — that is how you identify a spinning thread. JFR gives the same information with timeline context and lower manual effort.

Practical backend example: Correlating a hot thread with its stack:

```

jcmd $(pgrep -f myapp.jar) Thread.print > dump1.txt; sleep 5
jcmd $(pgrep -f myapp.jar) Thread.print > dump2.txt
top -H -p $(pgrep -f myapp.jar)          # note the top TID, e.g. 4812
printf '%x\n' 4812                        # -> 12cc, then grep 'nid=0x12cc' dump2.txt
grep -c 'java.lang.Thread.State: BLOCKED' dump2.txt

```

Common follow-ups:

- What indicates pool starvation? All worker threads busy or waiting while the queue grows, plus requests timing out before being processed.
- BLOCKED versus WAITING? BLOCKED means waiting to acquire a monitor; WAITING means parked until another thread signals, for example in a queue or lock condition.
- Why take several dumps? To distinguish stuck threads from threads that are simply doing work.

Mistakes to avoid: Drawing conclusions from one dump; ignoring thread names; assuming RUNNABLE means consuming CPU (a socket read can appear RUNNABLE); restarting the process before capturing evidence.

Production perspective: Make dumps easy to take in production — a documented `kubectl exec` command or an operator runbook — and store them alongside the incident timeline. Combined with connection pool and GC metrics, a dump usually identifies the bottleneck within minutes.

Related concepts covered: `jcmd` and `jstack`, thread states, connection pool starvation, CPU-to-thread correlation, JFR, incident runbooks.

Q072. What happens if a task throws inside execute versus submit?

Priority: Important

Why interviewers ask it: Silently swallowed background exceptions are a classic production blind spot.

Interview-ready answer: With `execute(Runnable)`, an uncaught exception propagates out of the worker, the thread's `UncaughtExceptionHandler` runs (the default prints to `stderr`), and the pool replaces the thread. With `submit(...)`, the exception is captured inside the returned `Future` and nothing is logged; if nobody calls `get()`, the failure is completely invisible. That is why background tasks should either use `execute`, or wrap the body in `try/catch` with logging and metrics, or always inspect the `Future`. For scheduled tasks the consequence is worse: with `scheduleAtFixedRate`, an uncaught exception cancels all future executions of that task, so a job can stop silently.

In-depth explanation: `submit` wraps the task in a `FutureTask` that stores the throwable and rethrows it, wrapped in `ExecutionException`, when `get()` is called. `CompletableFuture` behaves similarly — an exceptional completion is invisible unless a stage observes it, which is why every chain should end in `exceptionally`, `handle` or `whenComplete` that logs. You can also override `ThreadPoolExecutor.afterExecute` to centralise logging for both styles, or set a `Thread.UncaughtExceptionHandler` via the thread factory. In Spring, `@Async` methods returning `void` route exceptions to an `AsyncUncaughtExceptionHandler` you can configure, while methods returning a future behave like `submit`. Scheduled task cancellation deserves special emphasis because the symptom — “the nightly job stopped running three weeks ago” — is usually discovered far too late.

Practical backend example: Making background failures visible in Spring:

```

@Configuration
@EnableAsync
public class AsyncConfig implements AsyncConfigurer {
    @Override public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
        return (ex, method, params) ->
            LoggerFactory.getLogger(AsyncConfig.class)
                .error("async failure in {}", method.getName(), ex); // never silent
    }
}

@Scheduled(fixedDelay = 60_000)
public void reconcile() {
    try { reconciliationService.run(); }
    catch (Exception e) { log.error("reconciliation failed", e); } // keeps the schedule alive
}

```

Common follow-ups:

- How do you observe submit failures? Call `get()`, or wrap the task body, or override `afterExecute` in the executor.
- Why did my scheduled task stop? An uncaught exception from `scheduleAtFixedRate` cancels subsequent runs; catch inside the task.
- Does `CompletableFuture` log failures? No; attach `exceptionally/whenComplete` or the error disappears.

Mistakes to avoid: Fire-and-forget submit with no inspection; catching `Exception` and logging at debug level; assuming the framework reports background errors; letting a scheduled job die silently.

Production perspective: Add a metric counter for background task failures and alert on it. Logs alone are easy to miss; a counter at zero that suddenly moves is actionable, and a job that stopped running is detectable by a freshness metric on its output.

Related concepts covered: `FutureTask` semantics, `UncaughtExceptionHandler`, `@Async` error handling, scheduled task cancellation, background observability.

Q073. How do you prevent lost updates in a shared in-memory counter?

Priority: Important

Why interviewers ask it: It connects a small concurrency exercise to the distributed version of the same problem.

Interview-ready answer: In one JVM, the fix is to make the read-modify-write atomic: `AtomicLong.incrementAndGet`, `LongAdder.increment`, a synchronized block, or `ConcurrentHashMap.merge`. Across multiple instances, in-memory state cannot work at all, because each replica has its own copy. Then the counter must live somewhere shared: a single SQL statement such as `UPDATE ... SET count = count + 1` (atomic at the database), a Redis `INCR`, or an optimistic-locking update with a version column that retries on conflict. The general rule is that the update must happen where the data is authoritative, not in application memory.

In-depth explanation: Lost updates happen whenever a value is read into a variable, modified and written back without a guarantee that nothing changed in between. The three standard remedies map cleanly across layers: atomic instructions (CAS) in memory, atomic statements or row locks in the database, and atomic commands in Redis. Optimistic locking — `UPDATE ... WHERE version = :expected` — is preferable when conflicts are rare because it avoids holding locks; pessimistic locking with `SELECT ... FOR UPDATE` suits high-contention rows. A read-then-write in the application combined with `@Transactional` alone does not prevent lost updates unless the isolation level or an explicit lock enforces it, since at `READ COMMITTED` both transactions can read the same value. For counters that tolerate approximation, per-instance aggregation flushed periodically reduces contention significantly.

Practical backend example: The same increment at three layers:

```
private final AtomicLong inFlight = new AtomicLong(); // per JVM only
inFlight.incrementAndGet();
```

```
-- authoritative and atomic in PostgreSQL, no read-modify-write in the application
UPDATE inventory SET reserved = reserved + 1
WHERE sku = :sku AND reserved < available; -- also enforces the invariant
```

```
// optimistic locking with JPA; retry on OptimisticLockException
@Version private long version;
```

Common follow-ups:

- Is `@Transactional` enough to stop lost updates? No; at `READ COMMITTED` two transactions can read the same value. Use an atomic statement, a version check or a row lock.
- What about a distributed counter? Use Redis `INCR` or a database column; in-memory counters diverge per replica and vanish on restart.
- When is optimistic better than pessimistic? When conflicts are rare; it avoids lock holding but requires retry handling.

Mistakes to avoid: Using a local cache as the source of truth; select then save without a version; ignoring `OptimisticLockException` instead of retrying; assuming a synchronised method protects anything across replicas.

Production perspective: Counters that drive business outcomes — inventory, credit limits, quotas — must be enforced by a database constraint or an atomic statement. Application-level checks are advisory and race under concurrency.

Related concepts covered: Atomic operations, optimistic and pessimistic locking, isolation levels, Redis atomic commands, retry with backoff.

Q074. What are the trade-offs between blocking and nonblocking APIs?

Priority: Important

Why interviewers ask it: It tests architectural judgement and immunity to hype in either direction.

Interview-ready answer: Blocking, thread-per-request code is simple: the stack trace is the story, debugging and profiling work normally, and transactions and thread-bound context are straightforward. Its cost is a thread per in-flight request, which limits concurrency when work is I/O-bound. Non-blocking or reactive code multiplexes many in-flight operations onto few threads, giving higher concurrency per unit of memory, but it costs readability, harder debugging, a different programming model end to end, and it fails badly if any library in the chain blocks. Reactive improves throughput and resource efficiency; it does not reduce the latency of a single call. In Java 21, virtual threads offer much of the scalability benefit while keeping blocking code, which makes them the pragmatic default for many services.

In-depth explanation: The decision depends on where the bottleneck is. If a service is database-bound, the connection pool caps concurrency long before threads do, so reactive plumbing adds complexity without capacity. If a service fans out to many slow HTTP dependencies and must hold tens of thousands of concurrent connections, non-blocking pays off. Mixed stacks are the real danger: one blocking JDBC call inside a reactive pipeline blocks an event-loop thread and can stall the whole application, which is why reactive systems need R2DBC or a bounded dedicated scheduler for blocking work. Backpressure is a genuine advantage of reactive streams, since demand is signalled explicitly, whereas blocking systems rely on bounded pools and queues for the same effect. Observability differs too: stack traces in reactive code are fragmented, so tracing context propagation must be configured deliberately.

Practical backend example: Choosing the model per workload:

```
// Blocking, transactional, database-bound: simplest and perfectly scalable behind a bounded pool
@Transactional
public Order place(OrderRequest request) { return orderRepository.save(Order.from(request)); }

// Non-blocking client for a high-fan-out, latency-tolerant aggregation
Flux<Quote> quotes = Flux.fromIterable(suppliers)
    .flatMap(s -> webClient.get().uri(s.url()).retrieve().bodyToMono(Quote.class)
        .timeout(Duration.ofMillis(300))
        .onErrorResume(e -> Mono.empty()), 8); // bounded concurrency
```

Common follow-ups:

- Does reactive always improve latency? No; it improves resource efficiency and concurrency. A single request is not faster and may be marginally slower.
- What breaks a reactive pipeline? Any blocking call on an event-loop thread; use a dedicated scheduler or a non-blocking driver.
- Where do virtual threads fit? They give thread-per-request code much better scalability without the reactive programming model.

Mistakes to avoid: Adopting reactive for a database-bound CRUD service; mixing blocking JDBC into an event loop; claiming reactive reduces latency; ignoring the operational cost of a less debuggable model.

Production perspective: The scarce resource is usually the database connection pool or a downstream quota, not threads. Fix the real bottleneck first; the programming model should follow the measurement.

Related concepts covered: Thread-per-request versus event loop, backpressure, R2DBC, virtual threads, tracing in async code, bottleneck analysis.

Q075. How would you diagnose a service with high CPU but low throughput?

Priority: Bonus

Why interviewers ask it: It is an integrative question requiring JVM, concurrency and observability knowledge together.

Interview-ready answer: I start with the cheap signals: GC metrics, thread states and request latency percentiles. High CPU with low throughput usually means the CPU is doing work that is not progress — most often garbage collection thrash, lock contention with spinning, an accidental hot loop, or excessive serialisation and logging. I check the proportion of time in GC and heap after collection first, because a nearly full heap causes constant collections that look like application CPU. Then I take thread dumps and per-thread CPU to find the hot stacks, or attach a profiler such as JFR or async-profiler to get a flame graph. The fix follows the evidence: bound a cache, remove a hot allocation, reduce lock scope, or fix the loop.

In-depth explanation: Distinguish three CPU consumers: application code, GC threads and the JIT compiler. `top -H` plus a dump attributes them precisely; GC threads are typically named `GC Thread#n` or `G1 Conc#n`. Very frequent full collections with small reclaimed amounts indicate heap pressure; in that state throughput collapses while CPU stays pinned. Lock contention with synchronized shows as `BLOCKED` threads and moderate CPU, while spin-heavy CAS contention shows as high CPU in `Unsafe` or atomic operations. Other common causes are regular expression backtracking on user input, debug-level logging of large payloads, JSON serialisation of oversized responses, and per-request creation of expensive objects like `ObjectMapper`, `SecureRandom` or a compiled `Pattern`. Flame graphs make these obvious in minutes compared with hours of guessing.

Practical backend example: A profiling sequence and a typical fix:

```
kubectl top pod api-7d9f          # confirm the container is CPU-bound
jcmd 1 JFR.start name=diag settings=profile duration=60s filename=/tmp/diag.jfr
jcmd 1 GC.heap_info              # heap after GC rising? -> pressure, not code
```

```
// typical culprit: per-request construction of expensive, thread-safe objects
private static final ObjectMapper MAPPER = new ObjectMapper();           // reuse
private static final Pattern SKU = Pattern.compile("[A-Z]{3}-\\d{6}$");    // compile once
```

Common follow-ups:

- What metrics do you inspect first? GC time and heap after collection, thread pool saturation, latency percentiles, then CPU by thread.
- How do you find a hot method quickly? A flame graph from JFR or async-profiler; a single stack sample is not enough.
- Could the cause be outside the JVM? Yes — noisy neighbours, CPU throttling from cgroup quotas, or encryption overhead; check throttling counters in Kubernetes.

Mistakes to avoid: Scaling out before diagnosing, which multiplies the cost of the same waste; assuming high CPU means efficient work; profiling only in a development environment with unrepresentative data.

Production perspective: CPU throttling in containers is a frequently missed cause: a low CPU limit makes the application appear slow while the JVM still sees many cores and sizes its thread pools accordingly. Always check throttling metrics and align pool sizes with the actual CPU quota.

Related concepts covered: Flame graphs and JFR, GC thrash, lock contention, cgroup CPU limits, object reuse, latency percentiles.

Chapter 4. Spring Framework and Spring Boot

All answers assume Spring Boot 3.x on Spring Framework 6 with Java 17, which means the jakarta.* namespace, SecurityFilterChain-style configuration and Hibernate 6. Where behaviour is a default that can be changed, the text says so.

Q076. What does dependency injection solve, and why prefer constructor injection?

Priority: Must Know

Why interviewers ask it: It is the foundation of every Spring answer that follows, and constructor injection is a design decision, not a style preference.

Interview-ready answer: Dependency injection inverts control of object creation: a class declares what it needs and the container supplies it, so the class depends on abstractions rather than on constructing concrete collaborators. I prefer constructor injection because it makes dependencies explicit and mandatory, allows the fields to be final so the object is immutable and safely published, and lets me instantiate the class in a plain unit test with new and hand-written fakes — no Spring context required. It also surfaces design problems: a constructor with nine parameters is telling me the class does too much. Field injection hides dependencies, prevents final fields, and cannot be used without reflection in tests.

In-depth explanation: Spring resolves constructor parameters by type, falling back to @Qualifier or the parameter name when several candidates exist. Since Spring 4.3 a single constructor needs no @Autowired annotation at all. Setter injection remains useful for genuinely optional dependencies, and ObjectProvider<T> handles optional or multiple candidates cleanly. A practical benefit of constructor injection is that circular dependencies fail fast at startup rather than producing a half-initialised bean; since Boot 2.6 circular references are rejected by default, and re-enabling them with spring.main.allow-circular-references is a workaround, not a fix. The deeper point is that DI exists to decouple policy from wiring — the same service class should run in production with a Kafka publisher and in a test with an in-memory one, without conditional code inside the class.

Practical backend example: A service whose dependencies are visible and testable:

```
@Service
public class CheckoutService {
    private final OrderRepository orders;           // final: set once, safely published
    private final PaymentGateway payments;

    public CheckoutService(OrderRepository orders, PaymentGateway payments) { // no @Autowired needed
        this.orders = orders;
        this.payments = payments;
    }
}

// unit test: no Spring context at all
var service = new CheckoutService(new InMemoryOrderRepository(), new StubPaymentGateway());
```

Common follow-ups:

- What about circular dependencies? They are rejected by default in Boot 2.6+; extract the shared behaviour into a third bean or use an event instead of forcing @Lazy.
- When is setter injection acceptable? For genuinely optional collaborators, or when a framework requires a no-argument constructor.
- How do you inject one of several implementations? @Qualifier by bean name, @Primary for a default, or inject List<T>/Map<String, T> to get them all.

Mistakes to avoid: Using @Autowired on fields because it is shorter; calling new on a dependency inside a service and losing substitutability; using @Lazy to hide a circular design problem.

Production perspective: Explicit constructor dependencies make refactoring and dependency auditing far easier; you can see from the signature what a class touches. It also keeps startup failures loud and early, which is much better than discovering a missing bean on the first production request.

Related concepts covered: Inversion of control, bean qualification, immutability, testability, circular dependency handling.

Q077. How are component scanning and bean registration different?

Priority: Must Know

Why interviewers ask it: Knowing both routes explains how third-party classes join the context and why a bean sometimes "isn't found".

Interview-ready answer: Component scanning discovers your own classes annotated with `@Component` and its stereotypes — `@Service`, `@Repository`, `@Controller`, `@RestController` — within the packages Spring scans, which by default is the package of the `@SpringBootApplication` class and everything below it. Explicit registration with `@Bean` methods inside a `@Configuration` class is how I register classes I do not own, such as an `ObjectMapper`, a `RestClient` or a third-party client, and how I control construction when it needs parameters or conditional logic. Both produce singletons in the same container; scanning is convention, `@Bean` is explicit configuration.

In-depth explanation: The stereotypes are semantically meaningful: `@Repository` adds persistence exception translation, `@RestController` implies `@ResponseBody`, and `@Configuration` classes are proxied by default so that calling one `@Bean` method from another returns the shared singleton instead of a new instance. Setting `proxyBeanMethods = false` on `@Configuration` avoids that proxy for a small startup gain, but then inter-method calls create new objects — a subtle trap. Package placement matters: a `@Service` in a sibling package of the main class is never scanned, which is the usual cause of "required a bean of type X that could not be found". `@Import`, `@ComponentScan(basePackages = ...)` and auto-configuration classes listed in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` are the remaining registration routes.

Practical backend example: Registering a third-party client explicitly while scanning your own services:

```
@Configuration
public class HttpClientConfig {
    @Bean
    RestClient pricingRestClient(RestClient.Builder builder,
                               @Value("${pricing.base-url}") String baseUrl) {
        return builder.baseUrl(baseUrl)
            .requestFactory(new SimpleClientHttpRequestFactory())
            .build(); // a class you do not own -> @Bean
    }
}

@Service // your own class -> component scanning
public class PricingClient { /* ... */ }
```

Common follow-ups:

- How do you register a class from a library? With a `@Bean` method, since you cannot annotate its source.
- Why was my `@Service` not found? It is outside the scanned package tree, or its auto-configuration condition excluded it.
- What does `proxyBeanMethods = false` change? Inter-`@Bean`-method calls no longer return the container singleton; they execute the method again.

Mistakes to avoid: Putting components outside the main application package; using `@Component` on a class that also needs constructor arguments only known at runtime; scanning very broad packages, which slows startup and can register unwanted beans.

Production perspective: Over-broad component scanning lengthens startup and can pull in test or sample classes. Keep the application package tree deliberate, and prefer explicit `@Bean` definitions for anything with configuration-dependent construction.

Related concepts covered: Stereotype annotations, `@Configuration` proxying, auto-configuration imports, bean not found diagnosis, startup cost.

Q078. What happens during bean creation and lifecycle?

Priority: Important

Why interviewers ask it: Lifecycle knowledge explains initialisation failures, proxy behaviour and why `@PostConstruct` sometimes sees incomplete state.

Interview-ready answer: For each singleton, Spring instantiates the bean (constructor injection happens here), populates remaining dependencies, calls `*Aware` callbacks, runs `BeanPostProcessor` before-initialisation hooks, then `@PostConstruct`, then `InitializingBean.afterPropertiesSet`, then any custom init method, then post-initialisation hooks — which is where AOP proxies such as `@Transactional` are created. Once all singletons are ready the context publishes `ContextRefreshedEvent` and Boot fires `ApplicationRunner/CommandLineRunner`. On shutdown it runs `@PreDestroy` and `DisposableBean.destroy` in reverse dependency order. The key consequence is that inside a constructor or `@PostConstruct` the bean is not yet proxied, so self-calls there bypass transactional and async behaviour.

In-depth explanation: `BeanPostProcessor` is the extension point behind most Spring "magic": `AutowiredAnnotationBeanPostProcessor` performs injection, `AbstractAutoProxyCreator` wraps beans in proxies, and `ConfigurationPropertiesBindingPostProcessor` binds configuration. Because proxies are applied after initialisation, work that must run through the proxy belongs in an `ApplicationRunner` or an `ApplicationListener<ApplicationReadyEvent>` rather than in `@PostConstruct`. Prototype beans are not fully managed: Spring creates and configures them but does not call destruction callbacks, so resources must be released explicitly. `SmartLifecycle` gives ordered start and stop phases, which is how message listener containers are started after the web layer is ready and stopped first during shutdown.

Practical backend example: Choosing the right hook for warm-up work:

```
@Component
public class CacheWarmer {
    private final PricingService pricing;           // proxied bean
    public CacheWarmer(PricingService pricing) { this.pricing = pricing; }

    @PostConstruct
    void validateConfig() { /* cheap, local checks only - no proxied self-calls, no DB work */ }

    @EventListener(ApplicationReadyEvent.class)    // context fully initialised and proxied
    void warmUp() { pricing.preloadPopularSkus(); }
}
```

Common follow-ups:

- When does `@PostConstruct` run? After dependency injection but before AOP proxies are in place around that bean.
- Why is my `@Transactional` ignored during startup? The proxy is not applied to self-invocations inside initialisation; use `ApplicationReadyEvent`.
- Are prototype beans destroyed by Spring? No; destruction callbacks are not invoked for prototypes.

Mistakes to avoid: Doing network or database work in `@PostConstruct` and delaying startup; relying on bean initialisation order instead of dependencies; putting `@Transactional` on an init method.

Production perspective: Work performed at startup extends deployment time and can fail a rollout. Prefer readiness-gated warm-up, keep initialisation fast, and let health checks report when the application is ready rather than blocking the context.

Related concepts covered: `BeanPostProcessor`, AOP proxy timing, `ApplicationReadyEvent`, `SmartLifecycle`, prototype limitations.

Q079. What are bean scopes and singleton thread-safety implications?

Priority: Must Know

Why interviewers ask it: Shared mutable state in a singleton bean is one of the most common concurrency bugs in Spring applications.

Interview-ready answer: The default scope is singleton: one instance per application context, shared by every request thread. That is fine as long as the bean is stateless or only holds immutable or thread-safe state; a mutable instance field on a controller or service is shared across concurrent requests and will corrupt data. Prototype creates a new instance per lookup, and the web scopes — request, session, application — bind an instance to that lifetime. Injecting a shorter-lived scope into a singleton needs a proxy or an `ObjectProvider`, otherwise the first instance is captured forever. My default is stateless singletons with all request state passed as method parameters.

In-depth explanation: "Singleton" in Spring means one per container, not the Gang of Four singleton pattern, and it is not automatically thread-safe — the container makes no synchronisation promises. Request-scoped beans are implemented with a scoped proxy that resolves the current request from a `ThreadLocal`, which means they do not work on a different thread, such as inside `@Async` or a `CompletableFuture` continuation, without explicit context propagation. Prototype beans injected into singletons are resolved once at injection time; `@Lookup` methods, `ObjectProvider<T>.getObject()` or `@Scope(proxyMode = TARGET_CLASS)` obtain a fresh instance per call. Fields that are safe include immutable configuration values, thread-safe collaborators (repositories, `RestClient`, `ObjectMapper`) and concurrent data structures.

Practical backend example: An unsafe field and its stateless replacement:

```
@RestController
public class ReportController {
    // BROKEN: shared across all concurrent requests
    // private List<Row> rows = new ArrayList<>();

    private final ReportService reportService; // stateless collaborator - safe
    public ReportController(ReportService reportService) { this.reportService = reportService; }

    @GetMapping("/reports/{id}")
    public ReportDto get(@PathVariable UUID id) {
        List<Row> rows = reportService.rows(id); // request state stays on the stack
        return ReportDto.from(rows);
    }
}
```

Common follow-ups:

- Is a singleton bean thread-safe? Only if it holds no mutable state; Spring adds no synchronisation.
- How do you inject a prototype into a singleton? `ObjectProvider`, `@Lookup`, or a scoped proxy — plain injection captures one instance.
- Do request-scoped beans work in `@Async` methods? Not by default; the request context is thread-bound and must be propagated explicitly.

Mistakes to avoid: Caching per-user data in a service field; using a non-thread-safe `SimpleDateFormat` as a bean field (use `DateTimeFormatter`, which is immutable); assuming prototype scope solves a concurrency problem in a singleton caller.

Production perspective: Shared-state bugs appear only under concurrency, so they pass functional tests and surface as cross-user data leakage in production — a correctness and privacy problem. Code review should treat any mutable field in a singleton as suspicious.

Related concepts covered: Scoped proxies, `ThreadLocal` request context, stateless design, thread-safe collaborators, async context propagation.

Q080. How does Spring Boot auto-configuration decide what to create?

Priority: Must Know

Why interviewers ask it: Auto-configuration feels like magic until you can explain conditions and inspect the report.

Interview-ready answer: `@SpringBootApplication` includes `@EnableAutoConfiguration`, which loads auto-configuration classes listed in each starter's `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`. Each class is guarded by conditions: `@ConditionalOnClass` (the library is on the classpath), `@ConditionalOnMissingBean` (you have not defined your own), `@ConditionalOnProperty`, `@ConditionalOnWebApplication` and others. So adding `spring-boot-starter-data-jpa` plus a driver gives you a `DataSource`, an `EntityManagerFactory` and a transaction manager — unless you define your own, in which case `@ConditionalOnMissingBean` backs off. To see exactly what happened I run with `--debug` or open `/actuator/conditions`, which lists positive and negative matches with reasons.

In-depth explanation: Ordering matters: auto-configurations run after user configuration so that `@ConditionalOnMissingBean` sees your beans, and `@AutoConfigureBefore`/`@AutoConfigureAfter` sequence them relative to each other. Properties are bound through `@ConfigurationProperties` classes such as

`DataSourceProperties` and `ServerProperties`, which is why `application.yml` keys map so predictably. You can exclude a configuration with `@SpringBootApplication(exclude = ...)` or `spring.autoconfigure.exclude`. Note that `@ConditionalOnMissingBean` evaluates on bean type by default, so defining a bean of a different type will not disable the default. Boot 2.7 moved the registration file from `spring.factories` to the `AutoConfiguration.imports` file, and Boot 3 completed that move — relevant when reading older tutorials or writing your own starter.

Practical backend example: Overriding one auto-configured bean while keeping the rest:

```
@Configuration
public class JacksonConfig {
    @Bean
    ObjectMapper objectMapper() {
        // replaces the auto-configured ObjectMapper
        // because of @ConditionalOnMissingBean
        return JsonMapper.builder()
            .addModule(new JavaTimeModule())
            .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS)
            .serializationInclusion(JsonInclude.Include.NON_NULL)
            .build();
    }
}
```

```
java -jar app.jar --debug # prints the condition evaluation report
curl localhost:8080/actuator/conditions | jq '.contexts.application.positiveMatches | keys'
```

Common follow-ups:

- How do you inspect what was auto-configured? `--debug` at startup or the `/actuator/conditions` endpoint.
- How do you override a default bean? Define your own of the same type; most auto-configurations back off via `@ConditionalOnMissingBean`.
- How do you disable one entirely? `spring.autoconfigure.exclude` or the `exclude` attribute of `@SpringBootApplication`.

Mistakes to avoid: Assuming auto-configuration is reflection-based guesswork rather than conditional configuration; excluding a whole starter when one property would do; defining a bean of a subtly different type and wondering why the default is still active.

Production perspective: Startup behaviour differences between environments usually come from classpath or property differences that flip conditions. The condition report is the fastest way to prove which branch was taken in a given deployment.

Related concepts covered: Conditional annotations, starters, property binding, actuator conditions endpoint, custom starters.

Q081. How should profiles and external configuration be managed?

Priority: Must Know

Why interviewers ask it: Configuration mistakes cause outages and credential leaks, and precedence rules are frequently misunderstood.

Interview-ready answer: I keep one `application.yml` with safe defaults, profile-specific files such as `application-prod.yml` for structural differences, and put anything environment-specific or secret into environment variables or a secret manager. Spring Boot resolves properties in a defined order where later sources win: command-line arguments beat environment variables, which beat profile-specific files, which beat the base file. Relaxed binding means `SPRING_DATASOURCE_URL` maps to `spring.datasource.url`, which is exactly how containers supply configuration. Profiles should express environment topology, not business logic; scattering `@Profile` across services makes behaviour hard to reason about and untestable.

In-depth explanation: The full precedence list includes devtools settings, test properties, command-line arguments, `SPRING_APPLICATION_JSON`, servlet parameters, JNDI, OS environment variables, Java system properties, profile-specific files outside and inside the jar, then the base files. Boot 3 supports config trees (`spring.config.import=configtree:/etc/secrets/`) for Kubernetes secrets mounted as files, and `spring.config.import` for sources such as Vault or Config Server. Secrets must never be committed, and they should not be passed as command-line arguments because they appear in the process list. `@ConfigurationProperties` with `@Validated` turns configuration errors into startup failures, which is exactly what

you want — failing at boot is better than failing on the first request. Keep profile use minimal and prefer property values to conditional beans wherever possible.

Practical backend example: Typed, validated configuration with environment overrides:

```
# application.yml
payments:
  base-url: https://sandbox.payments.local
  timeout: 2s
  api-key: ${PAYMENTS_API_KEY}      # supplied by the environment, never committed
```

```
@Validated
@ConfigurationProperties(prefix = "payments")
public record PaymentsProperties(
    @NotBlank String baseUrl,
    @NotNull Duration timeout,
    @NotBlank String apiKey) {}
```

Common follow-ups:

- What wins in property precedence? Command-line arguments over environment variables over profile files over the base file; the reference documentation lists the exact order.
- How do you supply secrets in Kubernetes? Environment variables from a Secret, or mounted files read with a config tree import; never baked into the image.
- Should business rules live behind profiles? No; use feature flags or configuration values so behaviour is testable without switching profiles.

Mistakes to avoid: Committing credentials to application-prod.yml; using @Value everywhere instead of typed properties; relying on the default profile in production; duplicating whole configuration files per environment.

Production perspective: Validated configuration at startup converts a class of runtime incidents into deployment failures caught by the rollout. Also log the active profiles at startup — but never log property values that may contain secrets.

Related concepts covered: Property precedence, relaxed binding, @ConfigurationProperties validation, secret management, config trees.

Q082. What is the Spring MVC request lifecycle?

Priority: Must Know

Why interviewers ask it: Knowing the order of filters, handler mapping, argument resolution and exception handling makes debugging web behaviour systematic.

Interview-ready answer: The servlet container passes the request through the filter chain — including Spring Security's filter — to the DispatcherServlet. It asks the HandlerMapping which controller method matches, then the HandlerAdapter invokes it. Before invocation, HandlerInterceptor.preHandle runs, then argument resolvers bind path variables, query parameters and the body, with @Valid triggering Bean Validation at that point. The method returns an object, a return-value handler serialises it through an HttpMessageConverter (Jackson for JSON), and postHandle/afterCompletion run. If anything throws, the HandlerExceptionResolver chain handles it, which is where @ControllerAdvice and @ExceptionHandler participate. Errors thrown in filters are outside MVC and land on the servlet error page instead.

In-depth explanation: That last distinction matters in practice: a @ControllerAdvice cannot handle an exception thrown by a filter, because the dispatcher never ran; security authentication failures are therefore handled by Spring Security's own entry point and access-denied handler, not by your advice. Content negotiation selects the converter based on the Accept header and the return type. ResponseEntity gives explicit control over status and headers. Spring 6 adds ProblemDetail (RFC 7807) as a first-class error representation, and ResponseEntityExceptionHandler already maps common Spring exceptions to it. Async handling differs: returning Callable, DeferredResult or CompletableFuture releases the container thread while processing continues, with interceptors following the AsyncHandlerInterceptor contract.

Practical backend example: Validation plus a consistent error shape:

```

@RestController
@RequestMapping("/api/orders")
public class OrderController {
    @PostMapping
    public ResponseEntity<OrderResponse> create(@Valid @RequestBody CreateOrderRequest request) {
        Order order = orderService.place(request.toCommand());
        return ResponseEntity.created(URI.create("/api/orders/" + order.id()))
            .body(OrderResponse.from(order));
    }
}

@RestControllerAdvice
class ApiExceptionHandler {
    @ExceptionHandler(MethodArgumentNotValidException.class)
    ProblemDetail onInvalid(MethodArgumentNotValidException ex) {
        ProblemDetail problem = ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);
        problem.setTitle("Validation failed");
        problem.setProperty("errors", ex.getBindingResult().getFieldErrors().stream()
            .collect(Collectors.toMap(FieldError::getField, FieldError::getDefaultMessage, (a, b)
                -> a)));
        return problem; // problem-details body (RFC 9457)
    }
}

```

Common follow-ups:

- Where does validation run? During argument resolution, before the controller body executes; failures raise `MethodArgumentNotValidException`.
- Can `@ControllerAdvice` handle filter exceptions? No; those occur before the dispatcher, so security and servlet-level handlers deal with them.
- What is the difference between an interceptor and a filter here? Filters wrap the whole servlet request; interceptors run inside the dispatcher and know which handler was selected.

Mistakes to avoid: Expecting `@ExceptionHandler` to catch authentication failures; returning entities directly and triggering lazy loading during serialisation; ignoring content negotiation and hard-coding produces incorrectly.

Production perspective: A single consistent error contract across all endpoints — ideally `ProblemDetail` — makes client handling and log triage far simpler. Always include a correlation identifier in error responses so a user report maps to a log entry.

Related concepts covered: `DispatcherServlet` flow, argument resolvers, `HttpMessageConverter`, `ProblemDetail`, async request handling, filter versus interceptor.

Q083. How do `@RequestParam`, `@PathVariable` and `@RequestBody` differ?

Priority: Important

Why interviewers ask it: Correct binding choices reflect an understanding of HTTP semantics, not just Spring annotations.

Interview-ready answer: `@PathVariable` binds a segment of the URI template and identifies a resource — `/orders/{id}`. `@RequestParam` binds query string parameters or form fields, which suit filtering, sorting and pagination — `/orders?status=PAID&page=2`. `@RequestBody` deserialises the request body, normally JSON via Jackson, and is used for create and update payloads. Required-ness differs: path variables are inherently required, request parameters can have `required = false` and a `defaultValue`, and a missing body produces a 400. Header and cookie values have their own annotations. The rule of thumb is identity in the path, filters in the query, and state in the body.

In-depth explanation: Type conversion happens through Spring's `ConversionService`, so `UUID`, `LocalDate` and enums bind automatically; a malformed value produces `MethodArgumentTypeMismatchException`, which maps to 400 rather than 500 if you handle it. Validation applies differently: `@Valid @RequestBody` triggers `MethodArgumentNotValidException`, while constraints directly on `@RequestParam` require `@Validated` on the class and raise `ConstraintViolationException` — two different exception types that both deserve mapping in your advice. Jackson deserialisation of records works in Boot 3 without extra annotations in most cases. Sensitive data should never travel in query parameters because URLs are logged by proxies and stored in browser history; use the body or a header instead.

Practical backend example: One endpoint using each binding correctly:

```
@GetMapping("/api/customers/{customerId}/orders")
public Page<OrderSummary> list(
    @PathVariable UUID customerId,                // identity
    @RequestParam(defaultValue = "PAID") OrderStatus status, // filter, with default
    @RequestParam(required = false) @Past LocalDate since, // optional filter
    @PageableDefault(size = 20, sort = "createdAt") Pageable pageable) {
    return orderQueryService.search(customerId, status, since, pageable);
}

@PutMapping("/api/orders/{id}")
public OrderResponse update(@PathVariable UUID id, @Valid @RequestBody UpdateOrderRequest body) {
    return OrderResponse.from(orderService.update(id, body.toCommand()));
}
```

Common follow-ups:

- How are invalid bodies reported? Jackson failures surface as `HttpMessageNotReadableException` (400); constraint failures as `MethodArgumentNotValidException`.
- Can GET have a body? HTTP does not forbid it, but it is not interoperable — use query parameters, or POST for complex searches.
- Why not put a token in a query parameter? URLs are logged and cached; credentials belong in headers.

Mistakes to avoid: Using `@RequestParam` for resource identity; accepting entities as request bodies; forgetting `@Validated` when validating parameters; putting personally identifiable data in the URL.

Production perspective: Query parameters end up in access logs and traces; body content usually does not. That difference matters for both privacy compliance and log volume, and it should drive the API design rather than convenience.

Related concepts covered: HTTP semantics, type conversion, validation exception types, pagination parameters, logging and privacy.

Q084. How would you validate requests and return consistent errors?

Priority: Must Know

Why interviewers ask it: Error contracts are part of an API's public surface, and inconsistent ones are a real source of client bugs.

Interview-ready answer: I validate DTO shape declaratively with Jakarta Bean Validation — `@NotBlank`, `@Positive`, `@Email`, `@Size` — triggered by `@Valid` on the request body, and I use `@Valid` on nested objects and collection elements so the whole graph is checked. Business rules that need database state or cross-field logic stay in the service layer and throw domain exceptions. Both paths converge in a `@RestControllerAdvice` that produces one consistent error body, ideally `ProblemDetail` with a stable type, a human-readable title, a field-level error list and a correlation ID. Validation returns 400, domain conflicts 409, missing resources 404, and anything unexpected 500 without leaking a stack trace.

In-depth explanation: Group validation with constraint groups handles create-versus-update differences without duplicating DTOs, and custom constraints (`@Constraint` with a `ConstraintValidator`) keep complex rules declarative. Note the exception taxonomy: `MethodArgumentNotValidException` for bodies, `ConstraintViolationException` for parameter-level constraints, `HttpMessageNotReadableException` for malformed JSON, and `MethodArgumentTypeMismatchException` for bad path or query types. Extending `ResponseEntityExceptionHandler` gives sensible defaults for the Spring-raised set. Accumulate all field errors rather than failing on the first one, since clients want to show every problem at once. Never echo the raw input value of a sensitive field in the error message, and keep messages stable enough that clients can rely on codes rather than prose.

Practical backend example: Layered validation with a single error contract:

```

public record CreateOrderRequest(
    @NotNull UUID customerId,
    @NotEmpty @Valid List<LineRequest> lines,           // nested validation
    @Size(max = 280) String note) {
    public record LineRequest(@NotBlank String sku, @Positive int quantity) {}
}

@Service
public class OrderService {
    @Transactional
    public Order place(PlaceOrderCommand command) {
        if (!inventory.hasStock(command.lines())) {
            throw new InsufficientStockException(command.lines()); // business rule -> 409
        }
        return orderRepository.save(Order.from(command));
    }
}

@RestControllerAdvice
class ApiErrors extends ResponseEntityExceptionHandler {
    @ExceptionHandler(InsufficientStockException.class)
    ProblemDetail onStock(InsufficientStockException ex) {
        ProblemDetail p = ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT, ex.getMessage());
        p.setType(URI.create("https://api.example.com/problems/insufficient-stock"));
        p.setProperty("correlationId", MDC.get("correlationId"));
        return p;
    }
}

```

Common follow-ups:

- How do you validate nested objects? Annotate the field with `@Valid`; without it the nested constraints are skipped.
- Can `@Valid` replace service-layer validation? No; it checks shape and format, not invariants that depend on stored state.
- Which status code for a business conflict? 409 for state conflicts, 422 when the payload is syntactically valid but semantically unacceptable, 400 for malformed input.

Mistakes to avoid: Returning 500 for validation failures; exposing stack traces or SQL text to clients; validating only in the controller and trusting the service; inconsistent error shapes between endpoints.

Production perspective: A stable error contract reduces support load and makes client retries safe. Include a correlation ID in every error body so a user-reported failure can be found in logs and traces instantly.

Related concepts covered: Bean Validation, constraint groups, `ProblemDetail`, status code selection, domain exceptions, correlation IDs.

Q085. How does `@Transactional` work through proxies?

Priority: Must Know

Why interviewers ask it: Proxy semantics explain the single most common "why is my transaction not working?" bug in Spring applications.

Interview-ready answer: `@Transactional` is implemented with AOP: Spring creates a proxy around the bean, and the proxy opens a transaction before the method and commits or rolls back after it. Because the interception happens on the way in through the proxy, an internal call from one method of the bean to another — self-invocation — goes directly to the target object and is never intercepted, so the annotation is silently ignored. The same applies to private, static or final methods, since the proxy cannot intercept them. The fixes are to move the transactional method to another bean, inject a self-reference, or use `TransactionTemplate` for explicit control. Spring Boot uses CGLIB class proxies by default, so an interface is not required.

In-depth explanation: The transaction itself is bound to the thread through `TransactionSynchronizationManager`, which is also how the `EntityManager` and `JDBC Connection` are shared within a boundary. That thread binding is why transactions do not propagate into `@Async` methods or new threads. Rollback rules are another frequent surprise: by default Spring rolls back on unchecked exceptions and `Error`, but commits on checked exceptions unless you declare `rollbackFor`. Ordering with other aspects matters too — `@Transactional` and `@Async` on the same method create two proxies, and the effective behaviour depends on order, which is a good reason not to combine them. Since the proxy wraps the bean, any `@PostConstruct` work executes before the transactional proxy exists.

Practical backend example: The self-invocation trap and two correct alternatives:

```
@Service
public class ImportService {
    private final RowRepository rows;
    private final TransactionTemplate tx;           // explicit, no proxy involved

    public void importAll(List<Row> batch) {
        for (Row row : batch) {
            saveRow(row);                         // BROKEN: self-call bypasses the proxy
        }
        for (Row row : batch) {
            tx.executeWithoutResult(status -> rows.save(row)); // WORKS: explicit boundary
        }
    }

    @Transactional
    public void saveRow(Row row) { rows.save(row); } // only transactional when called from outside
}
```

Common follow-ups:

- Why does self-invocation bypass the transaction? The call never leaves the target object, so the proxy has no chance to intercept it.
- Do private methods work? No; CGLIB proxies cannot advise private, static or final methods.
- How do you verify a boundary exists? `TransactionSynchronizationManager.isActualTransactionActive()` in a test, or enable transaction debug logging.
- Are checked exceptions rolled back? Not by default; declare `@Transactional(rollbackFor = Exception.class)` if you need that.

Mistakes to avoid: Annotating a private method; wrapping an entire request in one long transaction; calling a transactional method from the same class; assuming a transaction spans threads.

Production perspective: A missing transaction boundary appears as partially applied writes — an order saved without its lines, or an event published without the row it describes. These bugs survive testing because they only manifest when a later step fails.

Related concepts covered: AOP proxies, transaction synchronisation, rollback rules, `TransactionTemplate`, `async` and transaction interaction.

Q086. What do propagation modes **REQUIRED** and **REQUIRES_NEW** actually do?

Priority: Must Know

Why interviewers ask it: Propagation determines what survives a failure, and **REQUIRES_NEW** has operational consequences people miss.

Interview-ready answer: **REQUIRED**, the default, joins an existing transaction if one is active and starts a new one otherwise; everything commits or rolls back together. **REQUIRES_NEW** suspends the current transaction and starts a genuinely independent one with its own database connection, so it can commit even if the outer transaction later rolls back — useful for audit records or failure logging that must survive. The catch is that it borrows a second connection from the pool while the first is held, so nesting it under load can exhaust the pool and deadlock. **NESTED** uses savepoints within one transaction, allowing partial rollback, and is supported only by some JDBC drivers and platforms. **MANDATORY**, **SUPPORTS**, **NOT_SUPPORTED** and **NEVER** express the remaining combinations.

In-depth explanation: An important consequence of joining in **REQUIRED** mode is that an inner failure marks the whole transaction rollback-only, so even if the caller catches the exception the commit fails with `UnexpectedRollbackException` — this confuses teams who expect a caught exception to be recoverable. **REQUIRES_NEW** avoids that but introduces a visibility subtlety: the inner transaction cannot see uncommitted changes from the outer one, so passing entity IDs created but not yet flushed will fail. The pool interaction is the practical risk: with a pool of 10 connections, ten concurrent requests each holding an outer transaction and opening an inner one will deadlock waiting for connections. Keep the inner transaction short, and size the pool with nesting in mind.

Practical backend example: Audit that must persist even when the business transaction fails:

```

@Service
public class PaymentService {
    @Transactional                                // REQUIRED (default)
    public void charge(Order order) {
        auditService.record(order.id(), "CHARGE_ATTEMPTED"); // committed independently
        gateway.capture(order.total());                     // may throw -> outer rolls back
        order.markPaid();
    }
}

@Service
public class AuditService {
    @Transactional(propagation = Propagation.REQUIRES_NEW) // own connection, own commit
    public void record(UUID orderId, String event) {
        auditRepository.save(new AuditRow(orderId, event, Instant.now()));
    }
}

```

Common follow-ups:

- Does REQUIRES_NEW share the same transaction? No; it suspends the outer one and uses a separate connection and commit.
- What is UnexpectedRollbackException? The outer transaction was marked rollback-only by an inner failure, so the commit could not proceed.
- When would you use NESTED? For partial rollback via savepoints within a single transaction, where the platform supports it.
- Can the inner transaction see the outer one's uncommitted rows? No, which can cause foreign key failures if you pass unflushed IDs.

Mistakes to avoid: Using REQUIRES_NEW casually inside loops; assuming catching an exception inside a joined transaction makes it recoverable; ignoring connection pool sizing when nesting transactions.

Production perspective: Connection pool exhaustion caused by nested transactions is a classic incident: requests hang, HikariCP logs timeouts, and CPU is idle. If audit or outbox writes must survive failures, consider the transactional outbox pattern instead of nested transactions.

Related concepts covered: Transaction propagation, savepoints, connection pool sizing, rollback-only marking, outbox pattern.

Q087. What determines transaction rollback and isolation in Spring?

Priority: Must Know

Why interviewers ask it: Default rollback behaviour surprises people, and isolation choices affect both correctness and throughput.

Interview-ready answer: By default Spring rolls back on RuntimeException and Error and commits on checked exceptions; you change that with rollbackFor and noRollbackFor. Once any participant marks the transaction rollback-only, the commit cannot succeed. Isolation is delegated to the database: @Transactional(isolation = ...) sets the level for that transaction, and PostgreSQL's default is READ COMMITTED, which prevents dirty reads but allows non-repeatable reads and phantoms. Raising the level to REPEATABLE READ or SERIALIZABLE increases consistency but also the chance of serialisation failures that the application must retry. I usually keep READ COMMITTED and enforce correctness with explicit locking, unique constraints or optimistic versioning where it matters.

In-depth explanation: The checked-exception default comes from EJB conventions; it is a frequent bug source in code that throws checked domain exceptions expecting a rollback. Note also that catching an exception inside the transactional method prevents the proxy from ever seeing it, so no rollback occurs at all unless you call TransactionAspectSupport.currentTransactionStatus().setRollbackOnly(). Isolation levels are enforced by the database engine, and their exact semantics vary between products: PostgreSQL implements REPEATABLE READ using snapshot isolation, which also prevents phantom reads, whereas the SQL standard permits them — a good example of distinguishing standard from implementation. SERIALIZABLE in PostgreSQL uses serialisable snapshot isolation and can abort transactions with SQLSTATE 40001, so retry logic is mandatory when you choose it. Isolation only applies within the database; it says nothing about external API calls.

Practical backend example: Explicit rollback rules and a retryable serialisable operation:

```

@Transactional(rollbackFor = InsufficientFundsException.class) // checked exception must roll back
public void withdraw(UUID accountId, Money amount) throws InsufficientFundsException {
    Account account = accountRepository.findById(accountId).orElseThrow();
    if (!account.canWithdraw(amount)) throw new InsufficientFundsException(accountId);
    account.debit(amount);
}

@Retryable(retryFor = CannotAcquireLockException.class, maxAttempts = 3,
    backoff = @Backoff(delay = 50, multiplier = 2.0))
@Transactional(isolation = Isolation.SERIALIZABLE)
public void reconcile(UUID ledgerId) { /* may abort with SQLSTATE 40001 and be retried */ }

```

Common follow-ups:

- Do checked exceptions roll back by default? No; only unchecked exceptions and errors do.
- What happens if you catch the exception inside the method? No rollback occurs unless you mark the transaction rollback-only explicitly.
- Does PostgreSQL REPEATABLE READ allow phantom reads? Its snapshot isolation prevents them, which is stronger than the SQL standard requires.
- Is SERIALIZABLE safe to use? It is correct but can abort transactions under contention, so it requires retry handling.

Mistakes to avoid: Assuming every exception rolls back; swallowing exceptions inside transactional methods; raising isolation globally to fix one race; expecting isolation to protect an external HTTP call.

Production perspective: Higher isolation shifts failures from silent data anomalies to visible serialisation errors — better, provided the application retries idempotently. Always log and meter retry counts so contention is visible before it becomes a latency problem.

Related concepts covered: Rollback rules, rollback-only marking, isolation levels, PostgreSQL snapshot isolation, retry strategies.

Q088. Why can a read-only transaction still write?

Priority: Important

Why interviewers ask it: It is a precise test of the difference between an optimisation hint and an enforced guarantee.

Interview-ready answer: `@Transactional(readOnly = true)` is primarily a hint. For Hibernate it sets the flush mode to manual, so the persistence context does not dirty-check or flush changes automatically, and it can avoid taking snapshots of loaded entities, which saves memory on large reads. Spring also passes the read-only flag to the JDBC connection, and whether that is enforced depends on the driver and database: PostgreSQL does honour a read-only transaction and rejects writes with an error, while some databases ignore it. So with JPA alone, an explicit `flush()` or a native query could still write; it is best treated as an optimisation and a statement of intent, not a security control.

In-depth explanation: The performance benefit is real for query-heavy endpoints: skipping dirty checking on thousands of loaded entities reduces CPU and memory. The semantic benefit is documentation — readers know the method is not supposed to modify state. Some setups also route read-only transactions to a read replica through a routing `DataSource`, which is where the annotation acquires real operational meaning, and where a stray write would fail outright. Note also that `readOnly` on a method that lazily initialises collections still works, since loading is not writing. If you truly need to guarantee no writes, rely on database permissions for the connection user rather than on the annotation.

Practical backend example: Read-only query services and replica routing:

```
@Service
@Transactional(readOnly = true)                // class-level default for queries
public class OrderQueryService {
    public Page<OrderSummary> search(OrderFilter filter, Pageable pageable) {
        return orderRepository.search(filter, pageable); // no dirty checking, no flush
    }

    @Transactional                            // explicit override where writes are intended
    public void archive(UUID id) { orderRepository.archive(id); }
}
```

Common follow-ups:

- Does readOnly enforce immutability? Not by itself; it is a hint to Hibernate and the driver. PostgreSQL does reject writes in a read-only transaction, but do not rely on it as a security boundary.
- What is the actual benefit? Skipped dirty checking and flushes, lower memory for large result sets, clear intent, and the ability to route to replicas.
- Can lazy loading still work? Yes; reading associations is not a write.

Mistakes to avoid: Treating readOnly as authorisation; expecting it to speed up every query; mixing writes into a class-level read-only service without overriding it.

Production perspective: Combining read-only transactions with replica routing is a common scaling step, but it introduces replication lag: a read immediately after a write may not see the new row. Route flows that need read-your-writes consistency to the primary.

Related concepts covered: Hibernate flush modes, dirty checking cost, read replicas and replication lag, JDBC connection flags, intent documentation.

Q089. How do Spring filters and interceptors differ?

Priority: Important

Why interviewers ask it: Placing cross-cutting logic at the wrong layer causes security gaps and duplicated work.

Interview-ready answer: A Filter is a servlet-level component that wraps the entire request, including static resources and error dispatches, and runs before Spring MVC decides which handler applies. It can modify the request and response streams, which is why authentication, correlation IDs, compression and CORS live there. A HandlerInterceptor runs inside the DispatcherServlet and knows which handler method was selected, so it suits handler-aware concerns such as per-endpoint auditing, rate limits keyed by route, or adding model attributes. Spring Security is entirely filter-based, so authentication decisions happen before any interceptor or controller code runs.

In-depth explanation: Ordering is explicit: filters run in registration order (@Order or FilterRegistrationBean), and OncePerRequestFilter ensures a filter is not executed twice on forwards or async dispatches. Interceptors have three hooks — preHandle, postHandle and afterCompletion — and postHandle is not called when an exception is thrown, so cleanup belongs in afterCompletion. Exceptions thrown in filters bypass @ControllerAdvice, which is why security error responses need their own AuthenticationEntryPoint and AccessDeniedHandler. For simple cross-cutting behaviour on service methods, AOP aspects are a third option, and for reactive applications WebFilter replaces the servlet filter. Choosing the outermost layer that has enough information is a good default rule.

Practical backend example: A filter for correlation and an interceptor for per-endpoint metrics:

```

@Component
@Order(Ordered.HIGHEST_PRECEDENCE)
public class CorrelationFilter extends OncePerRequestFilter { /* sets MDC, clears in finally */ }

@Component
public class AuditInterceptor implements HandlerInterceptor {
    @Override public boolean preHandle(HttpServletRequest req, HttpServletResponse res, Object
        handler) {
        if (handler instanceof HandlerMethod hm) { // knows the target method
            MDC.put("endpoint", hm.getBeanType().getSimpleName() + "#" + hm.getMethod().getName());
        }
        return true;
    }
    @Override public void afterCompletion(HttpServletRequest req, HttpServletResponse res,
        Object handler, Exception ex) { MDC.remove("endpoint"); }
}

```

Common follow-ups:

- Where should authentication live? In the filter chain, before handler resolution — this is what Spring Security does.
- Why was `postHandle` not called? An exception was thrown; use `afterCompletion` for cleanup that must always run.
- What guarantees a filter runs once? Extending `OncePerRequestFilter`, which guards against forwards and async re-dispatch.

Mistakes to avoid: Implementing authorisation in an interceptor that security filters already bypassed; forgetting to clear thread-bound state; assuming `@ControllerAdvice` covers filter exceptions; reading the request body in a filter without wrapping it, which leaves nothing for the controller.

Production perspective: Correlation IDs set in the outermost filter make every downstream log and trace joinable. Keep filters cheap — they run for every request including health checks — and exclude actuator paths from expensive logic.

Related concepts covered: Servlet filter chain, `OncePerRequestFilter`, `HandlerInterceptor` hooks, Spring Security placement, request body wrapping, MDC hygiene.

Q090. How do you avoid circular dependency and oversized services?

Priority: Important

Why interviewers ask it: A circular dependency is usually a symptom of a design problem, and the interviewer wants the diagnosis, not the workaround.

Interview-ready answer: A circular dependency means two beans each need the other during construction; since Boot 2.6 that fails at startup by default. The workarounds — `@Lazy`, setter injection, or enabling circular references — hide the real issue, which is that responsibilities are tangled. My first move is to look for the shared concept: usually a third collaborator should own the common logic, or one direction should become an event rather than a direct call. The same instinct applies to oversized services: when a class has a dozen dependencies it is doing several jobs, and splitting it along use cases makes both testing and reasoning simpler.

In-depth explanation: Three refactoring patterns cover most cases. First, extract a shared component that both beans depend on, turning a cycle into a tree. Second, invert one direction with an interface owned by the consumer, so the dependency points the way the domain does. Third, decouple with an application event when the second action is a reaction rather than a requirement — although in-process events are synchronous by default and are not durable. For large services, splitting by use case — `PlaceOrderService`, `CancelOrderService` — keeps each class small and gives transaction boundaries a natural home. Package-by-feature rather than package-by-layer reduces the temptation to create a giant shared service in the first place.

Practical backend example: Breaking a cycle by extracting the shared rule:

```
// before: OrderService -> PricingService -> OrderService (cycle, context fails to start)

@Component // extracted shared logic, no back-reference
public class DiscountPolicy {
    public Money apply(Money base, CustomerTier tier) { /* pure calculation */ }
}

@Service
public class PricingService {
    private final DiscountPolicy discountPolicy; // depends downward only
    public PricingService(DiscountPolicy discountPolicy) { this.discountPolicy = discountPolicy; }
}

@Service
public class OrderService {
    private final PricingService pricing; // one direction only
    public OrderService(PricingService pricing) { this.pricing = pricing; }
}
```

Common follow-ups:

- Does @Lazy solve the design problem? No; it defers resolution so startup succeeds, but the tangled responsibilities remain.
- Is an application event a good fix? Sometimes — it decouples the direction, but in-process events are synchronous and lost on crash unless you use the outbox pattern.
- How many dependencies are too many? There is no fixed number; when the class name no longer describes one job, split it.

Mistakes to avoid: Enabling `spring.main.allow-circular-references` as a permanent fix; creating a "CommonService" dumping ground; splitting by technical layer instead of by feature.

Production perspective: Large tangled services concentrate risk: every change touches the same file, merge conflicts increase, and transaction boundaries blur. Smaller use-case services make it clear what runs inside a transaction and what does not.

Related concepts covered: Dependency direction, extract-collaborator refactoring, application events, package-by-feature, transaction boundary clarity.

Q091. When would you use @ConfigurationProperties instead of @Value?

Priority: Important

Why interviewers ask it: Typed configuration is a small decision with a large effect on startup safety and maintainability.

Interview-ready answer: @Value injects a single property and is fine for one-off values, but it is stringly typed, scattered across classes, validated only when it is used, and awkward for lists or nested structures. @ConfigurationProperties binds a whole prefix to a typed object — records work well in Boot 3 — supports relaxed binding, Duration and DataSize conversion, nested objects, lists and maps, and can be validated with @Validated so a bad or missing value fails at startup rather than at the first request. I use @ConfigurationProperties for anything with more than one related key, and keep @Value for trivial single values.

In-depth explanation: Binding happens through Binder, which applies relaxed rules: `payments.api-key`, `payments.apiKey`, `PAYMENTS_APIKEY` and `PAYMENTS_API_KEY` all reach the same property. Constructor binding (used automatically for records and classes annotated with @ConfigurationProperties that have a single parameterised constructor) makes the object immutable. Registering the class requires either @EnableConfigurationProperties or @ConfigurationPropertiesScan. Validation uses Bean Validation annotations on the properties type; failures produce a clear startup error naming the property. A related benefit is discoverability: with the `spring-boot-configuration-processor` dependency, your properties appear in IDE auto-completion, and the `actuator configprops` endpoint documents effective values (with sanitisation for secrets).

Practical backend example: Grouped, validated, immutable configuration:

```
@Validated
@ConfigurationProperties(prefix = "orders.retry")
public record RetryProperties(
    @Min(1) @Max(10) int maxAttempts,
    @NotNull Duration initialBackoff,
    @DefaultValue("2.0") double multiplier) {}
```

```
orders:
  retry:
    max-attempts: 3
    initial-backoff: 250ms    # bound to Duration automatically
```

Common follow-ups:

- How do you handle a missing setting? Make it `@NotNull` so startup fails loudly, or give it a `@DefaultValue`.
- Does it support lists and maps? Yes, including nested objects, with indexed YAML syntax.
- How do you see effective values at runtime? The `/actuator/configprops` endpoint, which sanitises keys that look like secrets.

Mistakes to avoid: Sprinkling `@Value` across many classes; using `String` for durations and parsing manually; forgetting `@EnableConfigurationProperties` or `@ConfigurationPropertiesScan`; logging the whole properties object when it contains credentials.

Production perspective: Configuration errors caught at startup are caught by the deployment pipeline; configuration errors caught at first use become incidents. Typed, validated properties are a cheap way to shift that failure left.

Related concepts covered: Relaxed binding, constructor binding with records, startup validation, actuator `configprops`, configuration metadata.

Q092. How do application events work, and what do they not guarantee?

Priority: Important

Why interviewers ask it: In-process events are often mistaken for messaging, which leads to lost work after a crash.

Interview-ready answer: Publishing with `ApplicationEventPublisher` calls matching `@EventListener` methods synchronously on the same thread by default, inside the same transaction if one is active. That makes them useful for decoupling modules within one application, but they guarantee nothing about durability: if the process crashes after the listener starts, the work is gone, and there is no retry, no ordering across publishers and no delivery to other instances. `@TransactionalEventListener` improves correctness by deferring the listener until after commit, which prevents acting on data that later rolls back. If the side effect must survive a crash or reach another service, the event belongs in a broker with a transactional outbox, not in the application context.

In-depth explanation: By default `@TransactionalEventListener` runs in the `AFTER_COMMIT` phase; other phases are `BEFORE_COMMIT`, `AFTER_ROLLBACK` and `AFTER_COMPLETION`. An important subtlety is that a listener running after commit is outside the original transaction, so persisting something there needs `REQUIRES_NEW` or a new transaction — writes without one will silently do nothing in some setups. Adding `@Async` to a listener moves it to another thread, which loses transaction and request context and makes exception handling your responsibility. Exceptions in a synchronous listener propagate to the publisher and will roll back the caller's transaction, which is sometimes desirable and often surprising. Events are also invisible to tracing unless you propagate context, so cross-module debugging suffers if you overuse them.

Practical backend example: Sending a confirmation only after the order really committed:

```

@Service
public class OrderService {
    private final ApplicationEventPublisher events;
    @Transactional
    public Order place(PlaceOrderCommand cmd) {
        Order order = orderRepository.save(Order.from(cmd));
        events.publishEvent(new OrderPlacedEvent(order.id())); // published inside the transaction
        return order;
    }
}

@Component
public class ConfirmationListener {
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void onPlaced(OrderPlacedEvent event) {
        mailer.sendConfirmation(event.orderId()); // never runs if the transaction rolled back
    }
}

```

Common follow-ups:

- Can listeners run before commit? Yes, with `TransactionPhase.BEFORE_COMMIT`, but then the work can still be rolled back.
- Are events delivered to other instances? No; they are in-process only.
- What happens if a listener throws? A synchronous listener propagates the exception to the publisher and can roll back the transaction; an async listener does not.

Mistakes to avoid: Treating events as reliable messaging; writing to the database in an `AFTER_COMMIT` listener without a new transaction; chaining many events until the flow is untraceable; adding `@Async` without an error handler.

Production perspective: Events are excellent for modular monoliths and terrible as an integration guarantee. If a business outcome depends on the side effect — a payment, an email, a downstream update — persist the intent in the same transaction and publish from an outbox relay.

Related concepts covered: Transaction phases, async listeners, outbox pattern, modular monolith design, tracing context.

Q093. What are the risks of `@Async` in a web application?

Priority: Important

Why interviewers ask it: `@Async` looks like free concurrency and quietly breaks context, error handling and capacity assumptions.

Interview-ready answer: `@Async` is proxy-based like `@Transactional`, so self-invocation does not work and the method must be public and called from another bean. The method runs on a `TaskExecutor`; if you do not define one, Boot 3 provides a `SimpleAsyncTaskExecutor`-based default that is not a bounded pool in the traditional sense, so I always configure an explicit bounded executor. Thread-bound context does not follow the call: security context, MDC, request-scoped beans and the current transaction are all absent unless propagated. Exceptions from void async methods disappear unless you register an `AsyncUncaughtExceptionHandler`. And fire-and-forget work is lost on shutdown or crash, so anything that must happen needs persistence first.

In-depth explanation: The correct mental model is that `@Async` hands work to a different thread with a different context, and everything thread-bound must be rebuilt. `DelegatingSecurityContextAsyncTaskExecutor` propagates the security context, and a `TaskDecorator` can copy MDC entries for logging continuity. Transactions do not propagate: a `@Transactional` method calling an `@Async` method will commit independently of it, and the async method needs its own boundary. Capacity is the other half — an unbounded queue behind `@Async` turns a traffic spike into heap growth, so a bounded `ThreadPoolTaskExecutor` with a sensible rejection policy is essential. Spring Boot 3.2+ can back `@Async` with virtual threads, which removes the pool-sizing question for I/O-bound work but not the context and durability issues.

Practical backend example: Bounded executor with context propagation:

```

@Bean("notificationExecutor")
public ThreadPoolTaskExecutor notificationExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(4);
    executor.setMaxPoolSize(8);
    executor.setQueueCapacity(200); // bounded
    executor.setThreadNamePrefix("notify-");
    executor.setRejectedExecutionHandler(new ThreadPoolExecutor.CallerRunsPolicy());
    executor.setTaskDecorator(new MdcTaskDecorator()); // keeps correlation IDs in logs
    executor.setWaitForTasksToCompleteOnShutdown(true);
    executor.setAwaitTerminationSeconds(20); // drain on shutdown
    executor.initialize();
    return executor;
}

@Async("notificationExecutor")
public void sendReceipt(UUID orderId) { /* own transaction if it touches the database */ }

```

Common follow-ups:

- Are transactions propagated to `@Async` methods? No; the new thread has no transaction, so annotate the async method if it needs one.
- Why is my `@Async` method running synchronously? Self-invocation, a missing `@EnableAsync`, or a non-public method.
- Where do exceptions go? For void methods, to the `AsyncUncaughtExceptionHandler`; for future-returning methods, into the future.

Mistakes to avoid: Using the default executor in production; assuming the security context follows; fire-and-forget for work the user was told succeeded; unbounded queues.

Production perspective: Async work must still be observable: export queue depth, active threads and rejection counts, and propagate trace context so an async failure can be linked to the originating request.

Related concepts covered: Proxy semantics, `TaskExecutor` configuration, context propagation, graceful shutdown, durability of background work.

Q094. How should scheduled tasks be designed for multiple replicas?

Priority: Important

Why interviewers ask it: Almost every service is deployed with more than one instance, and naive `@Scheduled` jobs then run several times.

Interview-ready answer: `@Scheduled` runs in every instance, so with three replicas a nightly job runs three times. If the job is not idempotent that means duplicate emails, duplicate charges or conflicting updates. The options are to make the work idempotent and let duplicates be harmless, to use a distributed lock such as `ShedLock` backed by the database or Redis, to elect a leader, or to move the schedule outside the application into a Kubernetes `CronJob` or a scheduler service. I also remember that Spring's default `TaskScheduler` is single-threaded, so one long job delays the others, and that `fixedDelay` measures from the end of the previous run while `fixedRate` measures from the start and can overlap.

In-depth explanation: Idempotency is the strongest design: claim work with an atomic `UPDATE ... WHERE status = 'PENDING'` so exactly one instance takes each row, rather than relying on who runs the schedule. Distributed locks must handle the crash case with a lock TTL, otherwise an instance dying mid-job blocks the schedule; `ShedLock`'s `lockAtMostFor` exists for this. Time zones matter for cron expressions — `@Scheduled(cron = "0 0 3 * * *", zone = "UTC")` avoids daylight-saving surprises. Long-running jobs also need to interact correctly with shutdown: mark them as interruptible and let the graceful shutdown drain them, or they will be killed mid-write. Finally, jobs need observability: a last-success timestamp metric and an alert on staleness catch a silently dead schedule far faster than log inspection.

Practical backend example: Database-claimed work plus a distributed lock:

```
@Scheduled(cron = "0 */5 * * * *", zone = "UTC")
@SchedulerLock(name = "settlementJob", lockAtMostFor = "10m", lockAtLeastFor = "1m") // ShedLock
public void settleBatch() {
    settlementService.settleClaimedBatch();
}
```

```
-- atomic claim: only one instance can take each row, even without a lock library
UPDATE settlement
SET status = 'PROCESSING', claimed_by = :instanceId, claimed_at = now()
WHERE id IN (SELECT id FROM settlement WHERE status = 'PENDING'
            ORDER BY created_at LIMIT 100 FOR UPDATE SKIP LOCKED)
RETURNING id;
```

Common follow-ups:

- Can two pods run the same job? Yes, by default; use a distributed lock, leader election, or an external scheduler.
- What is the difference between `fixedRate` and `fixedDelay`? `fixedRate` starts runs on a fixed interval and can overlap; `fixedDelay` waits a fixed gap after the previous run finishes.
- Why did my scheduled task stop? An uncaught exception in a `scheduleAtFixedRate`-style task cancels future runs; catch inside the task.
- Is the default scheduler multi-threaded? No; configure a `ThreadPoolTaskScheduler` when jobs can overlap.

Mistakes to avoid: Assuming a single instance; using `SELECT ... FOR UPDATE` without `SKIP LOCKED` and serialising all workers; cron expressions in local time; jobs with no metrics.

Production perspective: `FOR UPDATE SKIP LOCKED` is a robust, dependency-free way to distribute work across replicas in PostgreSQL. Whatever mechanism you choose, expose a "last successful run" gauge and alert on it — silent schedule failure is common and expensive.

Related concepts covered: Distributed locking, idempotent jobs, `SKIP LOCKED` work queues, cron time zones, scheduler thread pools, job observability.

Q095. How does Spring Security's filter chain fit into a request?

Priority: Must Know

Why interviewers ask it: Understanding where authentication happens explains error handling, testing and most configuration mistakes.

Interview-ready answer: Spring Security installs a single servlet filter that delegates to an ordered chain of security filters. Each request passes through filters that extract credentials — a session cookie, a bearer token, HTTP Basic — authenticate them, and populate the `SecurityContextHolder`. Later filters perform authorisation based on the configured rules, and only then does the request reach the `DispatcherServlet` and your controller. Because all of this happens before MVC, authentication failures are handled by the `AuthenticationEntryPoint` and authorisation failures by the `AccessDeniedHandler`, not by `@ControllerAdvice`. In Spring Security 6 configuration is a `SecurityFilterChain` bean using the lambda DSL; `WebSecurityConfigurerAdapter` no longer exists.

In-depth explanation: Notable filters in order include `SecurityContextHolderFilter` (restores an existing context), `CsrfFilter`, authentication filters such as `UsernamePasswordAuthenticationFilter` or `BearerTokenAuthenticationFilter`, `ExceptionTranslationFilter` (converts security exceptions into responses or redirects), and `AuthorizationFilter` (applies `authorizeHttpRequests` rules) last. Method security with `@PreAuthorize` is a separate AOP mechanism applied at the service layer and requires `@EnableMethodSecurity`. The `SecurityContext` is stored in a `ThreadLocal`, so it does not propagate to async threads without a delegating executor. For stateless APIs you disable session creation and CSRF protection is generally unnecessary for token-authenticated endpoints, but that decision must be deliberate rather than copied from a tutorial.

Practical backend example: A stateless resource server configuration:

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity // enables @PreAuthorize
public class SecurityConfig {
    @Bean
    SecurityFilterChain api(HttpSecurity http) throws Exception {
        return http
            .securityMatcher("/api/**")
            .csrf(csrf -> csrf.disable()) // stateless bearer-token API
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.GET, "/api/public/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .oauth2ResourceServer(oauth -> oauth.jwt(Customizer.withDefaults()))
            .build();
    }
}

```

Common follow-ups:

- Where is the SecurityContext populated? In an authentication filter, before the dispatcher servlet runs.
- Why doesn't my @ControllerAdvice catch a 401? Because the failure happened in the filter chain, before MVC; configure an AuthenticationEntryPoint.
- How do you secure service methods as well? @EnableMethodSecurity plus @PreAuthorize, which applies AOP at the service layer.

Mistakes to avoid: Disabling CSRF without understanding whether cookies authenticate the request; relying only on URL rules while ignoring object-level ownership checks; assuming the security context is available in async threads.

Production perspective: Order and matchers are easy to get subtly wrong, so integration tests that assert 401, 403 and 200 for representative endpoints are worth more than reading the configuration. Log authentication failures with enough context to detect credential-stuffing patterns, but never log tokens.

Related concepts covered: Filter chain order, SecurityContextHolder, method security, stateless sessions, entry points and access-denied handling.

Q096. How do you test a Spring MVC controller slice?

Priority: Important

Why interviewers ask it: Test slicing shows whether you can get fast feedback without booting the whole application for every test.

Interview-ready answer: @WebMvcTest starts only the web layer — controllers, JSON converters, validation, exception handlers and, if configured, the security filter chain — and leaves out repositories and services, which I supply with @MockitoBean (Boot 3.4+; @MockBean in earlier versions). I drive requests with MockMvc, which dispatches through the real DispatcherServlet without a network socket, and assert status, headers and JSON. That exercises routing, binding, validation and serialisation, which unit-testing the controller class directly would skip entirely. For full-stack coverage I use @SpringBootTest with a random port and a Testcontainers database, but sparingly because it is much slower.

In-depth explanation: Slices work by limiting component scanning to relevant annotations, so a @Service will not be loaded and must be mocked. Security is included in @WebMvcTest when Spring Security is on the classpath, which means unauthenticated requests return 401 unless you annotate with @WithMockUser or configure test security — surprising if you expected 200. MockMvc does not run the servlet container, so container-specific behaviour such as compression or real HTTP semantics is not covered; @SpringBootTest(webEnvironment = RANDOM_PORT) with TestRestClient/TestRestTemplate does. JSON assertions with JSONPath keep tests focused on the contract rather than on object equality. Remember to test the failure paths — validation errors, not-found handling and authorisation — because those are the parts clients hit when something goes wrong.

Practical backend example: A focused controller test including a validation failure:

```

@WebMvcTest(OrderController.class)
class OrderControllerTest {
    @Autowired MockMvc mvc;
    @MockitoBean OrderService orderService; // collaborator is mocked

    @Test
    @WithMockUser(roles = "USER")
    void returns201AndLocationOnCreate() throws Exception {
        given(orderService.place(any())).willReturn(Order.sample());
        mvc.perform(post("/api/orders").contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"customerId":"3f1e...", "lines":[{"sku":"ABC-000001", "quantity":2}]}"""))
            .andExpect(status().isCreated())
            .andExpect(header().exists("Location"))
            .andExpect(jsonPath("$.status").value("NEW"));
    }

    @Test
    @WithMockUser
    void returns400WhenQuantityIsNotPositive() throws Exception {
        mvc.perform(post("/api/orders").contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"customerId":"3f1e...", "lines":[{"sku":"ABC-000001", "quantity":0}]}"""))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.errors").exists());
    }
}

```

Common follow-ups:

- Which beans are loaded by `@WebMvcTest`? Web-layer components only — controllers, advice, converters, filters and security; services and repositories are excluded.
- Why do my tests get 401? Spring Security is active in the slice; add `@WithMockUser` or configure test security explicitly.
- When do you use `@SpringBootTest` instead? For end-to-end behaviour across layers, real HTTP, or database integration with Testcontainers.

Mistakes to avoid: Calling controller methods directly and claiming to have tested the endpoint; mocking `MockMvc`; loading the full context for every test and slowing the build; asserting only the happy path.

Production perspective: Fast slice tests keep pull-request feedback short, which materially improves how often people run the suite. Keep a small number of full end-to-end tests for the critical flows and rely on slices for breadth.

Related concepts covered: Test slices, `MockMvc`, mocking collaborators, security in tests, `JsonPath` assertions, test pyramid.

Q097. What happens on application startup and how do you debug failure?

Priority: Important

Why interviewers ask it: Startup failures block deployments, and a structured diagnosis beats trial and error.

Interview-ready answer: `SpringApplication.run` creates the environment (loading properties and profiles), creates the application context, applies auto-configuration, instantiates and wires singletons, performs `@ConfigurationProperties` binding and validation, starts the web server, and finally publishes `ApplicationReadyEvent` and runs any runners. Failures cluster into a few families: a missing or ambiguous bean, a property binding or validation error, a port already in use, a failing external connection such as the database, and a classpath or version conflict. Boot's failure analysers usually print a clear description and action; if not, I run with `--debug` for the condition report, check `mvn dependency:tree` for conflicts, and read the first exception in the chain rather than the last.

In-depth explanation: Reading the stack trace from the bottom up is usually the fastest route, because the root cause — for example `NoSuchMethodError` from a version clash — sits deepest. Bean definition overriding is disabled by default in Boot, so two beans with the same name fail rather than silently replacing each other. Slow startup is a related concern: eager database connections, large component scans and heavy `@PostConstruct` work all add seconds; `spring.jmx.enabled=false`, lazy initialisation for development, and deferring warm-up to `ApplicationReadyEvent` help. In Kubernetes, startup problems interact with probes: use a startup probe or a generous `initialDelaySeconds`.

so a slow boot is not mistaken for a crash loop.

Practical backend example: A systematic triage sequence:

```
java -jar app.jar --debug 2>&1 | head -80      # condition evaluation report
mvn dependency:tree -Dincludes=com.fasterxml.jackson.core # version conflicts
curl localhost:8080/actuator/health | jq      # which component is down
curl localhost:8080/actuator/beans | jq 'keys' # what actually got registered
```

Typical messages and their meaning

Parameter 0 of constructor ... required a bean of type X	-> not scanned, or condition excluded it
Failed to bind properties under 'payments'	-> type mismatch or missing value
Web server failed to start. Port 8080 was already in use	-> port conflict
NoSuchMethodError / NoClassDefFoundError	-> dependency version clash

Common follow-ups:

- Where do you find condition reports? --debug output at startup or /actuator/conditions at runtime.
- What causes a bean definition conflict? Two definitions with the same name; overriding is disabled by default in Boot.
- How do you speed up startup? Trim component scanning, avoid eager remote calls, and move warm-up to ApplicationReadyEvent.

Mistakes to avoid: Reading only the last line of the stack trace; enabling bean overriding to silence a conflict; adding dependencies to fix a NoClassDefFoundError without resolving the version clash; heavy work in @PostConstruct.

Production perspective: Startup time affects rollout speed and autoscaling responsiveness. Track it as a metric, and ensure health and readiness endpoints distinguish "still starting" from "broken", so orchestrators wait instead of restarting.

Related concepts covered: Context refresh phases, failure analysers, condition reports, dependency conflicts, startup probes, actuator endpoints.

Q098. How do you troubleshoot a bean not found or ambiguous bean?

Priority: Important

Why interviewers ask it: It is an everyday Spring problem with a small set of causes, and a methodical answer shows practical experience.

Interview-ready answer: "No qualifying bean of type X" has three usual causes: the class is not in a scanned package, it lacks a stereotype annotation or @Bean method, or a conditional annotation excluded it — for example a missing property or an absent class. "Expected single matching bean but found 2" means several candidates exist, which I resolve with @Primary for a sensible default or @Qualifier at the injection point. Injecting List<T> or Map<String, T> is often better when all implementations are genuinely needed, such as a set of validators. I confirm the actual state with /actuator/beans and the condition report rather than guessing.

In-depth explanation: Conditional exclusion is the most confusing case, because nothing is wrong with your code: a bean guarded by @ConditionalOnProperty silently disappears when the property is absent in one environment. The condition report shows that as a negative match with the reason. Qualifiers can be by name, by a custom qualifier annotation, or by the parameter name, since Spring falls back to matching the bean name against the parameter name — which is why renaming a parameter can break wiring in unexpected ways. ObjectProvider<T> handles optional dependencies without failing startup, and @Autowired(required = false) is the older equivalent. For collections, ordering with @Order or Ordered is defined and useful when the sequence matters, such as a chain of validators.

Practical backend example: Multiple implementations selected cleanly:

```

public interface PaymentGateway { boolean supports(PaymentMethod method); Receipt capture(Order o); }

@Component class CardGateway implements PaymentGateway { /* ... */ }
@Component class WalletGateway implements PaymentGateway { /* ... */ }

@Service
public class PaymentRouter {
    private final List<PaymentGateway> gateways;           // inject all implementations
    public PaymentRouter(List<PaymentGateway> gateways) { this.gateways = gateways; }

    public Receipt pay(Order order, PaymentMethod method) {
        return gateways.stream().filter(g -> g.supports(method)).findFirst()
            .orElseThrow(() -> new UnsupportedOperationException(method))
            .capture(order);
    }
}

```

Common follow-ups:

- When should you use `@Primary`? When one implementation is the sensible default and the others are rare overrides.
- How do you make a dependency optional? `ObjectProvider<T>`, `Optional<T>` injection, or `@Autowired(required = false)`.
- Why did the bean disappear in one environment? A `@ConditionalOnProperty` or `@Profile` guard; check the condition report.

Mistakes to avoid: Adding `@Primary` to several beans; using field injection so the failure appears later; marking everything `@Component` including value objects; ignoring the actionable text Boot prints under "Action".

Production perspective: Environment-specific wiring bugs are best caught by a smoke test that starts the context with the production profile in CI. A context-loads test is cheap and catches an entire class of deployment failures.

Related concepts covered: Qualifiers and primary beans, conditional beans, collection injection, `ObjectProvider`, actuator beans endpoint.

Q099. How do you configure database connections and pool limits?

Priority: Must Know

Why interviewers ask it: The connection pool is usually the real capacity limit of a Spring service, and its defaults are frequently wrong.

Interview-ready answer: Spring Boot uses HikariCP by default. The settings that matter are `maximum-pool-size` (total connections this instance may hold), `connection-timeout` (how long a thread waits for a connection before failing, default 30 seconds), `max-lifetime` (recycle connections before the database or a proxy drops them), `idle-timeout` and `leak-detection-threshold`. Sizing is a whole-system decision: replicas times pool size must stay below the database's `max_connections`, leaving headroom for migrations and admin sessions. Bigger is not better — a pool larger than the database can serve just moves queueing into the database. When the pool is exhausted, requests block until the timeout and then fail with a Hikari timeout exception, which in traces looks like a slow endpoint with idle CPU.

In-depth explanation: Connection hold time is what determines the required pool size: a transaction that also makes an HTTP call holds its connection for the duration of that call, which is why external calls should never sit inside a transaction. `leak-detection-threshold` logs a stack trace when a connection is held longer than the configured time, which is the fastest way to find the offending code path. For PostgreSQL, connections are relatively expensive, so a connection pooler such as PgBouncer is common at scale; note that transaction-level pooling in PgBouncer is incompatible with session-level features such as prepared statement caching unless configured carefully. Long idle transactions are especially harmful in PostgreSQL because they hold back vacuum and cause table bloat; monitor idle in transaction sessions.

Practical backend example: A deliberate pool configuration with diagnostics:

```
spring:
  datasource:
    hikari:
      maximum-pool-size: 10          # 10 x replicas must stay under max_connections
      minimum-idle: 10              # avoid latency spikes from pool growth
      connection-timeout: 30000      # fail fast rather than queue for 30s
      max-lifetime: 1500000          # 25 min, below the database/proxy idle cut-off
      leak-detection-threshold: 20000
      pool-name: orders-pool
  jpa:
    properties:
      hibernate.jdbc.batch_size: 50
```

```
-- what to check during an incident
SELECT state, count(*) FROM pg_stat_activity GROUP BY state;
SELECT pid, now() - xact_start AS age, query FROM pg_stat_activity
WHERE state = 'idle in transaction' ORDER BY age DESC LIMIT 10;
```

Common follow-ups:

- What happens when all connections are borrowed? New requests wait up to connection-timeout and then fail; latency rises while CPU stays low.
- Should the pool be large? Only as large as the database can serve concurrently; oversizing shifts contention into the database.
- How do you find a connection leak? Enable leak-detection-threshold and read the logged stack traces.

Mistakes to avoid: Leaving the default 30-second connection timeout on a user-facing API; calling external services inside a transaction; ignoring max_connections when scaling replicas; sizing the pool by guesswork rather than from hold time and throughput.

Production perspective: Export Hikari metrics — active, idle, pending threads and timeout count. Pending threads above zero is the earliest signal of saturation and usually precedes user-visible errors by minutes.

Related concepts covered: HikariCP settings, connection hold time, PgBouncer, idle-in-transaction bloat, pool metrics, capacity planning.

Q100. What causes slow Spring endpoints besides slow Java code?

Priority: Must Know

Why interviewers ask it: Most latency in a backend service is not in the Java logic, and knowing where to look first is the skill being tested.

Interview-ready answer: In my experience the usual suspects, in order, are database work (too many queries, especially N+1, missing indexes, or large result sets), waiting on the connection pool, downstream HTTP calls without proper timeouts, serialisation of oversized payloads, GC pauses from memory pressure, and excessive logging. I start with a distributed trace for a slow request, which attributes time to spans, then look at the SQL count and duration for that endpoint, then the pool and GC metrics. Only after that do I profile Java code. Fixing the wrong layer is expensive: adding instances to a database-bound service makes the database slower, not the endpoint faster.

In-depth explanation: A trace usually narrows the problem in seconds: a single 900 ms span on a downstream call is a different problem from 300 spans of 3 ms each, which is N+1. Hibernate statistics or a query-count assertion in tests catch N+1 before production. Serialisation cost grows with payload size and is often invisible — returning an entity graph with lazy collections can trigger extra queries during JSON writing. Response compression, pagination and DTO projections address that. Thread pool saturation shows as queueing delay rather than processing time, which is why it is easy to miss without pool metrics. Finally, logging at debug level in production, or logging whole request bodies, can consume more CPU than the business logic.

Practical backend example: Measuring before optimising:

```
@Bean
ObservationRegistryCustomizer<ObservationRegistry> observationConfig() {
    return registry -> registry.observationConfig()
        .observationHandler(new DefaultTracingObservationHandler(tracer)); // spans per request
}
```

```
# development-time visibility into SQL volume (never enable verbose SQL logging in production)
spring.jpa.properties.hibernate.generate_statistics=true
logging.level.org.hibernate.stat=DEBUG
management.endpoints.web.exposure.include=health,metrics,prometheus
```

Common follow-ups:

- How do traces narrow it down? Each span shows where wall-clock time went — database, downstream call, or application code.
- What is the fastest check for N+1? Query count per request from Hibernate statistics, or a test that asserts the number of statements.
- Why is CPU low while latency is high? The service is waiting — on the database, a downstream service, or a pool.

Mistakes to avoid: Profiling Java code before checking I/O; enabling show-sql in production; scaling out a database-bound service; drawing conclusions from average latency instead of percentiles.

Production perspective: Alert on p95/p99 latency per endpoint rather than averages, and record SQL query counts per request in a development or staging profile. Most serious regressions announce themselves as a change in query count or payload size.

Related concepts covered: Distributed tracing, N+1 detection, pool saturation, payload size, GC pauses, latency percentiles.

Q101. How do you handle validation of business rules versus DTO shape?

Priority: Important

Why interviewers ask it: Confusing input validation with domain invariants leads to rules that can be bypassed.

Interview-ready answer: DTO validation checks the shape of the request — required fields, formats, ranges, sizes — and belongs at the controller boundary with Bean Validation, returning 400. Business rules depend on state the request cannot see: does this customer have credit, is the SKU in stock, is the order in a cancellable status. Those belong in the domain or service layer, execute inside the transaction that will act on them, and typically return 409 or 422. The distinction matters because the same service can be called from a message consumer or a scheduled job where no controller validation ran — so the invariant must be enforced where the state change happens, not only at the edge.

In-depth explanation: The strongest form is to make invalid states unrepresentable: value objects that validate in their constructor, an `Order` that only exposes legal transitions, and database constraints as the last line of defence. Note that even a service-layer check is advisory under concurrency — two requests can both see stock available. Genuine enforcement requires a database constraint, a conditional update (`UPDATE ... WHERE quantity >= :n`) or a lock. Keep error signalling consistent: domain exceptions that the advice layer maps to status codes, rather than returning booleans that callers forget to check. Also avoid duplicating rules in the DTO and the domain; the DTO checks format, the domain checks meaning.

Practical backend example: Three layers of defence for the same rule:

```
public record ReserveStockRequest(@NotBlank String sku, @Positive int quantity) {} // shape

@Transactional
public void reserve(String sku, int quantity) {
    int updated = inventoryRepository.reserveIfAvailable(sku, quantity); // meaning
    if (updated == 0) throw new InsufficientStockException(sku, quantity); // -> 409
}
```

```
-- conditional update: the database enforces the invariant atomically
UPDATE inventory SET reserved = reserved + :qty
WHERE sku = :sku AND available - reserved >= :qty;
-- plus a constraint as the final guarantee
ALTER TABLE inventory ADD CONSTRAINT reserved_not_negative CHECK (reserved >= 0);
```

Common follow-ups:

- Can `@Valid` replace service validation? No; it cannot see database state, and non-HTTP entry points skip it entirely.

- Which status code for a broken business rule? 409 for a state conflict, 422 for semantically invalid content; be consistent across the API.
- Where do cross-field checks go? Simple ones can be custom class-level constraints; anything needing stored state belongs in the service.

Mistakes to avoid: Trusting controller validation as the only guard; duplicating the same rule in three layers with different messages; returning 500 for rule violations; checking availability without an atomic update.

Production perspective: Rules enforced only in application code fail under concurrency and during data migrations. A database constraint is the cheapest guarantee you will ever write, and it protects against bugs in code paths you have not thought about.

Related concepts covered: Bean Validation, domain invariants, conditional updates, database constraints, HTTP status semantics.

Q102. What is the difference between @Controller and @RestController?

Priority: Important

Why interviewers ask it: It is a quick check of precision about response handling and content negotiation.

Interview-ready answer: @Controller is the general MVC stereotype: a handler method's return value is interpreted as a view name unless the method or class is annotated with @ResponseBody. @RestController is simply @Controller plus @ResponseBody at the class level, so every return value is serialised into the response body — JSON by default with Jackson on the classpath. ResponseEntity adds explicit control over the status code, headers and body, which is what I use whenever a method returns anything other than a plain 200, such as 201 with a Location header or 204 with no content.

In-depth explanation: The mechanism is the return-value handler: with @ResponseBody, the HttpMessageConverter chain selects a converter based on the return type and the client's Accept header, producing 406 if nothing matches. Without it, a ViewResolver maps the returned string to a template. Mixing both is legitimate in applications that serve server-rendered pages and JSON APIs; use separate controllers to keep it clear. ResponseEntity<Void> covers 204 responses, and ResponseEntity.created(uri) is the standard way to answer a successful POST. Spring also honours @ResponseStatus on exceptions and on handler methods, which is a lighter alternative when the status is always the same.

Practical backend example: Explicit status and headers where they matter:

```
@RestController
@RequestMapping("/api/invoices")
public class InvoiceController {

    @PostMapping // 201 + Location for creation
    public ResponseEntity<InvoiceResponse> create(@Valid @RequestBody CreateInvoice body) {
        Invoice invoice = invoiceService.create(body.toCommand());
        return ResponseEntity.created(URI.create("/api/invoices/" + invoice.id()))
            .body(InvoiceResponse.from(invoice));
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT) // 204, no body
    public void delete(@PathVariable UUID id) { invoiceService.delete(id); }

    @GetMapping(value = "/{id}/pdf", produces = MediaType.APPLICATION_PDF_VALUE)
    public ResponseEntity<Resource> pdf(@PathVariable UUID id) {
        return ResponseEntity.ok()
            .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=\"invoice.pdf\"")
            .body(invoiceService.renderPdf(id));
    }
}
```

Common follow-ups:

- What does ResponseEntity add? Full control of status, headers and body, including conditional responses and cache headers.
- Can one application mix both stereotypes? Yes; keep view controllers and API controllers separate for clarity.
- What produces a 406? Content negotiation failed — no converter can satisfy the Accept header.

Mistakes to avoid: Returning 200 for every outcome; forgetting the Location header on creation; annotating a @Controller method with @ResponseBody inconsistently across a class; hard-coding produces values that conflict with the actual payload.

Production perspective: Correct status codes are part of the API contract and drive client retry behaviour, caching and monitoring dashboards. A service that returns 200 with an error body defeats every generic alerting rule built on HTTP status.

Related concepts covered: HttpMessageConverter, content negotiation, ResponseEntity, @ResponseStatus, REST status semantics.

Q103. How do you implement graceful shutdown for a Boot service?

Priority: Important

Why interviewers ask it: Rolling deployments happen constantly, and ungraceful shutdown silently drops in-flight work.

Interview-ready answer: I enable `server.shutdown=graceful` and set `spring.lifecycle.timeout-per-shutdown-phase`, so on SIGTERM the web server stops accepting new connections and waits for in-flight requests to finish before the context closes. In Kubernetes I pair that with a readiness probe that fails immediately on shutdown, so the pod is removed from the load balancer before the server stops accepting, and with a `terminationGracePeriodSeconds` longer than the shutdown timeout. Background components need the same care: executors must be configured to wait for tasks, message consumers must stop polling and finish or re-deliver the current message, and scheduled jobs should be interruptible. Work that must not be lost has to be persisted, not held in memory.

In-depth explanation: The ordering problem is subtle: SIGTERM and endpoint removal happen concurrently, so a pod can receive new requests for a short period after it starts shutting down. Spring Boot's availability state support (`AvailabilityChangeEvent`, `/actuator/health/readiness`) plus a small `preStop` sleep gives the load balancer time to notice. Kafka consumers should be allowed to finish processing the current records and commit offsets; an abrupt stop means those records are re-delivered, which is only safe if the consumer is idempotent. Executors configured with `setWaitForTasksToCompleteOnShutdown(true)` and an `awaitTerminationSeconds` participate in the same lifecycle. If the process exceeds the grace period the orchestrator sends SIGKILL, so the sum of your timeouts must be smaller than the grace period.

Practical backend example: Coordinated shutdown configuration:

```
server:
  shutdown: graceful
spring:
  lifecycle:
    timeout-per-shutdown-phase: 25s
management:
  endpoint:
    health:
      probes:
        enabled: true           # /actuator/health/liveness and /readiness
```

```
# Kubernetes side
terminationGracePeriodSeconds: 40      # > 25s shutdown phase + preStop
lifecycle:
  preStop:
    exec: { command: ["sh", "-c", "sleep 5"] } # let endpoints propagate first
readinessProbe:
  httpGet: { path: /actuator/health/readiness, port: 8080 }
```

Common follow-ups:

- What about async consumers? Stop polling first, finish or abandon the in-flight message deliberately, and rely on idempotent processing for re-delivery.
- Why a `preStop` sleep? Endpoint removal is eventually consistent; the delay avoids receiving requests after the server stops accepting.

- What happens if shutdown exceeds the grace period? SIGKILL terminates the process immediately, losing in-flight work.

Mistakes to avoid: Relying on in-memory queues to drain; a grace period shorter than the shutdown timeout; ignoring background executors; assuming requests stop arriving the moment SIGTERM is received.

Production perspective: Graceful shutdown is what makes deployments invisible to users. Measure it: a spike in 502s or connection resets during rollouts is the signal that the shutdown sequence is misconfigured.

Related concepts covered: SIGTERM handling, readiness probes, executor draining, consumer rebalancing, idempotency, deployment safety.

Q104. How would you structure a small Spring feature end to end?

Priority: Must Know

Why interviewers ask it: It shows whether you can assemble the pieces into something maintainable rather than reciting annotations.

Interview-ready answer: I keep four layers with clear responsibilities. The controller handles HTTP: binding, validation of shape, status codes, and mapping to a command object — no business logic. The service owns the use case and the transaction boundary, orchestrating domain objects and repositories. The domain holds invariants and state transitions. The repository handles persistence. DTOs are separate from entities in both directions so the API contract and the schema can evolve independently. Cross-cutting concerns — error mapping, correlation IDs, security — live in advice and filters. For a feature like "cancel an order", that gives one endpoint, one service method with `@Transactional`, a domain method enforcing the state machine, and a repository call, plus tests at each level.

In-depth explanation: The transaction boundary belongs in the service, not the controller, because the controller is about transport and the repository is too granular. External calls should sit outside the transaction or be handled with an outbox, so a slow gateway never holds a database connection. Package by feature (orders/, payments/) rather than by layer keeps related code together and makes boundaries visible. Mapping can be manual for a handful of fields — explicit, debuggable and refactor-safe — or generated with MapStruct when the volume justifies it; reflection-based deep mappers tend to hide bugs. Idempotency deserves thought at design time for any state-changing endpoint that a client may retry.

Practical backend example: The full slice for one use case:

```
// web/OrderController.java
@PostMapping("/api/orders/{id}/cancellation")
public ResponseEntity<OrderResponse> cancel(@PathVariable UUID id,
                                             @RequestHeader("Idempotency-Key") String key,
                                             @Valid @RequestBody CancelOrderRequest body) {
    Order order = orderService.cancel(new CancelOrderCommand(id, body.reason(), key));
    return ResponseEntity.ok(OrderResponse.from(order));
}

// application/OrderService.java
@Service
public class OrderService {
    @Transactional // one boundary per use case
    public Order cancel(CancelOrderCommand command) {
        Order order = orders.findById(command.orderId())
            .orElseThrow(() -> new OrderNotFoundException(command.orderId()));
        order.cancel(command.reason()); // domain enforces the state machine
        events.publishEvent(new OrderCancelledEvent(order.id())); // after-commit listener
        return order; // dirty checking flushes the update
    }
}

// domain/Order.java
public void cancel(String reason) {
    if (status != OrderStatus.PAID && status != OrderStatus.NEW) {
        throw new IllegalOrderStateException(id, status); // invariant lives here
    }
    this.status = OrderStatus.CANCELLED;
    this.cancellationReason = reason;
}
```

Common follow-ups:

- Where is the transaction boundary? In the service method that represents the use case.
- Why not return entities from controllers? They leak schema details, can trigger lazy loading during serialisation, and couple the API to the database.
- Where does idempotency live? Usually a stored key with a unique constraint checked at the start of the use case.

Mistakes to avoid: Business logic in controllers; `@Transactional` on repository calls only; anaemic domain objects with all logic in a service and no invariants anywhere; entities as API payloads.

Production perspective: Clear layering pays off during incidents: you know where to add a log line, where the transaction starts, and which layer to test. It also makes it obvious when an external call has crept inside a transaction.

Related concepts covered: Layered architecture, transaction boundaries, DTO mapping, domain invariants, idempotency, package-by-feature.

Q105. What should you check before upgrading a Boot 2 app to Boot 3?

Priority: Important

Why interviewers ask it: Boot 3 is a real migration, and knowing its breaking changes shows current, practical stack knowledge.

Interview-ready answer: The three big items are Java 17 as the minimum, the move from `javax.*` to `jakarta.*` for servlet, persistence, validation and annotations, and third-party libraries that must be upgraded to Jakarta-compatible versions. Beyond that: Spring Security 6 removes `WebSecurityConfigurerAdapter` in favour of `SecurityFilterChain` beans and changes several defaults; Hibernate 6 changes some ID generation and type mapping behaviour; the trailing-slash URL matching default changed; actuator endpoint details and property names moved in places. My process is to upgrade to the latest Boot 2.7 first, remove deprecation warnings, run the Spring Boot migrator and OpenRewrite recipes, then move to Boot 3 with a thorough integration test suite and a staged rollout.

In-depth explanation: The `jakarta` namespace change is mechanical but wide: every `javax.persistence`, `javax.validation` and `javax.servlet` import changes, and any library that still ships `javax` classes must be replaced. Hibernate 6 is the subtlest part of the migration — sequence-based ID allocation defaults, `@Enumerated` handling and some SQL generation differ, so persistence tests with a real PostgreSQL through Testcontainers are worth far more than unit tests here. Security 6 requires rewriting configuration classes and `antMatchers` becomes `requestMatchers`. The trailing-slash matching change means `/orders/` no longer maps to `/orders` by default and can break existing clients. Observability also changed: Spring Boot 3 uses Micrometer Observation and Micrometer Tracing instead of Spring Cloud Sleuth.

Practical backend example: Typical mechanical changes:

<pre>// Boot 2 import javax.persistence.Entity; import javax.validation.Valid; import javax.servlet.Filter; // Security: adapter class // extends WebSecurityConfigurerAdapter // http.authorizeRequests().antMatchers(...) // .))</pre>	<pre>// Boot 3 import jakarta.persistence.Entity; import jakarta.validation.Valid; import jakarta.servlet.Filter; // Security 6: a bean @Bean SecurityFilterChain chain(HttpSecurity http) http.authorizeHttpRequests(a -> a.requestMatchers(...))</pre>
---	--

```
mvn -B org.openrewrite.maven:rewrite-maven-plugin:run \
  -Drewrite.activeRecipes=org.openrewrite.java.spring.boot3.UpgradeSpringBoot_3_0
```

Common follow-ups:

- Why do `javax` imports break? Jakarta EE renamed the namespace; Spring 6 targets `jakarta.*` exclusively.
- What usually breaks silently? Hibernate 6 mapping and ID generation differences, and trailing-slash URL matching.
- How do you de-risk the upgrade? Upgrade to the latest 2.7 first, rely on integration tests against a real database, and roll out behind a canary.

Mistakes to avoid: Upgrading Boot and business features in the same release; skipping the 2.7 stepping stone; trusting unit tests with mocks to catch persistence behaviour changes; forgetting third-party libraries that still target

javax.

Production perspective: Plan the upgrade as its own deployment with a rollback path, and compare key metrics — latency percentiles, error rates, query counts — before and after. Framework upgrades most often show up as subtle persistence or serialisation differences rather than outright failures.

Related concepts covered: Jakarta namespace migration, Spring Security 6 configuration, Hibernate 6 changes, OpenRewrite automation, canary deployment, integration testing with Testcontainers.

Chapter 5. SQL, transactions, JPA, and Hibernate

SQL examples target PostgreSQL 14+; where a feature is PostgreSQL-specific the text says so. JPA examples assume Spring Boot 3 with Hibernate 6 and the `jakarta.persistence` namespace. Query plans and optimiser choices are database-specific behaviour, not guarantees.

Q106. Write a join to find customers and their latest orders.

Priority: Must Know

Why interviewers ask it: "Latest row per group" is the most common practical SQL task, and there are several correct approaches with different costs.

Interview-ready answer: There are three good approaches. A window function with `ROW_NUMBER()` `OVER (PARTITION BY customer_id ORDER BY created_at DESC)` filtered to rank 1 is portable and clear. PostgreSQL's `DISTINCT ON` is the most concise and usually the fastest with a matching index. A `LATERAL` join is best when you need the top N per customer or extra columns from a correlated lookup. I would use a `LEFT JOIN` variant if customers without orders must still appear. The key detail is tie-breaking: if two orders share a timestamp, the ordering must include a unique column or the result is non-deterministic.

In-depth explanation: The naive version — grouping to get `MAX(created_at)` and joining back — is correct only if `(customer_id, created_at)` is unique; otherwise it returns duplicates for ties, which is exactly the bug interviewers look for. Performance depends on indexing: an index on `(customer_id, created_at DESC)` lets `DISTINCT ON` and `LATERAL` jump straight to the newest row per customer, while a window function over a large table may need a sort. For a top-N-per-group requirement, `LATERAL` with `LIMIT n` is typically the cheapest. Always confirm with `EXPLAIN ANALYZE` rather than assuming one form is faster.

Practical backend example: Three correct formulations of the same requirement:

```
-- 1) Portable: window function
SELECT c.id, c.name, o.id AS order_id, o.created_at, o.total_cents
FROM customer c
LEFT JOIN (
    SELECT o.*, ROW_NUMBER() OVER (PARTITION BY o.customer_id
                                   ORDER BY o.created_at DESC, o.id DESC) AS rn
    FROM "order" o
) o ON o.customer_id = c.id AND o.rn = 1;

-- 2) PostgreSQL: DISTINCT ON, concise and index-friendly
SELECT DISTINCT ON (o.customer_id) o.customer_id, o.id, o.created_at, o.total_cents
FROM "order" o
ORDER BY o.customer_id, o.created_at DESC, o.id DESC;

-- 3) LATERAL: best when you need the top N per customer
SELECT c.id, c.name, latest.id AS order_id, latest.created_at
FROM customer c
LEFT JOIN LATERAL (
    SELECT o.id, o.created_at FROM "order" o
    WHERE o.customer_id = c.id
    ORDER BY o.created_at DESC, o.id DESC
    LIMIT 1
) latest ON true;

CREATE INDEX idx_order_customer_created ON "order" (customer_id, created_at DESC, id DESC);
```

Common follow-ups:

- How do you include customers without orders? Use a `LEFT JOIN` (or `LEFT JOIN LATERAL ... ON true`), so the order columns come back null.
- What if two orders share the same timestamp? Add a unique tie-breaker such as `id` to the ordering, otherwise the result is arbitrary.
- Which is fastest? It depends on data distribution and indexes; measure with `EXPLAIN ANALYZE` on realistic volumes.

Mistakes to avoid: Grouping by customer and selecting non-aggregated order columns; assuming `MAX(created_at)` uniquely identifies a row; forgetting the supporting index; using `LIMIT 1` in a correlated subquery per column, which repeats the lookup.

Production perspective: This query often backs a dashboard, so it runs frequently. Index it deliberately, and if the customer list is long, paginate the outer query rather than computing the latest order for every customer on every request.

Related concepts covered: Window functions, DISTINCT ON, LATERAL joins, tie-breaking, composite indexes, outer joins.

Q107. How do INNER, LEFT and FULL joins differ?

Priority: Must Know

Why interviewers ask it: Join semantics plus null handling is where subtly wrong reports come from.

Interview-ready answer: INNER JOIN returns only rows with a match on both sides. LEFT JOIN returns every row from the left table, with nulls for right-side columns when there is no match — which is how you find "customers with no orders". RIGHT JOIN is the mirror image and is usually rewritten as a LEFT JOIN for readability. FULL OUTER JOIN returns unmatched rows from both sides. The classic trap is putting a condition on the right table in the WHERE clause of a LEFT JOIN: because the unmatched rows have null there, the predicate filters them out and the join silently becomes an inner join. Conditions on the optional side belong in the ON clause.

In-depth explanation: ON filters during the join; WHERE filters the result afterwards. `o.status = 'PAID'` in WHERE eliminates rows where `o.status` is null, so customers without paid orders disappear. Writing `AND o.status = 'PAID'` in the ON clause keeps them with null order columns. The exception is `WHERE o.id IS NULL`, which is the deliberate anti-join idiom for "left rows with no match". Also remember that joins can multiply rows: joining a one-to-many relationship returns one row per child, so aggregates on the parent side double-count unless you aggregate in a subquery or use `COUNT(DISTINCT ...)`. CROSS JOIN produces the Cartesian product and is occasionally intentional, for example against a generated date series.

Practical backend example: The same intent expressed correctly and incorrectly:

```
-- WRONG: silently becomes an INNER JOIN, customers with no paid order vanish
SELECT c.id, count(o.id)
FROM customer c
LEFT JOIN "order" o ON o.customer_id = c.id
WHERE o.status = 'PAID'
GROUP BY c.id;

-- RIGHT: the condition belongs in ON, so unmatched customers survive with count 0
SELECT c.id, count(o.id) AS paid_orders
FROM customer c
LEFT JOIN "order" o ON o.customer_id = c.id AND o.status = 'PAID'
GROUP BY c.id;

-- anti-join: customers who never ordered
SELECT c.id FROM customer c
LEFT JOIN "order" o ON o.customer_id = c.id
WHERE o.id IS NULL;
```

Common follow-ups:

- Can WHERE turn a LEFT JOIN into an INNER JOIN? Yes, whenever it tests a right-side column for a non-null value.
- How do you count matched children without losing parents? `COUNT(child.id)` counts non-null values, so unmatched parents correctly return 0 — unlike `COUNT(*)`.
- When is FULL OUTER JOIN useful? Reconciliation between two sources where either side may have unmatched rows.

Mistakes to avoid: Filtering the optional side in WHERE; using `COUNT(*)` with outer joins; joining one-to-many and summing parent columns; forgetting that `NULL = NULL` is unknown, not true.

Production perspective: Reporting bugs from join semantics are expensive because the numbers look plausible. Validate aggregate queries against a small hand-checked dataset, and write tests that include the "no child rows" case.

Related concepts covered: ON versus WHERE, anti-joins, null semantics, row multiplication, COUNT behaviour.

Q108. When do WHERE and HAVING apply in grouped SQL?

Priority: Must Know

Why interviewers ask it: It tests understanding of logical query processing order, which explains many "column does not exist" errors.

Interview-ready answer: Logically, SQL evaluates FROM and joins, then WHERE on individual rows, then GROUP BY, then HAVING on the resulting groups, then SELECT expressions, then ORDER BY, then LIMIT. So WHERE filters rows before aggregation and cannot reference aggregate functions; HAVING filters groups after aggregation and can. For efficiency I push every row-level predicate into WHERE so fewer rows are aggregated, and reserve HAVING for conditions on aggregates such as HAVING COUNT(*) > 3. It also explains why a SELECT alias is usable in ORDER BY but generally not in WHERE — the alias does not exist yet at that stage.

In-depth explanation: PostgreSQL does allow GROUP BY and ORDER BY to reference output column aliases and positions, which is a convenience, not standard behaviour everywhere. Grouping semantics also matter: every non-aggregated column in SELECT must appear in GROUP BY unless it is functionally dependent on the grouped primary key, which PostgreSQL supports. COUNT(*) counts rows including nulls; COUNT(column) counts non-null values; SUM of no rows returns null rather than zero, so wrap it in COALESCE when a numeric default is expected. FILTER (WHERE ...) is a clean PostgreSQL way to compute conditional aggregates in one pass, avoiding several correlated subqueries.

Practical backend example: A monthly report with row-level and group-level filters:

```
SELECT date_trunc('month', o.created_at) AS month,
       o.customer_id,
       count(*) AS order_count,
       count(*) FILTER (WHERE o.total_cents > 100000) AS large_orders,
       coalesce(sum(o.total_cents), 0) AS revenue_cents
FROM "order" o
WHERE o.status = 'PAID' -- row filter, before grouping
  AND o.created_at >= date_trunc('month', now()) - interval '12 months'
GROUP BY 1, 2
HAVING count(*) > 3 -- group filter, after aggregation
ORDER BY month DESC, revenue_cents DESC;
```

Common follow-ups:

- How do you filter on count > 3? With HAVING, because the count only exists after grouping.
- Why can't you use a SELECT alias in WHERE? WHERE is evaluated before the select list is computed.
- What does SUM return with no matching rows? Null; use COALESCE(sum(x), 0) when zero is the expected default.

Mistakes to avoid: Putting row filters in HAVING and aggregating unnecessary rows; forgetting COALESCE on sums; selecting ungrouped columns; assuming COUNT(column) and COUNT(*) are the same.

Production perspective: Aggregations over large tables belong on an index that supports the filter and grouping, or on a pre-aggregated summary table refreshed on a schedule. Running a twelve-month aggregate on every dashboard load is a common cause of database load spikes.

Related concepts covered: Logical query processing order, conditional aggregates with FILTER, null handling in aggregates, functional dependency, summary tables.

Q109. How do subqueries, EXISTS and joins compare?

Priority: Important

Why interviewers ask it: Choosing between them affects both correctness with nulls and the shape of the query plan.

Interview-ready answer: EXISTS expresses a semi-join: "does at least one matching row exist", without multiplying result rows, and it can stop at the first match. A JOIN is right when you actually need columns from the other table, but it duplicates parent rows when the relationship is one-to-many. IN with a subquery is similar to EXISTS for existence checks, but NOT IN is dangerous: if the subquery returns any null, the whole predicate evaluates to unknown and you get no rows at all. NOT EXISTS does not have that problem, so I default to NOT EXISTS or a LEFT JOIN ... WHERE right.id IS NULL anti-join. Modern optimisers often rewrite these forms into the same plan, so correctness and clarity should drive the choice.

In-depth explanation: The null behaviour of `NOT IN` follows from three-valued logic: `x NOT IN (1, NULL)` is `x <> 1` AND `x <> NULL`, and the second comparison is unknown, so the result can never be true. That single rule has caused countless silent production bugs. Correlated subqueries in the `SELECT` list run per output row and can be much slower than a join or a lateral, though PostgreSQL can sometimes flatten them. Scalar subqueries must return at most one row or the query errors at runtime — a real risk when data changes. For readability, common table expressions are excellent, and since PostgreSQL 12 they are inlined by default rather than acting as optimisation fences, unless declared `MATERIALIZED`.

Practical backend example: Existence, anti-join and the null trap:

```
-- semi-join: customers with at least one paid order (no row multiplication)
SELECT c.id, c.name FROM customer c
WHERE EXISTS (SELECT 1 FROM "order" o WHERE o.customer_id = c.id AND o.status = 'PAID');

-- safe anti-join
SELECT c.id FROM customer c
WHERE NOT EXISTS (SELECT 1 FROM "order" o WHERE o.customer_id = c.id);

-- DANGEROUS: returns zero rows if any order.customer_id is NULL
SELECT c.id FROM customer c
WHERE c.id NOT IN (SELECT o.customer_id FROM "order" o);
```

Common follow-ups:

- When is `NOT IN` dangerous with null? Always, when the subquery can produce null; the predicate becomes unknown and filters everything out.
- Does `EXISTS` need `SELECT 1` or `SELECT *`? It makes no difference to the plan; `SELECT 1` is conventional.
- When do you prefer a join? When you need columns from the other table; add `DISTINCT` or aggregate carefully if the relationship is one-to-many.

Mistakes to avoid: Using `NOT IN` on a nullable column; a correlated subquery per row where a join would do; assuming a CTE always materialises; scalar subqueries that can return multiple rows.

Production perspective: Query rewrites for readability are cheap; query rewrites for performance should be validated with `EXPLAIN ANALYZE` on production-like data. A form that is faster on 1,000 rows can be slower on 10 million.

Related concepts covered: Semi-joins and anti-joins, three-valued logic, correlated subqueries, CTE inlining, plan validation.

Q110. How would you page through a large table safely?

Priority: Must Know

Why interviewers ask it: Offset pagination is the default in most frameworks and degrades badly at scale.

Interview-ready answer: `LIMIT ... OFFSET ...` is simple but the database must still read and discard every skipped row, so page 10,000 is slow even with an index. It is also unstable: if rows are inserted or deleted between requests, items shift and users see duplicates or gaps. Keyset (or seek) pagination fixes both: order by a unique, indexed key and ask for rows after the last one the client saw. The trade-off is that you lose direct random access to page N, which is usually acceptable for infinite scroll and API cursors. Whichever I choose, the sort must include a unique tie-breaker, or the ordering is non-deterministic.

In-depth explanation: With a composite index matching the `ORDER BY`, a keyset query is an index range scan that starts exactly where the previous page ended — constant cost per page regardless of depth. The row-comparison syntax `(created_at, id) < (:lastCreatedAt, :lastId)` expresses the multi-column condition correctly; writing it as separate `AND/OR` clauses is error-prone. Counting total rows is a separate problem: `COUNT(*)` over a large filtered set is expensive, so many APIs return only `hasNext` (fetch `limit + 1` rows) or an approximate count from statistics. In Spring Data, `Slice` avoids the count query that `Page` performs. Cursors should be opaque to clients — base64-encode the keyset values — so the API can evolve without breaking them.

Practical backend example: Offset versus keyset, with the Spring Data equivalent:

```
-- offset: simple, but the database scans and discards 200,000 rows
SELECT id, created_at, total_cents FROM "order"
WHERE customer_id = :customerId
ORDER BY created_at DESC, id DESC
LIMIT 20 OFFSET 200000;
```

```
-- keyset: constant cost, stable under concurrent inserts
SELECT id, created_at, total_cents FROM "order"
WHERE customer_id = :customerId
  AND (created_at, id) < (:lastCreatedAt, :lastId)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

```
public interface OrderRepository extends JpaRepository<Order, UUID> {
    @Query("""
        select o from Order o
        where o.customerId = :customerId
            and (o.createdAt < :lastCreatedAt
                or (o.createdAt = :lastCreatedAt and o.id < :lastId))
        order by o.createdAt desc, o.id desc
        """)
    Slice<Order> nextPage(UUID customerId, Instant lastCreatedAt, UUID lastId, Pageable pageable);
}
```

Common follow-ups:

- What if rows are inserted between pages? Offset pagination shifts results and can duplicate or skip items; keyset pagination is stable.
- How do you show a total count? Either accept the cost of a separate count query, return only hasNext, or use an approximate count for large sets.
- Can you jump to page 500 with keyset pagination? Not directly; that is the trade-off, and it is rarely a real user need.

Mistakes to avoid: Sorting by a non-unique column only; deep offsets in public APIs; running COUNT(*) on every page request; exposing raw keyset values that leak internal ordering semantics.

Production perspective: Deep pagination is often a sign of a batch consumer using a UI endpoint. Provide a dedicated export or cursor-based API for those clients, and cap OFFSET to protect the database.

Related concepts covered: Index range scans, cursor APIs, Slice versus Page, count query cost, stable ordering.

Q111. How do B-tree indexes help and when are they not used?

Priority: Must Know

Why interviewers ask it: "Add an index" is the reflex answer; knowing when it does not help is the differentiator.

Interview-ready answer: A B-tree index is a sorted structure that lets the database find matching rows without scanning the table, and it also satisfies ORDER BY and range queries in index order. It is not used when the planner estimates a large fraction of the table matches — a sequential scan is genuinely cheaper then — when the predicate wraps the column in a function or a type cast, when a leading wildcard makes LIKE '%term' unindexable by a plain B-tree, or when statistics are stale so the estimate is wrong. Indexes also cost write throughput and disk, since every insert, update and delete maintains them. So I add indexes based on real query patterns and verify with EXPLAIN ANALYZE, rather than indexing every column.

In-depth explanation: Selectivity drives the decision: for a column where one value covers 40% of the table, random index lookups plus heap fetches are slower than a sequential scan, and choosing the scan is the optimiser being right. Expression indexes solve the function problem — CREATE INDEX ON customer (lower(email)) makes WHERE lower(email) = ? indexable. Partial indexes (WHERE status = 'PENDING') are ideal for work-queue tables where only a small subset is ever queried. For text search, trigram (pg_trgm) or full-text indexes handle patterns a B-tree cannot. PostgreSQL can perform index-only scans when all needed columns are in the index and the visibility map is current, which is why VACUUM affects read performance. Also, an index on a column with a different type than the parameter (for example bigint column versus numeric parameter) may prevent its use.

Practical backend example: Targeted indexes for real query patterns:

```
-- 1) case-insensitive lookup: index the expression actually used
CREATE INDEX idx_customer_lower_email ON customer (lower(email));
SELECT * FROM customer WHERE lower(email) = lower(:email);

-- 2) work queue: partial index keeps it small and hot
CREATE INDEX idx_outbox_pending ON outbox (created_at) WHERE status = 'PENDING';

-- 3) prefix search can use a B-tree; leading wildcard cannot
SELECT * FROM product WHERE name LIKE 'lap%';      -- indexable
SELECT * FROM product WHERE name LIKE '%top%';     -- needs pg_trgm or full-text search
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_product_name_trgm ON product USING gin (name gin_trgm_ops);
```

Common follow-ups:

- Does an index help LIKE '%x'? Not a plain B-tree; use a trigram (GIN) index or full-text search.
- Why is a sequential scan sometimes correct? When a large proportion of rows match, sequential I/O beats random lookups plus heap fetches.
- What is the cost of an index? Slower writes, more storage, and more work for vacuum and replication.
- Why did the index stop being used after a data load? Statistics are stale; run ANALYZE.

Mistakes to avoid: Indexing every column; wrapping indexed columns in functions in the WHERE clause; assuming an index guarantees a fast query; adding indexes without checking for an existing composite index that already covers the pattern.

Production perspective: Create indexes on large production tables with CREATE INDEX CONCURRENTLY so writes are not blocked. Track unused indexes via pg_stat_user_indexes and drop them; they cost write performance for no benefit.

Related concepts covered: Selectivity, expression and partial indexes, trigram search, index-only scans, statistics and ANALYZE, write amplification.

Q112. How do you read EXPLAIN ANALYZE without overreacting?

Priority: Must Know

Why interviewers ask it: Evidence-led tuning is the difference between fixing a query and cargo-culting hints.

Interview-ready answer: EXPLAIN shows the planned nodes and cost estimates; EXPLAIN ANALYZE actually runs the query and adds real timings and row counts. I read it from the innermost nodes outwards and compare estimated rows to actual rows — a large mismatch means the planner's statistics are wrong, which is the root cause of most bad plans. I look for nodes with high actual time, unexpected nested loops with many loops, sorts spilling to disk, and sequential scans on large tables where a selective predicate exists. A sequential scan on a small table is fine and not worth touching. EXPLAIN (ANALYZE, BUFFERS) adds I/O detail, showing whether the pages came from cache or disk.

In-depth explanation: Costs are in arbitrary planner units and are only meaningful relative to each other. The loops value matters: a node taking 0.2 ms with 5,000 loops accounts for a second of real time. Row estimate errors usually come from stale statistics, correlated columns the planner treats as independent, or expressions it cannot estimate; fixes include ANALYZE, extended statistics (CREATE STATISTICS), or restructuring the predicate. Note that EXPLAIN ANALYZE executes the statement, so wrap data-modifying statements in a transaction you roll back. Timing instrumentation itself adds overhead, which can exaggerate the cost of nodes called very frequently.

Practical backend example: Reading a plan and acting on the mismatch:

```
BEGIN;
EXPLAIN (ANALYZE, BUFFERS, VERBOSE)
SELECT o.id, o.total_cents FROM "order" o
WHERE o.customer_id = '8f1c...' AND o.status = 'PAID'
ORDER BY o.created_at DESC LIMIT 20;
ROLLBACK;
```

```

Limit (cost=0.43..8.50 rows=20 width=24) (actual time=0.031..0.112 rows=20 loops=1)
  -> Index Scan Backward using idx_order_customer_created on "order" o
      (cost=0.43..812.11 rows=2013 width=24) (actual time=0.029..0.104 rows=20 loops=1)
      Index Cond: (customer_id = '8f1c... '::uuid)
      Filter: (status = 'PAID'::text)
      Rows Removed by Filter: 4
      Buffers: shared hit=7
Planning Time: 0.140 ms
Execution Time: 0.139 ms
-- healthy: index scan, few buffers, estimate close to actual, no sort node

```

Common follow-ups:

- Why might a sequential scan be correct? On a small table, or when most rows match, it is cheaper than random access.
- What does a big estimate-versus-actual gap mean? Stale or insufficient statistics, or correlated predicates; run ANALYZE or add extended statistics.
- What do BUFFERS numbers tell you? Whether the data came from shared cache (hit) or disk (read), which explains variable latency.

Mistakes to avoid: Comparing cost units to milliseconds; optimising the node with the largest cost instead of the largest actual time; running EXPLAIN ANALYZE on an UPDATE outside a transaction; tuning on a development dataset with unrepresentative volumes.

Production perspective: Capture plans for slow queries automatically with `auto_explain` and log statements above a duration threshold. Plans change as data grows, so a query that was fast at launch can regress without any code change.

Related concepts covered: Planner statistics, buffers and caching, nested loops, `auto_explain`, plan regressions.

Q113. What are ACID properties in a money transfer?

Priority: Must Know

Why interviewers ask it: It grounds transaction theory in a scenario where mistakes are obviously unacceptable.

Interview-ready answer: Atomicity means the debit and the credit both happen or neither does — there is no state where money has left one account and not arrived at the other. Consistency means the transaction moves the database from one valid state to another, respecting constraints such as "balance must not go negative". Isolation means concurrent transfers do not observe each other's partial work; the level chosen determines which anomalies are possible. Durability means that once the commit returns, the change survives a crash, because it is in the write-ahead log on durable storage. The important caveat is that ACID applies inside the database: an HTTP call to a payment provider is not part of it, so distributed steps need idempotency and reconciliation instead.

In-depth explanation: Consistency in ACID is about constraints and application invariants, and is a different concept from the "C" in CAP, which is about replicas agreeing. Durability depends on configuration: PostgreSQL's `synchronous_commit` can be relaxed for throughput at the cost of losing recent commits after a crash, and replica-level durability depends on synchronous replication. Isolation is the property with real trade-offs, since stronger levels cost concurrency. The practical lesson for backend engineers is scope: keep transactions short, do not include network calls, and never rely on application-level checks alone for invariants that the database can enforce with a constraint.

Practical backend example: A transfer that is atomic and constraint-protected:

```

ALTER TABLE account ADD CONSTRAINT balance_non_negative CHECK (balance_cents >= 0);

BEGIN;
UPDATE account SET balance_cents = balance_cents - 5000 WHERE id = :from; -- fails the CHECK if
insufficient
UPDATE account SET balance_cents = balance_cents + 5000 WHERE id = :to;
INSERT INTO transfer (id, from_id, to_id, amount_cents, created_at)
VALUES (:transferId, :from, :to, 5000, now());
COMMIT;

```

```
@Transactional // one boundary, no external calls inside
public void transfer(UUID from, UUID to, long cents) {
    accountRepository.debit(from, cents); // conditional UPDATE enforcing the invariant
    accountRepository.credit(to, cents);
    transferRepository.save(new Transfer(from, to, cents));
}
```

Common follow-ups:

- Does ACID cover a remote API call? No; the external system has its own transaction. Use idempotency keys and reconciliation, or an outbox.
- Is durability absolute? It depends on configuration and hardware; asynchronous commit and asynchronous replication both trade durability for throughput.
- Where does the negative-balance rule belong? In a database constraint plus a conditional update, not only in Java code.

Mistakes to avoid: Holding a transaction open across a payment gateway call; relying on a read-then-write check for balances; assuming ACID gives distributed consistency; treating CAP consistency and ACID consistency as the same thing.

Production perspective: Money flows need an audit trail and reconciliation regardless of transactional correctness, because failures can occur between systems. Store an immutable ledger of attempted and completed movements rather than only the current balance.

Related concepts covered: Transaction boundaries, constraints as invariants, write-ahead logging, idempotency across systems, ledgers and reconciliation.

Q114. What anomalies do isolation levels prevent?

Priority: Must Know

Why interviewers ask it: It checks precise knowledge rather than a memorised table, especially the difference between the standard and PostgreSQL.

Interview-ready answer: The classic anomalies are dirty reads (seeing uncommitted data), non-repeatable reads (the same row changes between two reads in one transaction) and phantom reads (a repeated range query returns new rows). READ UNCOMMITTED allows all three, READ COMMITTED prevents dirty reads, REPEATABLE READ additionally prevents non-repeatable reads, and SERIALIZABLE prevents all of them plus write skew. PostgreSQL's default is READ COMMITTED, where each statement sees a fresh snapshot. Its REPEATABLE READ is implemented as snapshot isolation and also prevents phantoms, which is stronger than the standard requires, and its SERIALIZABLE adds predicate-based conflict detection that can abort transactions with a serialisation failure.

In-depth explanation: Write skew is the anomaly people forget: two transactions each read a consistent snapshot, check a condition that is still true, and write different rows — for example both doctors dropping off the on-call rota because each sees the other still on it. Snapshot isolation permits that; only SERIALIZABLE prevents it. In PostgreSQL, a serialisation failure appears as SQLSTATE 40001 and must be retried by the application, so choosing SERIALIZABLE is a design decision that includes retry logic. Note that isolation is per transaction, so raising the level on one transaction does not protect the others. A pragmatic approach in most services is READ COMMITTED with targeted protection: unique constraints, conditional updates, SELECT ... FOR UPDATE and optimistic version columns where invariants really matter.

Practical backend example: Demonstrating and preventing write skew:

```
-- both transactions run concurrently at REPEATABLE READ and both succeed: invariant broken
-- T1: SELECT count(*) FROM oncall WHERE active; -- 2
--     UPDATE oncall SET active = false WHERE doctor_id = 1;
-- T2: SELECT count(*) FROM oncall WHERE active; -- 2 (own snapshot)
--     UPDATE oncall SET active = false WHERE doctor_id = 2;

-- prevention 1: SERIALIZABLE (one transaction aborts with SQLSTATE 40001, retry it)
BEGIN ISOLATION LEVEL SERIALIZABLE;
-- prevention 2: explicit lock on the rows the decision depends on
SELECT * FROM oncall WHERE active FOR UPDATE;
```

```
@Transactional(isolation = Isolation.READ_COMMITTED) // Spring default follows the database
public void schedule(...) { /* protect invariants explicitly, not by raising isolation globally */ }
```

Common follow-ups:

- What does PostgreSQL READ COMMITTED do exactly? Each statement sees a snapshot taken at statement start, so two reads in one transaction can differ.
- Does REPEATABLE READ prevent phantoms in PostgreSQL? Yes, because it uses snapshot isolation, though the SQL standard does not require it.
- What is write skew? Two transactions read overlapping data and write disjoint rows, jointly violating an invariant; only SERIALIZABLE prevents it.

Mistakes to avoid: Assuming isolation level names mean the same thing across databases; raising isolation globally to fix a single race; using SERIALIZABLE without retry handling; believing isolation prevents lost updates at READ COMMITTED.

Production perspective: Higher isolation converts silent anomalies into explicit errors, which is an improvement only if the application retries safely. Measure serialisation failures; a rising rate indicates contention that may need a data model change rather than more retries.

Related concepts covered: Snapshot isolation, write skew, SQLSTATE 40001 retries, explicit locking, database-specific semantics.

Q115. How do database locks and deadlocks arise?

Priority: Must Know

Why interviewers ask it: Lock contention is a common production incident and the remedy is design, not luck.

Interview-ready answer: Writers take row-level locks: an UPDATE or DELETE locks the affected rows until commit, and SELECT ... FOR UPDATE takes the same lock explicitly. Readers in PostgreSQL are not blocked by writers thanks to MVCC, but writers block each other on the same row. A deadlock happens when two transactions hold locks the other needs — typically because they update the same rows in different orders. PostgreSQL detects the cycle and aborts one transaction with SQLSTATE 40P01, so the application must catch it and retry. The prevention is deterministic ordering of updates, short transactions, and avoiding user think-time or network calls between acquiring and releasing locks.

In-depth explanation: Lock escalation to table level does not happen in PostgreSQL the way it does in some other engines, but DDL takes heavy locks: ALTER TABLE needs an ACCESS EXCLUSIVE lock, which is why migrations on busy tables need care and CREATE INDEX CONCURRENTLY exists. Foreign keys take locks on the referenced row, so inserting many children of the same parent can serialise. Advisory locks (pg_advisory_xact_lock) are useful for application-level mutual exclusion keyed by an identifier, such as processing one account at a time. SELECT ... FOR UPDATE SKIP LOCKED turns a table into a work queue by letting each worker take different rows, and NOWAIT fails immediately rather than waiting. Monitoring pg_locks joined with pg_stat_activity shows who is blocking whom during an incident.

Practical backend example: Consistent ordering, plus a queue that never blocks:

```
-- deterministic order avoids the cycle entirely
SELECT * FROM account WHERE id = ANY(:ids) ORDER BY id FOR UPDATE;

-- work queue: each worker claims different rows, no contention
SELECT id FROM job WHERE status = 'PENDING'
ORDER BY created_at
FOR UPDATE SKIP LOCKED
LIMIT 50;

-- who is blocking whom, during an incident
SELECT blocked.pid AS blocked_pid, blocking.pid AS blocking_pid, blocked.query
FROM pg_stat_activity blocked
JOIN pg_locks bl ON bl.pid = blocked.pid AND NOT bl.granted
JOIN pg_locks gl ON gl.locktype = bl.locktype AND gl.relation IS NOT DISTINCT FROM bl.relation
AND gl.granted
JOIN pg_stat_activity blocking ON blocking.pid = gl.pid;
```

Common follow-ups:

- What SQLSTATE signals a deadlock? 40P01 in PostgreSQL (deadlock_detected); 40001 is a serialisation failure. Both are retryable.
- Do readers block writers? Not in PostgreSQL's MVCC model for ordinary reads; explicit FOR UPDATE reads do participate in locking.
- How do you avoid lock waits in a queue? FOR UPDATE SKIP LOCKED, so each worker takes unclaimed rows.

Mistakes to avoid: Updating rows in data-dependent order; long transactions that hold locks across external calls; retrying a deadlock without backoff or idempotency; running blocking DDL on large tables during peak hours.

Production perspective: Instrument deadlock and lock-wait counts, and log the SQL of blocked statements. Most deadlocks trace back to two code paths touching the same entities in different orders, which is fixable once you can see both statements.

Related concepts covered: MVCC, row locks, SKIP LOCKED queues, advisory locks, DDL locking, retry strategies.

Q116. When choose optimistic versus pessimistic locking?

Priority: Must Know

Why interviewers ask it: It is the practical answer to "how do you prevent lost updates" and shows awareness of contention trade-offs.

Interview-ready answer: Optimistic locking assumes conflicts are rare: JPA's @Version column is included in the UPDATE ... WHERE id = ? AND version = ?, and if no row matches, someone else changed it and you get an optimistic locking exception to retry or surface to the user. It costs nothing while there is no conflict and scales well. Pessimistic locking takes the row lock up front with SELECT ... FOR UPDATE, serialising access; it suits short, high-contention operations such as decrementing stock for a popular item, where retry storms would be worse. I default to optimistic for user-edited entities and use pessimistic for hot rows or where a retry would be expensive or confusing.

In-depth explanation: Optimistic locking is the only workable option when the read and the write are separated by user think-time, because you cannot hold a database lock across an HTTP round trip — that is the classic "edit form" case, where a version field in the payload detects concurrent edits. In JPA, @Version is checked at flush, so the exception (OptimisticLockException, surfaced by Spring as ObjectOptimisticLockingFailureException) may appear at commit rather than at the save call. Pessimistic modes are PESSIMISTIC_READ, PESSIMISTIC_WRITE and PESSIMISTIC_FORCE_INCREMENT, and a lock timeout should always be set so a blocked request fails rather than hanging. A third option often beats both: a single conditional UPDATE that performs the check and the change atomically, with no read-modify-write at all.

Practical backend example: All three strategies:

```
@Entity
public class Product {
    @Id private UUID id;
    @Version private long version;           // optimistic: checked on flush
    private int stock;
}

// pessimistic: serialise access to a hot row, with a timeout
@Lock(LockModeType.PESSIMISTIC_WRITE)
@QueryHints(@QueryHint(name = "jakarta.persistence.lock.timeout", value = "3000"))
@Query("select p from Product p where p.id = :id")
Optional<Product> findByIdForUpdate(UUID id);
```

```
-- often best: atomic conditional update, no lock held, no retry loop
UPDATE product SET stock = stock - :qty
WHERE id = :id AND stock >= :qty;    -- affected rows = 0 means insufficient stock
```

Common follow-ups:

- What happens on an optimistic conflict? The update affects zero rows and JPA throws an optimistic locking exception; retry the operation or return 409 to the client.
- Can you hold a pessimistic lock across a user's editing session? No; locks live only inside a transaction. Use a version field instead.

- Which is better for a hot row? Pessimistic or an atomic conditional update; optimistic retries would thrash.

Mistakes to avoid: Adding `@Version` and then catching and ignoring the exception; holding a pessimistic lock while calling an external service; assuming `@Transactional` alone prevents lost updates; omitting a lock timeout.

Production perspective: Surface optimistic conflicts to users as a meaningful message ("this record changed, review and retry") rather than a 500. Meter conflict rates: a rising rate signals that the entity is too coarse-grained and may need splitting.

Related concepts covered: Lost updates, `@Version` semantics, pessimistic lock modes, conditional updates, retry and conflict UX.

Q117. What are JPA entity lifecycle states?

Priority: Must Know

Why interviewers ask it: The state model explains why some changes persist automatically and others silently do nothing.

Interview-ready answer: A new object is transient: not associated with a persistence context and with no database row. After `persist` it is managed, meaning the persistence context tracks it and will synchronise changes at flush. When the transaction or the entity manager closes, it becomes detached: it still holds data but changes are no longer tracked. `remove` marks it removed, scheduling a delete at flush. `merge` takes a detached instance and returns a managed copy — crucially, it returns a new managed instance rather than making the argument managed, which is the most common misunderstanding. Changes to a managed entity need no explicit save call because dirty checking writes them at flush.

In-depth explanation: `persist` on an already-detached entity throws; `merge` handles both new and detached cases, which is why Spring Data's `save` delegates to `merge` when the entity is not new. Working with detached entities outside a transaction — for example populating a form object and saving it back — risks overwriting fields with stale values, because `merge` copies the whole state. Reattaching by merging a partially populated object silently nulls the fields you did not set. Removal requires a managed instance, so `remove(detachedEntity)` throws unless you `merge` first. Understanding these transitions also clarifies why an entity loaded in one transaction cannot lazily load a collection in another — the proxy has no session to use.

Practical backend example: State transitions in a typical service:

```
@Transactional
public Order update(UUID id, UpdateOrderCommand cmd) {
    Order order = orderRepository.findById(id).orElseThrow(); // managed
    order.changeNote(cmd.note()); // dirty checking, no save() needed
    return order; // flushed at commit
}

@Transactional
public Order importLegacy(OrderPayload payload) {
    Order detached = OrderMapper.toEntity(payload); // transient or detached
    return orderRepository.save(detached); // persist or merge -> returns managed
    // note: use the RETURNED instance; 'detached' may still be unmanaged
}
```

Common follow-ups:

- What does `merge` return? A managed copy; the instance you passed in stays detached, so always use the return value.
- Do you need to call `save` on a managed entity? No; dirty checking persists changes at flush within the transaction.
- Can you remove a detached entity? Not directly; `merge` it first, or load it by ID inside the transaction.

Mistakes to avoid: Ignoring the return value of `merge/save`; modifying detached entities and expecting changes to persist; calling `persist` on an entity with an assigned ID from a previous session; partial merges that overwrite fields with nulls.

Production perspective: Detached-entity bugs typically appear as silently lost updates that only a careful audit detects. A simple rule — load inside the transaction, mutate the managed instance, never `merge` half-populated objects — prevents almost all of them.

Related concepts covered: persist versus merge, dirty checking, detached state, Spring Data save semantics, lazy loading boundaries.

Q118. How do persistence context and dirty checking work?

Priority: Must Know

Why interviewers ask it: It explains "why did an UPDATE run when I never called save?" and the first-level cache's effect on query counts.

Interview-ready answer: The persistence context is a transaction-scoped identity map: within one transaction, loading the same entity ID twice returns the same instance, and no second SELECT is issued. It also keeps a snapshot of each managed entity's loaded state. At flush time Hibernate compares current values to the snapshot and generates UPDATE statements for anything that changed — that is dirty checking, and it is why mutating a managed entity persists without an explicit save. The cost is memory and CPU proportional to the number of managed entities, which is why long-running batch loops need periodic `flush()` and `clear()`, and why read-only transactions can skip snapshots.

In-depth explanation: The identity map guarantees reference equality for the same ID in one context, which is the correct mental model for JPA equality questions. Flush ordering is defined — inserts, updates, then deletes, with some ordering by entity type — which occasionally surprises people who expect statement order to follow their code. `saveAndFlush` forces it early. Hibernate can also generate an UPDATE for all columns or only changed ones depending on `@DynamicUpdate`; the default all-column update is cheaper to prepare and cache but writes more. Bulk JPQL update/delete statements bypass the persistence context entirely, so in-memory entities become stale — you must clear the context afterwards. The first-level cache is per transaction; the optional second-level cache is shared and introduces its own invalidation concerns.

Practical backend example: Dirty checking, and keeping a batch loop bounded:

```
@Transactional
public void applyDiscount(UUID orderId, int percent) {
    Order order = entityManager.find(Order.class, orderId); // managed, snapshot taken
    order.applyDiscount(percent);                             // no save() call required
}                                                            // flush at commit -> UPDATE

@Transactional
public void reindexAll() {
    int i = 0;
    for (Product p : productRepository.streamAll()) {        // avoid loading everything at once
        p.recomputeSearchVector();
        if (++i % 500 == 0) {
            entityManager.flush();                             // write pending changes
            entityManager.clear();                             // release managed entities
        }
    }
}
```

Common follow-ups:

- Does save always execute SQL immediately? No; for generated identifiers other than IDENTITY, the insert can be deferred to flush. With IDENTITY, Hibernate must insert immediately to obtain the ID.
- Why did loading the same row twice issue only one query? The identity map returned the already-managed instance.
- What happens after a bulk JPQL update? The persistence context is not updated; clear it to avoid stale entities.

Mistakes to avoid: Loading tens of thousands of entities in one transaction; expecting a bulk update to refresh managed instances; relying on the first-level cache across transactions; assuming no SQL runs until you call save.

Production perspective: Memory growth during batch jobs almost always traces back to an unbounded persistence context. Flush-and-clear in chunks, or use stateless sessions and plain SQL for large data movement.

Related concepts covered: Identity map, snapshots and dirty checking, flush ordering, bulk operations, batch memory management, second-level cache.

Q119. When does flush happen, and how is it different from commit?

Priority: Must Know

Why interviewers ask it: Confusing flush with commit leads to wrong assumptions about visibility and rollback.

Interview-ready answer: Flush pushes pending SQL from the persistence context to the database inside the current transaction. Commit ends the transaction and makes everything durable and visible to others. With the default AUTO flush mode, Hibernate flushes before a query whose results could be affected by pending changes, and always before commit. Crucially, flushed statements are still inside the transaction, so they can be rolled back, and other transactions cannot see them until commit. Constraint violations therefore surface at flush time, which may be in the middle of your method rather than at the save call — a frequent source of confusing stack traces.

In-depth explanation: Flush mode can be set to COMMIT to flush only at the end, which risks queries not seeing your own pending changes, and Hibernate uses a manual-style mode for read-only transactions. Explicit `flush()` is useful when you need a generated ID before continuing, or when you want a constraint violation to surface at a precise point so you can handle it. Note that `saveAndFlush` does not commit. In Spring, the transaction commits when the outermost `@Transactional` method returns, so exceptions thrown after a flush still roll back the flushed statements. One more subtlety: because flush writes rows, it can take locks that are then held for the rest of the transaction, so flushing early lengthens lock hold time.

Practical backend example: Controlling flush deliberately:

```
@Transactional
public Invoice issue(UUID orderId) {
    Invoice invoice = invoiceRepository.save(new Invoice(orderId)); // may not hit the DB yet
    entityManager.flush(); // force the INSERT now: we need the generated number
    pdfService.render(invoice.getNumber()); // uses the generated value
    if (!fraudCheck.passes(orderId)) {
        throw new FraudException(orderId); // rollback still undoes the flushed INSERT
    }
    return invoice; // commit happens when the method returns
}
```

Common follow-ups:

- Can flushed SQL be rolled back? Yes; it is inside the transaction until commit.
- When does AUTO flush occur? Before a query that could be affected by pending changes, and before commit.
- Why did a constraint violation appear at an unexpected line? The violating statement was flushed there, not where save was called.

Mistakes to avoid: Believing save commits; catching a constraint exception far from the offending statement without understanding flush; flushing in a tight loop and holding locks; disabling auto-flush without understanding query visibility.

Production perspective: Flush timing affects lock hold time and therefore contention. In write-heavy transactions, do the flushing work as late as possible and keep the transaction short so locks are released quickly.

Related concepts covered: Flush modes, generated identifiers, rollback semantics, constraint violation timing, lock duration.

Q120. How do lazy and eager loading affect endpoint performance?

Priority: Must Know

Why interviewers ask it: Fetch strategy is the single biggest ORM performance lever and the source of `LazyInitializationException`.

Interview-ready answer: `@ManyToOne` and `@OneToOne` default to EAGER in JPA, while collections default to LAZY. Eager associations are loaded on every fetch of the parent, including in queries where you do not need them, which quietly multiplies joins and data volume. Lazy associations are proxies loaded on first access, which is efficient but throws `LazyInitializationException` if accessed after the persistence context closed. My default is to make everything lazy — including `@ManyToOne(fetch = FetchType.LAZY)` — and then fetch exactly what each use case needs with a fetch join, an entity graph, or a DTO projection. That makes the fetch plan a property of the query rather than of the mapping.

In-depth explanation: `LazyInitializationException` typically appears when an entity is serialised in the controller after the transaction ended. Spring Boot enables `open-session-in-view` by default, which hides the problem by keeping the session open during rendering — but it also holds a database connection for the whole request and makes N+1 queries invisible. Disabling it (`spring.jpa.open-in-view=false`) surfaces the real fetch requirements and is widely recommended for APIs. Eager loading is not "faster": it makes every query load more, and multiple eager collections in one query produce a Cartesian product. The right tool per case is: `fetch join` for a single collection, `@EntityGraph` for declarative plans, batch fetching for many parents, and projections when you only need a few columns.

Practical backend example: Explicit fetch plan per use case:

```
@Entity
public class Order {
    @ManyToOne(fetch = FetchType.LAZY) private Customer customer; // override the EAGER default
    @OneToMany(mappedBy = "order", fetch = FetchType.LAZY) private List<OrderLine> lines = new
        ArrayList<>();
}

public interface OrderRepository extends JpaRepository<Order, UUID> {
    @EntityGraph(attributePaths = {"customer", "lines"}) // declarative fetch plan
    Optional<Order> findById(UUID id);

    @Query("select new com.example.OrderSummary(o.id, o.status, o.totalCents) from Order o "
        + "where o.customerId = :customerId") // projection: no entities at all
    List<OrderSummary> summaries(UUID customerId);
}
```

```
spring.jpa.open-in-view=false # fail fast on missing fetch plans instead of hiding them
```

Common follow-ups:

- Why does `LazyInitializationException` occur? The proxy was accessed after the persistence context closed, usually during serialisation.
- Is EAGER a safe default? No; it loads data you may not need on every query and can produce Cartesian products with multiple collections.
- Should `open-session-in-view` stay enabled? Generally disable it for APIs: it holds connections longer and conceals N+1 problems.

Mistakes to avoid: Making associations eager to fix a lazy exception; returning entities from controllers; fetching two collections in one join; assuming lazy loading is free — each access is a query.

Production perspective: Fetch strategy directly drives query count and connection hold time. Add a test that asserts the number of statements for critical endpoints so a mapping change cannot silently introduce N+1 queries.

Related concepts covered: Fetch types, entity graphs, `open-session-in-view`, projections, Cartesian products, query-count testing.

Q121. How do you diagnose and fix an N+1 query?

Priority: Must Know

Why interviewers ask it: It is the most frequent ORM performance defect and the fix requires understanding several tools.

Interview-ready answer: N+1 means one query loads N parents and then each parent triggers another query for its association — 101 statements for 100 orders. I detect it by counting statements per request: Hibernate statistics, a SQL logging profile in development, an assertion in an integration test, or a database span count in a trace. Fixes depend on the shape. For a single collection, a `JOIN FETCH` or `@EntityGraph` loads everything in one query. For many parents, `hibernate.default_batch_fetch_size` or `@BatchSize` turns N queries into a handful of `IN` queries. When I only need a few fields, a DTO projection avoids entities altogether. The wrong fix is switching the mapping to EAGER, which spreads the cost to every other query.

In-depth explanation: Fetch joining a collection has an important caveat: combining it with pagination cannot be done in SQL, so Hibernate loads all matching rows and paginates in memory, logging a warning (historically HHH000104). For paginated parents with children, the standard technique is two queries — page the parent IDs first, then fetch

children with where parent.id in (:ids) — or rely on batch fetching, which does exactly that automatically. Joining two collections in one query produces a Cartesian product, so fetch at most one collection per query (distinct in JPQL removes duplicate parent references but not the wasted database work). Batch size is a global tuning knob worth setting to a sensible value such as 25–100 in most applications.

Practical backend example: Detection and two correct fixes:

```
// detection in a test: fail the build if the query count regresses
@Test
void listingOrdersRunsAtMostTwoQueries() {
    Statistics stats = entityManagerFactory.unwrap(SessionFactory.class).getStatistics();
    stats.clear();
    orderQueryService.listWithLines(customerId, PageRequest.of(0, 20));
    assertThat(stats.getPrepareStatementCount()).isLessThanOrEqualTo(2);
}
```

```
// fix 1: entity graph for a single aggregate
@EntityGraph(attributePaths = "lines")
List<Order> findByCustomerId(UUID customerId);
```

```
# fix 2: batch fetching turns N lazy loads into ceil(N / size) IN queries
spring.jpa.properties.hibernate.default_batch_fetch_size=50
```

Common follow-ups:

- Can a fetch join break pagination? Yes, for collection joins: Hibernate paginates in memory. Page parent IDs first, then fetch children.
- What does batch fetching do? Loads pending proxies in groups with an IN clause instead of one query each.
- Does distinct in JPQL fix the duplicates? It removes duplicate parent references in the result list, but the database still returns the multiplied rows.

Mistakes to avoid: Switching to EAGER; fetch-joining two collections; adding distinct and assuming the performance problem is solved; measuring only in development where data volumes hide the cost.

Production perspective: N+1 usually scales with result size, so it passes tests with ten rows and fails with a thousand. Assert query counts in integration tests for the endpoints that matter, and alert on database calls per request in tracing.

Related concepts covered: Entity graphs, fetch joins, batch fetching, in-memory pagination warnings, DTO projections, query-count assertions.

Q122. How should entities and DTOs be separated?

Priority: Must Know

Why interviewers ask it: Returning entities from controllers couples the API to the schema and causes both performance and security problems.

Interview-ready answer: Entities model persistence: identity, associations, lifecycle. DTOs model the API contract: exactly the fields a client needs, in a shape that can evolve independently of the schema. Returning entities directly leaks internal fields, tempts the serialiser into triggering lazy loads, and means a column rename becomes a breaking API change. I map explicitly at the boundary — manual mapping for small objects, MapStruct when there are many — and for read-heavy endpoints I skip entities entirely with a projection query that selects only the needed columns. Separate request DTOs also prevent mass-assignment, where a client sets a field it should not control.

In-depth explanation: The serialisation hazard is concrete: Jackson walking an entity graph can trigger lazy loading (or a LazyInitializationException when the session has closed), and bidirectional associations cause infinite recursion unless annotated. Projections come in three forms in Spring Data: interface-based (closed projections are translated into narrower SQL), class-based (constructor expressions in JPQL) and dynamic projections via a generic return type. For write paths, a command object makes explicit which fields the use case accepts, so adding a column to the entity cannot accidentally become settable through the API. The cost of mapping is real but small; the cost of a leaked or coupled contract is much larger.

Practical backend example: Projection for reads, command for writes:

```
// read model: only what the client needs, one narrow SELECT
public record OrderSummary(UUID id, OrderStatus status, long totalCents, Instant createdAt) {}

@Query("select new com.example.order.OrderSummary(o.id, o.status, o.totalCents, o.createdAt) "
      + "from Order o where o.customerId = :customerId")
List<OrderSummary> findSummaries(UUID customerId);

// write model: explicit accepted fields, no mass assignment of entity internals
public record UpdateOrderRequest(@Size(max = 280) String note) {
    public UpdateOrderCommand toCommand(UUID id) { return new UpdateOrderCommand(id, note); }
}
```

Common follow-ups:

- Why not return entities directly? Schema coupling, accidental lazy loading, exposure of internal fields, and recursion in bidirectional graphs.
- Is mapping boilerplate worth it? Yes; use MapStruct or records to keep it small, and projections to skip mapping entirely for reads.
- What is mass assignment? Binding client input straight onto a persistent object so it can set fields such as role or balance.

Mistakes to avoid: Annotating entities with Jackson annotations to control the API; reusing one DTO for request and response; exposing database IDs and internal state you do not intend to support; deep reflection-based mappers that hide errors.

Production perspective: A stable API contract is what lets you refactor the schema without coordinating client releases. Projections also reduce payload size and query cost, which shows up directly in p99 latency for list endpoints.

Related concepts covered: Projections, MapStruct, mass assignment, serialisation pitfalls, API versioning, read models.

Q123. What do cascade and orphanRemoval mean?

Priority: Important

Why interviewers ask it: Cascades are easy to configure and dangerous to get wrong — they can delete data you did not intend.

Interview-ready answer: Cascade propagates entity manager operations from a parent to its associated entities: PERSIST, MERGE, REMOVE, REFRESH, DETACH, or ALL. `orphanRemoval = true` is different: it deletes a child when it is removed from the parent's collection, modelling true ownership. I use `CascadeType.ALL` with `orphanRemoval` only for genuine composition — order and order lines, where a line has no meaning without its order. I never put REMOVE on a @ManyToOne pointing at a shared entity, because deleting an order would then delete the customer. Cascades are a JPA-level mechanism; they do not replace database-level ON DELETE rules, and the two can conflict.

In-depth explanation: Cascading remove issues individual deletes per child, which is slow for large collections; a bulk DELETE ... WHERE parent_id = ? is far cheaper when the children have no further cascades. `orphanRemoval` also implies cascade remove for the association. A subtle failure is replacing a collection instance (`order.setLines(newList())`) instead of mutating it, which detaches Hibernate's tracked collection and can throw or silently skip orphan removal; mutate the existing collection instead. Bidirectional associations need a helper method that sets both sides, otherwise the foreign key is not written because the owning side was never updated. Foreign keys with ON DELETE CASCADE in the schema act independently of JPA and can remove rows Hibernate still has in its context.

Practical backend example: Aggregate ownership done correctly:

```

@Entity
public class Order {
    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true)
    private final List<OrderLine> lines = new ArrayList<>(); // composition: lines belong to the
        order

    public void addLine(OrderLine line) { lines.add(line); line.setOrder(this); } // both sides
    public void removeLine(OrderLine line) { lines.remove(line); line.setOrder(null); } // deleted at
        flush

    @ManyToOne(fetch = FetchType.LAZY) // NEVER cascade REMOVE here
    private Customer customer;
}

```

Common follow-ups:

- Should `CascadeType.REMOVE` be on a `@ManyToOne`? Almost never; it would delete the shared parent when a child is removed.
- What does `orphanRemoval` add over cascade remove? It deletes children removed from the collection, not only when the parent is deleted.
- Why was the foreign key null? The owning side was not set; use a helper method that updates both directions.

Mistakes to avoid: `CascadeType.ALL` on every association; replacing a managed collection instance; relying on cascade to delete thousands of rows; assuming JPA cascade and database `ON DELETE` do the same thing.

Production perspective: An accidental cascade delete is a data-loss incident, and backups are the only recovery. Model ownership explicitly, add foreign key constraints that make wrong deletes fail, and review cascade settings in code review as carefully as security rules.

Related concepts covered: Aggregate design, orphan removal, bidirectional association ownership, bulk deletes, database cascade rules.

Q124. How do one-to-many mappings affect SQL and ownership?

Priority: Important

Why interviewers ask it: Ownership determines which SQL is generated, and confusion here produces extra updates or missing foreign keys.

Interview-ready answer: In a bidirectional one-to-many, the many side owns the relationship because it holds the foreign key column; the one side is mapped with `mappedBy`. Hibernate writes the foreign key based on the owning side, so if you add a child to the parent's collection but never set `child.setParent(parent)`, the column stays null. A unidirectional `@OneToMany` without `@JoinColumn` defaults to a join table, which is usually not what people expect; adding `@JoinColumn` keeps the foreign key on the child table but generates extra `UPDATE` statements. That is why bidirectional with a synchronising helper method is the common, efficient choice.

In-depth explanation: Collection type matters too: `List` without an order column is treated as a bag and can generate less efficient SQL for updates; `Set` requires stable `equals`/`hashCode` on the child, which is awkward for entities with generated IDs — a common recommendation is to base equality on a business key or a UUID assigned in the constructor. `@OrderColumn` maintains an index column but adds write overhead. For very large collections, do not map them at all: query the children with pagination instead, since loading a 50,000-element collection into memory is never the right answer. Also, the parent's collection is only refreshed from the database when the context is cleared or reloaded, so after a bulk insert of children the in-memory collection may be stale.

Practical backend example: A correctly synchronised bidirectional mapping:

```

@Entity
public class Invoice {
    @OneToMany(mappedBy = "invoice", cascade = CascadeType.ALL, orphanRemoval = true)
    private final Set<InvoiceLine> lines = new LinkedHashSet<>();

    public void addLine(InvoiceLine line) { lines.add(line); line.setInvoice(this); }
}

@Entity
public class InvoiceLine {
    @Id private UUID id = UUID.randomUUID(); // assigned early -> stable equals/hashCode
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "invoice_id", nullable = false) // owning side: writes the FK
    private Invoice invoice;

    @Override public boolean equals(Object o) { return o instanceof InvoiceLine l && id.equals(l.id); }
    @Override public int hashCode() { return id.hashCode(); }
}

```

Common follow-ups:

- Which side writes the foreign key? The owning side — the one with `@JoinColumn`, normally the many side.
- What happens with a unidirectional `@OneToMany` and no `@JoinColumn`? JPA creates a join table, which is rarely intended.
- Why do extra UPDATE statements appear? A unidirectional mapping with `@JoinColumn` inserts children first and then updates the foreign key.

Mistakes to avoid: Updating only the parent collection; using Set with entity equality based on a generated ID that is null before persist; mapping huge collections; forgetting `nullable = false` and allowing orphan rows.

Production perspective: Large mapped collections are a common cause of slow endpoints and memory spikes. If a parent can have thousands of children, expose them through a paginated query rather than an association.

Related concepts covered: Owning side semantics, join tables, entity equality, order columns, collection size limits.

Q125. How do JPQL, native SQL and projections compare?

Priority: Important

Why interviewers ask it: Choosing the right query tool shows pragmatism rather than ORM purity.

Interview-ready answer: JPQL works on the entity model, is portable across databases, and integrates with the persistence context — good for most CRUD and aggregate loading. Native SQL gives full access to database features JPQL cannot express: window functions, CTEs, `DISTINCT ON`, full-text search, upserts. Projections — interface-based or constructor-based — return only the columns needed and skip entity management entirely, which is the best choice for read models and list endpoints. My rule is: JPQL for entity operations, projections for reads, native SQL for reporting and database-specific features, and Criteria API only for genuinely dynamic filters.

In-depth explanation: Native queries return unmanaged results unless you map them to entities, which is often an advantage for reporting because nothing is added to the persistence context. They do tie you to a dialect, which matters only if you actually plan to change databases — for most teams the portability argument is theoretical, and the real cost is losing compile-time checking. Spring Data can execute native queries with pagination if you supply a `countQuery`. Closed interface projections let Spring Data generate a narrower `SELECT`, whereas open projections with `@Value` SPEL expressions fetch the whole entity, defeating the purpose. For complex dynamic filtering, Criteria API or a library such as jOOQ or Querydsl is more maintainable than string concatenation — and string concatenation with user input is an injection risk.

Practical backend example: Three query styles for three needs:

```
// JPQL: entity operation, portable, participates in the persistence context
@Query("select o from Order o where o.status = :status and o.createdAt >= :since")
List<Order> findRecent(OrderStatus status, Instant since);

// Closed interface projection: narrow SELECT, no entity overhead
public interface CustomerRevenue { UUID getCustomerId(); long getRevenueCents(); }

// Native SQL: window function that JPQL cannot express
@Query(value = ""
    SELECT customer_id, SUM(total_cents) AS revenue_cents
    FROM "order"
    WHERE status = 'PAID' AND created_at >= :since
    GROUP BY customer_id
    ORDER BY revenue_cents DESC
    LIMIT :limit
    """, nativeQuery = true)
List<CustomerRevenue> topCustomers(Instant since, int limit);
```

Common follow-ups:

- When do you use a DTO projection? Whenever you only need a subset of columns, especially for list and report endpoints.
- Does a native query participate in dirty checking? Not unless it returns mapped entities; scalar and projection results are unmanaged.
- How do you paginate a native query? Provide a separate countQuery alongside the main query.

Mistakes to avoid: Building queries by string concatenation with user input; loading entities to compute an aggregate; using open projections and assuming they narrow the SQL; overusing Criteria API for static queries.

Production perspective: Reporting queries have very different performance characteristics from transactional ones. Keep them on read replicas where possible, give them their own timeouts, and make sure a slow report cannot exhaust the connection pool used by user-facing traffic.

Related concepts covered: JPQL versus native SQL, interface and constructor projections, dynamic queries, SQL injection risk, read replicas.

Q126. What happens when a transaction calls an external service?

Priority: Must Know

Why interviewers ask it: Mixing database transactions with network calls is a top cause of pool exhaustion and inconsistent state.

Interview-ready answer: The database connection is held for the whole transaction, so an HTTP call inside it keeps the connection busy for the duration of the remote call, including its timeout. Under load that exhausts the pool and causes cascading failures. There is also a correctness problem: the remote system has no idea about your transaction, so it cannot roll back. If the call succeeds and the transaction later fails, the two systems disagree. The pattern I use is to do the database work in a short transaction, commit, and then perform the external call — driven by a transactional outbox or an after-commit step — with idempotency so retries are safe. If the call must happen first, store the intent and reconcile.

In-depth explanation: The transactional outbox writes the domain change and an event row in the same local transaction, so they commit atomically. A relay then reads pending outbox rows and publishes them, retrying until acknowledged; consumers deduplicate because delivery is at-least-once. This converts a distributed-transaction problem into a local-transaction problem plus idempotent delivery. If you cannot use an outbox, the minimum discipline is: short transaction, call outside it, persist the outcome, and have a reconciliation job that finds records stuck in an intermediate state. Two-phase commit exists but is rarely justified in modern service architectures because of its operational cost and coupling.

Practical backend example: Outbox write plus a relay:

```
@Transactional // short, local, no network calls
public Order place(PlaceOrderCommand cmd) {
    Order order = orderRepository.save(Order.from(cmd));
    outboxRepository.save(new OutboxRow(order.id(), "OrderPlaced",
        json.write(OrderPlacedEvent.from(order)))); // same transaction, atomic
    return order;
}
```

```
-- relay: claim a batch without blocking other relay instances
UPDATE outbox SET status = 'SENDING', claimed_at = now()
WHERE id IN (SELECT id FROM outbox WHERE status = 'PENDING'
    ORDER BY created_at FOR UPDATE SKIP LOCKED LIMIT 100)
RETURNING id, payload;
```

Common follow-ups:

- How does the outbox pattern help? It makes the state change and the intent to publish atomic, removing the dual-write problem.
- Does it give exactly-once delivery? No; it gives at-least-once publication, so consumers must be idempotent.
- What if the call must happen before the commit? Make it idempotent, keep the transaction extremely short, and reconcile stuck records.

Mistakes to avoid: Calling a payment gateway inside `@Transactional`; assuming a rollback undoes a remote side effect; long timeouts on calls made inside transactions; publishing events before commit.

Production perspective: Connection pool exhaustion from in-transaction HTTP calls presents as widespread slow endpoints with healthy CPU. Look for `idle` in transaction sessions and check whether a remote client sits inside a transactional method.

Related concepts covered: Dual writes, transactional outbox, idempotent consumers, reconciliation jobs, connection hold time.

Q127. How do you batch writes without exhausting memory?

Priority: Important

Why interviewers ask it: Bulk jobs are common, and naive implementations either crawl or run out of heap.

Interview-ready answer: Two things must be bounded: the number of round trips and the size of the persistence context. For round trips I enable JDBC batching with `hibernate.jdbc.batch_size` plus `order_inserts` and `order_updates` so statements group well. For memory I flush and clear every batch, so managed entities are released. One important detail is that `GenerationType.IDENTITY` prevents insert batching, because Hibernate must execute each insert to obtain the generated key — using a sequence with a pooled optimiser allows batching. For very large loads, plain JDBC or PostgreSQL `COPY` is dramatically faster than the ORM, and I choose that when the job is pure data movement.

In-depth explanation: Batching only works when consecutive statements target the same table and shape, which is why ordering matters. `rewriteBatchedInserts=true` in the PostgreSQL JDBC URL rewrites batched inserts into multi-row `INSERT` statements, which is a substantial improvement. Reading also needs bounding: a `Stream` from a repository with a fetch size, or keyset pagination, avoids materialising an entire table. Transaction scope is a trade-off: one transaction for a million rows creates a huge undo footprint and long lock duration, while a transaction per row is slow and non-atomic; chunked transactions of a few hundred to a few thousand rows, with restartability, is the usual compromise. Spring Batch formalises this with readers, processors, writers and checkpointing.

Practical backend example: A chunked import with batching enabled:

```
spring.jpa.properties.hibernate.jdbc.batch_size=100
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true
spring.datasource.url=jdbc:postgresql://db:5432/app?rewriteBatchedInserts=true
```

```
@Transactional
public void importChunk(List<RowDto> chunk) {           // called per chunk, not once for the file
    int i = 0;
    for (RowDto row : chunk) {
        entityManager.persist(Product.from(row));
        if (++i % 100 == 0) { entityManager.flush(); entityManager.clear(); } // bound memory
    }
}
```

Common follow-ups:

- Does IDENTITY allow insert batching? No; Hibernate must run each insert to get the key. Use a sequence with a pooled optimiser.
- Why flush and clear? Flush sends the batch; clear releases managed entities so the persistence context does not grow without bound.
- When would you bypass JPA? For pure bulk loads, plain JDBC batching or COPY is far faster and uses much less memory.

Mistakes to avoid: One transaction for an entire file; forgetting `clear()` and exhausting the heap; leaving batching disabled and issuing a million round trips; ignoring restartability after a partial failure.

Production perspective: Long bulk jobs hold locks and generate large amounts of WAL, which affects replication lag and vacuum. Run them off-peak, chunk them, and monitor replica lag while they run.

Related concepts covered: JDBC batching, sequence generators, chunked transactions, streaming reads, Spring Batch, WAL and replication impact.

Q128. How should schema migrations be deployed?

Priority: Important

Why interviewers ask it: Schema changes are the riskiest part of most deployments and require a rollout strategy, not just a script.

Interview-ready answer: I use a versioned migration tool — Flyway or Liquibase — with migrations in source control, applied automatically at startup or by a dedicated job, never by hand. The key principle is expand and contract: make additive, backward-compatible changes first so the old and new application versions can both run against the same schema during a rolling deploy, then remove the old structures in a later release. That means adding a nullable column, backfilling in batches, switching the code to use it, and only then making it non-nullable or dropping the old column. I also avoid long-blocking DDL on large tables and use `CREATE INDEX CONCURRENTLY`.

In-depth explanation: During a rolling deploy both versions run simultaneously, so a migration that renames a column breaks whichever version is not updated. Expand-contract avoids that entirely. In PostgreSQL, adding a column with a non-volatile default is fast since version 11 because it no longer rewrites the table, but adding a CHECK constraint or changing a type may rewrite and lock it — `ADD CONSTRAINT ... NOT VALID` followed by `VALIDATE CONSTRAINT` avoids the long exclusive lock. Backfills should be batched with a delay so replication and vacuum keep up. Flyway migrations should be immutable once applied; fixing a mistake means a new migration, not editing history. `hibernate.ddl-auto` should be `validate` or `none` in production — never `update`, which makes uncontrolled changes.

Practical backend example: An expand-contract sequence across two releases:

```
-- V12__add_email_normalised.sql (release 1: expand, backward compatible)
ALTER TABLE customer ADD COLUMN email_normalised text;           -- nullable, no rewrite
CREATE INDEX CONCURRENTLY idx_customer_email_norm ON customer (email_normalised);

-- V13__backfill_email_normalised.sql (batched to avoid a long lock and replica lag)
UPDATE customer SET email_normalised = lower(email)
WHERE email_normalised IS NULL AND id IN (
    SELECT id FROM customer WHERE email_normalised IS NULL LIMIT 10000);

-- V20__enforce_email_normalised.sql (release 2: contract, after all instances use it)
ALTER TABLE customer ALTER COLUMN email_normalised SET NOT NULL;
ALTER TABLE customer DROP COLUMN email;
```

```
spring.jpa.hibernate.ddl-auto=validate      # never 'update' in production
spring.flyway.enabled=true
```

Common follow-ups:

- Why avoid destructive changes first? Both old and new application versions run during a rolling deploy; dropping something the old version needs breaks it.
- How do you add an index without downtime? `CREATE INDEX CONCURRENTLY` in PostgreSQL, outside a transaction.
- What about rollback? Prefer forward-only migrations with backward-compatible steps; restoring from backup is the real rollback for destructive changes.

Mistakes to avoid: `ddl-auto=update` in production; editing an applied migration; unbatched backfills on large tables; renaming columns in a single step; running migrations from multiple instances without the tool's locking.

Production perspective: Migrations should be tested against a production-sized copy, because a script that takes two seconds on a laptop can lock a 200 million-row table for minutes. Record migration duration in the deployment log so regressions are visible.

Related concepts covered: Flyway and Liquibase, expand-contract, concurrent index creation, batched backfills, `ddl-auto` settings, rollback strategy.

Q129. What do unique constraints protect that application checks cannot?

Priority: Must Know

Why interviewers ask it: It tests understanding of concurrency: an application-level check is inherently a race.

Interview-ready answer: A check like "does this email already exist?" followed by an insert is a check-then-act race: two concurrent requests can both see no row and both insert. Only a unique constraint enforced by the database prevents that, because the index guarantees it atomically. So I add the constraint, attempt the insert, and translate the resulting violation into a meaningful response — 409 with a clear message. The application-level check is still useful for a friendly error message in the common case, but it is not the guarantee. The same reasoning applies to idempotency keys, which are best implemented as a unique column.

In-depth explanation: In Spring, a constraint violation surfaces as `DataIntegrityViolationException` wrapping the driver exception; to map specific constraints you inspect the constraint name, which is why naming constraints explicitly in migrations pays off. PostgreSQL also offers `INSERT ... ON CONFLICT DO NOTHING/UPDATE` for upsert semantics, which turns a race into a defined outcome without exception handling. Partial unique indexes handle rules such as "only one active subscription per customer" (`WHERE status = 'ACTIVE'`). Case-insensitive uniqueness needs an index on a normalised expression, not just a constraint on the raw column. Remember that the constraint also fails for retries of the same logical request, which is exactly what makes it a good idempotency mechanism.

Practical backend example: Constraint plus upsert plus error translation:

```
CREATE UNIQUE INDEX uk_customer_email_norm ON customer (lower(email));
CREATE UNIQUE INDEX uk_subscription_active ON subscription (customer_id) WHERE status = 'ACTIVE';
CREATE UNIQUE INDEX uk_payment_idempotency ON payment (idempotency_key);

-- race-free upsert
INSERT INTO payment (id, idempotency_key, order_id, amount_cents)
VALUES (:id, :key, :orderId, :amount)
ON CONFLICT (idempotency_key) DO NOTHING
RETURNING id;
```

```
@ExceptionHandler(DataIntegrityViolationException.class)
ProblemDetail onConflict(DataIntegrityViolationException ex) {
    String constraint = ex.getMostSpecificCause().getMessage(); // name your constraints!
    HttpStatus status = constraint.contains("uk_customer_email_norm")
        ? HttpStatus.CONFLICT : HttpStatus.BAD_REQUEST;
    return ProblemDetail.forStatusAndDetail(status, "Record already exists");
}
```

Common follow-ups:

- How do you map constraint violations to responses? Catch `DataIntegrityViolationException` and branch on the constraint name, which you should set explicitly in migrations.
- Is an application check useless? No; it gives a better message in the normal case, but the constraint is the guarantee.
- How do you enforce "only one active per customer"? A partial unique index on the active rows.

Mistakes to avoid: Relying on `existsBy...` before save; auto-generated constraint names that are unreadable in logs; catching the violation and returning 500; assuming uniqueness across case or whitespace variants without normalising.

Production perspective: Duplicate records are hard to clean up later and often have downstream effects such as duplicate billing. Constraints are cheap insurance, and they also protect against bugs in code paths written years later by someone else.

Related concepts covered: Check-then-act races, upserts, partial indexes, idempotency keys, exception translation.

Q130. How do you make a transfer atomic in SQL and service code?

Priority: Must Know

Why interviewers ask it: It is the canonical correctness exercise combining transactions, locking, constraints and retries.

Interview-ready answer: One transaction, deterministic lock ordering, an invariant enforced by the database, and idempotency at the boundary. Concretely: begin a transaction, update both balances with conditional statements so a negative balance is impossible, insert a ledger row with a unique idempotency key so a retried request cannot double-apply, and commit. If I need to read both rows first I lock them with `SELECT ... FOR UPDATE ORDER BY id` to avoid deadlocks. No external calls inside the transaction. If the transfer must call a payment provider, the database part commits first and the provider call is driven by an outbox with reconciliation.

In-depth explanation: The conditional update is the important trick: `UPDATE account SET balance = balance - :amt WHERE id = :from AND balance >= :amt` performs the check and the change atomically, and zero affected rows means insufficient funds — no read-modify-write race. A `CHECK (balance >= 0)` constraint is the backstop. Under concurrency, two transfers touching the same pair of accounts in opposite directions can deadlock, which ordered locking prevents; if a deadlock still occurs, the driver reports `SQLSTATE 40P01` and the operation should be retried with backoff, which is safe because the idempotency key makes reapplication a no-op. A double-entry ledger — one row per movement rather than only mutable balances — is the design that makes auditing and reconciliation possible later.

Practical backend example: The full pattern:

```
BEGIN;
-- idempotency: a duplicate request inserts nothing and the transfer is skipped
INSERT INTO transfer (id, idempotency_key, from_id, to_id, amount_cents, created_at)
VALUES (:id, :key, :from, :to, :amount, now())
ON CONFLICT (idempotency_key) DO NOTHING;

UPDATE account SET balance_cents = balance_cents - :amount
WHERE id = :from AND balance_cents >= :amount;           -- 0 rows -> insufficient funds, roll back

UPDATE account SET balance_cents = balance_cents + :amount WHERE id = :to;
COMMIT;
```

```
@Retryable(retryFor = {CannotAcquireLockException.class, DeadlockLoserDataAccessException.class},
           maxAttempts = 3, backoff = @Backoff(delay = 50, multiplier = 2))
@Transactional
public void transfer(TransferCommand cmd) {
    if (transferRepository.insertIfAbsent(cmd) == 0) return;           // idempotent replay
    if (accountRepository.debitIfSufficient(cmd.from(), cmd.amount()) == 0) {
        throw new InsufficientFundsException(cmd.from());           // rolls back everything
    }
    accountRepository.credit(cmd.to(), cmd.amount());
}
```

Common follow-ups:

- What happens under concurrent transfers? Row locks serialise conflicting updates; deadlocks are possible with inconsistent ordering and must be retried.
- Why not read the balance first? Read-then-write is a race; the conditional update is atomic.
- How do you make retries safe? A unique idempotency key so a replay does not apply the movement twice.

Mistakes to avoid: Checking the balance in Java; two separate transactions for debit and credit; no idempotency key; calling a payment API inside the transaction; retrying without backoff.

Production perspective: Financial flows need an immutable ledger and a reconciliation job comparing internal state with the provider. Balances derived from a ledger are auditable; a mutable balance column alone leaves you unable to explain how it got there.

Related concepts covered: Conditional updates, deadlock retries, idempotency keys, double-entry ledgers, outbox for external calls.

Q131. Why can a composite index's column order matter?

Priority: Important

Why interviewers ask it: It is a precise, practical indexing question that separates surface knowledge from real tuning experience.

Interview-ready answer: A composite B-tree index is sorted by the first column, then the second within it, and so on, so it is usable for predicates that form a leftmost prefix. An index on (customer_id, created_at) supports WHERE customer_id = ?, WHERE customer_id = ? AND created_at > ?, and ORDER BY created_at within a customer. It generally does not help WHERE created_at > ? alone, because the leading column is unconstrained. The usual guideline is equality columns first, then the range or sort column. Column order also affects whether the index can satisfy an ORDER BY without a separate sort step.

In-depth explanation: PostgreSQL can sometimes use a non-leading column through an index-only scan or a bitmap scan when the index is small relative to the table, but you should not rely on that. Index direction matters for multi-column ordering: an index on (created_at DESC, id DESC) serves that exact ordering, and PostgreSQL can also scan backwards, but mixed directions such as ORDER BY a ASC, b DESC need a matching index definition. Adding a column to an existing composite index is often better than creating a second index, because each index costs write performance and storage. Covering indexes with INCLUDE add non-key columns so a query can be answered from the index alone without a heap fetch, at the cost of a larger index.

Practical backend example: One index serving several access patterns:

```
CREATE INDEX idx_order_customer_created ON "order" (customer_id, created_at DESC, id DESC);

SELECT * FROM "order" WHERE customer_id = :c ORDER BY created_at DESC LIMIT 20; -- uses it fully
SELECT * FROM "order" WHERE customer_id = :c AND created_at >= :from;           -- uses it
SELECT * FROM "order" WHERE created_at >= :from;                             -- leading column
                                                                    -- unconstrained:

-- likely a seq scan

-- covering index: answer a hot query without touching the heap
CREATE INDEX idx_order_status_covering ON "order" (status, created_at) INCLUDE (total_cents);
```

Common follow-ups:

- Does an index on (a, b) help WHERE b = ?? Usually not efficiently; the leading column must be constrained for a normal index scan.
- Equality or range column first? Equality first, then the range or sort column.
- When is a second index better than extending the first? When the access patterns genuinely differ in their leading column and both are frequent.

Mistakes to avoid: Creating one single-column index per column and expecting them to combine as well as a composite; ignoring sort direction; duplicating an index that an existing prefix already covers; adding indexes without checking write impact.

Production perspective: Review pg_stat_user_indexes for unused indexes and check index bloat periodically. Each additional index slows every write and increases WAL volume, which affects replication.

Related concepts covered: Leftmost prefix rule, index-only and covering indexes, sort avoidance, index maintenance cost, bitmap scans.

Q132. How do connection pools interact with database transactions?

Priority: Important

Why interviewers ask it: The link between transaction duration and pool capacity is the core of database-bound capacity planning.

Interview-ready answer: A transaction is bound to one connection for its entire life, so transaction duration directly determines how long a pool slot is occupied. Throughput is therefore roughly pool size divided by average transaction duration. If transactions do slow things — external calls, large scans, user think-time — the pool drains and requests queue for a connection, which looks like a slow application but is really saturation. Long idle-in-transaction sessions are especially bad in PostgreSQL because they hold locks and prevent vacuum from reclaiming dead tuples, causing table bloat. So short transactions are both a correctness and a capacity practice.

In-depth explanation: Autocommit matters: outside a transaction each statement runs and releases immediately, so read-only endpoints that do not need a transaction should not open one. In Spring, a `@Transactional(readOnly = true)` method still holds a connection for the whole method, so heavy in-memory processing inside it wastes a slot. Nested `REQUIRES_NEW` propagation takes a second connection while the first is held, which can deadlock the pool when concurrency approaches the pool size. Monitoring should include Hikari's pending-threads gauge and PostgreSQL's `state = 'idle in transaction'` count; `idle_in_transaction_session_timeout` is a useful database-side guardrail. For very high connection counts, a pooler such as PgBouncer in transaction mode multiplexes many clients onto fewer server connections.

Practical backend example: Restructuring to shorten connection hold time:

```
// before: connection held during the HTTP call and the report rendering
@Transactional
public Report generate(UUID id) {
    Data data = repository.load(id);
    Enrichment e = httpClient.fetch(id);           // remote latency inside the transaction
    return renderer.render(data, e);               // CPU work inside the transaction
}

// after: minimal transactional scope
public Report generate(UUID id) {
    Data data = txTemplate.execute(s -> repository.load(id)); // short transaction
    Enrichment e = httpClient.fetch(id);                       // outside
    return renderer.render(data, e);                           // outside
}
```

Common follow-ups:

- Why is a long idle transaction harmful? It holds locks and blocks vacuum, causing bloat and degrading performance for everyone.
- How do you size the pool? From target throughput and average transaction time, bounded by the database's `max_connections` across all replicas.
- What does a saturated pool look like? Rising pending threads, connection timeout exceptions, high latency with low CPU.

Mistakes to avoid: Wrapping whole request handlers in `@Transactional`; rendering or serialising inside a transaction; ignoring `REQUIRES_NEW` when sizing; unlimited statement timeouts.

Production perspective: Set `statement_timeout` and `idle_in_transaction_session_timeout` at the database level as a safety net. They turn an unbounded stall into a bounded, diagnosable error.

Related concepts covered: Connection hold time, Little's law, PgBouncer, vacuum and bloat, statement timeouts, pool metrics.

Q133. How do you detect duplicate rows from a join and fix the query?

Priority: Important

Why interviewers ask it: It is a realistic debugging scenario where the naive fix, `DISTINCT`, hides the real problem.

Interview-ready answer: Duplicates from a join mean the join matched more rows than expected — typically a one-to-many relationship where each parent row is repeated once per child. The diagnosis is to count rows before and after the join, or group by the parent key and look for counts greater than one. The fix depends on intent: if I only need existence, use EXISTS instead of joining; if I need an aggregate, aggregate in a subquery or use COUNT(DISTINCT ...); if I need one specific child, use a lateral join with LIMIT 1. SELECT DISTINCT can produce the right answer but it masks the cause, costs a sort or hash, and will silently give wrong sums if I later add an aggregate.

In-depth explanation: The failure is worst with aggregates: joining orders to both lines and payments multiplies rows and inflates SUM(order.total) by the number of payment rows. Aggregating each side separately in subqueries, or using FILTER-based conditional aggregates on a single join, avoids the fan-out. Another frequent cause is a join condition missing a tenant or version column, so rows match across partitions that should be isolated. Checking cardinality assumptions explicitly — "this join should not change the row count" — is a good habit; you can verify it in a test with a row-count assertion.

Practical backend example: Diagnosing the fan-out and fixing it properly:

```
-- symptom: revenue looks inflated
SELECT o.id, sum(o.total_cents)
FROM "order" o
JOIN payment p ON p.order_id = o.id      -- 3 payments -> order counted 3 times
GROUP BY o.id;

-- diagnose: which parents multiply?
SELECT o.id, count(*) FROM "order" o JOIN payment p ON p.order_id = o.id
GROUP BY o.id HAVING count(*) > 1 ORDER BY 2 DESC LIMIT 10;

-- fix: aggregate each side independently, no fan-out
SELECT o.id, o.total_cents,
       coalesce(p.paid_cents, 0) AS paid_cents
FROM "order" o
LEFT JOIN (SELECT order_id, sum(amount_cents) AS paid_cents FROM payment GROUP BY order_id) p
ON p.order_id = o.id;
```

Common follow-ups:

- Does DISTINCT fix the root cause? It removes duplicate rows but not the wasted work, and it will not fix inflated aggregates.
- How do you count children without duplicating parents? Aggregate in a subquery, or use COUNT(DISTINCT child.id).
- What if two one-to-many joins are needed? Aggregate each in its own subquery; joining both multiplies their cardinalities.

Mistakes to avoid: Reaching for DISTINCT first; summing parent columns across a fan-out; forgetting tenant columns in join conditions; assuming the ORM's distinct flag solves the database-side cost.

Production perspective: Inflated aggregates are dangerous because they look plausible on a dashboard. Add assertions on invariants — total paid never exceeds total billed — and reconcile against an independent calculation periodically.

Related concepts covered: Join cardinality, pre-aggregation, COUNT DISTINCT, correctness testing of reports, tenant isolation.

Q134. How would you store and query audit history?

Priority: Bonus

Why interviewers ask it: Auditing is a common requirement that exercises schema design, transactions and query patterns together.

Interview-ready answer: The simplest robust design is an append-only history table per audited entity, holding the entity ID, the changed fields or a full snapshot, the actor, the reason and a timestamp, written in the same transaction as the change so history and state cannot diverge. For "what did this row look like on a date" queries I store validity ranges or query by timestamp with a DISTINCT ON per entity. Triggers can populate history automatically and cannot be bypassed by application bugs, but they hide logic from the codebase and cannot record application-level context such as the user. Hibernate Envers automates entity auditing if the JPA-centric model fits. The choice depends on

whether completeness or context matters more.

In-depth explanation: Storage grows quickly, so decide retention and partitioning early — monthly partitions with a drop policy are far cheaper than deleting rows. If the audit trail is a compliance artefact it must be tamper-evident: append-only permissions, no updates, and possibly hash chaining. Application-written history can capture intent — who, why, from which request — which triggers cannot see unless you push the context into a session variable. Query patterns matter for indexing: (entity_id, changed_at DESC) supports the timeline view, and a GIN index on a JSONB diff column supports "which changes touched this field". PostgreSQL range types with an exclusion constraint can model non-overlapping validity periods for temporal tables.

Practical backend example: Append-only history written in the same transaction:

```
CREATE TABLE order_history (
  id          bigserial PRIMARY KEY,
  order_id    uuid          NOT NULL,
  changed_at  timestampz    NOT NULL DEFAULT now(),
  changed_by  text          NOT NULL,
  reason      text,
  snapshot    jsonb         NOT NULL
) PARTITION BY RANGE (changed_at);
CREATE INDEX idx_order_history_entity ON order_history (order_id, changed_at DESC);
```

```
@Transactional // history commits with the change, atomically
public void cancel(UUID orderId, String reason, String actor) {
    Order order = orderRepository.findById(orderId).orElseThrow();
    order.cancel(reason);
    historyRepository.append(orderId, actor, reason, snapshotJson(order));
}
```

Common follow-ups:

- Should history updates share the transaction? Yes; otherwise a failure can leave state and history inconsistent.
- Triggers or application code? Triggers are unby-passable but lack application context; application code captures intent but can be skipped by a stray query.
- How do you reconstruct state at a point in time? Take the latest history row per entity at or before the timestamp, typically with DISTINCT ON or a window function.

Mistakes to avoid: Writing audit rows after commit in a separate transaction; unbounded growth with no partitioning or retention; storing only "updated" without the actual values; allowing updates to audit rows.

Production perspective: Audit tables often become the largest tables in the database and can slow backups and vacuum. Partition by time, archive cold partitions, and keep the hot index narrow.

Related concepts covered: Append-only design, table partitioning, temporal queries, Hibernate Envers, retention policy, tamper evidence.

Q135. When would you choose SQL over JPA for a complex report?

Priority: Important

Why interviewers ask it: It rewards tool choice based on the problem rather than loyalty to an ORM.

Interview-ready answer: JPA is designed for loading and mutating object graphs in a transaction; reporting is a different workload — set-based aggregation, window functions, grouping sets, CTEs — that SQL expresses far better and executes far more efficiently. When a report joins several tables, aggregates millions of rows and returns a few hundred, I write SQL and map to a projection, because materialising entities would be both slower and pointless. The two coexist happily: JPA for the transactional domain, SQL (native queries, JdbcTemplate or jOOQ) for read models. The deciding factors are expressiveness, data volume, and whether the result is a report or a mutable aggregate.

In-depth explanation: Entity hydration costs memory and CPU per row: building objects, populating the persistence context, taking dirty-check snapshots. For a report that only reads, all of that is waste. SQL also gives access to features JPQL simply cannot express — GROUPING SETS, PERCENTILE_CONT, lateral joins, recursive CTEs, FILTER aggregates. The counter-arguments are portability, which for most teams is theoretical, and losing compile-time safety, which tools like jOOQ restore. For very heavy reporting, the right architecture may be different altogether: a read replica, a materialised view refreshed on a schedule, or a separate analytical store. Keep report queries out of the

transactional connection pool so a slow analyst query cannot affect checkout.

Practical backend example: A reporting query that JPQL cannot express:

```
SELECT date_trunc('week', o.created_at) AS week,
       count(*) AS orders,
       percentile_cont(0.5) WITHIN GROUP (ORDER BY o.total_cents) AS median_cents,
       sum(o.total_cents) FILTER (WHERE c.tier = 'GOLD') AS gold_revenue_cents,
       sum(sum(o.total_cents)) OVER (ORDER BY date_trunc('week', o.created_at)) AS running_total_cents
FROM "order" o
JOIN customer c ON c.id = o.customer_id
WHERE o.status = 'PAID' AND o.created_at >= now() - interval '90 days'
GROUP BY 1
ORDER BY 1;
```

```
// map straight to a record via JdbcTemplate or a native query projection - no entities involved
List<WeeklyRevenue> rows = jdbcTemplate.query(SQL, (rs, i) -> new WeeklyRevenue(
    rs.getObject("week", OffsetDateTime.class), rs.getLong("orders"),
    rs.getLong("median_cents"), rs.getLong("running_total_cents")));
```

Common follow-ups:

- Can both approaches coexist? Yes; use JPA for the transactional model and SQL for read models — this is effectively CQRS at the query level.
- What about portability? Native SQL ties you to a dialect; in practice most teams never change database engines, so weigh it honestly.
- When does a materialised view help? When the report is expensive, read often and tolerant of some staleness.

Mistakes to avoid: Loading entities to compute aggregates; building dynamic SQL by string concatenation with user input; running heavy reports on the primary transactional pool; recreating SQL features laboriously in Java.

Production perspective: Reporting queries should have their own timeouts, their own pool, and ideally their own replica. The most common reporting incident is an analyst query saturating the primary database during peak traffic.

Related concepts covered: Read models and CQRS, window functions, materialised views, read replicas, jOOQ and JdbcTemplate, workload isolation.

Chapter 6. HTTP, REST APIs, and microservices

Examples use Spring Boot 3 with RestClient/WebClient and Resilience4j where a resilience library is needed. HTTP semantics are quoted from the RFC definitions of the methods and status codes; anything that is convention rather than specification is labelled as such.

Q136. How do GET, POST, PUT, PATCH and DELETE differ?

Priority: Must Know

Why interviewers ask it: Method semantics drive caching, retries and proxy behaviour, so getting them wrong has real operational consequences.

Interview-ready answer: GET is safe and idempotent: it must not change state, and caches and crawlers may repeat it freely. POST is neither safe nor idempotent; it is the general "process this" method and is the right choice for creating a resource whose identifier the server assigns. PUT replaces a resource at a known URI and is idempotent — sending it twice leaves the same state. PATCH applies a partial modification and is not idempotent in general, although a well-designed patch usually is. DELETE is idempotent in effect: the first call removes the resource and later calls find nothing to remove, which typically returns 404 or 204. Idempotency is the property that determines whether a client may safely retry after a timeout.

In-depth explanation: Safety and idempotency are contracts you must actually honour: a GET with side effects breaks caches and prefetchers, and an idempotent PUT that appends rather than replaces will corrupt data on retry. PATCH has two common formats — JSON Merge Patch (RFC 7386), where null means "remove", and JSON Patch (RFC 6902), an operation list — and picking one explicitly avoids ambiguity about partial updates. For POST creation, return 201 with a Location header; for asynchronous processing, return 202 with a status URL. Note that idempotency in HTTP is about the resulting state, not the response body: two DELETE calls may return different statuses while leaving the same state. Requests that are not naturally idempotent can be made retry-safe with an idempotency key, which is the standard approach for payments.

Practical backend example: Method choice mapped to real endpoints:

```
@GetMapping("/api/orders/{id}")           // safe, cacheable, no side effects
public OrderResponse get(@PathVariable UUID id) { ... }

@PostMapping("/api/orders")               // server assigns the ID -> 201 + Location
public ResponseEntity<OrderResponse> create(@Valid @RequestBody CreateOrder body) { ... }

@PutMapping("/api/orders/{id}/shipping-address") // full replacement, idempotent
public ShippingAddress replaceAddress(@PathVariable UUID id, @Valid @RequestBody ShippingAddress a) {
    ... }

@PatchMapping(value = "/api/orders/{id}", consumes = "application/merge-patch+json")
public OrderResponse patch(@PathVariable UUID id, @RequestBody JsonMergePatch patch) { ... }

@DeleteMapping("/api/orders/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)      // 204; repeated calls leave the same state
public void delete(@PathVariable UUID id) { ... }
```

Common follow-ups:

- Is DELETE always repeat-safe? The resulting state is the same, though the status code may differ between the first and later calls.
- Can POST be made idempotent? Not by the method, but yes by an idempotency key stored with a unique constraint.
- When is PUT better than PATCH? When the client sends the full representation and you want replace semantics with natural idempotency.

Mistakes to avoid: Using GET for state changes because it is easy to call from a browser; POST /getOrders-style RPC over HTTP without a reason; PUT that merges instead of replacing; assuming PATCH is automatically idempotent.

Production perspective: Proxies, CDNs, service meshes and client libraries make retry decisions based on method semantics. If your POST is not idempotent and a gateway retries it after a timeout, you get duplicate orders — which is why idempotency keys exist.

Related concepts covered: Safety and idempotency, JSON Patch formats, 201 versus 202, idempotency keys, retry behaviour of intermediaries.

Q137. Which HTTP status codes should an API use for common outcomes?

Priority: Must Know

Why interviewers ask it: Status codes are the machine-readable part of your contract and drive client retries, alerts and dashboards.

Interview-ready answer: 200 for a successful read or update with a body, 201 with a Location header for creation, 202 when work is accepted for asynchronous processing, 204 for success with no body. On the client side: 400 for malformed input, 401 when authentication is missing or invalid, 403 when the caller is authenticated but not allowed, 404 for a missing resource, 409 for a state conflict such as a duplicate or a version mismatch, 422 when the payload is well-formed but semantically invalid, 429 for rate limiting with a Retry-After header. On the server side: 500 for unexpected errors, 502/504 for upstream failures and timeouts, and 503 when the service is deliberately unavailable or overloaded. The distinction that matters most is 4xx meaning "do not retry unchanged" versus 5xx and 429 meaning "retrying may help".

In-depth explanation: 401 versus 403 is precise: 401 means the request lacks valid credentials — and the response should include WWW-Authenticate — while 403 means the identity is known and still not permitted. 404 versus 403 also has a security dimension: returning 404 for resources the caller may not see avoids leaking their existence, which is often preferable for multi-tenant APIs. 409 versus 422 is conventional rather than strictly specified; the important thing is to be consistent across your API and document it. Never return 200 with an error body: it defeats generic monitoring, client error handling and gateway retry policies. Include Retry-After on 429 and 503 so clients back off intelligently rather than hammering.

Practical backend example: Mapping domain outcomes to codes in one place:

```
@RestControllerAdvice
class StatusMapping {
    @ExceptionHandler(OrderNotFoundException.class)
    ProblemDetail notFound(OrderNotFoundException e) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, e.getMessage()); // 404
    }
    @ExceptionHandler(DuplicateOrderException.class)
    ProblemDetail conflict(DuplicateOrderException e) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT, e.getMessage()); // 409
    }
    @ExceptionHandler(RateLimitExceededException.class)
    ResponseEntity<ProblemDetail> throttled(RateLimitExceededException e) {
        return ResponseEntity.status(HttpStatus.TOO_MANY_REQUESTS)
            .header(HttpHeaders.RETRY_AFTER, String.valueOf(e.retryAfterSeconds())) // 429
            .body(ProblemDetail.forStatus(HttpStatus.TOO_MANY_REQUESTS));
    }
}
```

Common follow-ups:

- When do you return 409 versus 422? 409 for a conflict with current state (duplicate, stale version); 422 for a syntactically valid payload that violates semantic rules. Be consistent and document the choice.
- 401 or 403 for an expired token? 401, because the credentials are no longer valid; include WWW-Authenticate.
- Should a missing resource the user cannot access return 404? Often yes, to avoid confirming that it exists.

Mistakes to avoid: 200 with {"error": ...}; 500 for validation failures; 403 for unauthenticated requests; omitting Retry-After on throttling responses; inventing non-standard codes.

Production perspective: Dashboards and alerts are usually built on status classes. If validation errors are reported as 500, every client bug pages your on-call engineer; if real failures are reported as 200, nothing pages anyone.

Related concepts covered: Retry semantics, Retry-After, information disclosure, ProblemDetail, monitoring by status class.

Q138. How would you design a resource-oriented order API?

Priority: Must Know

Why interviewers ask it: It shows whether you can translate a business process into a clean, evolvable HTTP interface.

Interview-ready answer: I model nouns as resources with stable URIs and use methods for the verbs: `/orders` for the collection, `/orders/{id}` for the item, `/orders/{id}/lines` for sub-resources. State transitions that are not simple updates become sub-resources rather than verbs in the path — `POST /orders/{id}/cancellation` instead of `POST /cancelOrder`. Representations are DTOs, not entities, with explicit field names, ISO-8601 timestamps in UTC and monetary amounts as integer minor units with a currency code. I design for pagination and filtering from the start, return 201 with a Location on creation, and keep the contract additive so it can evolve. Where a process genuinely does not map onto a resource, a well-named action endpoint is better than a contorted abstraction.

In-depth explanation: Consistency matters more than REST purism: plural collection names, the same pagination parameters everywhere, the same error shape and the same date format across every endpoint. Nesting should stop at one or two levels — `/customers/{id}/orders` is useful for scoping, but deep nesting makes URIs brittle when relationships change. Expose the identifiers clients need and avoid leaking internal sequence IDs when they can be enumerated; UUIDs or opaque IDs reduce that risk. Think about concurrency in the design: an ETag plus If-Match gives optimistic concurrency at the HTTP level. Also decide early whether partial responses (sparse fieldsets) or embedded sub-resources are supported, because adding them later changes response shapes.

Practical backend example: A coherent resource layout:

GET	<code>/api/orders?status=PAID&customerId=...&page=...&size=...</code>	list with filters
POST	<code>/api/orders</code>	create -> 201 + Location
GET	<code>/api/orders/{orderId}</code>	read one
PATCH	<code>/api/orders/{orderId}</code>	partial update
POST	<code>/api/orders/{orderId}/cancellation</code>	state transition as a sub-resource
GET	<code>/api/orders/{orderId}/lines</code>	sub-collection
PUT	<code>/api/orders/{orderId}/shipping-address</code>	replace a component

```
{
  "id": "1f0b2a5e-9c2d-4f7a-8f0e-8a1b2c3d4e5f",
  "status": "PAID",
  "createdAt": "2026-03-14T09:21:00Z",
  "total": { "amountMinor": 129900, "currency": "EUR" },
  "lines": [ { "sku": "ABC-000001", "quantity": 2 } ]
}
```

Common follow-ups:

- Should verbs appear in URLs? Prefer resources and sub-resources; a clearly named action endpoint is acceptable when no resource fits.
- How do you represent money? Integer minor units plus a currency code, never a floating-point number.
- How deep should nesting go? One or two levels; beyond that, use top-level resources with filters.

Mistakes to avoid: RPC-style endpoints such as `/api/doOrderStuff`; exposing database column names; inconsistent date formats; returning different error shapes per endpoint; sequential integer IDs that let clients enumerate other customers' data.

Production perspective: An API is a long-lived contract; clients you do not control will depend on details you considered incidental. Write the OpenAPI specification alongside the implementation and treat breaking changes as a release process, not a code change.

Related concepts covered: Resource modelling, sub-resources for transitions, representation design, OpenAPI, opaque identifiers, ETags.

Q139. How should validation and error responses be structured?

Priority: Must Know

Why interviewers ask it: Error design is where APIs most often become inconsistent, and clients suffer for years afterwards.

Interview-ready answer: One error shape for the whole API. In Spring Boot 3 I use `ProblemDetail`, the standard problem-details format defined by RFC 7807 and updated by RFC 9457, which gives type, title, status, detail and instance, and I add a machine-readable error code, a list of field errors for validation failures, and a correlation ID for support. Clients should branch on the status code and the stable error code, never on prose. Messages must be safe: no stack traces, no SQL, no internal hostnames, and no echoing of sensitive input values. Validation failures return 400 with every field error listed at once, so a form can display them all rather than one per round trip.

In-depth explanation: A stable code field matters because title and detail are human text that may be reworded or localised. Locale handling belongs in the client where possible; if the server localises, it should still keep the code stable. Distinguish client-visible detail from internal detail: log the exception with full context and the correlation ID, then return a short safe message with that same ID. For bulk operations, decide and document whether the response is all-or-nothing or a per-item result list — 207-style multi-status semantics need explicit design. Finally, version the error contract alongside the API: adding a field is safe, changing the meaning of code values is not.

Practical backend example: A consistent, safe error payload:

```
@ExceptionHandler(MethodArgumentNotValidException.class)
ProblemDetail onValidation(MethodArgumentNotValidException ex) {
    ProblemDetail p = ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);
    p.setType(URI.create("https://api.example.com/problems/validation-failed"));
    p.setTitle("Validation failed");
    p.setProperty("code", "VALIDATION_FAILED"); // stable, machine-readable
    p.setProperty("errors", ex.getBindingResult().getFieldErrors().stream()
        .map(fe -> Map.of("field", fe.getField(), "message", fe.getDefaultMessage()))
        .toList());
    p.setProperty("correlationId", MDC.get("correlationId"));
    return p;
}

@ExceptionHandler(Exception.class)
ProblemDetail onUnexpected(Exception ex) {
    String id = MDC.get("correlationId");
    log.error("unhandled error correlationId={}", id, ex); // full detail stays in the logs
    ProblemDetail p = ProblemDetail.forStatusAndDetail(HttpStatus.INTERNAL_SERVER_ERROR,
        "Unexpected error. Quote the correlation id when contacting support.");
    p.setProperty("correlationId", id);
    return p; // nothing internal leaks
}
```

Common follow-ups:

- How do you avoid leaking internals? Log the detail server-side and return a generic message plus a correlation ID.
- Should errors be localised? Keep a stable machine-readable code; localise only human-facing text, ideally in the client.
- One error or all errors for validation? Return all field errors in one response.

Mistakes to avoid: Different error shapes per controller; returning exception class names; embedding SQL or stack traces; using HTTP 200 with an error object; unstable error codes.

Production perspective: A correlation ID that appears in the response, the logs and the trace turns a vague user complaint into a two-minute investigation. Make it mandatory in every error path, including the security filter chain.

Related concepts covered: `ProblemDetail` (RFC 7807, updated by RFC 9457), error codes, correlation IDs, information disclosure, bulk operation semantics.

Q140. How would you implement pagination, filtering and stable sorting?

Priority: Must Know

Why interviewers ask it: List endpoints are where APIs meet database performance, and unstable paging is a classic bug.

Interview-ready answer: I expose a bounded page size with a server-side maximum, a documented default sort, and a tie-breaker on a unique column so ordering is deterministic. For shallow pages, offset-based paging is acceptable and simple; for deep or high-volume access I use cursor-based paging, where the cursor encodes the last seen sort key. Filters map to indexed columns and are validated against an allowlist so clients cannot sort or filter by arbitrary fields — that is both a performance and an injection concern. The response includes the items and paging metadata:

next cursor or page number, page size, and a total only if it can be produced cheaply.

In-depth explanation: Unstable sorting is the subtle failure: sorting only by `created_at` when several rows share a timestamp lets rows move between pages, so users see duplicates or miss items. Adding the primary key as a final sort key fixes it. Total counts are expensive on large filtered sets; many APIs return `hasMore` instead, or an approximate count. Allowlisting sort fields prevents a client from sorting by an unindexed column and triggering a full table sort. Cursors should be opaque — base64 of the keyset values — so the implementation can change. Also cap size: without a maximum, one client can request a million rows and exhaust memory on both sides.

Practical backend example: A cursor-paged endpoint with validated inputs:

```
private static final Set<String> SORTABLE = Set.of("createdAt", "totalCents");

@GetMapping("/api/orders")
public CursorPage<OrderSummary> list(
    @RequestParam(required = false) OrderStatus status,
    @RequestParam(defaultValue = "createdAt") String sort,
    @RequestParam(required = false) String cursor,
    @RequestParam(defaultValue = "20") @Min(1) @Max(100) int size) {    // hard cap
    if (!SORTABLE.contains(sort)) throw new InvalidSortException(sort);    // allowlist
    Cursor decoded = Cursor.decode(cursor);    // opaque to clients
    List<OrderSummary> rows = orderQueryService.page(status, sort, decoded, size + 1);
    boolean hasMore = rows.size() > size;    // avoids a COUNT query
    return CursorPage.of(rows.subList(0, Math.min(size, rows.size())), hasMore);
}
```

Common follow-ups:

- What happens if rows are inserted between pages? With offsets, items shift and can duplicate or disappear; cursors are stable.
- How do you return a total without a slow count? Return `hasMore`, or use an approximate count from statistics when an estimate suffices.
- Why allowlist sort fields? To keep queries on indexed columns and to avoid dynamic SQL built from user input.

Mistakes to avoid: Unbounded size; sorting by a non-unique column only; `COUNT(*)` on every request; exposing raw offsets in cursors; allowing arbitrary filter expressions from clients.

Production perspective: List endpoints are usually the highest-traffic part of an API and the easiest to regress. Load test them with production-like data volumes, and monitor page-size distribution — a client requesting size 100 every time may need a bulk export instead.

Related concepts covered: Keyset pagination, deterministic ordering, allowlists, count query cost, opaque cursors, rate limiting interaction.

Q141. How does idempotency make payment creation retry-safe?

Priority: Must Know

Why interviewers ask it: Network timeouts are inevitable, and duplicate payments are the textbook consequence of ignoring them.

Interview-ready answer: The client generates a unique idempotency key per logical operation and sends it as a header. The server stores that key with a unique constraint in the same transaction that creates the payment. If the same key arrives again, the insert conflicts and the server returns the original result instead of creating a second payment. The key must be tied to the request content so a different payload with the same key is rejected rather than silently returning the wrong result, and keys need a retention period long enough to cover client retry windows. This turns a non-idempotent POST into a retry-safe operation without changing HTTP semantics.

In-depth explanation: The critical detail is atomicity: recording the key and performing the effect must be in one transaction, otherwise a crash between them reopens the duplicate window. Concurrent retries need handling too — two simultaneous requests with the same key should not both proceed; the unique constraint resolves that by failing one, which then waits for or returns the stored result. Storing a hash of the request body lets you return 409 or 422 when the same key is reused with different content, which catches client bugs. Decide the response for a replay: returning the original 201 response body is the friendliest and is what major payment APIs do. Finally, downstream calls need their own idempotency: pass the key to the payment provider so their system also deduplicates.

Practical backend example: Key storage, replay and conflicting reuse:

```
CREATE TABLE idempotency_record (
  key          text PRIMARY KEY,
  request_hash text          NOT NULL,
  response_body jsonb        NOT NULL,
  status_code  int           NOT NULL,
  created_at   timestampz    NOT NULL DEFAULT now()
);
```

```
@PostMapping("/api/payments")
public ResponseEntity<PaymentResponse> create(@RequestHeader("Idempotency-Key") String key,
                                              @Valid @RequestBody CreatePayment body) {
    String hash = Hashing.sha256(body);
    Optional<IdempotencyRecord> existing = idempotencyRepository.find(key);
    if (existing.isPresent()) {
        if (!existing.get().requestHash().equals(hash)) {
            throw new IdempotencyKeyReusedException(key);           // 409: same key, different payload
        }
        return existing.get().toResponse();                         // replay the original result
    }
    return paymentService.createAtomically(key, hash, body);       // insert key + payment in one tx
}
```

Common follow-ups:

- What if the same key arrives with a different body? Reject it (409); silently returning the old result would hide a client bug.
- How long should keys be kept? Long enough to cover client retries and reconciliation — commonly 24 hours to a few days — then purge.
- Who generates the key? The client, once per logical operation, and it must reuse the same key for retries of that operation.

Mistakes to avoid: Generating the key on the server; storing the key after performing the effect; no unique constraint; treating a duplicate as an error rather than returning the original outcome; unbounded key tables.

Production perspective: Idempotency keys are what make automatic retries — by clients, gateways and service meshes — safe. Combine them with an outbox for downstream effects, and reconcile with the provider daily to catch the cases where you never learned the outcome.

Related concepts covered: Unique constraints, exactly-once effects, retry safety, request hashing, key retention, downstream idempotency.

Q142. What are ETag and Cache-Control used for?

Priority: Important

Why interviewers ask it: HTTP caching is free performance that most backend developers under-use, and ETags also provide concurrency control.

Interview-ready answer: Cache-Control tells clients and intermediaries whether and how long a response may be reused — max-age, no-store for sensitive data, private for per-user responses, must-revalidate for freshness. ETag is a validator: the server returns a version token for a representation, and the client sends it back with If-None-Match; if nothing changed the server responds 304 with no body, saving bandwidth. The same mechanism works for writes: If-Match with the current ETag gives optimistic concurrency, so a PUT fails with 412 if someone else modified the resource in the meantime. Last-Modified with If-Modified-Since is the coarser, timestamp-based alternative.

In-depth explanation: Strong ETags mean byte-identical representations; weak ETags (w/"...") mean semantically equivalent, which is usually what you want when the payload includes a rendering timestamp. Generating an ETag from an entity version column is cheap and stable; hashing the serialised body is simpler but costs the serialisation work you were trying to avoid. Caching rules are easy to get wrong for authenticated responses: mark them private and no-store when they contain personal data, and use Vary: Accept, Authorization so a shared cache does not serve one user's data to another. Note that 304 responses still require the request to reach the server unless a cache is allowed to serve without revalidation — ETags save bandwidth and rendering, not round trips.

Practical backend example: Conditional read plus optimistic write:

```

@GetMapping("/api/products/{id}")
public ResponseEntity<ProductResponse> get(@PathVariable UUID id,
                                           @RequestHeader(value = "If-None-Match", required = false)
                                           String inm) {
    Product product = productService.get(id);
    String etag = "\"v\" + product.getVersion() + \""; // from the @Version column
    if (etag.equals(inm)) {
        return ResponseEntity.status(HttpStatus.NOT_MODIFIED).eTag(etag).build(); // 304, no body
    }
    return ResponseEntity.ok()
        .eTag(etag)
        .cacheControl(CacheControl.maxAge(Duration.ofMinutes(5)).cachePublic())
        .body(ProductResponse.from(product));
}

@PutMapping("/api/products/{id}")
public ResponseEntity<ProductResponse> update(@PathVariable UUID id,
                                              @RequestHeader("If-Match") String ifMatch,
                                              @Valid @RequestBody UpdateProduct body) {
    long expectedVersion = parseVersion(ifMatch);
    return productService.update(id, expectedVersion, body) // throws -> 412 Precondition Failed
        .map(p -> ResponseEntity.ok(ProductResponse.from(p)))
        .orElseThrow(() -> new PreconditionFailedException(id));
}

```

Common follow-ups:

- What does 304 mean? Not Modified — the cached representation is still valid, and no body is sent.
- How do ETags help with concurrency? If-Match lets the server reject an update based on a stale version with 412.
- What should you never cache? Anything personal or authorised for a specific user in a shared cache; use private/no-store and Vary.

Mistakes to avoid: Caching authenticated responses publicly; generating ETags by hashing a payload that includes a timestamp; forgetting Vary; using no-cache when you mean no-store.

Production perspective: For read-heavy APIs behind a CDN, correct cache headers reduce origin load dramatically. Measure cache hit ratio and 304 rates; a caching layer that never hits is just latency.

Related concepts covered: Conditional requests, optimistic concurrency over HTTP, Vary headers, CDN behaviour, weak versus strong validators.

Q143. How do timeouts, retries and backoff interact?

Priority: Must Know

Why interviewers ask it: Naive retries turn a small downstream slowdown into a self-inflicted outage.

Interview-ready answer: Every outbound call needs a connect timeout and a read timeout, both shorter than the caller's own deadline, otherwise threads pile up waiting. Retries should only apply to safe or idempotent operations and to failures that could plausibly succeed on a second attempt — connection errors, 503, 429, and timeouts on idempotent requests. They must use exponential backoff with jitter to avoid synchronised retry storms, and a retry budget or attempt cap so the total work is bounded. Crucially, retries multiply load exactly when the downstream is struggling, so they belong behind a circuit breaker. I also propagate a deadline so that an inner retry never exceeds the time the caller is still willing to wait.

In-depth explanation: The mathematics matter: three attempts per call across three service hops is up to 27 requests for one user action. A retry budget — for example, retries may be at most 10% of requests — prevents amplification. Jitter is not optional; without it, all clients that failed at the same moment retry at the same moment. Timeouts should be derived from observed latency percentiles rather than guessed round numbers, and they should be tighter than the upstream deadline minus the time already spent. Non-idempotent operations can still be retried if they carry an idempotency key. Finally, distinguish fast failures from slow ones: a connection refused can be retried almost immediately, while a timeout suggests overload and deserves a longer wait.

Practical backend example: Bounded retries with jitter and a deadline:

```

@Bean
RestClient paymentsClient(RestClient.Builder builder) {
    var factory = new SimpleClientHttpRequestFactory();
    factory.setConnectTimeout(Duration.ofMillis(500));    // always set both
    factory.setReadTimeout(Duration.ofMillis(1500));
    return builder.baseUrl(paymentsBaseUrl).requestFactory(factory).build();
}

@Bean
RetryRegistry retryRegistry() {
    return RetryRegistry.of(RetryConfig.custom()
        .maxAttempts(3)
        .intervalFunction(IntervalFunction.ofExponentialRandomBackoff(
            Duration.ofMillis(100), 2.0, 0.5))    // exponential + jitter
        .retryExceptions(IOException.class, TimeoutException.class)
        .retryOnResult(r -> r instanceof ResponseEntity<?> re
            && (re.getStatusCode().value() == 503 || re.getStatusCode().value() == 429))
        .failAfterMaxAttempts(true)
        .build());
}

```

Common follow-ups:

- Which requests are safe to retry? Idempotent ones, or non-idempotent ones carrying an idempotency key.
- Why jitter? To de-synchronise clients that all failed at the same instant and would otherwise retry together.
- How do you stop retry amplification? A retry budget, a circuit breaker, and deadlines propagated across hops.

Mistakes to avoid: Retrying 4xx errors; infinite retries; the same timeout at every layer so nothing fails fast; retrying inside a database transaction; no jitter.

Production perspective: Instrument attempts, retry counts and timeout counts separately from errors. A retry spike is an early warning, and a retry rate that exceeds your budget during an incident is usually making the incident worse.

Related concepts covered: Deadlines, exponential backoff with jitter, retry budgets, circuit breakers, idempotency, timeout tuning from percentiles.

Q144. When should you add a circuit breaker or bulkhead?

Priority: Important

Why interviewers ask it: Resilience patterns are frequently name-dropped; the interviewer wants to know when each is warranted and what it costs.

Interview-ready answer: A circuit breaker protects a caller from a failing dependency: after a failure-rate threshold it opens and fails fast for a cool-down period, then allows trial calls in a half-open state before closing again. That preserves the caller's threads and gives the downstream room to recover. A bulkhead limits how much concurrency any one dependency can consume — a semaphore or a dedicated thread pool — so a slow dependency cannot starve the whole service. I add them for remote dependencies that are slow or unreliable and where a fallback or degraded response exists. They are not substitutes for timeouts: without a timeout, calls never fail, so the breaker never trips.

In-depth explanation: Configuration is where these go wrong. Thresholds based on too few calls flap; a minimum call count and a sliding window make the decision stable. The fallback must be genuinely acceptable — a cached value, an empty list, a "temporarily unavailable" field — because a fallback that fabricates data is worse than an error. Opening a breaker on a non-idempotent write is risky if the request may have succeeded; treat the outcome as unknown and reconcile. Bulkheads are often the simpler and more effective tool: a semaphore of 10 permits per dependency prevents resource monopolisation with far less configuration. Combine them with health-based routing at the infrastructure layer where available.

Practical backend example: Breaker plus bulkhead with a meaningful fallback:

```

@Bean
CircuitBreakerConfig recommendationsBreaker() {
    return CircuitBreakerConfig.custom()
        .slidingWindowType(SlidingWindowType.COUNT_BASED)
        .slidingWindowSize(50)
        .minimumNumberOfCalls(20)           // avoid flapping on tiny samples
        .failureRateThreshold(50)
        .waitDurationInOpenState(Duration.ofSeconds(10))
        .permittedNumberOfCallsInHalfOpenState(5)
        .build();
}

@CircuitBreaker(name = "recommendations", fallbackMethod = "popularFallback")
@Bulkhead(name = "recommendations", type = Bulkhead.Type.SEMAPHORE) // cap concurrency
public List<Product> recommend(UUID userId) { return recommendationClient.fetch(userId); }

private List<Product> popularFallback(UUID userId, Throwable t) {
    return popularProductsCache.get(); // degraded but honest: still useful content
}

```

Common follow-ups:

- Can a breaker replace a timeout? No; without timeouts, calls hang and failures are never counted.
- What is a good fallback? Cached or default data, or an explicit partial response — never fabricated business data.
- When is a bulkhead enough on its own? When the dependency is optional and you mainly need to protect shared capacity.

Mistakes to avoid: Breakers on every internal call regardless of need; thresholds that trip on three requests; fallbacks that silently return wrong data; forgetting that an open breaker on a write leaves an unknown outcome.

Production perspective: Export breaker state transitions as events and alert on prolonged open states — that is a dependency outage, not a blip. Test the fallback path deliberately, because untested fallbacks are usually broken when finally needed.

Related concepts covered: Failure isolation, semaphore bulkheads, fallback design, half-open probing, health-based routing.

Q145. How do synchronous calls compare with asynchronous messaging?

Priority: Must Know

Why interviewers ask it: Choosing the interaction style is an architecture decision with consequences for coupling, latency and consistency.

Interview-ready answer: A synchronous call gives an immediate answer and simple reasoning: the caller knows the outcome. The price is temporal coupling — if the callee is down or slow, the caller is affected — and cascading failures across a call chain. Asynchronous messaging decouples availability: the producer writes to a broker and continues, and the consumer processes when it can, which absorbs spikes and survives short outages. The price is eventual consistency, duplicate delivery to handle, out-of-order messages, and harder end-to-end debugging. My rule: use synchronous calls when the caller needs the result to continue — a price quote, an authorisation check — and messaging for side effects and notifications that can complete later.

In-depth explanation: Read operations that block a user response are naturally synchronous. Writes that trigger downstream work — send an email, update a search index, notify a partner — are naturally asynchronous, and doing them synchronously couples your checkout availability to an email provider. A hybrid pattern is common: accept the request synchronously, return 202 with a status URL, and complete the work asynchronously. Whatever the style, the client experience must be designed: what does the user see while the work is pending, and how do they learn the outcome? Messaging also shifts operational burden to the broker: partitions, consumer lag, dead-letter queues and replay all become things you must monitor.

Practical backend example: Synchronous where the answer is needed, asynchronous for the rest:

```
@Transactional
public OrderResponse place(PlaceOrderCommand cmd) {
    AuthorizationResult auth = paymentClient.authorize(cmd.payment()); // synchronous: needed now
    if (!auth.approved()) throw new PaymentDeclinedException(auth.reason());

    Order order = orderRepository.save(Order.from(cmd, auth));
    outbox.append(new OrderPlaced(order.id())); // asynchronous: email, invoicing, analytics
    return OrderResponse.from(order); // fast response, side effects follow
}
```

Common follow-ups:

- How do clients see the result of async work? Polling a status endpoint, a webhook, or a push notification; design it explicitly.
- Does messaging remove failure? No; it moves it to consumer lag, retries and dead letters, which need monitoring.
- What about request/reply over a broker? Possible, but it reintroduces temporal coupling with added complexity — usually prefer HTTP.

Mistakes to avoid: Making checkout depend synchronously on non-essential services; using messaging for something the user is waiting for without status feedback; assuming messaging guarantees ordering or exactly-once delivery.

Production perspective: Every synchronous dependency multiplies your availability: three dependencies at 99.9% give roughly 99.7% combined. Removing non-essential synchronous calls is often the cheapest reliability improvement available.

Related concepts covered: Temporal coupling, availability multiplication, 202 Accepted flows, eventual consistency, consumer lag monitoring.

Q146. How would you version an API without breaking clients?

Priority: Important

Why interviewers ask it: Versioning strategy shows whether you think about clients you cannot deploy.

Interview-ready answer: The cheapest version is the one you never need: design for additive change, so adding an optional field or a new endpoint does not break anyone, and require clients to ignore unknown fields. When a genuinely breaking change is unavoidable — removing a field, changing a type, changing semantics — I introduce a new version, most commonly as a URI prefix such as `/api/v2/orders` because it is explicit, easy to route and easy to log. Header or media-type versioning is more purist but harder to debug and cache. I then run both versions in parallel, publish a deprecation timeline with `Deprecation` and `Sunset` headers, measure usage per version, and remove the old one only when traffic has stopped.

In-depth explanation: Breaking versus non-breaking is worth being precise about. Non-breaking: adding an optional request field, adding a response field, adding an endpoint, adding an enum value only if clients tolerate unknown values. Breaking: removing or renaming fields, changing types or formats, tightening validation, changing default behaviour, adding a required request field, and — often overlooked — changing error codes or status semantics. Enum extension is the classic trap: clients that switch exhaustively on a value will fail when a new one appears, so document the tolerance requirement from day one. Internal service-to-service APIs can evolve faster with consumer-driven contract tests, while public APIs need long support windows. Whatever you choose, version the contract, not the internal code.

Practical backend example: Running two versions side by side with deprecation signalling:

```

@RestController
@RequestMapping("/api/v1/orders")
public class OrderControllerV1 {
    @GetMapping("/{id}")
    public ResponseEntity<OrderResponseV1> get(@PathVariable UUID id) {
        return ResponseEntity.ok()
            .header("Deprecation", "true")
            .header("Sunset", "Wed, 30 Sep 2026 23:59:59 GMT") // machine-readable timeline
            .header("Link", "</api/v2/orders>; rel=\"successor-version\"")
            .body(OrderResponseV1.from(orderService.get(id)));
    }
}

@RestController
@RequestMapping("/api/v2/orders") // v2: money as an object, status renamed
public class OrderControllerV2 { /* ... */ }

```

Common follow-ups:

- Is adding a field always safe? Adding an optional field is safe if clients ignore unknown fields; adding a required request field is breaking.
- URI or header versioning? URI is explicit and simple to operate; media-type versioning is cleaner in theory but harder to debug and cache.
- How do you know when to remove v1? Per-version usage metrics plus an announced sunset date.

Mistakes to avoid: Versioning every internal refactor; maintaining five versions indefinitely; changing semantics within a version; breaking clients by tightening validation without notice.

Production perspective: Track requests per version per client so deprecation is data-driven and you can contact the specific consumers still using v1. Contract tests against both versions prevent accidental regressions while they coexist.

Related concepts covered: Backward compatibility rules, deprecation headers, consumer-driven contract testing, enum evolution, client identification.

Q147. How do correlation IDs and trace context travel across services?

Priority: Important

Why interviewers ask it: Distributed debugging is impossible without propagated context, and this is a practical, testable skill.

Interview-ready answer: The entry point — gateway or first service — generates a trace ID if none is present and puts it in a standard header, today usually W3C traceparent, often alongside an application correlation ID. Every service reads it, adds it to the logging context (MDC), includes it in log lines and error responses, and forwards it on outbound calls, including message headers for asynchronous hops. In Spring Boot 3 this is handled by Micrometer Tracing with an exporter such as OpenTelemetry, which instruments the web layer, RestClient/WebClient and messaging automatically. The discipline required is to propagate context across thread boundaries — async executors and message consumers — and to clear it afterwards so pooled threads do not leak it.

In-depth explanation: A trace is a tree of spans; each span has a trace ID, a span ID and a parent. traceparent carries those, and baggage can carry a small amount of user-defined context — with caution, since baggage crosses trust boundaries and must never contain personal data. Sampling is the other practical concern: tracing every request is expensive at high volume, so head-based sampling with a percentage plus always-sample-on-error is typical. For logs to be joinable with traces, the trace ID must be in the log line, ideally in a structured JSON field. Asynchronous hops need explicit propagation: put the trace context in Kafka headers and restore it in the consumer, otherwise the trace ends at the producer.

Practical backend example: Structured logs carrying trace identifiers:

```
management:
  tracing:
    sampling:
      probability: 0.1          # sample 10% of traces; errors are typically always sampled
logging:
  pattern:
    level: "%5p [%${spring.application.name},%X{traceId:-},%X{spanId:-}]"
```

```
@Component
public class OrderEventProducer {
    private final KafkaTemplate<String, String> kafka;
    private final Propagator propagator;

    public void publish(OrderPlaced event) {
        ProducerRecord<String, String> record =
            new ProducerRecord<>("orders", event.orderId().toString(), json.write(event));
        propagator.inject(tracer.currentTraceContext().context(), record.headers(),
            (headers, key, value) -> headers.add(key, value.getBytes(UTF_8))); // async hop
        kafka.send(record);
    }
}
```

Common follow-ups:

- Why not log personal data alongside the trace? Logs are widely accessible and retained; identifiers should be opaque, and personal data needs minimisation and redaction.
- What happens on an async hop? Context must be injected into message headers and extracted by the consumer, or the trace breaks.
- How much should you sample? Enough to diagnose typical behaviour — often 1–10% — with errors and slow requests always captured.

Mistakes to avoid: Inventing a custom header when a standard exists; logging the trace ID in some services but not others; forgetting to clear MDC in pooled threads; putting sensitive data in baggage.

Production perspective: The single highest-value observability investment for a multi-service system is a trace ID that appears in every log line and every error response. It converts "some requests are slow" into "this span in this service is slow".

Related concepts covered: W3C trace context, MDC, sampling strategies, message header propagation, structured logging, data minimisation.

Q148. What failure modes occur with a chain of service calls?

Priority: Important

Why interviewers ask it: Reasoning about partial failure is the core skill of distributed backend work.

Interview-ready answer: The main modes are cascading failure, where a slow dependency consumes the caller's threads and spreads upward; partial failure, where some steps of a multi-step operation succeeded and others did not, leaving inconsistent state; timeout ambiguity, where you do not know whether the remote side applied the change; retry amplification, where every layer retries and multiplies load; and availability multiplication, where a chain of 99.9% services produces a much lower combined figure. The mitigations are deadlines propagated across hops, bulkheads and breakers, idempotency so retries are safe, compensating actions or reconciliation for partial failures, and removing unnecessary synchronous dependencies from critical paths.

In-depth explanation: Deadline propagation deserves emphasis: if the user-facing request has a 2-second budget and 1.2 seconds are already spent, the next hop should be given at most the remainder, not a fixed 3-second timeout. Without that, inner services keep working on requests nobody is waiting for. Partial failure needs a decision per workflow: compensate (undo the earlier steps), retry forward (keep trying until it succeeds, which requires idempotency), or reconcile later. Sagas formalise the compensating approach for long-running workflows. Graceful degradation is often better than failure: return the page without recommendations rather than a 500. Finally, test these paths — fault injection or chaos experiments reveal that most fallbacks have never actually run.

Practical backend example: Deadline-aware fan-out with degradation:

```

public Dashboard load(UUID userId, Duration budget) {
    Instant deadline = Instant.now().plus(budget);
    Duration remaining = () -> Duration.between(Instant.now(), deadline);

    Profile profile = call(() -> profileClient.fetch(userId), remaining.get().dividedBy(2),
        () -> Profile.unknown(userId)); // essential, has a fallback
    List<Offer> offers = remaining.get().isNegative()
        ? List.of() // budget exhausted: skip
        : call(() -> offerClient.fetch(userId), remaining.get(), List::of);
    return new Dashboard(profile, offers);
}

```

Common follow-ups:

- What is a deadline? The absolute time by which the whole operation must finish; each hop gets the remaining budget, not a fixed timeout.
- How do you handle a partially applied workflow? Compensate, retry forward with idempotency, or reconcile asynchronously — decide per workflow and document it.
- Why does a chain reduce availability? Serial dependencies multiply: three 99.9% services yield about 99.7%.

Mistakes to avoid: Fixed timeouts at every hop; assuming a failed call did nothing; unbounded retries; treating every dependency as essential; untested fallbacks.

Production perspective: Draw the dependency graph for your critical path and mark each dependency as essential or optional. Most teams discover that several "essential" calls can degrade gracefully, which is the cheapest reliability win available.

Related concepts covered: Deadline propagation, sagas and compensation, graceful degradation, availability math, chaos testing.

Q149. What is eventual consistency and when is it acceptable?

Priority: Important

Why interviewers ask it: It tests whether you can reason about consistency as a product decision, not just a technical one.

Interview-ready answer: Eventual consistency means that after a change, different parts of the system converge to the same state after some delay rather than instantly. It is the normal outcome when data is replicated or propagated by events: the order database commits, and the search index or the analytics store updates moments later. It is acceptable when the business tolerates a short window of staleness — search results, recommendations, dashboards, notification delivery. It is not acceptable where a stale read causes a wrong decision, such as an account balance used for authorisation or stock at the point of sale. The key design work is making the window visible and bounded, and giving users read-your-writes behaviour where they would otherwise be confused.

In-depth explanation: Read-your-writes is the most common usability requirement: after a user edits something, they must see their own change. Techniques include reading from the primary for a short period after a write, passing a version token the read path waits for, or optimistically rendering the local change. Monitoring must include the lag itself — replication lag, consumer lag, index freshness — with alerts, because "eventually" without a bound is not a guarantee. UI design matters too: showing "processing" rather than an incorrect value manages expectations. Also, eventual consistency interacts with idempotency and ordering: if events can arrive out of order, apply them with version checks so an older update cannot overwrite a newer one.

Practical backend example: Version-guarded projection updates plus a freshness metric:

```

@KafkaListener(topics = "orders")
public void onOrderChanged(OrderChanged event) {
    // ignore stale events: never let an older version overwrite a newer projection
    int updated = projectionRepository.updateIfNewer(event.orderId(), event.version(), event.payload());
    if (updated == 0) {
        log.debug("ignored stale event order={} version={}", event.orderId(), event.version());
    }
    freshnessGauge.set(Duration.between(event.occurredAt(), Instant.now()).toMillis());
}

```

```
UPDATE order_projection SET payload = :payload, version = :version, updated_at = now()  
WHERE order_id = :orderId AND version < :version;    -- monotonic, out-of-order safe
```

Common follow-ups:

- How do users see pending state? Show an explicit "processing" state, or read from the primary so they see their own writes.
- How do you bound the delay? Monitor replication and consumer lag with alerts, and define an SLO for freshness.
- What if events arrive out of order? Apply version or timestamp guards so older data cannot overwrite newer data.

Mistakes to avoid: Using eventually consistent data for authorisation or money; no lag monitoring; assuming ordering is guaranteed; showing stale data as if it were current.

Production perspective: Freshness is a service level objective like latency. Define it, measure it, and alert on it; otherwise a stalled consumer silently serves month-old data and nobody notices until a customer complains.

Related concepts covered: Read-your-writes, replication and consumer lag, out-of-order handling, projection design, freshness SLOs.

Q150. How would you design an order placement workflow end to end?

Priority: Must Know

Why interviewers ask it: It is an appropriately scoped design question that reveals how you combine API, transaction, payment and event concerns.

Interview-ready answer: The client sends POST /orders with an idempotency key. The service validates the request, reserves stock with an atomic conditional update, creates the order in PENDING_PAYMENT, and commits — all in one short local transaction with no external calls. Payment authorisation happens outside the transaction; on success a second short transaction marks the order PAID and writes an outbox event. Downstream effects — confirmation email, invoicing, analytics, fulfilment — consume that event asynchronously and idempotently. Failure paths are explicit: declined payment releases the reservation, an ambiguous payment timeout leaves the order in a pending state that a reconciliation job resolves against the provider. The API returns 201 with the order and its current status.

In-depth explanation: The critical decisions are where the transaction boundaries sit and where idempotency is enforced. Stock reservation must be atomic (UPDATE ... WHERE available >= :qty) or you oversell under concurrency. The payment call must not sit inside a database transaction. The outbox makes "order is paid" and "event published" atomic without distributed transactions. Reservations need a timeout so abandoned orders release stock, which is usually a scheduled sweep. For the user experience, decide whether to respond after authorisation (slower, definitive) or to accept and return 202 with a status endpoint (faster, requires client polling or push). Every consumer of the event must be idempotent because delivery is at-least-once.

Practical backend example: The skeleton of the flow:

```

public OrderResponse place(PlaceOrderCommand cmd, String idempotencyKey) {
    Optional<OrderResponse> replay = idempotency.find(idempotencyKey);
    if (replay.isPresent()) return replay.get(); // retry-safe

    Order order = tx.execute(s -> { // transaction 1: short, local
        inventory.reserveOrThrow(cmd.lines()); // atomic conditional update
        Order created = orderRepository.save(Order.pending(cmd));
        idempotency.record(idempotencyKey, created.id());
        return created;
    });

    AuthorizationResult auth = paymentClient.authorize( // outside any transaction
        cmd.payment(), order.id(), idempotencyKey); // key forwarded downstream

    return tx.execute(s -> { // transaction 2: short
        if (auth.approved()) {
            order.markPaid(auth.reference());
            outbox.append(new OrderPaid(order.id())); // atomic with the state change
        } else {
            order.markPaymentFailed(auth.reason());
            inventory.release(cmd.lines());
        }
        return OrderResponse.from(orderRepository.save(order));
    });
}

```

Common follow-ups:

- Where is idempotency enforced? At the API boundary with a stored key, and again downstream by passing the key to the payment provider.
- What if the payment call times out? The outcome is unknown: leave the order pending and reconcile against the provider rather than retrying blindly.
- How do reservations get released? A scheduled sweep expires reservations older than the hold window.

Mistakes to avoid: Holding a transaction across the payment call; checking stock with a read before writing; publishing events before commit; assuming a timeout means failure.

Production perspective: Instrument each state transition and alert on orders stuck in intermediate states — that metric catches nearly every failure mode in this workflow, including ones you did not anticipate.

Related concepts covered: Transaction boundaries, outbox pattern, idempotency keys, stock reservation, reconciliation, state machine monitoring.

Q151. How do API gateways differ from service-to-service concerns?

Priority: Important

Why interviewers ask it: It probes whether you can place cross-cutting responsibilities at the right layer.

Interview-ready answer: A gateway sits at the edge and handles concerns common to all external traffic: TLS termination, routing, authentication of end users, coarse rate limiting, request size limits, CORS and sometimes response caching. Service-to-service concerns are different: mutual authentication between services, per-dependency timeouts, retries, circuit breakers and fine-grained authorisation based on the resource being accessed. The mistake I avoid is putting business logic in the gateway, because it becomes a shared bottleneck that every team must change and nobody owns. Each service still validates its own input and enforces its own authorisation — the gateway is a filter, not a guarantee.

In-depth explanation: Defence in depth is the underlying principle: a service must not assume it is only reachable through the gateway, because internal callers, misconfigurations and lateral movement all bypass it. Service meshes move some of this into a sidecar — mTLS, retries, timeouts, traffic shifting — which centralises policy without embedding it in application code, at the cost of operational complexity. Backend-for-frontend is a useful pattern when different clients need different aggregations: a thin per-client layer keeps the core services clean. Gateways also become a single point of failure and a latency contributor, so they need the same capacity planning and observability as any service.

Practical backend example: Responsibility split:

```
Gateway / edge
- TLS termination, HTTP/2
- JWT validation (signature, issuer, audience, expiry)
- global rate limits per API key, body size limits
- CORS policy, routing to services, request ID injection

Service
- re-validates the token's claims it depends on
- authorises the specific resource (does this user own this order?)
- validates payloads, enforces business rules
- per-dependency timeouts, retries, bulkheads for its own outbound calls
```

Common follow-ups:

- Should business logic live at the gateway? No; it becomes a shared bottleneck and duplicates domain rules outside their owning service.
- Does the gateway remove the need for service-side authorisation? No; resource-level checks must happen where the data lives.
- What does a service mesh add? Uniform mTLS, retries and traffic policy at the infrastructure layer, with extra operational complexity.

Mistakes to avoid: Trusting gateway-injected headers without validation; implementing per-tenant business rules in gateway configuration; forgetting the gateway's own capacity limits; skipping internal authentication because "it is internal".

Production perspective: Gateways concentrate risk: a bad route or rate-limit change can affect every API at once. Treat gateway configuration as code with reviews, staged rollout and quick rollback.

Related concepts covered: Defence in depth, service mesh, backend-for-frontend, edge rate limiting, configuration as code.

Q152. How do you prevent duplicate requests from creating duplicate resources?

Priority: Must Know

Why interviewers ask it: Duplicate creation is one of the most common production data problems, and the fix must be at the right layer.

Interview-ready answer: The durable answer is a uniqueness guarantee in the database: a unique constraint on whatever identifies the logical operation — an idempotency key, a client-supplied reference, or a natural business key such as (customer, order number). Then either attempt the insert and translate a conflict into the original result, or use an upsert (`INSERT ... ON CONFLICT`). Application-level checks like "does it already exist?" before inserting are races under concurrency and will eventually produce duplicates. A client-generated ID helps, because it lets the client retry with the same identifier, but only if the server enforces uniqueness on it.

In-depth explanation: There are two distinct duplicate sources: the same request retried (network timeout, client bug, gateway retry) and two genuinely concurrent requests representing the same intent. A unique constraint handles both. Choosing the key matters: an idempotency key is per-attempt-group and expires; a natural business key is permanent and also prevents duplicates created months apart, which may or may not be desirable. For resources created from events, the event ID is the natural deduplication key. Where a resource legitimately allows duplicates (two identical orders placed deliberately), the client must supply distinct keys, which is why the key should express client intent rather than payload content alone.

Practical backend example: Client-supplied reference with a database guarantee:

```
ALTER TABLE "order" ADD CONSTRAINT uk_order_client_reference
    UNIQUE (customer_id, client_reference);
```

```

@PostMapping("/api/orders")
public ResponseEntity<OrderResponse> create(@Valid @RequestBody CreateOrder body) {
    try {
        Order order = orderService.create(body); // insert relies on the constraint
        return ResponseEntity.created(location(order)).body(OrderResponse.from(order));
    } catch (DataIntegrityViolationException e) {
        Order existing = orderRepository
            .findByCustomerIdAndClientReference(body.customerId(), body.clientReference())
            .orElseThrow(() -> e);
        return ResponseEntity.ok(OrderResponse.from(existing)); // return the original, not an
            error
    }
}

```

Common follow-ups:

- Is a client-generated ID enough? Only if the server enforces uniqueness on it; otherwise two inserts still succeed.
- What if the duplicate arrives concurrently? The unique index serialises them; one insert fails and reads the winner's row.
- Should a duplicate be an error? Usually not for a retry — return the original resource; for a genuinely different payload with the same key, return 409.

Mistakes to avoid: existsBy before save; relying on the application to be the only writer; unique constraints only in code comments; returning 500 on constraint violations.

Production perspective: Cleaning up duplicates after the fact is expensive and often user-visible (duplicate charges, duplicate shipments). A unique index costs almost nothing and prevents the entire class of problem.

Related concepts covered: Unique constraints, upserts, idempotency keys, natural keys, conflict translation, concurrent inserts.

Q153. What should happen when a downstream service times out after committing?

Priority: Must Know

Why interviewers ask it: Timeout ambiguity is the defining problem of distributed systems, and the answer reveals real experience.

Interview-ready answer: A timeout tells you nothing about the remote outcome — the request may have succeeded, failed, or still be in progress. So the operation must be treated as unknown, not failed. If the call is idempotent or carries an idempotency key, a bounded retry is safe and will either reapply harmlessly or return the original result. If it is not idempotent, retrying risks a duplicate side effect; instead I record the attempt in a pending state and resolve it by querying the provider's status endpoint or reconciling later. What I never do is silently assume failure and roll back local state, because that creates a mismatch the customer eventually discovers.

In-depth explanation: The design implication is that every non-idempotent remote call should have a way to ask "did this happen?" — a status endpoint keyed by your reference, or a reconciliation feed. Store your own reference before the call, so the record exists even if the response never arrives. A pending state plus a scheduled resolver is the standard pattern: check the provider, then either complete or fail the local record. Time limits matter: define how long a record may stay pending before it escalates to a human. The same logic applies to your own API: give clients an idempotency mechanism and a status endpoint so they can resolve their own ambiguous timeouts.

Practical backend example: Pending state plus a resolver:

```

@Transactional
public void startCapture(UUID paymentId) {
    payments.markPending(paymentId); // recorded BEFORE the call, so it always exists
}

public void capture(UUID paymentId) {
    startCapture(paymentId);
    try {
        CaptureResult result = gateway.capture(paymentId, idempotencyKeyFor(paymentId));
        payments.complete(paymentId, result);
    } catch (TimeoutException e) {
        log.warn("capture outcome unknown payment={}", paymentId, e); // do NOT assume failure
    }
}

@Scheduled(fixedDelay = 60_000)
public void resolvePending() {
    for (Payment p : payments.findPendingOlderThan(Duration.ofMinutes(2))) {
        gateway.lookup(p.reference()).ifPresent(status -> payments.settle(p.id(), status));
    }
}

```

Common follow-ups:

- Should you immediately retry? Only if the operation is idempotent or key-protected; otherwise query the status first.
- How do you know what happened? A provider status endpoint keyed by your reference, or a reconciliation file or feed.
- How long can a record stay pending? Define a limit, alert when exceeded, and have a manual resolution path.

Mistakes to avoid: Treating a timeout as a failure; rolling back local state without confirmation; retrying a non-idempotent capture; leaving pending records unmonitored.

Production perspective: Pending-state counts and ages are among the most valuable business-level metrics you can expose. A rising count of unresolved operations is an early indicator of an integration problem long before customers call.

Related concepts covered: Timeout ambiguity, idempotency keys, status lookup endpoints, reconciliation jobs, pending-state monitoring.

Q154. How do you make a webhook consumer safe?

Priority: Important

Why interviewers ask it: Webhooks are a common integration point with security, ordering and reliability pitfalls all at once.

Interview-ready answer: Four things. Verify authenticity: check the HMAC signature over the raw body with the shared secret, using a constant-time comparison, and reject stale timestamps to prevent replay. Deduplicate: store the provider's event ID with a unique constraint, because webhooks are delivered at least once. Respond quickly: acknowledge with 2xx and process asynchronously, since most providers retry on slow responses and may disable endpoints that time out. Handle ordering: events can arrive out of order, so apply them with version or timestamp guards rather than assuming sequence. I also make the endpoint's failure behaviour deliberate — return 5xx only when I genuinely want a retry.

In-depth explanation: Signature verification must use the exact raw bytes received; parsing and re-serialising JSON changes the payload and breaks the HMAC, which is why you need access to the raw body. Constant-time comparison avoids timing side channels. Timestamp tolerance (commonly five minutes) limits replay windows. For deduplication, a unique index on the event ID is both the check and the guarantee. Asynchronous processing means writing the event to a table or queue and returning 200 immediately; the worker then handles retries, dead-lettering and alerting. Because retries are inevitable, the handler must be idempotent at the effect level, not just the ingestion level. Finally, webhook endpoints are public: apply rate limits and body size limits, and never trust fields in the payload over your own records.

Practical backend example: Verify, deduplicate, enqueue:

```

@PostMapping(value = "/webhooks/payments", consumes = MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<Void> receive(@RequestHeader("X-Signature") String signature,
                                   @RequestHeader("X-Timestamp") long timestamp,
                                   @RequestBody byte[] rawBody) { // raw bytes, not a DTO
    if (Math.abs(Instant.now().getEpochSecond() - timestamp) > 300) {
        return ResponseEntity.badRequest().build(); // replay window
    }
    byte[] expected = hmacSha256(secret, (timestamp + ".").getBytes(UTF_8), rawBody);
    if (!MessageDigest.isEqual(expected, Base64.getDecoder().decode(signature))) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build(); // constant-time compare
    }
    WebhookEvent event = json.read(rawBody, WebhookEvent.class);
    inbox.insertIgnoreDuplicate(event.id(), rawBody); // unique index on event
    id
    return ResponseEntity.ok().build(); // fast ack, async
    processing
}

```

Common follow-ups:

- What if events arrive out of order? Apply them with a version or timestamp guard so an older event cannot overwrite newer state.
- Why acknowledge before processing? Providers retry slow endpoints and may disable them; processing asynchronously keeps acknowledgement fast and reliable.
- How do you handle a poison event? Dead-letter it after bounded retries and alert, rather than blocking the whole inbox.

Mistakes to avoid: Verifying the signature against re-serialised JSON; using equals for signature comparison; doing heavy work synchronously in the handler; trusting payload amounts without verifying against your own records.

Production perspective: Monitor inbox lag, duplicate rate and signature failures. A spike in signature failures usually means a secret rotation went wrong, and it is much easier to see on a graph than in logs.

Related concepts covered: HMAC verification, replay protection, inbox pattern, at-least-once delivery, dead-letter handling, async acknowledgement.

Q155. How would you rate-limit an API?

Priority: Important

Why interviewers ask it: Rate limiting protects availability and shows understanding of distributed counters.

Interview-ready answer: I choose an algorithm first: fixed windows are simple but allow bursts at boundaries; sliding windows smooth that; token bucket is the usual choice because it allows a controlled burst while enforcing an average rate. Then the scope: per API key, per user, per IP, or per endpoint, often several tiers together. In a multi-instance deployment the counter must be shared, so Redis with atomic operations or a Lua script is the standard implementation, or the limit is enforced at the gateway. Responses must be informative: 429 with Retry-After and rate-limit headers so well-behaved clients can adapt. I also make sure the limiter fails open or closed deliberately if Redis is unavailable.

In-depth explanation: Atomicity is the correctness crux — check-then-increment across a network is a race, so use INCR with EXPIRE set atomically, or a Lua script implementing the token bucket in one round trip. Per-IP limits are coarse behind NAT and proxies, so authenticated identity is a better key where available; X-Forwarded-For must be trusted only from known proxies. Consider what happens to legitimate bursts: a hard limit can break batch clients that could be served fine, which is why token bucket with a burst capacity is popular. Distinguish rate limiting from quota (long-term usage) and from concurrency limiting (in-flight requests) — they solve different problems, and a concurrency limit is often the better protection for expensive endpoints.

Practical backend example: Redis token bucket with informative headers:

```
// Lua keeps check-and-consume atomic in one round trip
private static final String TOKEN_BUCKET = ""
    local tokens = tonumber(redis.call('HGET', KEYS[1], 'tokens') or ARGV[1])
    local last   = tonumber(redis.call('HGET', KEYS[1], 'ts') or ARGV[4])
    local refill = math.min(tonumber(ARGV[1]), tokens + (ARGV[4] - last) * ARGV[2])
    if refill < 1 then return {0, refill} end
    redis.call('HSET', KEYS[1], 'tokens', refill - 1, 'ts', ARGV[4])
    redis.call('EXPIRE', KEYS[1], ARGV[3])
    return {1, refill - 1}
    """;

if (!allowed) {
    return ResponseEntity.status(HttpStatus.TOO_MANY_REQUESTS)
        .header("Retry-After", "1")
        .header("RateLimit-Limit", "100")
        .header("RateLimit-Remaining", "0")
        .build();
}
```

Common follow-ups:

- What response headers help clients? Retry-After plus limit/remaining/reset headers so clients can self-throttle.
- Why not count in application memory? Each replica would allow the full limit; the counter must be shared.
- What if Redis is down? Decide explicitly: fail open (accept traffic, risk overload) or fail closed (reject, risk an outage). Most APIs fail open with a local fallback limit.

Mistakes to avoid: Non-atomic check-then-increment; per-IP limits as the only tier; no Retry-After; limiting cheap and expensive endpoints identically; trusting X-Forwarded-For from anywhere.

Production perspective: Rate limits are a customer-facing contract: document them, expose usage headers, and alert when a major client starts hitting limits, because that usually signals a client bug or a legitimate need for a higher tier.

Related concepts covered: Token bucket, Redis atomic scripts, distributed counters, concurrency limiting versus rate limiting, fail-open decisions.

Q156. How do content negotiation and media types affect an API?

Priority: Bonus

Why interviewers ask it: It checks protocol precision that becomes important for file endpoints, versioning and integrations.

Interview-ready answer: The client states what it can accept with the Accept header, and describes what it is sending with Content-Type. The server picks a representation it can produce; if it cannot satisfy Accept it returns 406, and if it cannot parse the request body's media type it returns 415. In Spring, produces and consumes on a mapping participate in that negotiation. Beyond JSON, this matters for CSV or PDF exports, for versioned media types such as application/vnd.example.order.v2+json, and for application/merge-patch+json on PATCH endpoints. Charset should be UTF-8 everywhere, stated explicitly where the type allows it.

In-depth explanation: Negotiation also drives caching: a shared cache must vary on Accept (and often Accept-Encoding) or it will serve the wrong representation, which is what the Vary header is for. Media-type versioning keeps URIs stable and is elegant, but it is harder to test in a browser, harder to route at a gateway and harder to read in logs — a trade-off worth stating. For file downloads, Content-Disposition controls whether the browser displays or downloads the file and provides the filename; it must be sanitised because header injection through filenames is a real vulnerability. Compression is negotiated with Accept-Encoding, and enabling gzip on JSON responses is usually a large, cheap bandwidth win.

Practical backend example: Two representations of one resource:

```

@GetMapping(value = "/api/reports/{id}", produces = MediaType.APPLICATION_JSON_VALUE)
public ReportDto json(@PathVariable UUID id) { return reportService.get(id); }

@GetMapping(value = "/api/reports/{id}", produces = "text/csv") // same URI, different Accept
public ResponseEntity<StreamingResponseBody> csv(@PathVariable UUID id) {
    return ResponseEntity.ok()
        .contentType(MediaType.parseMediaType("text/csv; charset=UTF-8"))
        .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=\"report.csv\"")
        .header(HttpHeaders.VARY, HttpHeaders.ACCEPT) // caches must distinguish
        .body(out -> reportService.streamCsv(id, out)); // streamed, not buffered
}

```

Common follow-ups:

- What does 415 mean? Unsupported Media Type — the server cannot process the request body's Content-Type.
- What does 406 mean? Not Acceptable — the server cannot produce any representation matching Accept.
- Why set Vary? So shared caches store and serve the correct representation per Accept value.

Mistakes to avoid: Ignoring Accept and always returning JSON with a 200; unsanitised filenames in Content-Disposition; buffering large exports in memory; omitting charset on text types.

Production perspective: Export endpoints are a common source of memory incidents. Stream them, cap the result size or make them asynchronous with a download link, and keep them off the same connection pool as interactive traffic.

Related concepts covered: Accept and Content-Type, 406 versus 415, Vary and caching, media-type versioning, streaming responses, content disposition.

Q157. What is the difference between liveness and readiness?

Priority: Important

Why interviewers ask it: Confusing the two causes restart loops and outages during dependency blips.

Interview-ready answer: Liveness answers "is this process broken beyond recovery?" — if it fails, the orchestrator restarts the container. Readiness answers "should this instance receive traffic right now?" — if it fails, the instance is removed from the load balancer but keeps running. The critical rule is that a failing external dependency should usually not fail liveness, because restarting the pod will not fix the database and a restart loop makes things worse. It may legitimately fail readiness if the instance genuinely cannot serve requests. Spring Boot exposes both through /actuator/health/liveness and /actuator/health/readiness, and a startup probe covers slow initialisation.

In-depth explanation: Health indicators aggregate component checks; by default many are included in the overall health group, so it is important to configure which components belong to which probe. A readiness check that queries the database on every probe adds load and can amplify an incident — a cheap cached check is usually better. During shutdown, readiness should fail immediately so traffic drains before the server stops accepting connections. Startup probes exist because a slow-booting application would otherwise be killed by an aggressive liveness probe. Also consider partial readiness: if one of several features is degraded, it may be better to serve the rest than to remove the instance entirely.

Practical backend example: Probe groups configured deliberately:

```

management:
  endpoint:
    health:
      probes:
        enabled: true
      group:
        liveness:
          include: livenessState # process health only - no external dependencies
        readiness:
          include: readinessState,db # may include critical dependencies
        show-details: never # do not leak internals publicly

```

```

livenessProbe: { httpGet: { path: /actuator/health/liveness, port: 8080 }, periodSeconds: 10 }
readinessProbe: { httpGet: { path: /actuator/health/readiness, port: 8080 }, periodSeconds: 5 }
startupProbe: { httpGet: { path: /actuator/health/liveness, port: 8080 }, failureThreshold: 30 }

```

Common follow-ups:

- Should a failed database fail liveness? Generally no; restarting will not fix the database, and a restart loop removes capacity. Readiness may fail instead.
- What is a startup probe for? To give a slow-starting application time to boot without an aggressive liveness probe killing it.
- Why fail readiness during shutdown? So the load balancer stops sending traffic before the server stops accepting connections.

Mistakes to avoid: Pointing both probes at the same deep health check; probing expensive dependencies every second; exposing detailed health information publicly; ignoring the startup case.

Production perspective: A dependency outage combined with a dependency-checking liveness probe turns a partial failure into a full one as every pod restarts simultaneously. Keep liveness cheap and local.

Related concepts covered: Kubernetes probes, health groups, graceful shutdown, cascading failure, actuator configuration.

Q158. How do you handle file upload and download safely?

Priority: Important

Why interviewers ask it: File endpoints combine memory, security and storage concerns that trip up many services.

Interview-ready answer: For uploads I enforce a maximum size at both the framework and the proxy, stream to storage rather than buffering in memory, validate the content type by inspecting the bytes rather than trusting the client, and generate my own storage key instead of using the client's filename. For downloads I stream from storage with the correct Content-Type and a sanitised Content-Disposition, and I authorise every request rather than relying on an unguessable URL. Large files are better handled with pre-signed URLs to object storage so the bytes never pass through the application at all. Uploaded content should be stored outside the web root and scanned when the risk profile requires it.

In-depth explanation: Path traversal is the classic vulnerability: a filename like ../../etc/passwd must never influence a storage path, which is why a generated UUID key with the original name kept only as metadata is the safe design. Content-type sniffing matters because a file named .jpg can contain anything; libraries that inspect magic bytes are more trustworthy than the declared type. Serving user-uploaded content from your primary domain risks stored XSS, so a separate domain or Content-Disposition: attachment plus X-Content-Type-Options: nosniff is recommended. On the resource side, multipart parsing can buffer to disk or memory depending on configuration, so set thresholds explicitly. Pre-signed URLs shift bandwidth and memory to the object store and are the standard approach above a few megabytes.

Practical backend example: Bounded, streamed upload with a generated key:

```
spring:
  servlet:
    multipart:
      max-file-size: 10MB
      max-request-size: 12MB
      file-size-threshold: 1MB      # buffer to disk beyond this
```

```
@PostMapping(value = "/api/documents", consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
public DocumentResponse upload(@RequestPart("file") MultipartFile file) throws IOException {
    if (file.getSize() > MAX_BYTES) throw new PayloadTooLargeException();
    String detected = tika.detect(file.getInputStream());           // inspect bytes, not the name
    if (!ALLOWED_TYPES.contains(detected)) throw new UnsupportedFileTypeException(detected);

    String key = "documents/" + UUID.randomUUID();               // never use the client filename
    try (InputStream in = file.getInputStream()) {
        objectStorage.put(key, in, file.getSize(), detected);    // streamed, not loaded into heap
    }
    return documentService.record(key, sanitize(file.getOriginalFilename()), detected);
}
```

Common follow-ups:

- How do you avoid path traversal? Never build paths from client input; generate the storage key yourself.

- Can Content-Type be trusted? No; detect from the file's bytes and validate against an allowlist.
- How do you serve very large files? Pre-signed URLs to object storage, so the application does not proxy the bytes.

Mistakes to avoid: Reading the whole file into a byte array; trusting the declared content type or filename; serving uploads from the main domain without nosniff and attachment disposition; no size limits at the proxy.

Production perspective: File endpoints are a common cause of heap exhaustion and of security findings. Limit sizes, stream everything, and keep upload traffic isolated so a large transfer cannot starve interactive requests.

Related concepts covered: Multipart configuration, streaming I/O, content sniffing, path traversal, pre-signed URLs, stored XSS.

Q159. How would you design a small notification service?

Priority: Important

Why interviewers ask it: It is a scoped design exercise touching queues, retries, templating, idempotency and provider failure.

Interview-ready answer: The service consumes notification requests from a queue, renders a template per channel, sends through a provider, and records the outcome. Key design points: deduplicate with a notification key so the same event cannot send twice; retry with backoff for transient provider failures and dead-letter after a bounded number of attempts; respect user preferences and quiet hours; and store a delivery record with status so support can answer "was this sent?". Templates live in a versioned store with locale support. I also isolate providers behind an interface so a failing email provider can be swapped or failed over, and rate-limit per provider to respect their quotas.

In-depth explanation: Idempotency is the crux: at-least-once delivery from the queue means the consumer will occasionally see the same message twice, so the send must be keyed — typically (eventId, channel, recipient) with a unique constraint checked before sending, and the provider's own idempotency key passed through where supported. Ordering is usually not required for notifications, which simplifies scaling. Provider failures need classification: 4xx from the provider (bad address) is permanent and should not be retried, while 5xx and timeouts are transient. Bounce and complaint webhooks should feed back into suppression lists, otherwise reputation suffers. Finally, notifications carry personal data, so payload retention and logging need care.

Practical backend example: Consumer with dedupe, classification and dead-lettering:

```
@KafkaListener(topics = "notifications", concurrency = "4")
public void handle(NotificationRequested msg, Acknowledgment ack) {
    String key = msg.eventId() + ":" + msg.channel() + ":" + msg.recipientHash();
    if (deliveryRepository.insertIfAbsent(key) == 0) { ack.acknowledge(); return; } // duplicate

    RenderedMessage rendered = templateService.render(msg.templateId(), msg.locale(), msg.params());
    try {
        ProviderResult result = providers.forChannel(msg.channel()).send(rendered, key);
        deliveryRepository.markSent(key, result.providerMessageId());
    } catch (PermanentProviderException e) { // invalid address: never retry
        deliveryRepository.markFailed(key, e.getMessage());
    } catch (TransientProviderException e) { // retry with backoff, then dead-letter
        throw e; // handled by the error handler / DLT
    }
    ack.acknowledge();
}
```

Common follow-ups:

- How do you avoid duplicate emails? A unique delivery key per event, channel and recipient, inserted before sending.
- What goes to a dead-letter topic? Messages that failed after the retry budget, with enough context to replay them later.
- How do you handle bounces? Consume provider webhooks and maintain a suppression list to stop sending to bad addresses.

Mistakes to avoid: Retrying permanent failures; no deduplication; templates hard-coded in application code; logging full message bodies containing personal data; unlimited retries against a failing provider.

Production perspective: Track sent, failed, retried and suppressed counts per channel and template. Notification bugs are highly visible to customers, and a delivery record per attempt is what makes support requests answerable.

Related concepts covered: At-least-once consumers, deduplication keys, error classification, dead-letter topics, suppression lists, template versioning.

Q160. When should you split a service, and when keep a modular monolith?

Priority: Important

Why interviewers ask it: It tests architectural judgement and resistance to following trends without reasons.

Interview-ready answer: I split when there is a concrete forcing reason: independent scaling needs, different availability or compliance requirements, genuinely separate ownership by different teams, or an isolation requirement for risky workloads. I keep a modular monolith when the main goal is clean boundaries, because modules give you those without network calls, distributed transactions, versioned contracts and multi-service debugging. A network boundary costs real money: partial failure, latency, serialisation, deployment coordination, distributed tracing and operational overhead. The usual sensible path is a well-modularised monolith with clear internal APIs, extracting a service when a specific pressure justifies it.

In-depth explanation: Splitting on the wrong seam creates a distributed monolith: services that must be deployed together and cannot function independently, which is worse than either alternative. Good seams follow business capabilities with their own data, minimal synchronous chatter, and stable interfaces. Data ownership is the hardest part: two services sharing a database are not independent, and splitting the schema requires deciding how they exchange data — events, APIs, or replicated read models. Conway's law is real: service boundaries that do not match team boundaries create constant coordination. Before extracting, it is worth enforcing module boundaries in the monolith (package rules, ArchUnit tests, Spring Modulith) to prove the seam works when calls are still in-process.

Practical backend example: Enforcing a boundary before extracting it:

```
// ArchUnit test: the order module may not reach into billing internals
@ArchTest
static final ArchRule modules_respect_boundaries = classes()
    .that().resideInAPackage("..order..")
    .should().onlyDependOnClassesThat()
    .resideInAnyPackage("..order..", "..billing.api..", "java..", "org.springframework..");
```

```
Signals that justify extraction
- one component needs 10x the instances of the rest
- a compliance requirement demands isolated data and access control
- an independent team owns the capability and is blocked by shared deploys
- a risky or resource-hungry workload threatens the rest of the application
```

Common follow-ups:

- What is the cost of a network boundary? Partial failure, latency, serialisation, contract versioning, distributed debugging, and more infrastructure.
- Can a monolith scale? Often yes — horizontally behind a load balancer; the usual bottleneck is the database, which splitting does not automatically fix.
- How do you prepare for a future split? Enforce module boundaries in code, keep data ownership clean, and avoid shared mutable tables across modules.

Mistakes to avoid: Splitting by technical layer; services sharing one database; extracting before the boundary is understood; treating microservices as a goal rather than a trade-off.

Production perspective: Each additional service adds deployment, monitoring, on-call and dependency management overhead. Count the total operational cost, not just the code, and make sure the organisation is ready to run what it builds.

Related concepts covered: Modular monolith, distributed monolith anti-pattern, data ownership, Conway's law, ArchUnit boundary tests, extraction criteria.

Chapter 7. Security

Examples use Spring Security 6 (Spring Boot 3) and the `SecurityFilterChain` style of configuration. Security advice here is the standard, widely accepted guidance for backend APIs; specific regulatory obligations depend on your jurisdiction and industry and are out of scope.

Q161. How do authentication and authorization differ?

Priority: Must Know

Why interviewers ask it: It is the foundation for everything else, and conflating the two is how broken access control happens.

Interview-ready answer: Authentication establishes who the caller is — validating a password, a token signature, a client certificate. Authorization decides what that identity may do: which endpoints it can call and which specific records it can see or change. They fail differently: missing or invalid credentials give 401, an authenticated but not permitted caller gives 403. The most common real-world mistake is doing only coarse authorization — "this endpoint requires `ROLE_USER`" — without checking ownership, so any logged-in user can read another user's order by changing an ID. Both layers are necessary, and the resource-level check must happen on the server for every request.

In-depth explanation: Least privilege applies at every level: tokens should carry the narrowest scopes needed, database users should not be superusers, and service accounts should not share credentials. Authorization models range from role-based (roles grant permissions), to attribute-based (decisions from attributes such as tenant, ownership, time), to relationship-based for complex sharing scenarios. A practical middle ground for most backends is roles for coarse capability plus an explicit ownership or tenant predicate in the query itself, so a missing check cannot return another tenant's data. Authorization decisions should also be auditable: log who did what to which resource, because that record is what makes an incident investigable.

Practical backend example: Both layers applied together:

```
@PreAuthorize("hasRole('USER')")           // coarse: authenticated with a role
@GetMapping("/api/orders/{id}")
public OrderResponse get(@PathVariable UUID id, @AuthenticationPrincipal Jwt jwt) {
    UUID customerId = UUID.fromString(jwt.getSubject());
    return orderRepository.findByIdAndCustomerId(id, customerId) // fine-grained: ownership in the
        .query
        .map(OrderResponse::from)
        .orElseThrow(() -> new OrderNotFoundException(id));    // 404, not 403: no existence leak
}
```

Common follow-ups:

- Why check ownership on every request? Because the client controls the identifier; without it, any authenticated user can read any record.
- 401 or 403? 401 when credentials are missing or invalid, 403 when the authenticated caller is not permitted.
- Where should authorization live? At the edge for coarse rules and in the service or query for resource-level decisions.

Mistakes to avoid: Relying on the UI hiding a button; checking roles but not ownership; trusting an identifier from the request body; scattering authorization logic so no one can audit it.

Production perspective: Broken access control is consistently among the most common serious web vulnerabilities. Add tests that call each endpoint as a different tenant and assert 403 or 404 — they are cheap and catch regressions that code review misses.

Related concepts covered: Least privilege, RBAC and ABAC, tenant isolation, IDOR, audit logging, status code semantics.

Q162. When choose server sessions versus JWT bearer tokens?

Priority: Must Know

Why interviewers ask it: It is the clearest test of whether a candidate repeats "JWT is stateless and therefore better" or reasons about trade-offs.

Interview-ready answer: Server-side sessions keep state on the server and give the client an opaque cookie. Revocation is immediate — delete the session — and the cookie carries no data, but you need shared session storage such as Redis when running multiple instances. JWTs are self-contained and verified by signature, so any service can validate them without a lookup, which suits distributed systems and third-party APIs. The cost is revocation: a signed token is valid until it expires, so you need short lifetimes plus refresh tokens, or a denylist that reintroduces the state you were avoiding. For a browser application with one backend, sessions are often simpler and safer; for multi-service or third-party access, tokens usually win.

In-depth explanation: A JWT is signed, not encrypted, by default — anyone holding it can read its claims, so it must never contain secrets. Storage in the browser matters: an `httpOnly`, `Secure`, `SameSite` cookie is not readable by JavaScript, whereas `localStorage` is exposed to any XSS. Cookie-based auth then needs CSRF protection, which bearer headers avoid. Token lifetime is the key control: short access tokens (minutes) with refresh tokens held securely limit the damage of a leak, and refresh token rotation with reuse detection catches theft. Sessions also have practical advantages — you can list and revoke active sessions per user, and you can change permissions instantly, whereas a JWT carries the permissions it was minted with until expiry.

Practical backend example: Choosing per context:

```
// Browser app, single backend: session cookie, CSRF protection on
http.sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED))
    .csrf(Customizer.withDefaults());

// Service-to-service or SPA with a token: stateless resource server
http.sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .csrf(csrf -> csrf.disable()) // no ambient credentials to abuse
    .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()));
```

```
# short-lived access tokens limit the revocation gap
security.jwt.access-token-ttl=10m
security.jwt.refresh-token-ttl=14d
```

Common follow-ups:

- Is a JWT encrypted by default? No; it is signed and base64url-encoded, so the claims are readable. Use JWE if confidentiality is required.
- How do you revoke a JWT? Short expiry plus refresh rotation, or a denylist keyed by token ID — which makes it stateful again.
- Where should a browser store a token? An `httpOnly Secure` cookie is safer than `localStorage`, which is readable by any injected script.

Mistakes to avoid: Claiming JWT is always better; long-lived access tokens; putting personal data or secrets in claims; storing tokens in `localStorage` without accepting the XSS risk; forgetting that permissions inside a token go stale.

Production perspective: Whichever you choose, you need a way to force logout — after a password change, a compromise, or a permissions change. Design that path deliberately rather than discovering during an incident that tokens cannot be revoked.

Related concepts covered: Session storage, refresh token rotation, cookie flags, CSRF interaction, revocation strategies, claim staleness.

Q163. How should passwords be stored and verified?

Priority: Must Know

Why interviewers ask it: Credential handling is non-negotiable knowledge, and the wrong answer is a serious red flag.

Interview-ready answer: Never store passwords in reversible form. Use an adaptive, deliberately slow hash designed for passwords — Argon2id, bcrypt or scrypt — with a per-password random salt, which those algorithms generate and embed in the output. Verification re-hashes the candidate with the stored parameters and compares in constant time, which the encoder handles. General-purpose hashes such as SHA-256 are unsuitable because they are fast, so an attacker with the database can try billions of candidates per second on a GPU. The work factor should be tuned so

hashing takes a noticeable fraction of a second on your hardware, and increased over time as hardware improves.

In-depth explanation: In Spring Security, `DelegatingPasswordEncoder` prefixes stored hashes with the algorithm id (for example `{bcrypt}`), which allows transparent migration: verify with the old algorithm and re-hash on successful login with the new one. Around the hash, other controls matter: rate limiting and lockout on repeated failures, a generic error message so you do not reveal which accounts exist, and multi-factor authentication for sensitive systems. Password reset must use a single-use, short-lived, high-entropy token stored hashed, not a password sent by email. Also check candidate passwords against known-breached lists rather than enforcing baroque composition rules, which mostly produce predictable substitutions.

Practical backend example: Encoder configuration and safe verification:

```
@Bean
PasswordEncoder passwordEncoder() {
    // delegating encoder: new hashes use the default, old ones still verify
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
}

public void login(String email, String rawPassword) {
    User user = users.findByEmail(email).orElse(User.DUMMY); // avoid timing-based enumeration
    if (!passwordEncoder.matches(rawPassword, user.passwordHash())) {
        loginAttempts.recordFailure(email); // rate limit / lockout
        throw new BadCredentialsException("Invalid email or password"); // generic message
    }
    if (passwordEncoder.upgradeEncoding(user.passwordHash())) {
        users.updateHash(user.id(), passwordEncoder.encode(rawPassword)); // transparent upgrade
    }
}
```

Common follow-ups:

- Why not SHA-256 alone? It is fast by design, so offline brute force is cheap; password hashes must be deliberately slow and memory-hard.
- Where does the salt go? Modern encoders generate a random salt per password and embed it in the stored string.
- How do you handle password reset? A single-use, short-lived, random token stored hashed; never email the password.

Mistakes to avoid: Custom hashing schemes; a single application-wide salt; encryption instead of hashing; revealing whether an email exists; unlimited login attempts.

Production perspective: Plan for a breach: with adaptive hashing and unique salts, stolen hashes are expensive to crack, which buys time to force resets. Keep the work factor configurable so it can be raised without a code change.

Related concepts covered: Adaptive hashing, salts, encoder migration, account enumeration, rate limiting, reset token design.

Q164. What are OAuth2 and OpenID Connect at a practical level?

Priority: Important

Why interviewers ask it: Most services integrate with an identity provider, and the OAuth2/OIDC distinction is widely misunderstood.

Interview-ready answer: OAuth2 is a delegated authorization framework: it lets an application obtain a token to access resources on behalf of a user or of itself, without handling the user's credentials. It does not, by itself, tell you who the user is. OpenID Connect is a thin layer on top that adds authentication: an ID token, a standard `userinfo` endpoint and standard claims. The grants that matter today are authorization code with PKCE for user-facing applications, including SPAs and mobile apps, and client credentials for service-to-service calls. The implicit and resource-owner-password grants are discouraged. In Spring Boot, the API is usually an OAuth2 resource server validating access tokens, while the login flow is handled by the identity provider.

In-depth explanation: Three token types appear: the access token, presented to APIs and typically short-lived; the refresh token, used to obtain new access tokens and kept secret; and the ID token, which is for the client to learn about the user and must not be used as an API credential. PKCE protects the authorization code exchange for clients that cannot keep a secret. Scopes express what the token is allowed to do, and are not the same thing as application roles

— scopes are delegated permissions, roles are properties of the user. Redirect URI validation is security-critical: allowing wildcards enables token theft. Understanding these distinctions is more valuable in an interview than memorising the protocol's message flow.

Practical backend example: A resource server validating tokens and mapping scopes:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://id.example.com/realms/app      # discovery + JWKS
          # 'audiences' (Boot 2.7+) rejects tokens minted for other APIs;
          # see Q169 for the programmatic AudienceValidator equivalent
          audiences: orders-api
```

```
http.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.GET, "/api/orders/**").hasAuthority("SCOPE_orders.read")
    .requestMatchers(HttpMethod.POST, "/api/orders/**").hasAuthority("SCOPE_orders.write")
    .anyRequest().authenticated())
.oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()));
```

Common follow-ups:

- Is OAuth2 itself authentication? No; it is authorization delegation. OIDC adds authentication on top with an ID token.
- Which grant for a single-page app? Authorization code with PKCE; implicit is deprecated.
- Can an ID token be used to call an API? No; APIs must accept access tokens, and the audience check enforces that.

Mistakes to avoid: Using the ID token as an API credential; skipping audience validation; wildcard redirect URIs; treating scopes as roles; implementing your own OAuth server without a strong reason.

Production perspective: Cache the provider's JWKS and handle key rotation gracefully, because a failed key fetch means every request fails authentication. Also monitor token validation failures — a spike usually signals a clock skew or rotation issue rather than an attack.

Related concepts covered: Authorization code with PKCE, client credentials, token types, scopes versus roles, JWKS rotation, audience validation.

Q165. How do CORS and CSRF differ?

Priority: Must Know

Why interviewers ask it: These are routinely confused, and confusion leads to disabling the wrong protection.

Interview-ready answer: CORS is a browser mechanism that relaxes the same-origin policy: it lets a server declare which other origins may read its responses from browser JavaScript. It is not a server-side protection — a non-browser client such as curl or a backend service ignores CORS entirely. CSRF is an attack where a malicious site causes the victim's browser to send a state-changing request to your site using credentials the browser attaches automatically, typically cookies. CSRF protection therefore matters whenever authentication is ambient: cookies or HTTP Basic. An API authenticated with a bearer token in a header is not automatically vulnerable, because the browser will not attach that header for a cross-site request. So the rule is: configure CORS for legitimate browser clients, and enable CSRF protection when you use cookie-based sessions.

In-depth explanation: CORS has two request classes: simple requests, which the browser sends directly and then blocks the response if the origin is not allowed, and preflighted requests (custom headers, JSON content type, non-simple methods), where the browser first sends an OPTIONS request. Note the implication: for simple requests the server has already executed the action even if the browser hides the response, which is precisely why CORS is not a substitute for authorization. Access-Control-Allow-Origin: * cannot be combined with credentials, so cookie-based APIs must echo specific allowed origins. For CSRF, the standard defences are a synchroniser token, SameSite=Lax or Strict cookies (which modern browsers largely default to Lax), and checking Origin. Disabling CSRF protection is only acceptable when no ambient credential can be used.

Practical backend example: Correct configuration for each case:

```

@Bean
CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of("https://app.example.com")); // never "*" with credentials
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
    config.setAllowedHeaders(List.of("Authorization", "Content-Type", "Idempotency-Key"));
    config.setAllowCredentials(true);
    config.setMaxAge(Duration.ofMinutes(30)); // cache preflights
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/api/**", config);
    return source;
}

// cookie-session app: keep CSRF protection on
http.csrf(csrf -> csrf.csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse()));
// stateless bearer-token API: no ambient credentials, CSRF protection is not applicable

```

Common follow-ups:

- Does CORS protect an API from curl? No; it is enforced by browsers only. Server-side authorization is the protection.
- When can you disable CSRF protection? When authentication cannot be sent ambiently — a pure bearer-token API with no cookie sessions.
- What does SameSite do? It tells the browser not to attach the cookie on cross-site requests, mitigating CSRF at the cookie level.

Mistakes to avoid: allowedOrigins("*") with credentials; disabling CSRF protection on a cookie-authenticated application; believing CORS prevents attacks; reflecting the Origin header back without validation.

Production perspective: CORS misconfiguration is a frequent finding in security reviews, especially origin reflection, which effectively allows every site. Keep the allowlist in configuration, review it, and test it with an automated check per environment.

Related concepts covered: Same-origin policy, preflight requests, SameSite cookies, synchroniser tokens, ambient authority, origin validation.

Q166. How do you prevent SQL injection and unsafe deserialization?

Priority: Must Know

Why interviewers ask it: Both are injection-class vulnerabilities with catastrophic impact and well-known, simple defences.

Interview-ready answer: For SQL, never build statements by concatenating user input: use parameterised queries — JDBC prepared statements, JPA named parameters, or a query builder — so the value is never parsed as SQL. Parts of a query that cannot be parameterised, such as a column name in ORDER BY, must come from a strict allowlist. For deserialization, do not deserialise untrusted data into arbitrary types: avoid Java native serialization entirely for external input, and never enable Jackson polymorphic type handling with a permissive base type, because that lets an attacker choose which classes are instantiated. Deserialise into explicit DTOs, validate them, and reject unknown fields where appropriate.

In-depth explanation: ORMs reduce SQL injection risk but do not eliminate it: a native query built with string concatenation is just as vulnerable, and so is a dynamically constructed JPQL fragment. Least privilege limits the blast radius — the application's database user should not be able to drop tables or read other schemas. For deserialization, the danger is gadget chains: existing classes on the classpath whose construction triggers side effects. Jackson's default typing (enableDefaultTyping) has produced many CVEs, which is why polymorphic deserialization should use a closed allowlist of subtypes. The same class of problem exists for XML (XXE — disable external entities), YAML (unsafe constructors) and for any library that turns data into objects. Dependency scanning matters too, since these vulnerabilities often arrive through transitive dependencies.

Practical backend example: Parameterisation, allowlisting and safe JSON handling:

```
// SAFE: parameters are bound, never parsed as SQL
@Query(value = "SELECT * FROM \"order\" WHERE customer_id = :customerId AND status = :status",
      nativeQuery = true)
List<Order> find(UUID customerId, String status);

// UNSAFE: never do this
// entityManager.createNativeQuery("SELECT * FROM \"order\" WHERE status = '" + status + "'");

// dynamic ORDER BY cannot be parameterised -> allowlist
private static final Map<String, String> SORT = Map.of(
    "createdAt", "created_at", "total", "total_cents");
String column = SORT.getOrDefault(requestedSort, "created_at"); // never interpolate raw input
```

```
ObjectMapper mapper = JsonMapper.builder()
    .disable(MapperFeature.ALLOW_FINAL_FIELDS_AS_MUTATORS)
    .enable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES) // explicit contract
    .build(); // no default typing enabled
```

Common follow-ups:

- Is escaping sufficient for SQL? No; escaping is error-prone and context-dependent. Parameterisation is the reliable defence.
- What about dynamic sorting or table names? Use an allowlist mapping client values to known identifiers.
- Why is Java native serialization risky? Deserialising untrusted bytes can instantiate arbitrary classes and trigger gadget chains; avoid it for external data.

Mistakes to avoid: String concatenation "just this once"; trusting an ORM to prevent injection in native queries; enabling Jackson default typing; parsing XML without disabling external entities; running the application as a database superuser.

Production perspective: Add static analysis and dependency scanning to CI so both classes of issue are caught automatically. Also log and alert on database errors that look like injection attempts — they often indicate active probing.

Related concepts covered: Prepared statements, allowlists, gadget chains, XXE, least-privilege database users, dependency scanning.

Q167. How would you secure a Spring Boot API endpoint?

Priority: Must Know

Why interviewers ask it: It connects security theory to the concrete configuration you would actually write.

Interview-ready answer: I start from deny by default: `anyRequest().authenticated()`, then explicitly permit the few public endpoints. Authentication is a bearer token validated as an OAuth2 resource server, or a session for browser flows. Authorization is layered: URL rules for coarse capability, method security with `@PreAuthorize` for service-level rules, and an ownership predicate in the query for resource-level access. I disable CSRF protection only for genuinely stateless token APIs, enable HTTPS with HSTS at the edge, and add the standard security headers. Actuator endpoints are restricted, error responses avoid leaking internals, and I write tests asserting 401, 403 and 200 for representative requests.

In-depth explanation: Order matters in `authorizeHttpRequests`: the first matching rule wins, so a broad `permitAll` placed early can accidentally expose everything after it. `securityMatcher` lets you define separate chains for the API and for a browser-facing application with different policies. Method security requires `@EnableMethodSecurity`, and `@PreAuthorize` can reference method arguments and the authenticated principal, which makes ownership checks expressive. Remember that CORS is configured separately from authorization and that preflight `OPTIONS` requests must be permitted. Finally, security configuration is code that deserves review and tests; a one-line change can silently open an endpoint, and nothing in the application will fail at startup.

Practical backend example: A layered configuration with tests:

```

@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
    return http
        .securityMatcher("/api/**")
        .csrf(csrf -> csrf.disable()) // stateless bearer tokens only
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .cors(Customizer.withDefaults())
        .headers(h -> h.httpStrictTransportSecurity(hsts -> hsts.maxAgeInSeconds(31536000))
            .contentTypeOptions(Customizer.withDefaults())
            .frameOptions(HeadersConfigurer.FrameOptionsConfig::deny))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.OPTIONS, "**").permitAll() // CORS preflight
            .requestMatchers("/api/public/**").permitAll()
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated() // deny by default
        .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults())))
        .build();
}

@PreAuthorize("hasRole('SUPPORT') or #customerId == authentication.name") // method-level rule
public CustomerDto get(String customerId) { ... }

```

Common follow-ups:

- What about method-level security? `@EnableMethodSecurity` plus `@PreAuthorize`/`@PostAuthorize` for rules that need arguments or the principal.
- How do you test it? Slice tests asserting 401 for anonymous, 403 for the wrong role, and 200 for the right one.
- Why permit `OPTIONS`? Browsers send unauthenticated preflight requests; blocking them breaks legitimate CORS clients.

Mistakes to avoid: A broad `permitAll` early in the chain; disabling CSRF protection on cookie-authenticated applications; exposing actuator publicly; relying on URL rules alone for data access; never testing the negative cases.

Production perspective: Add an automated check that lists every endpoint and its required authority, and review it on each release. Endpoints added without security rules are the most common way an internal API becomes public.

Related concepts covered: Deny-by-default, filter chain matching, method security, security headers, CORS preflight, actuator exposure.

Q168. How do you design resource-level authorization?

Priority: Must Know

Why interviewers ask it: Insecure direct object reference is one of the most frequently exploited real-world flaws.

Interview-ready answer: Every read or write of a specific record must verify that the authenticated principal is allowed to touch that record, not just that they are logged in. The most robust technique is to make the ownership or tenant condition part of the query itself — `findByIdAndCustomerId` rather than `findById` followed by a check — because then a missing check cannot return data. For shared resources I model permissions explicitly (owner, collaborator, viewer) and evaluate them centrally rather than scattering `if` statements. Responding 404 instead of 403 for records the caller may not see avoids confirming that they exist. And identifiers should be unguessable, though unguessability is defence in depth, never the control itself.

In-depth explanation: Multi-tenancy deserves special discipline: a tenant column on every table plus a mandatory predicate, or PostgreSQL row-level security, so a forgotten filter fails closed rather than leaking. Hibernate filters or a repository base class that always applies the tenant condition reduce the chance of an oversight. Bulk endpoints are a common gap — a request containing 100 IDs must check every one, not just the first. So are nested resources: `/orders/{orderId}/lines/{lineId}` must verify that the line belongs to the order and the order to the caller. Write paths deserve extra care because mass assignment can let a client change the owner field itself. Finally, authorization decisions belong in tests: a parameterised test that replays each endpoint as a second tenant is one of the highest-value test suites you can have.

Practical backend example: Ownership in the query plus centralised policy:

```
// 1) ownership is part of the query - a forgotten check cannot leak data
Optional<Order> findByIdAndCustomerId(UUID id, UUID customerId);

// 2) shared resources: a central policy component, evaluated by method security
@Component("documentPolicy")
public class DocumentPolicy {
    public boolean canRead(UUID documentId, Authentication auth) {
        return permissions.hasAny(documentId, auth.getName(), Set.of(OWNER, COLLABORATOR, VIEWER));
    }
}

@PreAuthorize("@documentPolicy.canRead(#documentId, authentication)")
@GetMapping("/api/documents/{documentId}")
public DocumentDto get(@PathVariable UUID documentId) { ... }
```

```
-- 3) defence in depth for multi-tenancy
ALTER TABLE "order" ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON "order"
    USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

Common follow-ups:

- Why is a hidden UI not enough? The API is directly callable; the client is not a security boundary.
- Should you return 403 or 404? 404 avoids confirming the resource exists, which is usually preferable for other tenants' data.
- How do you handle bulk operations? Authorize every identifier in the request, and fail the whole operation or report per-item results explicitly.

Mistakes to avoid: `findById` followed by a forgettable check; trusting a tenant ID from the request body; sequential integer IDs as the only obstacle; skipping checks on nested and bulk endpoints.

Production perspective: Access-control regressions are silent — nothing errors, data simply leaks. Automated cross-tenant tests and periodic access reviews are the practical defences, along with logging denied attempts so probing is visible.

Related concepts covered: IDOR, multi-tenancy, row-level security, permission models, mass assignment, negative testing.

Q169. What should an access token contain and how is it validated?

Priority: Important

Why interviewers ask it: Token validation is easy to implement partially, and a partial implementation is an authentication bypass.

Interview-ready answer: A JWT access token carries registered claims — issuer, subject, audience, expiry, not-before, issued-at, and a token id — plus scopes or roles and any minimal application claims. Validation must check all of it: the signature against the issuer's published keys, that the algorithm is the expected asymmetric one rather than whatever the token says, the issuer, the audience (so a token for another API is rejected), and the time claims with a small clock-skew allowance. Keys are fetched from the JWKS endpoint and selected by the `kid` header, with caching and graceful handling of rotation. Tokens should be short-lived, and anything sensitive must stay out of the claims because they are readable.

In-depth explanation: The historical alg: none attack and key-confusion attacks (signing with the public key as an HMAC secret) are why the verifier must fix the expected algorithm rather than trusting the header. Clock skew is a real operational issue: allowing 30–60 seconds avoids spurious failures between machines. Rotation works through `kid`: the provider publishes both old and new keys during an overlap period, and your JWKS cache must refresh when it sees an unknown `kid` — with rate limiting so an attacker cannot force constant fetches. Opaque tokens are the alternative: they carry no data and are validated by introspection against the authorization server, which gives instant revocation at the cost of a network call per request. Spring Security implements all of this when you configure `issuer-uri`, but you should be able to explain what it is doing.

Practical backend example: Explicit validators, including audience:

```
@Bean
JwtDecoder jwtDecoder(@Value("${security.issuer-uri}") String issuer) {
    NimbusJwtDecoder decoder = JwtDecoders.fromIssuerLocation(issuer); // discovery + JWKS caching
    decoder.setJwtValidator(new DelegatingOAuth2TokenValidator<>{
        JwtValidators.createDefaultWithIssuer(issuer), // iss, exp, nbf
        new JwtClaimValidator<List<String>>("aud", // audience
            aud -> aud != null && aud.contains("orders-api")), // clock skew
        new JwtTimestampValidator(Duration.ofSeconds(30))));
    return decoder;
}
```

Common follow-ups:

- How does key rotation work? The provider publishes multiple keys; the token's kid selects one, and clients refresh the JWKS cache when they see an unknown key id.
- Why validate the audience? Otherwise a token issued for a different API of the same issuer would be accepted here.
- Opaque tokens or JWTs? Opaque tokens give instant revocation through introspection; JWTs avoid a network call per request. Choose based on revocation needs and latency budget.

Mistakes to avoid: Trusting the alg header; skipping audience or issuer checks; no clock-skew tolerance; caching JWKS forever; putting personal data in claims; accepting expired tokens because "the gateway checked already".

Production perspective: Alert on authentication failure spikes and on JWKS fetch errors. A provider key rotation combined with an unrefreshable cache is a classic total-outage scenario that is easy to prevent and hard to diagnose under pressure.

Related concepts covered: JWT claims, JWKS and kid rotation, algorithm confusion, clock skew, token introspection, audience scoping.

Q170. How do you manage secrets across environments?

Priority: Important

Why interviewers ask it: Leaked credentials are among the most common causes of real breaches, and prevention is mostly process plus tooling.

Interview-ready answer: Secrets never live in source control or container images. They are supplied at runtime — environment variables from a secret manager, mounted files, or a direct integration such as Vault or a cloud secret service — and referenced from configuration with placeholders. Each environment has its own secrets, and production secrets are not accessible from developer machines. Rotation must be possible without a code change and ideally without downtime, which means the application reads the secret at startup or refresh rather than baking it in, and supports an overlap period for credential change. Secret scanning runs in CI so an accidental commit is caught immediately, and any leaked secret is rotated, not just deleted from history.

In-depth explanation: Kubernetes Secrets are base64-encoded, not encrypted, unless encryption at rest is enabled, and they are visible to anyone who can read the namespace — so RBAC matters as much as the storage. Mounted files are often better than environment variables because environment variables leak into child processes, crash dumps and some logging frameworks. Short-lived dynamic credentials — for example a database credential issued per pod with a TTL — are stronger than long-lived static ones. For local development, use separate non-production credentials and a .env file that is gitignored, never a copy of production values. Also secure the audit trail: who read which secret and when is valuable during an incident.

Practical backend example: Configuration referencing runtime-provided secrets:

```
spring:
  datasource:
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD} # injected from the secret store, never committed
  config:
    import: "configtree:/etc/secrets/" # Kubernetes secret mounted as files
```

```
# Kubernetes: mount as files, restrict with RBAC, enable encryption at rest
volumes:
  - name: app-secrets
    secret: { secretName: orders-api-secrets, defaultMode: 0400 }
```

```
# CI: block commits containing credentials
gitLeaks detect --no-git --redact --exit-code 1
```

Common follow-ups:

- Should secrets be in images? No; images are shared, cached and often pushed to registries with broad read access.
- How do you rotate without downtime? Support two valid credentials during an overlap window, or restart with the new value behind a rolling deploy.
- What if a secret leaks? Rotate immediately and audit usage; removing the commit does not invalidate the credential.

Mistakes to avoid: Committing application-prod.yml with real values; passing secrets as command-line arguments (visible in the process list); sharing one credential across environments; treating base64 as encryption.

Production perspective: Make rotation routine rather than an emergency procedure — a credential that has never been rotated cannot be rotated quickly under pressure. Test the rotation path in a lower environment on a schedule.

Related concepts covered: Secret managers, config trees, dynamic credentials, secret scanning, Kubernetes RBAC, rotation procedures.

Q171. How do you handle sensitive data in logs and responses?

Priority: Important

Why interviewers ask it: Logs are widely readable and long-lived, so they are a common and overlooked source of data exposure.

Interview-ready answer: The principle is minimisation: log what is needed to operate the system and nothing more. Never log credentials, tokens, full card numbers, or entire request and response bodies containing personal data. Use identifiers instead of values — a customer ID rather than an email — and mask where a value must be partially visible. In responses, return only the fields the client needs, and keep internal details such as stack traces, SQL and hostnames out entirely. For debugging, rely on correlation IDs plus targeted, time-boxed verbose logging in a controlled environment rather than logging everything permanently. Redaction should be enforced structurally, for example a serialiser that masks annotated fields, because relying on every developer to remember is not a control.

In-depth explanation: Structured logging makes redaction practical: with key-value fields you can apply an allowlist, whereas free-text messages are hard to filter reliably. Log retention matters as much as content: if personal data ends up in logs, it inherits the retention and access policy of your logging system, which is usually broader than the database's. Third-party log aggregation and error trackers extend that reach further. Exceptions are a frequent leak path — an exception message containing a query with parameter values, or a validation error echoing the rejected value. Also consider derived exposure: a log line that records "user X requested document Y" may itself be sensitive in some domains.

Practical backend example: Structural redaction rather than developer discipline:

```
public record PaymentRequest(
    @JsonSerialize(using = MaskedSerializer.class) String cardNumber, // "**** * 4242"
    @JsonIgnore String cvv, // never serialised at all
    long amountMinor, String currency) {}

// logging: identifiers and outcomes, never payloads
log.info("payment processed paymentId={} customerId={} amountMinor={} currency={} outcome={}",
    payment.id(), payment.customerId(), payment.amountMinor(), payment.currency(), outcome);
// avoid: log.debug("payment request {}", request); // would serialise the whole object
```

```
<!-- Logback: last-resort pattern-based masking for known token formats -->
<replace regex="(Bearer\s+)[A-Za-z0-9._-]+" replacement="$1[REDACTED]"/>
```

Common follow-ups:

- How do you debug without logging tokens? Log a hash or the last four characters, plus a correlation ID that ties the request together.
- What about request bodies? Log them only in non-production with synthetic data, or log a redacted subset of fields.
- Who can read your logs? Usually many more people than can read the database — treat that as part of the threat model.

Mistakes to avoid: log.debug of whole DTOs in production; exception messages containing parameter values; stack traces in API responses; assuming log retention is short when it is not.

Production perspective: Add a CI check or code review rule for logging statements that pass whole objects, and periodically grep logs for patterns such as Bearer or card-shaped digits. Finding a leak yourself is much cheaper than having it reported.

Related concepts covered: Data minimisation, structured logging, redaction strategies, correlation IDs, log retention and access, exception hygiene.

Q172. What security checks belong in a public file-upload endpoint?

Priority: Important

Why interviewers ask it: Upload endpoints combine several attack surfaces and are a realistic test of defensive thinking.

Interview-ready answer: Authenticate and authorize the upload, then enforce a size limit at the proxy and in the application so a huge file cannot exhaust memory or disk. Determine the type from the file's bytes, not the declared Content-Type or the extension, and accept only an allowlist. Generate the storage key yourself so the client's filename can never influence a path — that removes traversal entirely. Store outside the web root, ideally in object storage, and serve downloads through an authorized endpoint or a short-lived pre-signed URL with Content-Disposition: attachment and X-Content-Type-Options: nosniff. Where the risk warrants it, scan for malware and re-encode images to strip embedded payloads, and rate-limit uploads per user.

In-depth explanation: Serving user content from your main domain is the subtle risk: an uploaded HTML or SVG file can execute in the origin's context and steal sessions, which is why a separate domain or forced download is recommended. Archive files introduce decompression bombs, so limit the expanded size and entry count if you extract them. Metadata in images can contain location data you may not want to retain, and re-encoding removes both metadata and many embedded exploits. Quotas matter operationally: without per-user limits, storage cost becomes an availability problem. Finally, log uploads with the user, size and detected type — that record is what makes an abuse investigation possible.

Practical backend example: Layered checks in one handler:

```
@PostMapping(value = "/api/attachments", consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
@PreAuthorize("hasRole('USER')")
public AttachmentDto upload(@RequestPart MultipartFile file, @AuthenticationPrincipal Jwt jwt) throws
    IOException {
    if (file.getSize() > 10L * 1024 * 1024) throw new PayloadTooLargeException();           // size
    quotaService.assertWithinQuota(jwt.getSubject(), file.getSize());                    // abuse
    String detected = contentTypeDetector.detect(file.getInputStream());                  // magic
    bytes
    if (!Set.of("image/png", "image/jpeg", "application/pdf").contains(detected)) {
        throw new UnsupportedFileTypeException(detected);                               //
        allowlist
    }
    byte[] safe = detected.startsWith("image/")
        ? imageService.reencode(file.getBytes(), detected) // strips metadata and payloads
        : file.getBytes();
    String key = "attachments/" + jwt.getSubject() + "/" + UUID.randomUUID();           //
    generated key
    objectStorage.put(key, safe, detected);
    return attachmentService.record(key, sanitizeName(file.getOriginalFilename()), detected, safe
        .length);
}
```

Common follow-ups:

- Can Content-Type be trusted? No; it is client-supplied. Detect from the content and validate against an allowlist.
- How do you prevent path traversal? Generate the storage key server-side; never concatenate client-supplied names into paths.
- Why serve uploads from a different domain? So that a malicious HTML or SVG file cannot execute in your application's origin.

Mistakes to avoid: Trusting the extension; storing files under the web root; no per-user quota; extracting archives without limits; serving user content inline on the main domain.

Production perspective: Uploads are also a cost and capacity concern, not only a security one. Set quotas, monitor storage growth, and keep upload traffic on a path that cannot starve interactive requests.

Related concepts covered: Content sniffing, allowlists, generated storage keys, pre-signed URLs, decompression bombs, image re-encoding, quotas.

Chapter 8. Testing, debugging, and coding exercises

Examples use JUnit 5, Mockito, AssertJ, Spring Boot 3 test slices and Testcontainers. Coding exercises state their assumptions explicitly, because in an interview the assumptions you name are part of the answer.

Q173. How do unit, slice, integration and end-to-end tests differ?

Priority: Must Know

Why interviewers ask it: A sensible test strategy is a design skill; it shows whether you optimise for feedback speed or for false confidence.

Interview-ready answer: A unit test exercises one class with its collaborators replaced by fakes or mocks; it runs in milliseconds and pinpoints failures, but it cannot prove that the pieces fit together. A slice test boots a narrow part of the Spring context — `@WebMvcTest` for the web layer, `@DataJpaTest` for persistence — so it verifies framework behaviour such as routing, serialisation and query mapping without the whole application. An integration test with `@SpringBootTest` wires real components together, ideally against a real database from Testcontainers, and that is where transaction and SQL behaviour is proven. End-to-end tests drive the deployed system through its public interface and are the slowest and most brittle, so I keep only a few for critical journeys. The shape is a pyramid: many fast tests, fewer slow ones.

In-depth explanation: The value of each level is different. Unit tests protect logic and edge cases; slice tests protect the contract with the framework; integration tests protect assumptions about the database, transactions and wiring; end-to-end tests protect the assembled system and deployment. Mocked unit tests are the easiest to write and the easiest to make meaningless — they can pass while the application is completely broken, because the mocks encode your assumptions rather than reality. Conversely, an all-integration suite is slow enough that people stop running it. Practical guidance: test business rules as units, test persistence against a real engine, test the API contract with slices, and reserve end-to-end for a handful of journeys. Also decide what you are not testing: framework behaviour itself does not need your tests.

Practical backend example: The same feature at three levels:

```
// unit: pure rule, no Spring
@Test void cancellingAPaidOrderIsAllowed() {
    Order order = Order.paid();
    order.cancel("customer request");
    assertThat(order.status()).isEqualTo(OrderStatus.CANCELLED);
}

// slice: HTTP contract only, service mocked
@WebMvcTest(OrderController.class)
class OrderControllerTest { /* MockMvc asserts status, JSON, validation */ }

// integration: real Postgres, real transaction, real SQL
@SpringBootTest
@Testcontainers
class OrderServiceIT {
    @Container static PostgreSQLContainer<?> db = new PostgreSQLContainer<>("postgres:16-alpine");
    @DynamicPropertySource static void props(DynamicPropertyRegistry r) {
        r.add("spring.datasource.url", db::getJdbcUrl);
        r.add("spring.datasource.username", db::getUsername);
        r.add("spring.datasource.password", db::getPassword);
    }
}
```

Common follow-ups:

- When do you use `@SpringBootTest`? When the behaviour under test spans layers — transactions, wiring, real SQL — not for logic you can test directly.
- Is 100% coverage a good goal? No; coverage measures execution, not assertions. Target meaningful coverage of business rules and boundaries.
- Why Testcontainers instead of H2? Because production runs PostgreSQL, and dialect, constraint and function differences make H2 results unreliable.

Mistakes to avoid: Mock-heavy tests that mirror the implementation; only end-to-end tests; treating coverage as the goal; testing getters and framework internals.

Production perspective: Test speed determines how often the suite runs, which determines how early defects are caught. A suite over ten minutes gets skipped locally; invest in slices and parallelism before adding more end-to-end tests.

Related concepts covered: Test pyramid, test slices, Testcontainers, coverage limitations, feedback loops.

Q174. How would you test a service with Mockito without over-mocking?

Priority: Must Know

Why interviewers ask it: Mockito misuse produces tests that are expensive to maintain and prove nothing.

Interview-ready answer: I mock collaborators that cross a boundary — repositories, HTTP clients, message publishers — and use real objects for value types, DTOs and pure logic. I stub only what the test actually needs, assert on the outcome rather than on every interaction, and use `verify` sparingly for effects that have no observable return value, such as publishing an event. `ArgumentCaptor` checks what was passed when that matters. I avoid deep stubs and mocking types I do not own; for a third-party client I wrap it in my own interface and mock that. Strict stubbing in `MockitoExtension` catches unused stubs, which usually indicate the test has drifted from the code.

In-depth explanation: The failure mode of over-mocking is a test that encodes the implementation: it breaks on every refactor and passes when behaviour is wrong. A useful heuristic is that a test should survive a rewrite of the method's internals as long as the observable behaviour is unchanged. Mocking a value object such as `Money` is a smell — construct a real one. Mocking static methods or final classes is possible with the inline mock maker but usually signals a design problem; injecting a `Clock` is better than mocking `Instant.now()`. For repositories, mocking is reasonable in a unit test, but query correctness must then be covered by a persistence test, because a mocked repository cannot tell you that your JPQL is wrong. Where several collaborators are involved, a hand-written fake can be clearer and more reusable than five stubs.

Practical backend example: Focused stubbing with outcome-based assertions:

```
@ExtendWith(MockitoExtension.class)           // strict stubs: unused stubbing fails the test
class CheckoutServiceTest {
    @Mock OrderRepository orders;              // boundary: mocked
    @Mock PaymentGateway payments;            // boundary: mocked
    @InjectMocks CheckoutService service;

    @Test
    void publishesPaidEventAndPersistsReceipt() {
        Order order = Order.pending(Money.of(1999, "EUR")); // real value objects
        given(orders.findById(order.id())).willReturn(Optional.of(order));
        given(payments.capture(any())).willReturn(new Receipt("rcp_1"));

        CheckoutResult result = service.checkout(order.id());

        assertThat(result.status()).isEqualTo(PAID); // assert the outcome
        ArgumentCaptor<Order> saved = ArgumentCaptor.forClass(Order.class);
        verify(orders).save(saved.capture()); // verify the effect that matters
        assertThat(saved.getValue().receiptId()).isEqualTo("rcp_1");
    }
}
```

Common follow-ups:

- Should you mock a value object? No; construct a real one. Mock behaviour, not data.
- When is `verify` appropriate? For side effects with no return value — a publish, a send, a delete — not for every call.
- What about static methods and `Instant.now()`? Inject a `Clock` instead of mocking statics; it is simpler and makes time explicit.

Mistakes to avoid: Stubbing everything including unused paths; `RETURNS_DEEP_STUBS`; verifying interactions that are implementation details; mocking the class under test; mocking types you do not own without a wrapper.

Production perspective: Tests are read far more often than they are written, and they are the documentation people trust. A test that states a business rule clearly is worth several that assert interaction sequences.

Related concepts covered: Test doubles, strict stubbing, ArgumentCaptor, hand-written fakes, Clock injection, behaviour versus implementation.

Q175. How do you test repository queries and database constraints?

Priority: Must Know

Why interviewers ask it: Persistence bugs are invisible to mocked tests, and the choice of test database determines whether they are caught.

Interview-ready answer: I test repositories against the same database engine as production, using Testcontainers to start PostgreSQL for the test run. `@DataJpaTest` gives a fast persistence slice, but by default it replaces the `datasource` with an embedded database, so I disable that with `@AutoConfigureTestDatabase(replace = NONE)` and point it at the container. Then I test what mocks cannot: that the JPQL or native query returns the right rows, that mappings and column types are correct, that unique and check constraints actually fire, and that migrations apply cleanly. Because `@DataJpaTest` rolls back each test, I use `TestTransaction` or explicit flushes when I need to observe constraint violations or committed state.

In-depth explanation: H2 in PostgreSQL compatibility mode is convenient but differs in dialect, function availability, constraint behaviour, type coercion and locking, so a test can pass on H2 and fail in production — the classic case is a native query using `DISTINCT ON`, `jsonb` or `ON CONFLICT`. Container startup cost is manageable: reuse one container per suite (a static container is initialised once per JVM), and enable Testcontainers reuse locally. Running Flyway migrations in the test gives the additional benefit of verifying the migration scripts themselves. For constraint tests, remember that violations surface at flush, not at save, so an explicit `flush()` makes the assertion land where you expect.

Practical backend example: Real database, real constraint, real query:

```
@DataJpaTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE) // no in-memory swap
@Testcontainers
class OrderRepositoryTest {
    @Container
    static PostgreSQLContainer<?> db = new PostgreSQLContainer<>("postgres:16-alpine");

    @DynamicPropertySource
    static void datasource(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", db::getJdbcUrl);
        registry.add("spring.datasource.username", db::getUsername);
        registry.add("spring.datasource.password", db::getPassword);
    }

    @Autowired OrderRepository orders;
    @Autowired TestEntityManager em;

    @Test
    void uniqueClientReferenceIsEnforcedByTheDatabase() {
        orders.save(Order.withReference("ref-1"));
        em.flush();
        orders.save(Order.withReference("ref-1"));
        assertThatThrownBy(em::flush) // violation surfaces at flush
            .isInstanceOf(PersistenceException.class);
    }

    @Test
    void searchAppliesStatusFilterAndOrdering() {
        em.persist(Order.paid(Instant.parse("2026-01-01T00:00:00Z")));
        em.persist(Order.cancelled(Instant.parse("2026-02-01T00:00:00Z")));
        em.flush();
        assertThat(orders.search(OrderStatus.PAID, PageRequest.of(0, 10)))
            .extracting(Order::status).containsExactly(OrderStatus.PAID);
    }
}
```

Common follow-ups:

- Why can H2 differ from PostgreSQL? Different dialect, functions, constraint enforcement and type handling; native SQL especially will not match.
- Why did the constraint violation not appear at save? It appears at flush, when the statement reaches the database.

- How do you keep container tests fast? One static container per suite, Testcontainers reuse locally, and run the slice rather than the full context.

Mistakes to avoid: Testing queries against mocks; H2 for anything using native SQL; ignoring migration scripts in tests; assuming a rolled-back test proves committed behaviour.

Production perspective: Repository tests against a real engine also validate your migrations, which is where many production incidents originate. Running them in CI on every pull request is cheap insurance.

Related concepts covered: @DataJpaTest, Testcontainers, flush timing, migration verification, dialect differences, TestEntityManager.

Q176. How do you test MVC status, validation and security?

Priority: Must Know

Why interviewers ask it: The API contract is what clients depend on, and these tests are what stop it changing by accident.

Interview-ready answer: I use @WebMvcTest with MockMvc to exercise the real dispatcher: routing, argument binding, Bean Validation, message conversion, exception handling and the security filter chain. Then I assert the things clients rely on — status codes, headers such as Location, and the JSON shape with JSONPath — for the happy path and for failures: 400 with field errors, 404 for missing resources, 409 for conflicts. Security is part of the contract, so I assert 401 for anonymous requests, 403 for an insufficient role and success for the right one, using @WithMockUser or JWT post-processors. Calling the controller method directly would skip all of that, which is why it is not a substitute.

In-depth explanation: MockMvc runs the Spring MVC infrastructure without a servlet container or network, so it is fast but does not cover container-level behaviour such as compression or connection handling; @SpringBootTest(webEnvironment = RANDOM_PORT) with a real client covers that when needed. For OAuth2 resource servers, SecurityMockMvcRequestPostProcessors.jwt() builds an authenticated request with chosen authorities and claims, which is cleaner than constructing tokens. Validation tests should assert the error body structure, not just the status, because clients parse it. It is also worth asserting that unknown or extra fields behave as documented, and that a malformed body returns 400 rather than 500. Keep a small set of these tests per controller — they are the contract's regression suite.

Practical backend example: Contract tests including negative cases:

```
@WebMvcTest(OrderController.class)
class OrderControllerContractTest {
    @Autowired MockMvc mvc;
    @MockitoBean OrderService orderService;          // Boot 3.4+; @MockBean in earlier versions

    @Test void anonymousIsRejected() throws Exception {
        mvc.perform(get("/api/orders/{id}", UUID.randomUUID()))
            .andExpect(status().isUnauthorized());      // 401
    }

    @Test void wrongScopeIsForbidden() throws Exception {
        mvc.perform(get("/api/orders/{id}", UUID.randomUUID())
            .with(jwt().authorities(new SimpleGrantedAuthority("SCOPE_other"))))
            .andExpect(status().isForbidden());          // 403
    }

    @Test void validationErrorsAreReportedPerField() throws Exception {
        mvc.perform(post("/api/orders").with(jwt().authorities(new SimpleGrantedAuthority(
            "SCOPE_orders.write"))))
            .contentType(MediaType.APPLICATION_JSON)
            .content("{\"lines\":[]}")
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.errors[*].field").value(hasItem("lines")))
            .andExpect(jsonPath("$.correlationId").exists());
    }
}
```

Common follow-ups:

- How do you test unauthorized requests? Perform the request without authentication and assert 401; with the wrong authority and assert 403.

- Does MockMvc use a real server? No; it dispatches through Spring MVC in-process. Use RANDOM_PORT when you need real HTTP.
- Should you assert the whole JSON body? Assert the fields that form the contract; whole-body snapshots are brittle.

Mistakes to avoid: Calling controller methods directly; disabling security in tests and never testing it; asserting only 200; ignoring the error body structure that clients parse.

Production perspective: These tests are your defence against silent contract changes — a renamed field or a changed status code is caught in CI rather than by a client at 2 a.m. Pair them with an OpenAPI diff check on the specification.

Related concepts covered: MockMvc, JSONPath assertions, security test support, error contract testing, OpenAPI validation.

Q177. How would you test transaction rollback and concurrency?

Priority: Important

Why interviewers ask it: These behaviours are where subtle production bugs live, and testing them requires understanding the test infrastructure itself.

Interview-ready answer: The first thing to know is that a `@Transactional` test method runs inside a transaction that is rolled back at the end, which can mask real behaviour: constraint violations that would occur at commit, the effect of `REQUIRES_NEW`, and anything another thread should see. To test rollback honestly, I let the service manage its own transaction and assert from outside it — with a non-transactional test that calls the service, then verifies the state in a fresh transaction. For concurrency, I run two real threads with a latch to control interleaving, each using its own transaction, and assert the outcome: an optimistic lock exception, a conditional update affecting zero rows, or a correct final balance.

In-depth explanation: `TestTransaction` gives fine control inside a transactional test — you can flag rollback, end the transaction and start a new one — which is useful for checking what was actually committed. For concurrency tests, the important detail is that each thread needs its own transaction and its own connection, so the pool must be large enough or the test deadlocks; and since the main thread's transaction would not be visible to them, the test should not be transactional itself. Deterministic interleaving with `CountDownLatch` is far better than `Thread.sleep`, which produces flaky tests. Clean-up matters too: because these tests commit, they must delete their data or use a fresh schema, otherwise subsequent tests see leftovers.

Practical backend example: A genuine concurrency test with two transactions:

```
@SpringBootTest                                // NOT @Transactional: we need real commits
@Testcontainers
class TransferConcurrencyIT {
    @Autowired TransferService transfers;
    @Autowired AccountRepository accounts;

    @Test
    void concurrentWithdrawalsCannotOverdraw() throws Exception {
        UUID account = accounts.save(Account.with(100_00)).getId(); // 100.00
        CountDownLatch start = new CountDownLatch(1);
        ExecutorService pool = Executors.newFixedThreadPool(2);

        Callable<Boolean> withdraw = () -> {
            start.await(); // interleave deterministically
            try { transfers.withdraw(account, 80_00); return true; }
            catch (InsufficientFundsException e) { return false; }
        };
        Future<Boolean> a = pool.submit(withdraw);
        Future<Boolean> b = pool.submit(withdraw);
        start.countDown();

        assertThat(List.of(a.get(), b.get())).containsExactlyInAnyOrder(true, false);
        assertThat(accounts.findById(account).orElseThrow().balanceMinor()).isEqualTo(20_00);
        pool.shutdown();
    }
}
```

Common follow-ups:

- Why can a test transaction mask behaviour? It rolls back and never commits, so commit-time constraints, other transactions and after-commit listeners do not behave realistically.
- How do you assert a rollback happened? Call the service non-transactionally, then read the state in a new transaction and assert nothing was written.
- Why not use `Thread.sleep` to interleave? It is timing-dependent and flaky; latches make the ordering deterministic.

Mistakes to avoid: Annotating concurrency tests with `@Transactional`; a connection pool smaller than the thread count; leaving committed test data behind; testing rollback by inspecting the same transaction that will be rolled back anyway.

Production perspective: A single concurrency test around the money-critical operation is worth dozens of unit tests. It is also the test most likely to fail after a seemingly unrelated change to locking or isolation.

Related concepts covered: `TestTransaction`, commit visibility, optimistic locking failures, latch-based interleaving, test data cleanup.

Q178. What makes a flaky test and how do you diagnose one?

Priority: Important

Why interviewers ask it: Flaky tests erode trust in the suite, and the causes are a good proxy for understanding shared state and timing.

Interview-ready answer: The usual causes are shared mutable state between tests, dependence on execution order, real time and time zones, asynchronous work awaited with sleeps, random data without a fixed seed, and external dependencies such as networks or ports. Diagnosis starts with reproduction: run the test in isolation, then repeatedly, then in a random order, and check whether it fails only in parallel execution. The fixes are structural — isolate state, inject a fixed `clock`, poll with `Awaitility` instead of sleeping, seed randomness, use dynamic ports and containers. Quarantining a flaky test is acceptable briefly, but retry-on-failure as a permanent policy hides real race conditions that also exist in production.

In-depth explanation: Asynchronous assertions are the most common source: the test asserts immediately after triggering async work, so it passes on a fast machine and fails on a loaded CI runner. `Awaitility's await().atMost(...).untilAsserted(...)` polls until the condition holds or the timeout expires, which is both faster and more reliable than a fixed sleep. Time-dependent failures cluster around midnight, month ends, daylight-saving transitions and leap days — an injected `clock` makes these testable instead of accidental. Shared state includes static fields, caches, the database, message queues and files; with parallel execution, each test class needs its own data namespace. Finally, a flaky test sometimes reveals a genuine race in the code, so investigate before disabling.

Practical backend example: Replacing sleeps and wall-clock time:

```
// flaky: timing-dependent
// service.publishAsync(event); Thread.sleep(500); assertThat(repository.count()).isEqualTo(1);

// stable: poll until the condition holds, fail fast when it does not
service.publishAsync(event);
await().atMost(Duration.ofSeconds(5))
    .pollInterval(Duration.ofMillis(50))
    .untilAsserted(() -> assertThat(repository.count()).isEqualTo(1));
```

```
@TestConfiguration
static class FixedClockConfig {
    @Bean Clock clock() { return Clock.fixed(Instant.parse("2026-03-14T09:00:00Z"), ZoneOffset.UTC); }
}
```

Common follow-ups:

- Should you add a sleep? No; it makes tests slower and still flaky. Poll for the condition or make the operation synchronous in tests.
- How do you find order dependence? Run the suite in random order and in isolation; a test that only passes in one order shares state.
- Is automatic retry acceptable? As a short-term mitigation with tracking, not as a policy — it hides real races.

Mistakes to avoid: `Thread.sleep` everywhere; static mutable fixtures; `LocalDate.now()` in assertions; hard-coded ports; disabling a flaky test without investigating whether the code is racy.

Production perspective: Track flaky tests explicitly — a rate, an owner and a deadline. A suite that fails randomly trains engineers to re-run rather than investigate, and eventually a real regression is dismissed as flakiness.

Related concepts covered: Test isolation, Awaitility, Clock injection, parallel execution, random seeds, quarantine policy.

Q179. Write a function to find the first non-repeating character.

Priority: Important

Why interviewers ask it: It is a compact exercise that reveals collection choice, complexity reasoning and whether you think about Unicode.

Interview-ready answer: Count occurrences in one pass into a `LinkedHashMap`, which preserves insertion order, then scan the entries for the first with a count of one — two passes, $O(n)$ time and $O(k)$ space in the number of distinct characters. An alternative is to store first-seen indices and take the minimum index among characters seen once. The assumption worth stating is what "character" means: iterating `char` values breaks on characters outside the Basic Multilingual Plane, such as emoji, which are surrogate pairs. If the input may contain those, iterate code points instead. I would ask the interviewer whether the input is ASCII, in which case a 128-entry array is faster and simpler.

In-depth explanation: The `LinkedHashMap` approach is attractive because the ordering is explicit rather than implicit in a second scan of the string. If the string is very long and the alphabet small, an `int[]` counting array plus one re-scan of the string avoids hashing entirely. For code points, `String.codePoints()` gives an `IntStream`, and results should be returned as a code point or a `String` rather than a `char`, since a `char` cannot hold a supplementary character. Case sensitivity and locale are also worth clarifying: "first non-repeating" under case-insensitive comparison is a different problem, and case folding is locale-dependent for some languages. Stating these assumptions explicitly is what distinguishes a senior answer.

Practical backend example: Both variants, with the Unicode-safe one preferred:

```
/** ASCII/BMP-safe version: counts char values. O(n) time, O(k) space. */
static Optional<Character> firstNonRepeating(String input) {
    Map<Character, Integer> counts = new LinkedHashMap<>();
    for (char c : input.toCharArray()) counts.merge(c, 1, Integer::sum);
    return counts.entrySet().stream()
        .filter(e -> e.getValue() == 1)
        .map(Map.Entry::getKey)
        .findFirst();
}

/** Unicode-safe: iterates code points, so emoji and supplementary characters work. */
static Optional<String> firstNonRepeatingCodePoint(String input) {
    Map<Integer, Integer> counts = new LinkedHashMap<>();
    input.codePoints().forEach(cp -> counts.merge(cp, 1, Integer::sum));
    return counts.entrySet().stream()
        .filter(e -> e.getValue() == 1)
        .map(e -> new String(Character.toChars(e.getKey())))
        .findFirst();
}
```

Common follow-ups:

- What changes for code points? Supplementary characters occupy two `char` values, so `char`-based counting splits them; use `codePoints()`.
- Can you do it in one pass? You can track first index and count together and take the minimum index at the end, which is still two traversals overall but only one over the string.
- What if the alphabet is known and small? Use a fixed-size `int[]` and re-scan the input; no hashing, better cache behaviour.

Mistakes to avoid: Using `HashMap` and then assuming iteration order; comparing with `==` on boxed `Integer` counts; ignoring Unicode; returning `char` when the answer may be a supplementary character; `indexOf/lastIndexOf` inside a loop, which is $O(n^2)$.

Production perspective: In real backend code this appears as deduplication or validation over user-supplied text, where Unicode correctness matters — names, addresses and messages routinely contain characters outside ASCII.

Related concepts covered: LinkedHashMap ordering, merge idiom, code points versus chars, complexity analysis, clarifying assumptions.

Q180. Write a function to merge overlapping intervals.

Priority: Important

Why interviewers ask it: It tests sorting, boundary conditions and the discipline of clarifying ambiguous requirements.

Interview-ready answer: Sort the intervals by start, then sweep: keep the current interval, and for each next one, if it starts at or before the current end, extend the end to the maximum of the two; otherwise emit the current and start a new one. That is $O(n \log n)$ for the sort plus $O(n)$ for the sweep. The question to clarify first is whether touching intervals — one ending exactly where the next begins — should merge; for half-open ranges such as time slots they usually should, for inclusive integer ranges it depends. I also handle the empty input, single-element input and fully contained intervals, which the maximum handles naturally.

In-depth explanation: The correctness of the sweep depends on sorting by start: without it, a later interval could overlap an already-emitted one. Using `Comparator.comparing` on a `long` or `Instant` avoids the subtraction overflow trap. For half-open intervals `[start, end)` the merge condition is `next.start <= current.end`; for closed intervals where adjacency should not merge, it is `next.start < current.end`. Mutating the input list is usually undesirable, so copy before sorting. In backend systems this problem appears as merging availability slots, coalescing maintenance windows, or compressing time-series ranges, and there the values are usually `Instant` with time zone questions behind them — another assumption worth naming.

Practical backend example: A generic, documented implementation:

```
public record Interval(Instant start, Instant end) {
    public Interval {
        if (end.isBefore(start)) throw new IllegalArgumentException("end before start");
    }
}

/** Merges half-open intervals [start, end); touching intervals are merged. O(n log n). */
static List<Interval> merge(List<Interval> input) {
    if (input.size() < 2) return List.copyOf(input);
    List<Interval> sorted = new ArrayList<>(input);           // do not mutate the
    caller's list
    sorted.sort(Comparator.comparing(Interval::start).thenComparing(Interval::end));

    List<Interval> merged = new ArrayList<>();
    Instant start = sorted.get(0).start();
    Instant end = sorted.get(0).end();
    for (Interval next : sorted.subList(1, sorted.size())) {
        if (!next.start().isAfter(end)) {                    // overlap or touch
            end = next.end().isAfter(end) ? next.end() : end; // handles containment
        } else {
            merged.add(new Interval(start, end));
            start = next.start();
            end = next.end();
        }
    }
    merged.add(new Interval(start, end));
    return merged;
}
```

Common follow-ups:

- Are touching intervals merged? Here yes, because the intervals are half-open; for closed ranges, clarify and change the comparison to strict.
- What about a fully contained interval? The max on the end handles it — the wider interval's end is kept.
- Can it be done without sorting? Only with extra structure, such as an interval tree, which is worthwhile when intervals arrive incrementally.

Mistakes to avoid: Forgetting to emit the last interval; sorting by end instead of start; mutating the input; integer subtraction in the comparator; not clarifying the touching-interval rule.

Production perspective: In scheduling and availability features this logic often runs on every request. If the interval set is large and stable, precompute the merged set and cache it rather than merging per call.

Related concepts covered: Sweep algorithms, comparator construction, half-open ranges, defensive copying, requirement clarification.

Q181. Write SQL for top customers by monthly order total.

Priority: Must Know

Why interviewers ask it: It combines grouping, date handling, ordering and the judgement to ask what counts as revenue.

Interview-ready answer: Group paid orders by month and customer, sum the totals, order by the sum descending and limit. The clarifying questions matter as much as the SQL: which statuses count, are refunds subtracted, which time zone defines a month, and should customers with no orders appear. In PostgreSQL I use `date_trunc('month', created_at AT TIME ZONE :tz)` so the month boundary matches the business time zone rather than UTC, filter on an indexed date range, and use a window function if I need the top N per month rather than overall. Money is summed in integer minor units to avoid floating point error.

In-depth explanation: Filtering with `created_at >= :from AND created_at < :to` keeps the predicate sargable so an index can be used, whereas wrapping the column in a function such as `date_trunc` in the WHERE clause usually prevents index use unless an expression index exists. Refunds are the most common correctness issue: either join a refunds table and subtract, or model all movements in one ledger table with signed amounts, which makes the aggregate trivial. For top N per month, `ROW_NUMBER() OVER (PARTITION BY month ORDER BY revenue DESC)` in a CTE is the standard approach. If this query runs frequently, a materialised view or a summary table refreshed nightly is usually the right production answer.

Practical backend example: The full query with refunds and top N per month:

```
WITH monthly AS (
    SELECT date_trunc('month', o.created_at AT TIME ZONE 'Europe/Berlin') AS month,
           o.customer_id,
           coalesce(sum(o.total_cents) FILTER (WHERE o.status = 'PAID'), 0) AS gross_cents,
           coalesce(sum(r.refunded_cents), 0) AS refunded_cents
    FROM "order" o
    -- LATERAL returns exactly one row per order, so joining refunds cannot
    -- fan out and inflate sum(o.total_cents)
    LEFT JOIN LATERAL (
        SELECT coalesce(sum(amount_cents), 0) AS refunded_cents
        FROM refund WHERE refund.order_id = o.id
    ) r ON true
    WHERE o.created_at >= :from AND o.created_at < :to      -- sargable range, uses the index
    GROUP BY 1, 2
), ranked AS (
    SELECT month, customer_id,
           (gross_cents - refunded_cents) AS net_cents,
           ROW_NUMBER() OVER (PARTITION BY month
                               ORDER BY (gross_cents - refunded_cents) DESC, customer_id) AS rn
    FROM monthly
)
SELECT r.month, c.name, r.net_cents
FROM ranked r
JOIN customer c ON c.id = r.customer_id
WHERE r.rn <= 10
ORDER BY r.month DESC, r.net_cents DESC;

CREATE INDEX idx_order_created_status ON "order" (created_at, status) INCLUDE (customer_id,
total_cents);
```

Common follow-ups:

- How do refunds affect the total? Subtract them, or model all movements as signed rows in one ledger so the sum is naturally net.
- Which time zone defines a month? Convert explicitly; using UTC when the business operates elsewhere shifts revenue across month boundaries.
- What if two customers tie? Add a deterministic tie-breaker such as `customer_id` so the result is stable.

Mistakes to avoid: Summing floating-point money; applying `date_trunc` to the column in the `WHERE` clause and losing the index; joining a one-to-many refunds table directly and inflating the order totals through fan-out; ignoring refunds and cancellations; unstable ordering; running an unbounded date range.

Production perspective: Revenue reports are checked by humans who notice discrepancies. Document exactly which statuses and adjustments are included, and validate the query against an independently calculated figure before anyone builds a dashboard on it.

Related concepts covered: Window functions, sargable predicates, time zones in aggregation, ledger modelling, covering indexes, report validation.

Q182. How do you debug a slow endpoint systematically?

Priority: Must Know

Why interviewers ask it: They want a repeatable method, not a list of guesses, because method is what works under pressure.

Interview-ready answer: I work outside in. First, quantify: which endpoint, which percentile, since when, and for all users or a subset — averages hide the problem. Second, locate the time with a distributed trace for a slow request: it attributes latency to spans, so I can see whether it is the database, a downstream call, or in-process work. Third, drill into the dominant span: for the database, look at query count and slow statements and run `EXPLAIN ANALYZE`; for downstream calls, check timeouts and retries; for in-process work, profile with JFR. Fourth, check the infrastructure layer — connection pool saturation, GC pauses, CPU throttling. Then fix one thing and verify with the same metric, rather than changing several things at once.

In-depth explanation: The order matters because the cheapest evidence comes first. Query count per request is particularly informative: 300 small queries means $N+1$, one slow query means a plan or index issue. Pool saturation appears as long waits before any query starts, which a trace shows as a gap. GC pauses appear as latency spikes affecting all endpoints simultaneously. Container CPU throttling appears as slowness that does not correlate with any span. It is also worth distinguishing a regression from a scaling limit: if the endpoint was fast last month with less data, the plan may have changed as the table grew. Always capture a before-and-after measurement; without it, you cannot tell whether the change helped or the load simply dropped.

Practical backend example: A concrete triage sequence:

```
# 1) quantify: percentile latency for the endpoint, before and after the change point
curl -s localhost:8080/actuator/metrics/http.server.requests \
  | jq '.availableTags, .measurements'

# 2) locate: open a slow trace and read the span breakdown (Tempo/Jaeger/Zipkin)

# 3) database: which statements and how many
# (development profile) hibernate statistics -> statement count per request
# (production) pg_stat_statements -> total_exec_time, calls, mean_exec_time
psql -c "SELECT calls, mean_exec_time, rows, query FROM pg_stat_statements
        ORDER BY mean_exec_time * calls DESC LIMIT 10;"

# 4) infrastructure
curl -s localhost:8080/actuator/metrics/hikaricp.connections.pending
curl -s localhost:8080/actuator/metrics/jvm.gc.pause
```

Common follow-ups:

- What do you measure first? The latency distribution for the specific endpoint, then the span breakdown of a slow request.
- How do you tell a database problem from a pool problem? A pool problem shows waiting before the query starts; the query itself is fast.
- What if nothing looks slow inside the application? Check container CPU throttling, network latency and the client's own processing.

Mistakes to avoid: Optimising the first thing you notice; using averages; changing several things at once; profiling in an environment with unrepresentative data; skipping the before-and-after comparison.

Production perspective: Write the sequence down as a runbook with the exact commands and dashboards. During an incident, a checklist beats improvisation, and it lets someone less familiar with the service make progress.

Related concepts covered: Latency percentiles, distributed tracing, `pg_stat_statements`, pool and GC metrics, CPU throttling, controlled experiments.

Q183. How do you investigate a sporadic 500 response?

Priority: Important

Why interviewers ask it: Intermittent failures require evidence-gathering discipline rather than reproduction luck.

Interview-ready answer: I start from the evidence I already have: the error rate over time, whether it correlates with a deploy, a specific instance, a specific tenant or a particular input, and what the exception actually is. Structured logs with a correlation ID let me pull the full request context for a failing call, including the trace. Then I look for patterns — all failures on one pod suggests a node or configuration issue; failures for one customer suggests data; failures at a fixed time suggests a scheduled job or a cache expiry. Once I have a hypothesis I confirm it with targeted logging or a metric before changing code. Meanwhile I make sure the response itself does not leak internals and that the client-visible behaviour is acceptable.

In-depth explanation: Aggregated error tracking with grouped stack traces is the most efficient starting point, because it shows frequency, first-seen time and affected users per error type. Frequency shape is diagnostic: a constant low rate suggests a data-dependent bug, a step change suggests a deploy or configuration change, and periodic spikes suggest a job or a cache stampede. Instance-specific failures often mean a partially applied configuration or a bad node. A failure affecting a single tenant usually means data that violates an assumption — a null where the code expects a value, an unusually large payload, an unexpected enum. Beware of the survivorship trap: if the error handler swallows exceptions, the true cause never reaches the logs, so check the logging path itself.

Practical backend example: Enriched logging and a targeted metric:

```
@ExceptionHandler(Exception.class)
ProblemDetail handle(Exception ex, HttpServletRequest request) {
    String correlationId = MDC.get("correlationId");
    log.error("unhandled endpoint={ } method={ } tenant={ } correlationId={ }",
        request.getRequestURI(), request.getMethod(), MDC.get("tenantId"), correlationId, ex);
    meterRegistry.counter("app.errors",
        "endpoint", request.getRequestURI(),
        "exception", ex.getClass().getSimpleName()).increment(); // groupable signal
    ProblemDetail p = ProblemDetail.forStatusAndDetail(HttpStatus.INTERNAL_SERVER_ERROR,
        "Unexpected error"); // no stack trace to the client
    p.setProperty("correlationId", correlationId);
    return p;
}
```

Common follow-ups:

- How do you avoid leaking stack traces? Log them server-side and return a generic message plus a correlation ID.
- What if it only affects one customer? Suspect data; fetch a failing request's payload shape from the trace, not from the logs if it contains personal data.
- How do you confirm a fix for something intermittent? Watch the error-rate metric for the specific exception over a period covering its previous frequency.

Mistakes to avoid: Reproducing blindly without reading the existing evidence; catching and swallowing exceptions; logging without correlation IDs; deploying a speculative fix and declaring victory before the next occurrence window.

Production perspective: Errors need an owner and a rate-based alert, not just a log line. An error budget framing — "this endpoint may fail 0.1% of requests" — turns intermittent failures into a tracked, prioritised problem instead of background noise.

Related concepts covered: Error aggregation, correlation IDs, failure pattern analysis, metrics by exception type, information disclosure, error budgets.

Q184. What test data and clock strategies make tests deterministic?

Priority: Important

Why interviewers ask it: Deterministic tests are a prerequisite for a trustworthy suite, and the techniques are concrete.

Interview-ready answer: Inject a `Clock` bean and use `Clock.fixed` in tests, so anything time-dependent is controllable; never call `Instant.now()` or `LocalDate.now()` directly in business code. Build test data with builders or object mothers that supply valid defaults and let each test override only what it cares about, which keeps the intent visible. Seed any randomness so a failure is reproducible. Isolate data per test — unique keys, a fresh schema, or cleanup — so tests can run in any order and in parallel. And pin the time zone in test configuration, because a test that passes in UTC can fail in a `+02:00` environment when a date boundary shifts.

In-depth explanation: Time is the most common non-determinism. Business rules such as "expires after 30 days" or "invoices on the first of the month" cannot be tested reliably against the real clock; with a fixed `Clock` you can assert exact boundaries and test daylight-saving transitions deliberately. For data, the object-mother pattern reduces noise: `OrderMother.paid()` conveys intent better than fifteen lines of setup. Randomised property-based testing is valuable but must log the seed on failure so it can be replayed. For database isolation, a per-test transaction rollback is fastest but misleading for commit behaviour (see Q177), so committing tests should use unique identifiers or truncate tables between runs. Parallel execution multiplies all of these requirements, which is why isolation should be designed in rather than retrofitted.

Practical backend example: Clock injection and a data builder:

```
@Service
public class SubscriptionService {
    private final Clock clock; // injected, never Instant.now()
    public SubscriptionService(Clock clock) { this.clock = clock; }

    public boolean isExpired(Subscription s) {
        return s.expiresAt().isBefore(Instant.now(clock));
    }
}

@Bean
Clock systemClock() { return Clock.systemUTC(); } // production

@Test
void expiresExactlyAtTheBoundary() {
    Clock fixed = Clock.fixed(Instant.parse("2026-03-14T00:00:00Z"), ZoneOffset.UTC);
    SubscriptionService service = new SubscriptionService(fixed);
    assertThat(service.isExpired(SubscriptionMother.expiringAt("2026-03-14T00:00:00Z")), isFalse());
    assertThat(service.isExpired(SubscriptionMother.expiringAt("2026-03-13T23:59:59Z")), isTrue());
}
```

Common follow-ups:

- How do you test time zones? Run the logic with explicit zones and assert the converted values; pin the JVM default in test configuration so results do not depend on the machine.
- How do you keep fixtures readable? Builders or object mothers with valid defaults and per-test overrides.
- What about random data? Seed the generator and log the seed so failures are reproducible.

Mistakes to avoid: `Instant.now()` inside business logic; tests that assume today is a weekday; shared mutable fixtures; relying on the machine's default time zone; unseeded randomness.

Production perspective: Injecting a clock also pays off in production: it makes time-travel debugging and simulation possible, and it lets you run scheduled logic deterministically in a staging environment.

Related concepts covered: Clock injection, object mothers, seeded randomness, test isolation, time zone pinning, parallel execution.

Q185. How do you review a PR for correctness and maintainability?

Priority: Bonus

Why interviewers ask it: Review quality is visible evidence of engineering judgement at the three-to-four-year level.

Interview-ready answer: I read the description and the tests first, because they tell me what the change is supposed to do. Then I check correctness at the boundaries: null and empty cases, concurrency, transaction scope, error handling and idempotency. Next, the contract: does this change break API consumers or the database schema for a rolling deploy? Then security: authorization checks, input validation, injection risk, logging of sensitive data. Then maintainability: naming, cohesion, whether the abstraction earns its keep. I separate blocking comments from suggestions, explain the reasoning rather than just the rule, and prefer asking a question when I might be missing context. Small, focused pull requests get better reviews, so I also push back on large mixed changes.

In-depth explanation: The highest-value comments cluster in a few areas: data loss and correctness risks, security, backward compatibility, and missing tests for the failure paths. Style issues should be automated with a formatter and linter so humans spend their attention elsewhere. It is worth explicitly checking things that are easy to miss in a diff: a new endpoint without authorization rules, a migration that is not backward compatible, a `@Transactional` boundary that now wraps an HTTP call, a new dependency with licensing or vulnerability implications, and logging that dumps whole objects. Tone matters for throughput as much as for morale: reviews that read as collaboration rather than gatekeeping get faster iterations and better outcomes.

Practical backend example: A concise review checklist:

Correctness	null/empty/boundary cases, concurrency, idempotency, transaction scope
Contract	API compatibility, schema migration safety for rolling deploys, event schema
Security	authz on new endpoints, input validation, injection, secrets, sensitive logging
Failure modes	timeouts, retries, what happens when the dependency is down
Data	indexes for new query patterns, N+1 risk, unbounded result sets
Tests	failure paths covered, no sleeps, deterministic data, meaningful assertions
Operability	metrics/logs for the new path, feature flag or rollback plan
Maintainability	naming, cohesion, dead code, abstraction that earns its keep

Common follow-ups:

- What feedback is highest priority? Anything that risks data loss, a security hole or a broken contract; style comes last and should be automated.
- How do you review a very large PR? Ask for it to be split; beyond a few hundred lines, review quality drops sharply.
- How do you disagree productively? State the risk and the reasoning, propose an alternative, and be explicit about whether it blocks merging.

Mistakes to avoid: Rubber-stamping; bikeshedding formatting; demanding changes without explaining why; reviewing only the diff without the surrounding context; ignoring missing tests because the code "looks right".

Production perspective: Review is one of the cheapest defect-detection mechanisms available, and the defects it catches best — missing authorization, unsafe migrations, absent error handling — are exactly the ones that cause incidents rather than test failures.

Related concepts covered: Backward compatibility, migration safety, security review, test adequacy, operability, review process and PR size.

Chapter 9. Messaging and caching

Examples use Apache Kafka with Spring for Apache Kafka, and Redis with Spring Data Redis. The principles — at-least-once delivery, idempotent consumers, invalidation, stampede control — apply to other brokers and caches, but the configuration names are Kafka- and Redis-specific.

Q186. What do Kafka topics, partitions and consumer groups do?

Priority: Must Know

Why interviewers ask it: These three concepts determine ordering, parallelism and scaling limits, and most Kafka mistakes come from misunderstanding them.

Interview-ready answer: A topic is a named log of records, split into partitions. Each partition is an append-only, ordered sequence, and ordering is guaranteed only within a partition — never across a topic. The producer's partitioner decides where a record goes, normally by hashing the key, so all records with the same key land in the same partition and stay ordered relative to each other. A consumer group is a set of consumers that share the work: each partition is assigned to exactly one consumer in the group, so the partition count is the upper bound on parallelism for that group. Different groups consume the same topic independently, each with its own committed offsets, which is how several services react to the same events.

In-depth explanation: Offsets are per group, per partition, and committing an offset means "I have processed everything up to here". Rebalancing reassigns partitions when a consumer joins, leaves or is deemed dead; during a rebalance, processing for the affected partitions pauses, and a consumer that takes longer than `max.poll.interval.ms` to process a batch is evicted, which produces the classic cycle of rebalance and duplicate processing. Choosing the partition count is a capacity decision made early: increasing it later changes the key-to-partition mapping, so records for an existing key may start landing in a different partition and ordering across the change is not preserved. Keys should be chosen for both ordering and distribution — an entity ID is usually right, while a low-cardinality key such as country creates hot partitions.

Practical backend example: Keyed production and a group consumer:

```
// same orderId -> same partition -> ordered for that order
kafkaTemplate.send("orders", order.id().toString(), new OrderPlaced(order));

@KafkaListener(topics = "orders", groupId = "billing") // billing has its own offsets
public void onOrderPlaced(ConsumerRecord<String, OrderPlaced> record) {
    log.info("partition={} offset={} key={}", record.partition(), record.offset(), record.key());
    billingService.handle(record.value());
}
```

```
spring.kafka.consumer.max-poll-records=50
spring.kafka.consumer.properties.max.poll.interval.ms=300000 # must exceed worst-case batch time
```

Common follow-ups:

- Is ordering global? No; only within a partition. Use a key to keep related records ordered.
- What limits consumer parallelism? The partition count: extra consumers in a group sit idle.
- What happens during a rebalance? Partitions are reassigned and processing pauses; slow processing can trigger repeated rebalances.

Mistakes to avoid: Assuming topic-wide ordering; one partition "to keep it ordered" and then wondering why throughput is low; a low-cardinality key producing hot partitions; changing partition count without considering key mapping; processing longer than the poll interval.

Production perspective: Monitor consumer lag per partition, not just per topic — a single lagging partition usually means a hot key or a stuck consumer, and the topic-level average hides it.

Related concepts covered: Partitioning and keys, consumer groups and offsets, rebalancing, parallelism limits, consumer lag.

Q187. What does at-least-once delivery require from a consumer?

Priority: Must Know

Why interviewers ask it: Duplicate handling is the single most important correctness requirement in message-driven systems.

Interview-ready answer: At-least-once means a message can be delivered more than once — after a rebalance, a redelivery, or a crash between processing and offset commit — so the consumer must be idempotent. The practical pattern is a processed-messages table keyed by a stable identifier from the message, written in the same database transaction as the business change: if the key already exists, skip; otherwise apply and record. Where the operation is naturally idempotent — setting a status, an upsert keyed by a business identifier — you get it for free. Offsets are committed after successful processing, never before, and if processing fails the message is retried. Exactly-once across a broker and an external database is not something you get by configuration; you build it from at-least-once plus deduplication.

In-depth explanation: Kafka's transactional producer plus `read_process_write` gives exactly-once semantics within Kafka, but the moment a side effect lands in PostgreSQL or an external API, the guarantee no longer covers it. Deduplication therefore lives in your database, where it can share the transaction with the business write. Ordering interacts with retries: if a failing message is retried while later messages proceed, ordering for that key is broken, which is why per-key blocking retry or a dead-letter path is a deliberate choice rather than a detail. Manual acknowledgement mode makes the commit point explicit, and the dedupe table needs a retention policy — a partitioned table or a TTL index — because it grows with every message.

Practical backend example: Transactional deduplication in the consumer:

```
@KafkaListener(topics = "payments", groupId = "ledger")
public void onPayment(PaymentCaptured event, Acknowledgment ack) {
    try {
        ledgerService.apply(event);           // @Transactional: dedupe insert + business write together
        ack.acknowledge();                   // commit the offset only after success
    } catch (TransientException e) {
        throw e;                             // let the error handler retry / route to DLT
    }
}

@Transactional
public void apply(PaymentCaptured event) {
    int inserted = processedRepository.insertIfAbsent(event.eventId()); // ON CONFLICT DO NOTHING
    if (inserted == 0) {
        log.debug("duplicate eventId={ } ignored", event.eventId());
        return; // idempotent no-op
    }
    ledger.record(event.accountId(), event.amountMinor());
}
```

```
CREATE TABLE processed_event (
    event_id      uuid PRIMARY KEY,
    processed_at  timestamptz NOT NULL DEFAULT now()
);
-- prune with a scheduled delete or partition by month
```

Common follow-ups:

- When can duplicates occur? Crash after processing but before the offset commit, rebalances, and producer retries.
- Is exactly-once achievable? Within Kafka, yes with transactions; across an external database, you build it from at-least-once plus deduplication.
- Where does the dedupe key come from? A producer-assigned event ID, or a business key such as payment reference — never the offset alone.

Mistakes to avoid: Committing offsets before processing; auto-commit with non-idempotent handlers; deduplicating in memory only; an unbounded dedupe table; assuming a database transaction rolls back an already-sent email.

Production perspective: Test duplicate delivery deliberately — replay the same message and assert the state is unchanged. It is the one failure mode that will certainly occur and is rarely covered by tests.

Related concepts covered: Idempotent consumers, offset commit timing, dedupe tables, Kafka transactions, retention of dedupe state.

Q188. How do retries and dead-letter handling work for poison messages?

Priority: Important

Why interviewers ask it: Without a considered retry and dead-letter policy, one bad message can stall a partition indefinitely.

Interview-ready answer: First distinguish transient failures — a timeout, a temporarily unavailable dependency — from permanent ones such as a deserialisation error or a validation failure. Transient failures deserve a bounded number of retries with exponential backoff and jitter. Permanent failures should go straight to a dead-letter topic, because retrying them wastes capacity and blocks the partition. In Spring for Apache Kafka, `DefaultErrorHandler` with an `ExponentialBackOff` plus classified non-retryable exceptions handles both, and `DeadLetterPublishingRecoverer` publishes the failed record with headers describing the original topic, partition, offset and exception. The dead-letter topic must be monitored and have an owner — an unwatched DLT is a silent data-loss queue.

In-depth explanation: Blocking retries hold up the partition, which preserves ordering but hurts throughput; non-blocking retries via `@RetryableTopic` republish to delay topics so the main partition continues, at the cost of reordering. That is a genuine trade-off: choose blocking when per-key ordering matters, non-blocking when throughput and isolation matter more. Deserialisation errors need `ErrorHandlingDeserializer`, otherwise the failure happens before your listener and cannot be handled. Replaying from a DLT should be a deliberate, tooling operation: fix the cause, then republish selected records to the main topic, ideally with an audit of what was replayed. And remember that the retry budget interacts with `max.poll.interval.ms` — long blocking retries can trigger a rebalance.

Practical backend example: Classified retries and a monitored dead-letter topic:

```
@Bean
DefaultErrorHandler errorHandler(KafkaTemplate<Object, Object> template, MeterRegistry meters) {
    ExponentialBackOff backOff = new ExponentialBackOff(500L, 2.0);    // 0.5s, 1s, 2s ...
    backOff.setMaxElapsedTime(30_000L);                               // bounded

    DeadLetterPublishingRecoverer recoverer = new DeadLetterPublishingRecoverer(template,
        (record, ex) -> new TopicPartition(record.topic() + ".DLT", record.partition()));

    DefaultErrorHandler handler = new DefaultErrorHandler(
        (record, ex) -> { meters.counter("kafka.dlt", "topic", record.topic()).increment();
                        recoverer.accept(record, ex); },
        backOff);
    handler.addNotRetryableExceptions(                                // permanent -> straight to DLT
        DeserializationException.class, ValidationException.class, IllegalArgumentException.class);
    return handler;
}
```

```
spring.kafka.consumer.value-deserializer=org.springframework.kafka.support.serializer
    .ErrorHandlingDeserializer
spring.kafka.consumer.properties.spring.deserializer.value.delegate.class=org.springframework.kafka
    .support.serializer.JsonDeserializer
```

Common follow-ups:

- Can retries make things worse? Yes; retrying a failing call against a struggling dependency amplifies load. Bound the retries and back off.
- Blocking or non-blocking retries? Blocking preserves per-key ordering but stalls the partition; non-blocking keeps throughput but reorders.
- What do you do with a DLT message? Alert, diagnose, fix the cause, then replay deliberately with an audit trail.

Mistakes to avoid: Infinite retries; retrying deserialisation failures; a DLT nobody monitors; retry loops longer than `max.poll.interval.ms`; discarding the original headers so replay is impossible.

Production perspective: Alert on any message reaching the dead-letter topic, with the exception type as a tag. The first DLT message of a new type is usually the earliest signal of a contract change upstream.

Related concepts covered: Error classification, exponential backoff, DeadLetterPublishingRecoverer, ErrorHandlingDeserializer, replay procedures, poll interval interaction.

Q189. How does the transactional outbox reduce dual-write risk?

Priority: Must Know

Why interviewers ask it: Dual writes are the most common source of inconsistency between a database and a broker, and the outbox is the standard remedy.

Interview-ready answer: The problem is writing to the database and publishing to a broker as two separate operations: if the publish fails after the commit, consumers never learn about the change; if it succeeds before a rollback, they learn about something that never happened. The outbox pattern makes it one atomic write: the business change and a row in an outbox table are committed in the same database transaction. A separate relay polls unpublished rows, sends them to the broker and marks them sent. Because the relay can crash between sending and marking, delivery is at-least-once, so consumers must still be idempotent. What the outbox guarantees is that a committed change is eventually published, and that a rolled-back change is never published.

In-depth explanation: Two relay styles exist. Polling reads unsent rows with `SELECT ... FOR UPDATE SKIP LOCKED` so multiple instances can share the work without conflict; it is simple and sufficient for most systems, with latency set by the poll interval. Change-data-capture reads the database's write-ahead log — Debezium is the common implementation — which reduces latency and load but adds operational complexity. Ordering deserves thought: if per-aggregate ordering matters, include a sequence or publish with the aggregate ID as the Kafka key and have a single relay process rows per aggregate in ID order. The outbox table needs pruning, since it grows at the rate of business events. A lightweight alternative for some cases is `@TransactionalEventListener(phase = AFTER_COMMIT)`, but that publishes in-process and loses the event if the application crashes at the wrong moment — the outbox survives that.

Practical backend example: Atomic write plus a competing-consumer relay:

```
@Transactional
public Order placeOrder(PlaceOrderCommand command) {
    Order order = orderRepository.save(Order.from(command));
    outboxRepository.save(new OutboxMessage(
        UUID.randomUUID(), "orders", order.id().toString(),
        json.write(new OrderPlaced(order)));
    return order;
}
```

```
-- relay: several instances, no double-sending, no blocking
SELECT id, topic, message_key, payload
FROM outbox
WHERE published_at IS NULL
ORDER BY created_at
LIMIT 100
FOR UPDATE SKIP LOCKED;
```

```
@Scheduled(fixedDelay = 500)
@Transactional
public void relay() {
    for (OutboxMessage m : outboxRepository.lockUnpublishedBatch(100)) {
        kafkaTemplate.send(m.topic(), m.messageKey(), m.payload()); // may duplicate after a crash
        m.markPublished(Instant.now(clock));
    }
}
```

Common follow-ups:

- Does it give exactly-once delivery? No; it gives at-least-once with atomicity against the database. Consumers must deduplicate.
- Polling or CDC? Polling is simpler and adequate at moderate volume; CDC lowers latency and database load at the cost of operational complexity.
- How do you keep the table small? Delete or archive published rows on a schedule, or partition by day.

Mistakes to avoid: Publishing inside the transaction and assuming rollback un-sends it; a relay without SKIP LOCKED that serialises or double-sends; never pruning the table; assuming the outbox removes the need for idempotent consumers.

Production perspective: Alert on outbox lag — the age of the oldest unpublished row. A stalled relay is invisible from the API's perspective while downstream systems quietly fall behind.

Related concepts covered: Dual-write problem, atomic commit, SKIP LOCKED relays, CDC, ordering by key, outbox pruning and lag.

Q190. How do you preserve useful event ordering?

Priority: Important

Why interviewers ask it: Ordering assumptions that hold in testing and break under load cause data corruption that is hard to trace.

Interview-ready answer: Decide which ordering you actually need. Global ordering across a topic is usually unnecessary and very expensive — it requires a single partition. What matters is ordering per entity, and you get that by using the entity ID as the partition key, so all its events go to the same partition and are consumed in sequence. Beyond that, make consumers robust to out-of-order arrival: include a version or timestamp in the event and ignore anything older than what you have already applied. That defends against redelivery, retries that reorder, and producers racing each other. In short: key for the ordering you need, and design the consumer so that ordering is an optimisation rather than a correctness requirement.

In-depth explanation: Several mechanisms break ordering even with correct keying. Non-blocking retries move a failed record to a delay topic while later records proceed. Multi-threaded consumption within a partition reorders unless you partition the work by key again in the consumer. Producer retries with `max.in.flight.requests.per.connection` above 1 can reorder on retry unless idempotent production is enabled (`enable.idempotence=true`, the default since the 3.0 clients, which preserves ordering with up to five in-flight requests). Repartitioning changes key placement, so events for one key can briefly exist in two partitions. The monotonic guard — `UPDATE ... WHERE version < :version` — covers all of these cases with one mechanism, and it composes with idempotent consumption.

Practical backend example: Keyed production plus a version guard in the consumer:

```
// produce: entity id as key -> per-entity ordering
kafkaTemplate.send("inventory", sku, new StockChanged(sku, quantity, version));
```

```
@KafkaListener(topics = "inventory", groupId = "catalog-projection")
@Transactional
public void onStockChanged(StockChanged event) {
    int updated = projectionRepository.applyIfNewer(event.sku(), event.quantity(), event.version());
    if (updated == 0) {
        log.debug("stale event sku={} version={} ignored", event.sku(), event.version());
    }
}
```

```
UPDATE stock_projection
SET quantity = :quantity, version = :version, updated_at = now()
WHERE sku = :sku AND version < :version; -- older events are no-ops
```

Common follow-ups:

- What if two producers race? The broker's arrival order decides, so the consumer's version guard, not the producer, must enforce correctness.
- Does a single partition guarantee global order? Yes, but it caps throughput at one consumer and is rarely the right trade.
- How do non-blocking retries affect ordering? They reorder by design; use blocking retries when per-key ordering must be preserved.

Mistakes to avoid: Assuming events arrive in production order; keying by something too coarse or too fine; multi-threading within a partition without re-partitioning by key; relying on event timestamps from different machines as

an ordering source.

Production perspective: Out-of-order bugs surface as rare, inexplicable state — a cancelled order showing as active. The version guard turns that class of incident into a logged no-op, which is why it is worth adding before you need it.

Related concepts covered: Partition keys, idempotent producers, in-flight requests, monotonic version guards, retry-induced reordering, projection correctness.

Q191. What is cache-aside and when should Redis be used?

Priority: Must Know

Why interviewers ask it: Caching is the most common performance tool and the easiest one to apply in the wrong place.

Interview-ready answer: Cache-aside means the application checks the cache, and on a miss loads from the database, stores the value with a TTL and returns it. The database stays the source of truth; the cache is a disposable copy. Redis is the right choice when the same data is read far more often than it changes, when the value is expensive to compute, and when it must be shared across instances — a local in-process cache would otherwise give each instance a different view. It is the wrong choice when data changes on almost every read, when staleness is unacceptable, or when the query is already fast and the cache only adds a network hop and an invalidation problem. I always measure the hit rate afterwards, because a cache with a low hit rate is pure overhead.

In-depth explanation: The alternatives are read-through and write-through caching, where the cache library owns loading and writing, and write-behind, which buffers writes and risks loss. Cache-aside is the most common because it is explicit and keeps failure handling in your code. That failure handling matters: if Redis is down, the application should fall back to the database rather than fail — unless the database cannot survive the load, in which case failing fast protects it. A local cache such as Caffeine in front of Redis gives a two-level design, with the caveat that local entries are invalidated independently and can be stale for their TTL. Serialisation format matters too: storing Java-serialised objects couples the cache to your class layout, so JSON or a compact binary format is preferable, and you should version the key namespace so a format change does not read old entries.

Practical backend example: Explicit cache-aside with degradation:

```
public ProductView getProduct(String sku) {
    String key = "product:v2:" + sku; // versioned namespace
    try {
        ProductView cached = redis.opsForValue().get(key);
        if (cached != null) { hits.increment(); return cached; }
    } catch (RedisConnectionFailureException e) {
        log.warn("cache unavailable, falling back to database"); // degrade, do not fail
    }
    misses.increment();
    ProductView loaded = productRepository.load(sku);
    try {
        redis.opsForValue().set(key, loaded,
            Duration.ofMinutes(10).plusSeconds(ThreadLocalRandom.current().nextInt(60))); // TTL
        jitter
    } catch (RedisConnectionFailureException ignored) { /* cache write is best effort */ }
    return loaded;
}
```

Common follow-ups:

- What happens if Redis is down? Fall back to the source and keep serving, unless the database cannot absorb the load.
- Local cache or Redis? Local is faster but per-instance and harder to invalidate; Redis is shared and consistent across instances.
- How do you know it is working? Hit rate, latency percentiles before and after, and load on the source system.

Mistakes to avoid: Caching data that changes constantly; no TTL; unbounded key growth; treating the cache as the source of truth; caching personal data without considering access and retention; Java serialisation as the value format.

Production perspective: Size and evict deliberately: set maxmemory with an eviction policy so Redis degrades predictably instead of failing writes. Track hit rate and evictions as first-class metrics.

Related concepts covered: Cache-aside versus read-through, TTL jitter, graceful degradation, two-level caching, key versioning, eviction policy.

Q192. How do TTL and invalidation affect correctness?

Priority: Must Know

Why interviewers ask it: Invalidation determines how stale your reads can be, and the ordering of cache and database operations is a classic source of bugs.

Interview-ready answer: TTL bounds staleness: with a five-minute TTL, a read can be up to five minutes out of date, and that must be an accepted business decision rather than an accident. Explicit invalidation on write reduces that window but introduces an ordering problem. The safest simple pattern is to write to the database first, commit, then delete the cache entry — deleting rather than updating, so the next read repopulates from the source. Deleting before the commit risks another request repopulating the cache with the old value before the new one is visible. Even then a race is possible, so the TTL remains the backstop. For data that must never be stale, do not cache it, or use a very short TTL and accept the cost.

In-depth explanation: Delete-after-commit is important in a Spring application because `@CacheEvict` on a `@Transactional` method runs when the method returns, which is before the transaction commits in some arrangements; binding eviction to `AFTER_COMMIT` avoids publishing a gap where a concurrent read repopulates stale data. Updating the cache with the new value instead of deleting it looks efficient but is riskier under concurrency, because two writers can interleave and leave the older value in place. Multi-instance deployments add another dimension: local caches need an invalidation broadcast (a Redis pub/sub message or a Kafka topic) or they will serve stale data for their full TTL. Finally, consider the cost of a mass invalidation: clearing a whole namespace can produce a stampede against the database, which is the subject of the next question.

Practical backend example: Eviction after commit, not before:

```
@Transactional
public void updatePrice(String sku, long priceMinor) {
    productRepository.updatePrice(sku, priceMinor);
    events.publishEvent(new ProductChanged(sku));    // handled after commit
}

@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void evict(ProductChanged event) {
    redis.delete("product:v2:" + event.sku());    // delete, do not overwrite
    localCache.invalidate(event.sku());           // plus broadcast to other instances
}
```

```
// Spring Cache equivalent, with the same after-commit concern in mind
@CacheEvict(cacheNames = "products", key = "#sku")
public void refresh(String sku) { /* called from the after-commit listener */ }
```

Common follow-ups:

- Should every write invalidate? Only the keys affected; clearing whole namespaces on each write turns the cache into overhead and risks a stampede.
- Why delete instead of update? Deletion is idempotent and race-tolerant; concurrent updates can otherwise leave the older value cached.
- How do you invalidate local caches on other instances? Broadcast the invalidation over pub/sub or a topic; otherwise they stay stale for their TTL.

Mistakes to avoid: Evicting before commit; caching with no TTL and relying entirely on invalidation; invalidating with the wrong key format; ignoring other instances' local caches; caching a value derived from data you do not invalidate on.

Production perspective: Write down the acceptable staleness for each cached dataset — it is a product decision, and once it is explicit the TTL and invalidation design follows. Undocumented staleness surfaces later as a support ticket about "wrong" data.

Related concepts covered: Staleness budgets, delete-versus-update, after-commit eviction, distributed invalidation, key hygiene, Spring Cache semantics.

Q193. How do you prevent a cache stampede or hot key?

Priority: Important

Why interviewers ask it: Stampedes turn a cache from a protection into an amplifier, usually at the worst possible moment.

Interview-ready answer: A stampede happens when many requests miss at the same time — a popular key expires, or the cache is cleared or restarted — and they all hit the database simultaneously. Three defences work together. First, TTL jitter, so keys created together do not expire together. Second, single-flight: only one caller recomputes a given key while the others wait or serve the previous value, implemented with a short-lived Redis lock keyed on the entry. Third, early or background refresh, recomputing a hot key before it expires so there is never a miss. For a genuinely hot key, add a short local cache in front of Redis so most requests never leave the process, and consider sharding the key if it is a counter.

In-depth explanation: Probabilistic early expiration is an elegant variant: each read recomputes with a probability that rises as the entry approaches expiry, so refreshes are spread out and one unlucky request does the work before the crowd arrives. The lock-based approach needs care — the lock must have a TTL so a crashed holder does not block the key forever, and waiters need a bounded wait with a fallback rather than an unbounded block. Serving stale-while-revalidate is often the best user experience: return the expired value immediately and refresh in the background, which requires storing a soft expiry alongside the value. Hot keys also create a Redis-side problem, since one key lives on one shard and can saturate it; a local cache or key sharding (counter:{id}:{0..9} summed on read) spreads that load.

Practical backend example: Single-flight with a lock and a stale fallback:

```
public ProductView get(String sku) {
    String key = "product:v2:" + sku;
    ValueOperations<String, CachedValue<ProductView>> values = redis.opsForValue();
    CachedValue<ProductView> entry = values.get(key);
    if (entry != null && !entry.isSoftExpired()) return entry.value();

    // only one caller recomputes; the lock has a TTL so a crash cannot wedge the key
    boolean acquired = Boolean.TRUE.equals(
        redis.opsForValue().setIfAbsent("lock:" + key, "1", Duration.ofSeconds(10)));
    if (!acquired && entry != null) {
        return entry.value(); // stale-while-revalidate: serve the old value
    }
    try {
        ProductView fresh = productRepository.load(sku);
        values.set(key, CachedValue.of(fresh, Duration.ofMinutes(10)),
            Duration.ofMinutes(10).plusSeconds(ThreadLocalRandom.current().nextInt(120)));
        return fresh;
    } finally {
        if (acquired) redis.delete("lock:" + key);
    }
}
```

Common follow-ups:

- What if Redis restarts? Every key misses at once; rate-limit or queue the repopulation, and warm critical keys before taking traffic.
- Why jitter the TTL? Entries written together otherwise expire together and produce a synchronised miss.
- How do you handle a single very hot key? A short local cache in front of Redis, and sharding the key if it is a counter.

Mistakes to avoid: Identical TTLs everywhere; locks without a TTL; unbounded waiting for the lock holder; clearing the entire cache during a deploy; assuming the database can absorb a full-miss period.

Production perspective: Rehearse a cache-loss scenario in a load test — restart Redis under traffic and watch the database. Many systems are quietly dependent on a warm cache and discover it during an incident.

Related concepts covered: TTL jitter, single-flight locking, stale-while-revalidate, probabilistic early expiry, hot-key sharding, cache warming.

Q194. How do Redis data structures and atomic operations help?

Priority: Important

Why interviewers ask it: Using Redis only as a string cache misses most of its value, and atomicity is where correctness lives.

Interview-ready answer: Redis has purpose-built structures: strings and counters for simple values and rate limits, hashes for grouped fields you update individually, sets for membership and deduplication, sorted sets for leaderboards, rankings and time-ordered windows, lists for simple queues, and streams for durable message-like consumption. The important property is that single commands are atomic, so INCR, SETNX and EXPIRE-based patterns give you counters and locks without a read-modify-write race. When several commands must be atomic together, a Lua script runs on the server as a single unit — that is how correct rate limiters and check-and-set patterns are built. Choosing the right structure usually replaces application logic with one server-side operation.

In-depth explanation: INCR is atomic, but "increment and set a TTL only on first creation" is two commands and therefore racy — hence the Lua script. Sorted sets solve sliding windows neatly: add an entry scored by timestamp, remove entries older than the window with ZREMRANGEBYSCORE, and count the rest. Redis is single-threaded for command execution, which is why atomicity is straightforward, but it also means an expensive command such as KEYS on a large database blocks everything — use SCAN instead. Keys need TTLs or explicit lifecycle management, because a structure that only grows will eventually hit maxmemory and trigger evictions across the whole instance. For distributed locks, SET key value NX PX ttl with a unique value and a Lua-guarded release is the minimum; even then, locks in Redis are advisory and can be lost during failover, so they must not be the only protection for money-critical operations.

Practical backend example: Atomic counter with TTL, and a sliding window:

```
// atomic: increment and set expiry only when the key is created
private static final String INCR_WITH_TTL = """
    local current = redis.call('INCR', KEYS[1])
    if current == 1 then redis.call('PEXPIRE', KEYS[1], ARGV[1]) end
    return current
""";

public long recordAttempt(String userId) {
    return redis.execute(RedisScript.of(INCR_WITH_TTL, Long.class),
        List.of("attempts:" + userId, String.valueOf(Duration.ofMinutes(15).toMillis())));
}

// sliding window with a sorted set
public long requestsInLastMinute(String apiKey) {
    long now = clock.millis();
    String key = "rl:" + apiKey;
    redis.opsForZSet().add(key, UUID.randomUUID().toString(), now);
    redis.opsForZSet().removeRangeByScore(key, 0, now - 60_000); // drop entries outside the window
    redis.expire(key, Duration.ofMinutes(2));
    return redis.opsForZSet().zCard(key);
}
```

Common follow-ups:

- Is INCR atomic? Yes, but "increment and expire" is two commands; use a Lua script to make the pair atomic.
- When would you use a sorted set? Leaderboards, sliding windows, and anything needing range queries by score.
- Are Redis locks safe? They are advisory and can be lost during failover; never rely on them alone for money-critical invariants.

Mistakes to avoid: KEYS in production; read-modify-write instead of an atomic command; keys without TTLs; storing large blobs; assuming persistence guarantees equivalent to a database; long-running Lua scripts blocking the server.

Production perspective: Watch memory, evicted keys and command latency. Because command execution is single-threaded, one slow command or one very large value degrades every client at once.

Related concepts covered: Redis data structures, atomic commands, Lua scripting, sliding windows, SCAN versus KEYS, lock caveats, memory limits.

Q195. How do you choose between an event and a synchronous request?

Priority: Important

Why interviewers ask it: It is an architecture judgement question, and the reasoning matters more than the choice.

Interview-ready answer: I ask whether the caller needs the result to continue. If it does — a payment authorisation, a stock check before confirming — it is a synchronous request with a timeout, because the answer is part of the response. If the work is a consequence rather than a precondition — sending a confirmation email, updating a search index, notifying analytics — an event is better: it decouples the services, keeps the API fast, and lets the receiver fail and retry without affecting the caller. The trade-off is consistency and observability: events make the system eventually consistent and harder to trace, so the caller must not assume the effect has happened. Ownership is the other factor: if the receiver decides what to do, an event fits; if the caller needs a specific action performed, a command or request fits.

In-depth explanation: Availability coupling is the strongest argument for events. A synchronous chain multiplies failure probabilities, and a slow dependency consumes your threads. An event decouples in time: the broker absorbs the outage and the consumer catches up. But the costs are real — eventual consistency requires read-your-writes handling in the UI, duplicate handling in the consumer, schema evolution discipline for the event payload, and a tracing setup that propagates context through the broker. A common hybrid works well: do the minimum synchronous work needed to answer the caller correctly, then emit an event for everything else. It is also worth distinguishing event types: a notification ("order placed") that receivers interpret, versus an event carrying state that receivers store — the latter reduces callbacks but couples the schema more tightly.

Practical backend example: Synchronous where the answer is needed, asynchronous for consequences:

```
@Transactional
public OrderResponse placeOrder(PlaceOrderCommand command) {
    // synchronous: the caller cannot proceed without this answer
    ReservationResult reservation = inventoryClient.reserve(command.items()); // timeout + retry
    budget
    if (!reservation.successful()) {
        throw new OutOfStockException(reservation.unavailableSkus());
    }
    Order order = orderRepository.save(Order.from(command, reservation.id()));

    // asynchronous consequences: email, analytics, search index - via the outbox
    outboxRepository.save(OutboxMessage.of("orders", order.id().toString(), new OrderPlaced(order)));
    return OrderResponse.from(order);
}
```

Common follow-ups:

- How does the caller learn the outcome of async work? Polling a status endpoint, a webhook, or a push channel — and the API must expose a pending state.
- What does an event cost you? Eventual consistency, duplicate handling, schema evolution and harder debugging without trace propagation.
- Can you mix them? Yes, and usually should: synchronous for the decision, events for the consequences.

Mistakes to avoid: Making everything asynchronous and then needing the result immediately; synchronous chains three or four services deep; events with no schema discipline; assuming the consumer processed the event because it was published.

Production perspective: Whichever you choose, make the outcome observable: a trace that spans the broker, and a metric for events published versus processed. Silent event loss is much harder to notice than a failed HTTP call.

Related concepts covered: Temporal decoupling, availability multiplication, eventual consistency, commands versus events, hybrid designs, trace propagation.

Chapter 10. Deployment, observability, and production troubleshooting

The final five questions are integrative: they cross layers on purpose, because that is how senior interviewers separate a candidate who has operated a service from one who has only written code for it. Examples assume Spring Boot 3 on containers with Micrometer, but the reasoning transfers.

Q196. What belongs in a production-ready Docker image?

Priority: Must Know

Why interviewers ask it: The image is the deployment unit, and its contents reveal how someone thinks about size, security and reproducibility.

Interview-ready answer: A multi-stage build: one stage with the JDK and build tool to produce the artefact, a final stage with only a JRE and the application. Use a pinned base image rather than a floating tag, run as a non-root user, and keep the image minimal — no build tools, no shell utilities you do not need, no secrets. Layer for cache efficiency: dependencies change rarely, so extract the Spring Boot layers and copy dependency layers before application classes, which makes rebuilds and pulls much faster. Configuration and secrets come in at runtime through environment variables or mounted files, never baked in. Set container-aware JVM options such as `-XX:MaxRAMPercentage`, expose the actuator health endpoints for probes, and scan the image for vulnerabilities in CI.

In-depth explanation: Layered jars matter more than they first appear: without them, every rebuild produces one fat jar layer of a hundred megabytes that must be pushed and pulled in full. With `-Djarmode=layertools` extract, dependencies, Spring Boot loader, snapshot dependencies and application classes land in separate layers, and only the last changes on a typical commit. Modern JVMs are container-aware and respect cgroup limits, so avoid fixed `-Xmx` and use `MaxRAMPercentage`, leaving headroom for metaspace, thread stacks and native memory. Signal handling matters for graceful shutdown: the JVM must be PID 1 or run under an init that forwards `SIGTERM`, otherwise the container is killed abruptly. Reproducibility comes from pinning base image digests and dependency versions, and from building in CI rather than on a developer machine.

Practical backend example: A layered, non-root Dockerfile:

```
# ---- build ----
FROM eclipse-temurin:21-jdk-alpine AS build
WORKDIR /build
COPY gradle/ gradle/
COPY gradlew build.gradle.kts settings.gradle.kts ./
RUN ./gradlew dependencies --no-daemon # cached unless the build files change
COPY src/ src/
RUN ./gradlew bootJar --no-daemon
RUN java -Djarmode=layertools -jar build/libs/*.jar extract --destination extracted

# ---- runtime ----
FROM eclipse-temurin:21-jre-alpine
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app
COPY --from=build --chown=app:app /build/extracted/dependencies/ ./
COPY --from=build --chown=app:app /build/extracted/spring-boot-loader/ ./
COPY --from=build --chown=app:app /build/extracted/snapshot-dependencies/ ./
COPY --from=build --chown=app:app /build/extracted/application/ ./
USER app
ENV JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75 -XX:+ExitOnOutOfMemoryError"
EXPOSE 8080
# launcher class is org.springframework.boot.loader.JarLauncher before Boot 3.2
ENTRYPOINT ["java", "org.springframework.boot.loader.launch.JarLauncher"]
```

Common follow-ups:

- How should secrets enter the container? At runtime from a secret store, as environment variables or mounted files; never in the image or the build args.
- Why not a fixed `-Xmx`? `MaxRAMPercentage` adapts to the container's memory limit, so the same image works across environments.
- Why run as non-root? It limits what a compromised process can do, and many platforms require it.

Mistakes to avoid: JDK in the runtime image; latest tags; running as root; secrets in layers (they remain in the history); one fat jar layer; ignoring the container memory limit; no vulnerability scanning.

Production perspective: Image size affects deployment speed and therefore rollback speed. When an incident requires a rollback, a small, cached image is the difference between seconds and minutes of impact.

Related concepts covered: Multi-stage builds, layered jars, container-aware JVM flags, non-root users, image scanning, reproducible builds.

Q197. Which logs, metrics and traces would you add to an API?

Priority: Must Know

Why interviewers ask it: Observability decides whether an incident takes five minutes or five hours, and the three signals have distinct jobs.

Interview-ready answer: Metrics answer "is something wrong and how bad": request rate, error rate and latency percentiles per endpoint, plus resource signals — connection pool usage, thread pool queue depth, GC pause time, cache hit rate, consumer lag. Traces answer "where is the time going or where did it fail" by following one request across services with spans. Logs answer "what exactly happened" for a specific request, and should be structured JSON with a correlation and trace ID so they join up with the other two. I instrument the business events that matter too — orders placed, payments failed — because technical metrics can look healthy while the business outcome is broken. Cardinality discipline is essential: never put user IDs or raw paths with identifiers into metric tags.

In-depth explanation: Percentiles matter more than averages because tail latency is what users experience; Micrometer can publish histograms so percentiles are computed correctly at the aggregation layer rather than averaged from per-instance values, which is mathematically wrong. The USE and RED framings are useful shorthands: for resources, utilisation, saturation and errors; for request-driven services, rate, errors and duration. Sampling keeps tracing affordable — a small percentage of normal requests plus, where supported, retention of slow or failed traces. Log levels need discipline: INFO for state changes worth keeping, WARN for recoverable anomalies, ERROR for things needing attention, DEBUG off in production but switchable at runtime through the actuator loggers endpoint. Finally, an alert should map to a human action; alerts nobody acts on train people to ignore the pager.

Practical backend example: Correlated structured logs plus business metrics:

```
// MDC filter: every log line carries the ids needed to join logs, traces and metrics
@Component
class CorrelationFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        String correlationId = Optional.ofNullable(req.getHeader("X-Correlation-Id"))
            .orElseGet(() -> UUID.randomUUID().toString());
        MDC.put("correlationId", correlationId);
        res.setHeader("X-Correlation-Id", correlationId);
        try { chain.doFilter(req, res); } finally { MDC.clear(); }
    }
}

// business metric with bounded cardinality
meterRegistry.counter("orders.placed", "channel", order.channel(), "currency", order.currency())
    .increment();
// NOT: .tag("customerId", id) -> unbounded cardinality, will break the metrics backend
```

```
management:
  endpoints.web.exposure.include: health,info,metrics,prometheus,loggers
  metrics.distribution.percentiles-histogram.http.server.requests: true
  tracing.sampling.probability: 0.1
```

```
logging pattern: %d %-5level [%X{traceId:-},%X{spanId:-},%X{correlationId:-}] %logger{36} - %msg%n
```

Common follow-ups:

- What must not be logged? Credentials, tokens, full card numbers, personal data beyond identifiers — logs are broadly readable and long-lived.
- Why are averages insufficient? They hide the tail; p95 and p99 describe what slow users actually experience.

- What is the cardinality risk? Tags with unbounded values create a time series per value and can overwhelm the metrics backend.

Mistakes to avoid: Logging whole request bodies; user IDs as metric tags; traces without propagation across the broker; alerting on every error rather than on rates; DEBUG logging left on in production.

Production perspective: The test of an observability setup is a dry run: pick a plausible failure and see whether the dashboards and logs would identify it. If you cannot answer "which dependency is slow" in under a minute, the instrumentation is incomplete.

Related concepts covered: RED and USE metrics, histograms and percentiles, trace sampling, MDC correlation, cardinality control, actionable alerting.

Q198. How do readiness, liveness and graceful shutdown work together?

Priority: Important

Why interviewers ask it: Rolling deployments drop requests when these are misconfigured, and the failure is subtle enough to be blamed on the network.

Interview-ready answer: Liveness answers "is this process broken beyond recovery" and its failure causes a restart, so it must check only the process itself — a liveness probe that checks the database will restart every instance during a database outage and turn a degradation into an outage. Readiness answers "can this instance serve traffic right now" and its failure removes the instance from the load balancer without restarting it; that is the right place for dependency checks and for the draining signal. Graceful shutdown ties them together: on SIGTERM the application marks itself unready, waits for the load balancer to notice, then stops accepting new requests and lets in-flight ones finish within a timeout before closing. Spring Boot provides all three — the health groups for probes and `server.shutdown=graceful`.

In-depth explanation: The ordering problem is that removal from the load balancer is eventually consistent: the platform notices unreadiness after a probe interval, so if the process stops accepting connections immediately on SIGTERM, requests routed in that gap fail. The standard fix is a `preStop` hook that sleeps a few seconds — longer than the readiness probe period — before the signal reaches the application, or an explicit `unready-then-wait` sequence. Kafka consumers need their own handling: on shutdown the container should stop polling and finish the current batch, so a message is not left half-processed, and the offset commit reflects reality. Startup probes deserve mention because a JVM application can take tens of seconds to warm up; without one, an aggressive liveness probe kills the pod in a restart loop. Finally, the shutdown timeout must exceed your longest legitimate request, or graceful shutdown truncates it anyway.

Practical backend example: Probes, graceful shutdown and a drain delay:

```
server:
  shutdown: graceful                # stop accepting, finish in-flight
spring:
  lifecycle:
    timeout-per-shutdown-phase: 30s # must exceed the longest legitimate request
management:
  endpoint.health.probes.enabled: true
  endpoint.health.group.liveness.include: livenessState # process only - no dependencies
  endpoint.health.group.readiness.include: readinessState,db,redis
```

```
# Kubernetes: drain before the process stops accepting connections
lifecycle:
  preStop:
    exec: { command: ["sh", "-c", "sleep 8"] } # longer than the readiness probe period
startupProbe: { httpGet: { path: /actuator/health/liveness, port: 8080 }, failureThreshold: 30,
  periodSeconds: 5 }
livenessProbe: { httpGet: { path: /actuator/health/liveness, port: 8080 }, periodSeconds: 10 }
readinessProbe: { httpGet: { path: /actuator/health/readiness, port: 8080 }, periodSeconds: 5 }
terminationGracePeriodSeconds: 45 # > preStop + shutdown timeout
```

Common follow-ups:

- What if a consumer is mid-message? The listener container should stop polling and finish the in-flight batch before the process exits, so offsets match the work done.

- Why must liveness exclude dependencies? Otherwise a shared dependency outage restarts every instance simultaneously, removing any chance of partial service.
- Why a startup probe? JVM startup can exceed a liveness threshold, producing a restart loop before the application ever becomes ready.

Mistakes to avoid: The same endpoint for liveness and readiness; database checks in liveness; no drain delay; a termination grace period shorter than the shutdown timeout; ignoring background workers and consumers during shutdown.

Production perspective: Verify it empirically: run a load test and perform a rolling deploy. Zero failed requests is achievable, and if you see a burst of connection errors at each pod replacement, the drain sequence is wrong.

Related concepts covered: Health groups, startup probes, drain delays, in-flight request completion, consumer shutdown, termination grace periods.

Q199. How would you triage rising latency and errors after a deploy?

Priority: Must Know

Why interviewers ask it: It tests incident behaviour: whether you stabilise first and investigate second.

Interview-ready answer: Mitigate first, diagnose second. If latency and errors rose right after a deploy, the deploy is the prime suspect, and rolling back is usually faster and safer than debugging in production — the investigation can continue from logs and traces afterwards. Before rolling back I take a quick snapshot for later: the error signature, a slow trace, thread and heap information if the JVM looks unhealthy. In parallel I check whether the change is really the cause: compare error rate, latency percentiles and resource metrics before and after the deploy boundary, confirm whether all instances or only the new ones are affected, and check whether a config or feature flag changed rather than the code. Communication matters throughout — declare the incident, state impact, and keep a timeline.

In-depth explanation: The evidence usually points somewhere specific. Connection pool saturation shows as hikaricp.connections.pending above zero with connection acquisition timeouts, while individual queries remain fast — often caused by a new query holding connections longer, or a transaction now wrapping a slow remote call. A database problem shows as increased mean execution time in pg_stat_statements, frequently after a plan change or a missing index on a new query. Memory pressure shows as rising GC time and falling throughput, ending in OutOfMemoryError. A downstream dependency shows as one span dominating traces and rising timeout counts. If only the new instances are affected, it is the code or its configuration; if all are, suspect a shared dependency, a data change or a coincidental event. Canary or blue-green deployment makes this comparison direct, which is the strongest argument for them.

Practical backend example: The first five minutes, concretely:

```
# 1) is it the deploy? compare the boundary
kubectl rollout history deployment/orders-api
kubectl get pods -l app=orders-api -o wide           # new vs old instances still serving?

# 2) snapshot evidence BEFORE rolling back
kubectl logs deploy/orders-api --since=10m | grep -c "ERROR"
kubectl exec deploy/orders-api -- jcmd 1 Thread.print > /tmp/threads.txt
kubectl exec deploy/orders-api -- jcmd 1 GC.heap_info

# 3) narrow the layer
curl -s localhost:8080/actuator/metrics/hikaricp.connections.pending # pool saturation
curl -s localhost:8080/actuator/metrics/jvm.gc.pause                 # memory pressure
psql -c "SELECT calls, mean_exec_time, query FROM pg_stat_statements ORDER BY mean_exec_time DESC
LIMIT 5;"

# 4) mitigate: roll back, or disable the new path behind its flag
kubectl rollout undo deployment/orders-api
```

Common follow-ups:

- What evidence points to database pool saturation? Pending connection requests and acquisition timeouts while the queries themselves are fast.
- When do you roll back rather than fix forward? Whenever the rollback is safe and quick — restore service first, unless a schema change makes rollback unsafe.

- What if the schema changed? Expand-contract migrations keep the previous version working, which is exactly what makes rollback possible.

Mistakes to avoid: Debugging while users are affected; rolling back without capturing evidence; changing several things at once; assuming correlation with the deploy proves causation; forgetting that a feature flag flip is also a change.

Production perspective: Write the blameless postmortem and fix the detection gap, not just the bug. If the deploy caused a ten-minute outage before anyone noticed, the alerting is as much the finding as the code.

Related concepts covered: Rollback versus fix-forward, evidence capture, pool and GC metrics, canary comparison, expand-contract migrations, incident communication.

Q200. How would you investigate an intermittent production order failure end to end?

Priority: Must Know

Why interviewers ask it: It is the integrative question: it touches HTTP, transactions, databases, messaging, concurrency and prevention in one narrative.

Interview-ready answer: I start by defining the failure precisely: what "failure" means to the customer, how often it happens, since when, and whether it correlates with a tenant, a payment method, a time of day or an instance. Then I follow one failing order end to end using its correlation and trace IDs — API request, transaction, payment call, outbox row, Kafka event, consumer processing — and find the first step where reality diverges from the expected sequence. Each layer has a characteristic signature: a timeout to the payment provider leaves an ambiguous pending state; a deadlock or serialisation failure shows as a rolled-back transaction under concurrency; a stalled outbox relay leaves committed orders with unpublished events; a duplicate or out-of-order event leaves an inconsistent projection. Once I can explain the mechanism, I fix it, add a regression test, and close the observability gap that let it stay hidden.

In-depth explanation: The value of the end-to-end trace is that it converts a vague report into a specific divergence point. If the API returned 201 but no event exists, look at the outbox: is the relay running, is the oldest unpublished row growing, did a serialisation error roll back the insert? If the event exists but the projection is wrong, look at the consumer: duplicate processing without deduplication, or an older event applied after a newer one. If the order is stuck in PENDING, look at the external call: a timeout leaves the outcome unknown, and without a reconciliation job those orders never resolve. Concurrency-specific failures — two clicks creating two orders, a deadlock under load — reproduce only with parallel tests, so a targeted concurrency test is usually part of the fix. Prevention is the other half of the answer: a unique constraint instead of check-then-act, idempotency keys on the write endpoint, a monotonic version guard in the projection, alerts on outbox lag and dead-letter arrivals, and a scheduled reconciliation for ambiguous states.

Practical backend example: The investigation path and the fixes it typically produces:

```
1. Define      failure = order paid by customer, never confirmed. ~12/day, started 9 days ago.
2. Correlate   find failing orderIds -> traceId -> full span timeline
3. API layer   201 returned? duplicate submissions? idempotency key present?
4. Transaction rolled back? SQLSTATE 40001 (serialisation) / 40P01 (deadlock) in logs?
5. External    payment call timed out -> outcome unknown -> order left PENDING
6. Outbox      SELECT min(created_at) FROM outbox WHERE published_at IS NULL;  -- relay lag
7. Messaging   consumer lag per partition; DLT contents; duplicate or stale events
8. Projection  applied out of order? version guard present?
```

```
-- the two highest-value structural fixes that usually come out of this
ALTER TABLE "order" ADD CONSTRAINT uq_order_client_reference
    UNIQUE (customer_id, client_reference);      -- duplicates become a conflict, not two orders

UPDATE order_projection SET status = :status, version = :version
WHERE order_id = :id AND version < :version;    -- stale events become no-ops
```

```
// ambiguous external outcomes get resolved, not abandoned
@Scheduled(fixedDelay = 60_000)
void resolvePendingPayments() {
    for (Order order : orderRepository.findPendingOlderThan(Duration.ofMinutes(2))) {
        paymentGateway.lookup(order.paymentReference()) // provider is the source of truth
            .ifPresent(status -> orderService.applyPaymentOutcome(order.id(), status));
    }
}
```

Common follow-ups:

- How do you prevent recurrence? Database constraints instead of application-level checks, idempotency keys, version guards, reconciliation jobs, and alerts on lag and dead-letter arrivals.
- What if you cannot reproduce it? Add targeted instrumentation around the suspected step and wait for the next occurrence with better evidence; concurrency issues often need a parallel test rather than manual reproduction.
- How do you know the fix worked? Track the specific failure rate as a metric over a window longer than its previous recurrence interval.

Mistakes to avoid: Restarting the service and calling it fixed; patching the symptom in the API while the data stays inconsistent; skipping the regression test because the bug is rare; failing to check whether affected customers need remediation.

Production perspective: Finish by asking two questions: what would have detected this sooner, and what would have prevented it structurally. Intermittent failures usually indicate a missing invariant — a constraint, an idempotency key, a guard — and adding it is worth more than the individual fix.

Related concepts covered: End-to-end tracing, transaction failure signatures, outbox lag, consumer deduplication and ordering, reconciliation of ambiguous states, structural prevention, regression testing.

Top 40 Questions to Revise First

These forty questions are the ones to rehearse if your interview is soon. The selection is a study judgement about coverage and consequence — questions whose answers unlock several others, and topics where a weak answer is expensive. It is not a claim about measured interview frequency, and no such statistics are used anywhere in this book.

Each row gives the question ID and the reason it earns a place. Speak each answer aloud in 45–90 seconds and answer at least one follow-up before moving on.

ID	Why start here
Q004	The equals and hashCode contract silently breaks sets, maps and caches, and it is asked constantly.
Q008	Checked versus unchecked exceptions shapes every API boundary you design.
Q012	Immutability is the cheapest defence against shared-state bugs, and it leads into records and thread safety.
Q019	Date, time and time-zone handling is where real APIs quietly produce wrong data.
Q020	Money requires deliberate precision and rounding; floating point here is a visible correctness failure.
Q031	Collection choice by semantics rather than habit is the entry point to the whole collections chapter.
Q032	HashMap internals explain collisions, resizing and why broken keys lose data.
Q038	Grouping and aggregating with streams is the most common practical stream task in backend code.
Q041	Knowing when not to use parallel streams shows judgement rather than enthusiasm.
Q051	Separating visibility from atomicity is the foundation of every concurrency answer.
Q053	Volatile is the most misused keyword in Java; explaining its limits is a strong signal.
Q055	Thread-pool sizing, queueing and rejection are daily production concerns in a Spring service.
Q058	Deadlock detection and prevention comes up whenever concurrency is discussed seriously.
Q064	Virtual threads in Java 21 are current, frequently asked, and easy to get subtly wrong.
Q076	Dependency injection and constructor injection underpins everything in the Spring chapter.
Q080	Auto-configuration explains why a Spring Boot application behaves as it does, and how to debug it.
Q082	The MVC request lifecycle is the map you need for error handling, filters and security questions.
Q085	Proxy-based <code>@Transactional</code> and self-invocation is one of the most common real Spring bugs.
Q086	<code>REQUIRED</code> versus <code>REQUIRES_NEW</code> decides whether your rollback behaviour is what you think it is.
Q099	Connection pool configuration connects Spring, the database and production latency in one answer.
Q111	Why an index exists but is not used separates people who read plans from people who guess.
Q112	Reading <code>EXPLAIN ANALYZE</code> calmly is the single most useful database skill in an interview.
Q114	Isolation levels and the anomalies they prevent is the classic transaction question.
Q116	Optimistic versus pessimistic locking is a decision you will be asked to justify, not just define.
Q121	N+1 diagnosis and repair is the most frequently encountered Hibernate performance problem.
Q126	Calling an external service inside a transaction is a design mistake interviewers love to probe.
Q137	Status code choices are a quick, high-signal test of API literacy.
Q141	Idempotency for payment creation is the retry-safety question, and it recurs in Chapters 6 and 9.
Q143	Timeouts, retries and backoff interact in ways that cause outages when misunderstood.
Q150	The end-to-end order workflow is the integrative design question for the REST chapter.
Q152	Preventing duplicate resources from duplicate requests tests constraints, not just controller code.
Q162	Sessions versus JWT is where candidates repeat slogans; a trade-off answer stands out immediately.
Q165	CORS and CSRF are routinely confused, and the confusion leads to disabling the wrong protection.
Q168	Resource-level authorization is the flaw class most often found in real systems.
Q173	A coherent test strategy is a design skill, and it frames every other testing question.
Q182	A systematic method for debugging a slow endpoint is what senior interviewers are listening for.

ID	Why start here
Q187	At-least-once delivery and idempotent consumers is the core correctness rule of messaging.
Q189	The transactional outbox is the standard answer to dual writes and comes up in system design too.
Q197	Logs, metrics and traces — knowing what each signal is for shows you have operated a service.
Q200	The final integrative question rehearses an end-to-end investigation across every layer.

Using this list

- **First pass:** read the interview-ready answer only, and speak it back. Roughly two hours for all forty.
- **Second pass:** answer the follow-ups and read the in-depth explanation for anything that felt shaky.
- **Third pass:** cover the page and answer from the ID alone. Anything you cannot start within five seconds goes on a short list for the day before the interview.

If you have time for only ten, take Q004, Q051, Q085, Q112, Q114, Q121, Q137, Q143, Q173 and Q197 — they span every chapter's core reasoning.

7-Day Interview Preparation Plan

Seven sessions of roughly 90–120 focused minutes: about 40 minutes reading, 30 minutes answering aloud, 30 minutes hands-on, and 10 minutes writing down what you missed. Every one of the 200 questions is assigned exactly once as a primary reading block; the spoken-practice IDs repeat deliberately, because retrieval is what makes an answer available under pressure.

If you have fewer than seven days, run Day 1, Day 4, Day 5 and Day 6 — they carry the largest chapters — and use `top-40.md` for the rest.

Day 1 — Core Java, OOP and exceptions

Read: Chapter 1, Q001–Q030.

Speak aloud (45–90 s each): Q001, Q004, Q008, Q012, Q019, Q020, Q027.

Hands-on: Implement an immutable Money value object with `BigDecimal`, correct `equals/hashCode`, and a scale and rounding policy. Write a test that puts it in a `HashSet` and proves lookup works. Then write a small exception hierarchy for a service and map it to HTTP status codes on paper.

Check yourself: Can you explain why a mutable field in a hash key loses entries, and why `BigDecimal.equals` is not the same as `compareTo == 0`?

Day 2 — Collections, generics, streams, and two coding exercises

Read: Chapter 2, Q031–Q050. Then Chapter 8, Q179 and Q180.

Speak aloud: Q031, Q032, Q036, Q038, Q039, Q041, Q046.

Hands-on: Code the first non-repeating character (Q179) and merge intervals (Q180) without an IDE, on paper or in a plain editor, then run them against edge cases: empty input, all duplicates, touching intervals, a string containing an emoji. Then write a `groupBy` pipeline that produces a summary per customer and handles a null grouping key deliberately.

Check yourself: Can you say what `Collectors.toMap` does on a duplicate key, and when a plain loop is the better choice?

Day 3 — Concurrency, async and the JVM

Read: Chapter 3, Q051–Q075.

Speak aloud: Q051, Q053, Q055, Q056, Q058, Q064, Q067.

Hands-on: Reproduce a lost update with two threads incrementing a shared `int`, then fix it three ways — `synchronized`, `AtomicInteger`, `LongAdder` — and explain when each is appropriate. Then create a `ThreadPoolExecutor` with a bounded queue and a `CallerRunsPolicy`, saturate it, and observe the behaviour. Finally, take a thread dump with `jcmd <pid> Thread.print` and find the state of your pool threads.

Check yourself: Can you explain the core-pool, queue, max-pool ordering, and what happens to an exception thrown inside `submit`?

Day 4 — Spring Framework and Spring Boot

Read: Chapter 4, Q076–Q105.

Speak aloud: Q076, Q080, Q082, Q085, Q086, Q087, Q099, Q104.

Hands-on: Build one vertical slice end to end: controller, service, repository, DTO, Bean Validation, a `@ControllerAdvice` returning `ProblemDetail`, and a transaction boundary in the service. Then deliberately break it — call a `@Transactional` method from within the same bean and watch the transaction not start. Open `/actuator/conditions` and find why one auto-configuration did or did not apply.

Check yourself: Can you draw the request path from the servlet container to your controller method and back, naming where validation, security and exception handling sit?

Day 5 — SQL, transactions, JPA, Hibernate, and a SQL exercise

Read: Chapter 5, Q106–Q135. Then Chapter 8, Q181.

Speak aloud: Q111, Q112, Q114, Q116, Q119, Q121, Q126, Q129.

Hands-on: In a local PostgreSQL instance, write the latest-order-per-customer query (Q106) and the monthly top-customers query (Q181). Run EXPLAIN (ANALYZE, BUFFERS) on both, add an index, and compare. Then write a JPA test that proves an N+1 problem exists by counting statements, fix it with an entity graph, and assert the count dropped.

Check yourself: Can you explain what flush does that commit does not, and name two reasons an existing index is not used?

Day 6 — HTTP, REST, microservices and security

Read: Chapter 6, Q136–Q160. Then Chapter 7, Q161–Q172.

Speak aloud: Q137, Q141, Q143, Q144, Q150, Q162, Q165, Q168.

Hands-on: Design a retry-safe POST /payments on paper: idempotency key storage, the conflict rule, the timeout and retry policy with a circuit breaker, and the error body shape. Then write the Spring Security configuration for it — deny by default, scope-based rules, and an ownership check in the query — and list the three tests that prove it: 401, 403, 200.

Check yourself: Can you explain why CORS does not protect an API, and what makes a retry safe rather than merely repeated?

Day 7 — Testing, messaging, caching, production, and consolidation

Read: Chapter 8, Q173–Q178 and Q182–Q185. Then Chapter 9, Q186–Q195, and Chapter 10, Q196–Q200.

Speak aloud: Q173, Q174, Q182, Q187, Q189, Q191, Q197, Q199, Q200.

Hands-on: Write one @DataJpaTest against a Testcontainers PostgreSQL instance that proves a unique constraint fires. Then rehearse Q200 out loud as a five-minute incident narrative — definition, correlation, layer-by-layer investigation, fix, prevention — because that is the shape of most final-round questions. Finish by reviewing every question you marked as weak on Days 1–6.

Check yourself: Can you give the first four things you would check when latency rises after a deploy, and say which evidence distinguishes pool saturation from a slow query?

Coverage of the plan

Day	Primary reading	Question IDs	Count
1	Chapter 1	Q001–Q030	30
2	Chapter 2 + coding exercises	Q031–Q050, Q179, Q180	22
3	Chapter 3	Q051–Q075	25
4	Chapter 4	Q076–Q105	30
5	Chapter 5 + SQL exercise	Q106–Q135, Q181	31
6	Chapters 6 and 7	Q136–Q160, Q161–Q172	37
7	Chapters 8, 9 and 10	Q173–Q178, Q182–Q185, Q186–Q195, Q196–Q200	25
		Total	200

Every ID from Q001 to Q200 appears exactly once as a primary assignment. The day before the interview, do not read anything new: speak the Top 40 aloud and re-read only your own notes on what you missed.

Practical Mini-Exercises

Twelve exercises that turn reading into recall. Each is tied to the questions it exercises, and each has a solution outline rather than a finished implementation — write your own first, then compare. Most take 15–30 minutes; the last three are design exercises you can do on paper.

1. A safe value key (Q004, Q012, Q025, Q026)

Implement a key type for `(tenantId, externalOrderId)` that is used as a `HashMap` key and a `Set` member. Explain why a mutable field in the key would be unsafe, and what happens to an entry whose key mutates after insertion.

Solution outline: A record gives value equality and a consistent `hashCode` from its components: `record OrderKey(long tenantId, String externalOrderId) { OrderKey { Objects.requireNonNull(externalOrderId); } }`. Both components must be immutable, and the canonical constructor is the place to reject blank or oversized values. If a component could mutate, the entry's stored hash no longer matches its bucket, so lookups miss and the entry is effectively lost while still occupying memory — a leak and a correctness bug at once. Extension: add a `normalise` step that trims and lowercases the external identifier, and write a test proving two differently cased inputs collapse to one key.

2. First non-repeating character, Unicode-aware (Q047, Q179)

Return the first non-repeating character of a string, or an empty result. Then adapt it so that a string containing an emoji behaves correctly.

Solution outline: One pass into a `LinkedHashMap<Character, Integer>` with `merge(c, 1, Integer::sum)`, then take the first entry whose count is one — $O(n)$ time, $O(k)$ space, and insertion order comes free from the map. For Unicode, `char` iteration splits supplementary characters into surrogate pairs, so switch to `input.codePoints()` with `Integer` keys and return the result with `Character.toChars`. Write tests for: empty string, all duplicates, single character, a string whose only unique character is last, and a string containing an emoji. Say the assumption out loud before coding — case sensitivity and what counts as a character are both interviewer questions in disguise.

3. Merge overlapping intervals (Q049, Q180)

Merge a list of intervals, deciding explicitly whether touching intervals merge. Input `[1,3]`, `[2,5]`, `[8,9]` should produce `[1,5]`, `[8,9]`.

Solution outline: Copy the input, sort by start (then by end), then sweep: if the next start is not after the current end, extend the end to the maximum of the two; otherwise emit and restart. Remember to emit the final interval after the loop — that omission is the most common bug. Test containment (`[1,10]`, `[2,3]`), touching (`[1,2]`, `[2,3]`), an unsorted input and a single-element list. Extension: rewrite it for `Instant` values and state which time zone the inputs are in.

4. Prove and fix an N+1 query (Q120, Q121, Q175)

Take an endpoint that returns orders with their line items. Write a test that counts the SQL statements executed, assert the count is wrong, then fix it and assert the new count.

Solution outline: Enable Hibernate statistics in the test profile, reset them, call the service inside a transaction, and assert on `getPrepositionedQueryExecutionCount` — in practice `statistics.getQueryExecutionCount()` plus `getEntityLoadCount()` is enough to expose the pattern. The fix is an `@EntityGraph` on the repository method or a fetch join; `default_batch_fetch_size` is the blunt alternative that turns N queries into N/batch . Run it against Testcontainers PostgreSQL, not H2, and keep the assertion — it is what stops the regression coming back.

5. Keyset pagination over a large table (Q110, Q140)

Replace OFFSET-based paging with keyset pagination on an order table sorted by `created_at` descending, with a stable tiebreaker.

Solution outline: The predicate is `WHERE (created_at, id) < (:lastCreatedAt, :lastId) ORDER BY created_at DESC, id DESC LIMIT :size`, supported by an index on `(created_at DESC, id DESC)`. Encode the cursor as an opaque base64 string so clients cannot construct arbitrary predicates, and return `hasMore` rather than a total count, which is expensive on large tables. Compare `EXPLAIN (ANALYZE, BUFFERS)` for OFFSET 100000 against the keyset version and note where the time goes.

6. Make a transfer atomic and concurrency-safe (Q113, Q116, Q130, Q177)

Implement `transfer(from, to, amountMinor)` so that concurrent transfers cannot overdraw an account, then write a test with two real threads that proves it.

Solution outline: Either a conditional update — `UPDATE account SET balance_cents = balance_cents - :amount WHERE id = :from AND balance_cents >= :amount` checking the affected row count — or `SELECT ... FOR UPDATE` on both accounts in a deterministic order (lowest ID first) to avoid deadlocks. Amounts are integer minor units. The test must not be `@Transactional`: run two threads released by a `CountDownLatch`, assert exactly one succeeds and the final balance is correct. Add a retry on `SQLSTATE 40001` and `40P01` and explain why a retry is safe here.

7. Idempotent payment creation (Q141, Q152, Q153)

Design and implement `POST /payments` so that a client retry after a timeout cannot create two payments.

Solution outline: An `idempotency_key` table with the key as primary key, a hash of the request body, the resulting status and the stored response. Insert the key in the same transaction as the payment; a duplicate key with a matching hash returns the stored response, and a duplicate key with a different hash returns 409. Back it with a database unique constraint on `(customer_id, client_reference)` so correctness does not depend on application timing. For the timeout case, model a `PENDING` state and a scheduled resolver that asks the provider for the real outcome rather than guessing.

8. Idempotent Kafka consumer (Q187, Q189, Q190)

Write a consumer that survives duplicate delivery and out-of-order arrival.

Solution outline: A `processed_event` table keyed by the producer-assigned event ID, inserted with `ON CONFLICT DO NOTHING` in the same transaction as the business write; zero rows inserted means skip. For ordering, include a version in the event and apply with `UPDATE ... WHERE version < :version`, so stale events become no-ops. Commit the offset only after the transaction succeeds — manual acknowledgement mode makes that explicit. Test by replaying the same record twice and by applying version 3 before version 2, asserting the final state both times.

9. Cache-aside with stampede protection (Q191, Q192, Q193)

Add a Redis cache in front of a slow product lookup, with a correct invalidation path and protection against a synchronised miss.

Solution outline: Versioned key namespace (`product:v2:{sku}`), TTL with random jitter, and a `try/catch` that falls back to the database if Redis is unavailable. Invalidate by deleting the key in an `AFTER_COMMIT` listener, not inside the transaction. Add single-flight: `SET lock:{key} NX PX 10000`, with waiters serving the previous value when one exists. Measure the hit rate before and after — if it is low, the cache is overhead, and removing it is the correct outcome of the exercise.

10. Secure an endpoint properly (Q161, Q167, Q168, Q176)

Take `GET /api/orders/{id}` and make it correct for authentication, coarse authorization and resource-level authorization, then prove it with tests.

Solution outline: A `SecurityFilterChain` with `anyRequest().authenticated()` and explicit permits; scope or role rules per method; and the ownership condition inside the repository query (`findByIdAndCustomerId`) so a missing check cannot leak. Return 404 rather than 403 for another tenant's record. Tests: anonymous request expects 401, wrong authority expects 403, correct principal but another tenant's order expects 404, owner expects 200. That fourth test is the one most teams do not have.

11. Design a resilient outbound call (Q143, Q144, Q148, Q153)

On paper, specify the full policy for calling a payment provider from your order service.

Solution outline: A connect timeout of about one second and a read timeout derived from the remaining request deadline; retries only for connection failures, 429 and 5xx, and only for idempotent or idempotency-keyed requests; exponential backoff with jitter and a hard cap of two or three attempts; a circuit breaker with a minimum call count before it can open and a half-open probe; a bulkhead so this dependency cannot consume every thread; and an honest fallback — a pending state and a resolver, not a fabricated success. Write down what the caller sees in each

failure mode, including the ambiguous timeout-after-commit case.

12. Run an incident end to end (Q182, Q183, Q199, Q200)

Rehearse, out loud and in five minutes, the investigation of "some customers are charged but their order never confirms".

Solution outline: Define the failure and its rate; find affected order IDs and pull one full trace; walk the layers in order — API response, transaction outcome, external call, outbox row, Kafka event, consumer projection — and name the signature you would expect at each: SQLSTATE 40001 for a serialisation failure, a growing oldest-unpublished-row age for a stalled relay, consumer lag or dead-letter arrivals for the messaging layer, a stale projection for missing version guards. Finish with prevention: a unique constraint, an idempotency key, a version guard, a reconciliation job, and alerts on outbox lag and dead-letter volume. Then state what would have detected it sooner — that sentence is what interviewers remember.

Glossary

Short definitions for terms used across the book. Where a term is version-dependent or commonly misused, the entry says so.

Java language and runtime

Autoboxing: Automatic conversion between a primitive and its wrapper type. Introduces null risk on unboxing and identity surprises with `==` outside the cached range.

Class loading: The process of locating, linking and initialising a class. Static initialisers run once, on first active use, and exceptions there surface as `ExceptionInInitializerError`.

Erasure: Generic type information is removed at compile time, so `List<String>` and `List<Integer>` share one runtime class. Explains why some overloads clash and why reified type checks are unavailable.

Escape analysis: A JIT optimisation that can avoid heap allocation for objects that do not escape a method. A reason microbenchmarks mislead.

Fail-fast iterator: An iterator that throws `ConcurrentModificationException` when the backing collection is structurally modified during iteration, detected through a modification counter.

Functional interface: An interface with a single abstract method, usable as a lambda target.

Happens-before: The Java memory model relation that guarantees one action's effects are visible to another. Established by synchronisation, volatile access, thread start and join, and similar actions.

JIT compilation: Runtime compilation of hot bytecode to machine code. Causes warm-up effects that invalidate naive benchmarks.

Record: A concise, shallowly immutable carrier for data with generated `equals`, `hashCode` and accessors. Suitable for DTOs and value objects, not for JPA entities.

Sealed hierarchy: A type whose permitted subtypes are fixed at compile time, enabling exhaustive pattern matching.

Structural modification: A change to a collection's size or internal structure, as opposed to replacing a value.

Virtual thread: A lightweight thread scheduled by the JVM, finalised in Java 21. Makes blocking I/O cheap to scale; does not make CPU-bound work faster and can pin to a carrier thread in some cases.

Collections and streams

Load factor: The fill ratio at which a hash table resizes, 0.75 by default in `HashMap`.

Short-circuiting operation: A stream operation that can finish without consuming the whole source, such as `findFirst` or `anyMatch`.

Splititerator: The traversal and splitting abstraction behind streams; its characteristics determine how well a source parallelises.

Stable sort: A sort that preserves the relative order of equal elements. Java's object sort is stable.

Treeify: `HashMap`'s conversion of a long collision chain into a balanced tree, at eight entries in a bin when the table has at least 64 slots.

Concurrency

ABA problem: A compare-and-swap succeeds because a value returned to its original state, hiding intermediate changes. Addressed with version stamps.

Backpressure: Limiting or slowing input when a consumer cannot keep up, rather than buffering without bound.

Bulkhead: An isolation limit — a separate pool or semaphore — so one slow dependency cannot consume all resources.

CAS (compare-and-swap): An atomic instruction that updates a value only if it still holds an expected value. The basis of the atomic classes.

Lost update: Two concurrent read-modify-write sequences where one overwrites the other's change.

Memory visibility: Whether one thread's write is observable by another. Distinct from atomicity: `volatile` gives visibility, not compound atomicity.

Safe publication: Making an object visible to other threads in a fully constructed state, through final fields, `volatile`, synchronisation or a concurrent collection.

Work stealing: A scheduling strategy in which idle workers take tasks from busy ones; used by the common `ForkJoinPool` that backs parallel streams.

JVM operation

GC pause: A stop-the-world interval during garbage collection. Contributes directly to tail latency.

Heap dump: A snapshot of heap contents, taken with `jcmd GC.heap_dump` and analysed with a tool such as MAT.

JFR (Java Flight Recorder): Low-overhead JVM event recording used for profiling allocation, locks, GC and I/O.

Metaspace: Native memory holding class metadata; exhausting it produces an `OutOfMemoryError` distinct from a heap one.

Thread dump: A snapshot of all thread stacks and states, obtained with `jcmd Thread.print`; the first tool for deadlocks and latency stalls.

Spring and application framework

Auto-configuration: Conditional bean definitions contributed by Spring Boot starters, applied when their conditions match and typically backing off when you define your own bean.

Bean: An object whose lifecycle is managed by the Spring application context.

Constructor injection: Supplying dependencies through the constructor, which makes them mandatory, final and testable without the framework.

Filter versus interceptor: A servlet filter wraps the whole request outside Spring MVC; a `HandlerInterceptor` runs inside MVC with knowledge of the resolved handler.

Profile: A named configuration set activated per environment.

Propagation: How a transactional method joins or creates a transaction — `REQUIRED` joins, `REQUIRES_NEW` suspends and starts another, `NESTED` uses a savepoint.

Proxy: The wrapper Spring creates to apply behaviour such as transactions or security. Explains why self-invocation bypasses `@Transactional`.

Slice test: A test that starts only part of the application context, such as `@WebMvcTest` or `@DataJpaTest`.

Databases, transactions and persistence

ACID: Atomicity, consistency, isolation and durability — the guarantees a transaction provides.

Covering index: An index that contains every column a query needs, allowing an index-only scan.

Dirty checking: Hibernate's detection of changes to managed entities, producing `UPDATE` statements at flush time.

Entity states: Transient, managed, detached and removed — the JPA lifecycle states that determine what persist, merge and remove do.

Flush: Synchronising pending persistence-context changes to the database. Not the same as commit; a flush can be rolled back.

Isolation level: The rules defining which concurrent effects a transaction can observe — read committed, repeatable read, serializable, and the anomalies each prevents.

Keyset pagination: Paging by a predicate on the last seen sorted key rather than `OFFSET`, which keeps cost constant as pages deepen.

N+1 problem: One query for a collection followed by one query per element, usually from lazy associations accessed in a loop.

Optimistic locking: Detecting concurrent modification with a version column at write time, failing the loser.

Persistence context: The first-level cache and unit of work holding managed entities for a transaction.

Pessimistic locking: Taking database locks up front with `SELECT ... FOR UPDATE` so conflicting transactions wait.

Sargable predicate: A condition an index can be used for, typically because the column is not wrapped in a function.

Serialisation failure: `SQLSTATE 40001` — a transaction aborted to preserve isolation. Usually retried.

Skip locked: `FOR UPDATE SKIP LOCKED`, which lets competing workers claim different rows without blocking each other.

Write skew: An anomaly where two transactions each read a consistent state and write non-conflicting rows that jointly violate an invariant.

HTTP, REST and distributed systems

Circuit breaker: A component that stops calling a failing dependency for a period, then probes it, preventing cascading failure.

Content negotiation: Selecting a representation from the client's Accept header; a mismatch yields 406, an unsupported request body type yields 415.

Deadline propagation: Passing the remaining time budget down a call chain so downstream work is not started when the caller has already given up.

ETag: A representation validator enabling conditional requests — If-None-Match for caching (304) and If-Match for optimistic concurrency (412).

Eventual consistency: A model where replicas or projections converge after a delay rather than updating atomically.

Idempotency: Repeating an operation has the same effect as performing it once. A property of the implementation, not only of the HTTP method.

Idempotency key: A client-supplied identifier stored server-side so a retried request returns the original outcome instead of creating a duplicate.

Problem Detail: The standard JSON error format for HTTP APIs, defined by RFC 7807 and updated by RFC 9457, available in Spring 6 as `ProblemDetail`.

Retry budget: A cap on the proportion of traffic that may be retries, preventing retry storms during an outage.

Safe method: An HTTP method with no intended side effects — GET, HEAD, OPTIONS.

Trace context: The W3C traceparent header propagating trace and span identifiers across services.

Messaging and caching

At-least-once delivery: Messages may be delivered more than once, so consumers must be idempotent.

Cache-aside: The application reads the cache, loads from the source on a miss, then populates the cache. The database remains the source of truth.

Consumer group: A set of Kafka consumers sharing a topic's partitions, each partition assigned to one member, with offsets tracked per group.

Consumer lag: How far behind a consumer is from the latest offset; the primary health metric for a consumer.

Dead-letter topic: A destination for messages that cannot be processed, so a poison message does not block a partition.

Dual write: Writing to two systems without a shared transaction, risking inconsistency when one fails.

Partition key: The value whose hash decides a record's partition, and therefore what ordering is preserved.

Rebalance: Reassignment of partitions when consumers join or leave; processing pauses for affected partitions.

Stampede: Many simultaneous cache misses hitting the source at once, typically after synchronised expiry or a cache restart.

Transactional outbox: Writing an event row in the same database transaction as the business change, with a relay publishing it afterwards.

TTL: Time to live — the bound on how stale a cached value may be.

Security

Argon2id / bcrypt / scrypt: Adaptive password hashing algorithms with tunable cost, designed to be slow. Appropriate for passwords; general-purpose hashes are not.

CORS: A browser mechanism letting a server declare which origins may read its responses. Not a server-side access control.

CSRF: An attack causing a victim's browser to send a state-changing request using ambient credentials such as cookies.

IDOR: Insecure direct object reference — accessing another user's record by supplying its identifier, because ownership is not checked.

JWKS: The JSON Web Key Set published by an issuer, from which a verifier selects a key by `kid` to validate token signatures.

JWT: A signed, base64url-encoded token. Signed means tamper-evident, not confidential — claims are readable.

OAuth2 / OpenID Connect: OAuth2 delegates authorization; OIDC adds authentication on top with an ID token and standard claims.

PKCE: Proof Key for Code Exchange, protecting the authorization code flow for clients that cannot keep a secret.

Scope: A delegated permission carried by a token; distinct from a role, which is a property of the user.

Testing and operations

Blue-green / canary deployment: Release strategies that run the new version alongside the old, enabling direct comparison and fast rollback.

Expand-contract migration: Adding schema changes in backward-compatible steps so old and new application versions can run simultaneously.

Flaky test: A test that passes and fails without code changes, usually because of shared state, timing or real clocks.

Liveness probe: A check whose failure restarts the process. Should test the process only, never its dependencies.

Readiness probe: A check whose failure removes the instance from load balancing without restarting it.

RED metrics: Rate, errors, duration — the standard signals for a request-driven service.

Testcontainers: A library that runs real dependencies in containers for tests, avoiding in-memory substitutes that behave differently.

USE metrics: Utilisation, saturation, errors — the standard signals for a resource such as a pool or a disk.

Final Coverage Checklist

This checklist records what the book contains and how that is verified. Every line is checked mechanically by `scripts/validate.py`; nothing here is asserted by hand.

Chapter counts

#	Chapter	File	Questions	IDs
1	Core Java, OOP and exceptions	chapters/01-core-java.md	30	Q001–Q030
2	Collections, generics, streams and functional Java	chapters/02-collections-and-streams.md	20	Q031–Q050
3	Concurrency, async and the JVM	chapters/03-concurrency-and-jvm.md	25	Q051–Q075
4	Spring Framework and Spring Boot	chapters/04-spring.md	30	Q076–Q105
5	SQL, transactions, JPA and Hibernate	chapters/05-sql-jpa-hibernate.md	30	Q106–Q135
6	HTTP, REST and microservices	chapters/06-rest-and-microservices.md	25	Q136–Q160
7	Security	chapters/07-security.md	12	Q161–Q172
8	Testing, debugging and coding exercises	chapters/08-testing-debugging-coding.md	13	Q173–Q185
9	Messaging and caching	chapters/09-messaging-and-caching.md	10	Q186–Q195
10	Deployment, observability and production troubleshooting	chapters/10-production-operations.md	5	Q196–Q200
	Total		200	Q001–Q200

Numbering and uniqueness

- [x] Question numbering is continuous from Q001 to Q200 with no gaps.
- [x] Each ID appears exactly once as a main question heading across all chapters.
- [x] No main question title is duplicated, in the plan or in the chapters.
- [x] Chapter question titles match the master plan in `question-plan.md` verbatim.
- [x] Each chapter's question count matches the required distribution above.

Required elements per question

Every one of the 200 questions contains all ten required elements:

- [x] The interview question itself, phrased as an interviewer would ask it (the `## Qnnn . heading`).
- [x] **Priority** — Must Know, Important or Bonus.
- [x] **Why interviewers ask it** — one sentence.
- [x] **Interview-ready answer** — a spoken answer of roughly 45–90 seconds.
- [x] **In-depth explanation** — the reasoning behind the short answer.
- [x] **Practical backend example** — a compact snippet or worked example where one helps.
- [x] **Common follow-ups** — two to four, each with a brief answer.
- [x] **Mistakes to avoid**.
- [x] **Production perspective**.
- [x] **Related concepts covered**.

Extras

- [x] **A. Top 40 to revise first** — `extras/top-40.md`, 40 distinct real question IDs, each with a reason.
- [x] **B. How to use this book** — `extras/how-to-use-this-book.md`.

- [x] **C. 7-day study plan** — extras/study-plan.md, Day 1 to Day 7, every ID assigned exactly once as primary reading.
- [x] **D. Practical mini-exercises** — extras/mini-exercises.md, 12 exercises tied to question IDs with solution outlines.
- [x] **E. Glossary** — extras/glossary.md, grouped by topic.
- [x] **F. Final coverage checklist** — this file.

Content quality rules applied

- [x] No fabricated statistics, benchmark numbers, citations or interview-frequency claims anywhere in the book.
- [x] Version-dependent behaviour is attributed to its Java or Spring Boot version rather than stated as universal.
- [x] Guarantees are distinguished from implementation details (for example ordering guarantees versus observed iteration order).
- [x] Where a question involves a choice, the answer explains when each option is appropriate rather than declaring a universal winner.
- [x] No unfinished-draft or stub text anywhere in the sources; the validator rejects the usual draft markers.
- [x] All fenced code blocks are balanced; Java, SQL, YAML and configuration samples were reviewed for correctness against the stated stack.

How to verify

From the repository root:

```
python3 java-backend-interview-book/scripts/validate.py
```

The validator checks the master plan's integrity, per-chapter counts and numbering, the presence of all nine labelled sections and at least two follow-up bullets per question, minimum answer length, duplicate or drifting titles, draft markers, balanced code fences, the extras' references to real question IDs, the Top 40 table size, the Day 1–7 study plan, and the combined manuscript's heading sequence and required sections. It exits non-zero if anything fails.